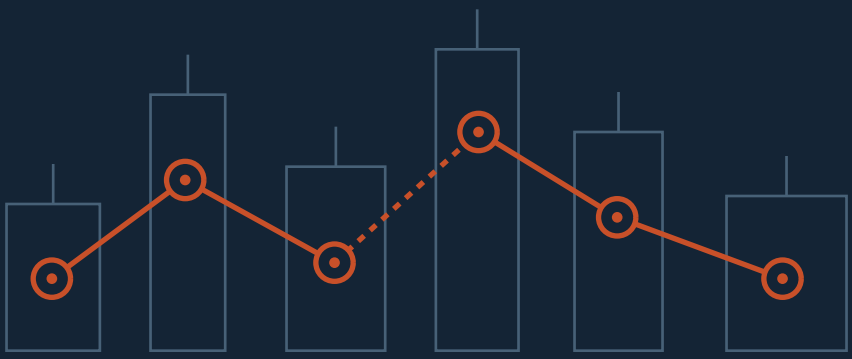
THE FERRUM EXPEDITION

UNDERSTANDING RUST

CAMPAIGN EDITION

42 levels / 4 paths / 5 zones

Advanced concepts for experienced developers



Choose a route. Build an artifact. Defend the invariant.

Rust 2024 edition / November 2026

Welcome to Ferrum

Ferrum was built by people who trusted their systems a little too much. The archive kept references to records after their storage was recycled. The foundry let incompatible components claim the same interface. Dispatch towers accepted work faster than the city could finish it. Deep below them, a reclamation engine treated an address as proof that an object still existed.

You arrive after the failures have stopped looking unrelated. Your contract is to restore the city's systems and leave behind an explanation another developer can defend. The compiler is a demanding instrument on your workbench. It can detect many invalid relationships. It cannot decide what a payment retry means or whether a shutdown protocol eventually finishes. Those are your encounters.

This is an advanced Rust book with a campaign structure. The technical field manuals retain their full explanations, worked examples, exercises, and solutions. The campaign adds choices that matter: which systems you study first, which constraints you accept, when you ask for a hint, and what evidence you produce before marking a level complete.

Choose Your Expedition

The Architect path explores trait relationships, type erasure, API evolution, and library boundaries. The Warden path explores ownership across threads, scheduling, cancellation, and service survival. The Delver path descends into layout, pointer validity, initialization, and unsafe abstractions. The Pathfinder path connects storage choices to complete parsers, stores, pipelines, and protocols.

These are routes through one world, not exclusive classes. Follow a path, leave it for a side quest, and return when a later encounter exposes something you need. Every numbered level on the map is a link. The suggested loadout on a level identifies useful prior knowledge. It does not lock the door or pretend to know what you already understand.

The complete campaign is Levels 1 through 42. If you already know a topic, take its boss encounter first. If you can produce the artifact, justify its

invariants, and explain the failure cases, mark the level mastered. If you get stuck, enter the field manual at the exact level rather than rereading material at random.

The Three Marks of Mastery

Scout earns 20 XP. Read the mission and inspect at least one worked example. Before compiling or reading its explanation, predict the important outcome: which value moves, which borrow remains active, which trait obligation applies, or which event makes progress possible. Write a prediction that could be wrong. Correct it after checking.

Forge earns 40 XP. Complete the level's boss encounter and produce the requested artifact. This can be an implementation, a deliberately rejected program, a schedule, an API signature, a failure table, or an unsafe proof argument. Satisfy the listed win conditions. Code that compiles is evidence for some conditions; it does not automatically satisfy every condition.

Explain earns 40 XP. Write a short debrief naming the owner or capability, the invariant, a failure path, and one tradeoff. Where relevant, distinguish memory safety from application correctness and local completion from a remote effect. Compare your reasoning with the vault debrief after making your own attempt.

The first three marks complete a level for 100 XP. A side quest adds 25 XP after you produce its additional evidence. The full 42-level campaign has 4,200 core XP and up to 1,050 side-quest XP. XP is a self-assessment record, not a certificate or an automatic code judge. The useful reward is the capability you can now explain and use.

Use the Save Room

The save room contains clickable checkboxes for Scout, Forge, Explain, and Side on every level, plus fields for your callsign, active route, and XP total. Mark the boxes and save a personal copy of the PDF in a reader that preserves form fields. If your viewer does not retain form edits, use the text quest journal in the field kit or print the ledger.

Tally XP from the column weights. The PDF does not execute your Rust code or calculate completion for you. Keep implementation notes, rejected

attempts, measurements, and debriefs in your own repository or the supplied journal. A checked box should point to evidence you can revisit.

There is no penalty for revealing a hint. Hint one redirects attention. Hint two gives a more concrete reasoning step. The debrief describes a defensible solution and its limits. It is often worth returning after a few days and rebuilding the argument without the vault open.

Read the Level Interface

An entry briefing gives the situation, objective, suggested loadout, and mastery targets. The field manual contains the complete technical material. Training encounters and their debriefs preserve the original exercises. The boss encounter is a new challenge that asks you to apply the idea under concrete constraints. The exit routes offer two meaningful branches into other systems.

The Map, Save, and Vault links remain in the page footer throughout the expedition. The topic index links directly to technical field manuals. The campaign directory links to level entries. A field manual's number matches its level number, so a reference to Level 24 means the pinning material even when the mission has a different narrative name.

The three branching rooms are optional diagnostic encounters. Choose an action before reading its consequence. Several actions can be defensible under different requirements. The interesting discovery is which assumption makes your choice valid. The world does not reward confidence without an ownership or protocol argument.

Carry the Right Tools

Open the field kit ZIP from your PDF viewer's attachments panel or download it from the World Map in the interactive reader. It contains Rust listings, rejected examples, sketches, campaign routes, and an editable quest journal. Listing 24.3 belongs to Level 24 and retains its chapter number in the filename.

The examples use Rust edition 2024. The first compiler pass used rustc 1.97.1, and the November 2026 review targets stable Rust 1.99.1. Some examples define main, and others define library items. Twelve listings are

deliberately rejected. The reader project's Rust audit records the checks for the supplied examples.

Work with stable Rust unless an encounter explicitly asks you to investigate a tool or an unstable experiment. Runtime-specific and foreign-library sketches identify the context they need. The campaign does not require a particular async runtime or an external service merely to follow its reasoning.

The source notes link to primary Rust documentation. Use those contracts when a precise rule matters. Passing tests, a successful benchmark, or a clean Miri run contributes evidence within its scope; it does not replace the invariant argument for an unsafe abstraction. Ferrum's most expensive failures came from treating one kind of evidence as if it proved everything.

Start Before You Feel Finished

Choose a path and enter its first unfamiliar level. Predict one example. Attempt one boss. Leave a record of what you believed and what changed. When another route looks interesting, take it. The map is built for revisiting ideas from a different direction.

World Map

Every numbered node is an unlocked door. Follow the whole campaign, choose a path, or jump to a system you want to repair.

01 The Borrowed Citadel

Ownership and resource logic

1 2 3 4 5 6 7 8

02 The Trait Foundry

Types and public interfaces

9 10 11 12 13 14 15 16 17 18

03 The Clockwork Wilds

Concurrency and asynchronous systems

19 20 21 22 23 24 25 26 27 28

04 The Unsafe Depths

Memory representation and proof obligations

29 30 31 32 33 34 35 36

05 The Endgame

Complete systems and their failure modes

37 38 39 40 41 42

OPTIONAL PORTALS

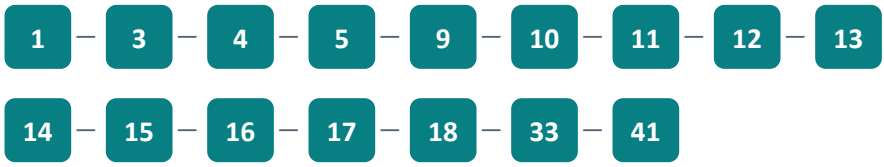
Choose a path Branching rooms The hint vault

Choose a Path

Click any node to enter. Change paths when a different problem catches your attention.

The Architect

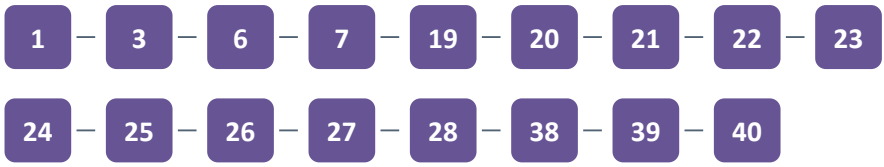
You rebuild the contracts between districts. Follow trait obligations, borrowing interfaces, type erasure, and API evolution until another team can use your library without sharing your implementation assumptions. Your final contract is a crate whose feature combinations and public promises remain coherent.



Route artifact A public API dossier with accepted and rejected examples, an error policy, and a compatibility argument.

The Warden

You restore the towers that move work through Ferrum. Follow capabilities across threads, queues, suspension points, wakeups, and cancellation. Your final contract is a service whose accepted work has an explicit fate during overload, failure, and shutdown. A process that merely stays memory safe is not yet rescued.



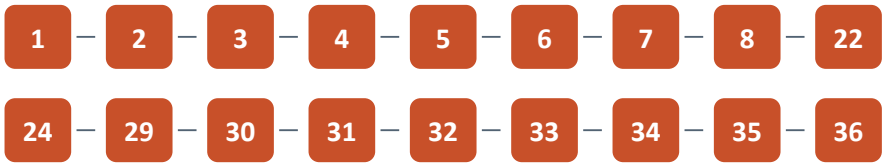
Route artifact A wait graph, resource budget, shutdown protocol, and fault-injection record for one concurrent service.

Two More Paths

Click any node to enter. Change paths when a different problem catches your attention.

The Delver

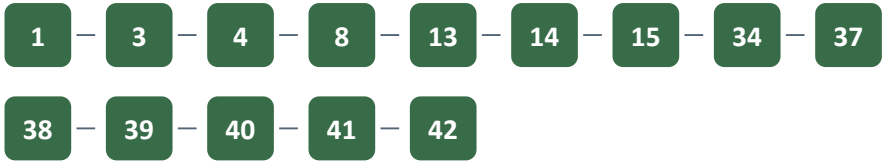
You descend beneath the safe interfaces to inspect representation, validity, initialization, and reclamation. Take the safe alternative whenever it meets the contract. When unsafe operations are necessary, make each obligation and owner visible. Your final contract is an abstraction whose implementation can be audited beyond the happy path.



Route artifact An unsafe proof ledger plus boundary, panic, and initialization tests and a measured cost comparison.

The Pathfinder

You connect isolated repairs into working systems. Follow retained bytes through a parser, stale decisions through a store, and accepted jobs through a pipeline. Your final contract is a design review that explains what types guarantee, what runtime checks establish, and what depends on another component.



Route artifact One complete design study with an ownership diagram, failure table, and defended tradeoffs.

Save Room One

Mark evidence, then save your own copy. Core marks earn 20 + 40 + 40 XP; a completed side quest earns 25. Tally XP yourself.

CALLSIGN	ACTIVE PATH	XP TOTAL

LEVEL AND MISSION	SCOUT 20 XP	FORGE 40 XP	EXPLAIN 40 XP	SIDE 25 XP
01 The Courier and the Sealed Cargo	☐	☐	☐	☐
02 Three Crates One Exit	☐	☐	☐	☐
03 The Bridge of Disjoint Regions	☐	☐	☐	☐
04 The Observatory That Keeps No Guests	☐	☐	☐	☐
05 The Three Mirror Portals	☐	☐	☐	☐
06 The Airlock Ledger	☐	☐	☐	☐
07 The Document That Answers Back	☐	☐	☐	☐
08 The Shifting Street Atlas	☐	☐	☐	☐
09 The Foundry Charter	☐	☐	☐	☐
10 The Hall of Useful Masks	☐	☐	☐	☐
11 The Anvil That Already Belongs to Someone	☐	☐	☐	☐
12 The Planner With a Memory	☐	☐	☐	☐
13 The Library of Borrowed Pages	☐	☐	☐	☐
14 The Repair Report That Can Be Used	☐	☐	☐	☐
15 The Gate of False Credentials	☐	☐	☐	☐
16 The Museum of Broken Promises	☐	☐	☐	☐
17 The Workshop of Borrowed Spells	☐	☐	☐	☐
18 The Frozen Signal Vault	☐	☐	☐	☐
19 The Surveyors Who Share a Horizon	☐	☐	☐	☐
20 The Observatory of Sleeping Readers	☐	☐	☐	☐
21 The Courier With Two Destinations	☐	☐	☐	☐

Core mastery = Scout + Forge + Explain. Side quests are optional.

Save Room Two

Mark evidence, then save your own copy. Core marks earn 20 + 40 + 40 XP; a completed side quest earns 25. Tally XP yourself.

Continue your marks. Keep evidence in the quest journal.

LEVEL AND MISSION	SCOUT 20 XP	FORGE 40 XP	EXPLAIN 40 XP	SIDE 25 XP
22 The Observatory Between Two Ticks	☐	☐	☐	☐
23 The Engine That Yields No Ground	☐	☐	☐	☐
24 The House Whose Rooms Must Stay Still	☐	☐	☐	☐
25 The Bell Thief's Last Message	☐	☐	☐	☐
26 The Bridge With a Missing Middle	☐	☐	☐	☐
27 The Guild of Unnamed Futures	☐	☐	☐	☐
28 The Last Light in the Wilds	☐	☐	☐	☐
29 The Shape Shifter	☐	☐	☐	☐
30 The Window With No Owner	☐	☐	☐	☐
31 The Trusted Label	☐	☐	☐	☐
32 The Scattered Assembly	☐	☐	☐	☐
33 The Two Allocators	☐	☐	☐	☐
34 The Archive That Remembers Too Much	☐	☐	☐	☐
35 The Benchmark Crown	☐	☐	☐	☐
36 The Fault Vault	☐	☐	☐	☐
37 The Living Document	☐	☐	☐	☐
38 The Transaction Within	☐	☐	☐	☐
39 The Collector Goes Dark	☐	☐	☐	☐
40 The Lost Acknowledgment	☐	☐	☐	☐
41 The Downstream Gauntlet	☐	☐	☐	☐
42 Restore Ferrum	☐	☐	☐	☐

Core mastery = Scout + Forge + Explain. Side quests are optional.

The Branching Rooms

Three optional diagnostic encounters. Pick an action first, then follow its link to the consequence. Afterward change one requirement and see whether your choice still holds.

The Vanishing Archive

A log service parses a 256 MiB input buffer. Downstream work retains only a 24-byte identifier from each selected record for several minutes. The input must then be reused for the next batch. Your benchmark rewards low allocation counts, but the incident report complains about peak live memory. Choose a representation before you inspect the consequences. Assume the selected identifiers are a small fraction of the original buffer and do not need to preserve the original allocation identity.

1 Keep slices into the reusable buffer

2 Retain an Arc to the whole batch

3 Copy the selected identifiers at the queue boundary

Remix Downstream analysis now retains nearly every byte. The producer may rotate to another receive buffer while readers retain the old one. Re-evaluate the three actions and explain how retention density and the number of live batches affect your choice.

The Ghost Payment

A request future times out after sending a payment command. The connection then disappears. Your local request value is dropped cleanly. The remote service may have processed the command, but no acknowledgement reached you. The business requires that one purchase must not be charged twice. Choose your next action. The service has a status lookup and an idempotency-key mechanism, but their retention and deduplication contracts must be checked.

1 Retry immediately with a fresh operation identifier

2 Reuse the key and treat deduplication as automatic

3 Record an uncertain outcome and reconcile under the peer contract

Remix Remove the status endpoint and give idempotency records a finite lifetime. Design an escalation path for an unresolved purchase approaching expiry. Identify what must become a human or business decision instead of an unbounded automatic retry.

The Hollow Lock

A shared cache calls a slow user-supplied loader while holding its global mutex. The loader can recursively request another key, and two requests for the same key can trigger an expensive external operation. Product wants low contention and one active loader per key. Choose an initial redesign. A valid answer must account for reentrancy, failure, and abandoned loads rather than merely moving a lock statement.

1 Keep the global lock and forbid nothing else

2 Release the lock and allow every miss to load

3 Install a per-key loading state and coordinate waiters

Remix Let callers cancel while waiting and permit the initiating loader to disappear. Explain how waiters get a terminal outcome and when they can retry. Test two simultaneous loaders that request each other's keys and identify which request must fail.

Campaign Directory

Welcome to Ferrum 2

World Map 6

Choose a Path 7

Two More Paths 8

Save Room One 9

Save Room Two 10

The Branching Rooms 11

Level 01 The Courier and the Sealed Cargo 17

Level 02 Three Crates One Exit 28

Level 03 The Bridge of Disjoint Regions 39

Level 04 The Observatory That Keeps No Guests 50

Level 05 The Three Mirror Portals 60

Level 06 The Airlock Ledger 70

Level 07 The Document That Answers Back 82

Level 08 The Shifting Street Atlas 93

Level 09 The Foundry Charter 105

Level 10 The Hall of Useful Masks 117

Level 11 The Anvil That Already Belongs to Someone 129

Level 12 The Planner With a Memory 140

Level 13 The Library of Borrowed Pages 151

Level 14 The Repair Report That Can Be Used 162

Level 15 The Gate of False Credentials 173

Level 16 The Museum of Broken Promises 185

Level 17 The Workshop of Borrowed Spells 196

Level 18 The Frozen Signal Vault 208

Level 19 The Surveyors Who Share a Horizon 220

Level 20 The Observatory of Sleeping Readers 230

Level 21 The Courier With Two Destinations 241

Level 22 The Observatory Between Two Ticks 251

Level 23 The Engine That Yields No Ground 261

Level 24 The House Whose Rooms Must Stay Still 272

Level 25 The Bell Thief's Last Message 283

Level 26 The Bridge With a Missing Middle 294

Level 27 The Guild of Unnamed Futures 305

Level 28 The Last Light in the Wilds 316

Level 29 The Shape Shifter 329

Level 30 The Window With No Owner 341

Level 31 The Trusted Label 353

Level 32 The Scattered Assembly 365

Level 33 The Two Allocators 376

Level 34 The Archive That Remembers Too Much 388

Level 35 The Benchmark Crown 400

Level 36 The Fault Vault 413

Level 37 The Living Document 427

Level 38 The Transaction Within 439

Level 39 The Collector Goes Dark 450

Level 40 The Lost Acknowledgment 461

Level 41 The Downstream Gauntlet 473

Level 42 Restore Ferrum 484

The Hint Vault 495

The Nudge Gallery 496

The Tactics Workshop 506

The Debrief Archive 519

Consequences in the Branching Rooms 540

Appendix A A Diagnostic Reasoning Guide 543

Appendix B A Glossary of Relationships 549

Topic Index 558

ZONE 1 / THE BORROWED CITADEL / LEVEL 01

The Courier and the Sealed Cargo

The Borrowed Citadel still has power, but its couriers disagree about who owns each shipment. One console marks a cargo capsule ready while another continues editing its contents. You arrive at a loading platform covered in contradictory receipts. Your repair is an ownership protocol: every capsule needs one responsible owner, inspection must grant temporary access, and handing a capsule onward must leave no usable delivery authority behind. The city is fictional; the ownership argument you build is the real equipment you take away.

Mission Design a cargo API that makes inspection, editing, acceptance, and rejected handoff express different ownership relationships.

Mastery Targets

- Identify the owner of the payload before, during, and after each handoff.
- Show that an accepted handoff prevents reuse of the original capsule in safe caller code.
- Return the original owned capsule on rejection without cloning its payload.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 1

Ownership as Permissions and State Transitions

The question behind an ownership error

An experienced developer usually meets Rust with an existing model of objects: an object has an identity, several parts of the program can refer to it, and the runtime or the programmer arranges its eventual cleanup. Rust asks for a more explicit account. Which operation currently has permission to destroy the value? Which operations may inspect its storage? Which operation may change that storage? What must remain true until those permissions expire? An ownership error is often the compiler reporting that two proposed answers cannot both be true.

This perspective is more useful than memorizing a collection of prohibitions. A database transaction, a file descriptor, an allocated buffer, and a lock guard have different behavior, but all can be represented by a value whose owner controls a permission. Passing that value by value transfers the permission. Borrowing makes a temporary promise about how the owner will behave. Destruction releases whatever the permission represents.

There are three distinct things to keep separate: the value, the storage that currently contains it, and the resource represented by it. A `Vec<u8>` value contains management information; its elements usually occupy another allocation. Moving the vector can change where the management information resides while preserving ownership of its allocation. A file value represents an operating system resource; moving the Rust value does not reopen the file. Ownership concerns the program's authority over these resources, rather than a requirement that every value occupy a permanent address.

The useful invariant is that an owned resource has one active destruction responsibility at a time. There may be many readers, several weak identifiers, or reference counted ownership represented by multiple handles, but the resource's implementation must still establish when final cleanup is authorized. Rust makes many of those responsibilities visible in types and signatures.

Values make protocols explicit

Consider a component that prepares bytes and hands them to another component for submission. A design in which every method takes a mutable reference can work, but the object remains available in states where its old operations no longer make sense. Consuming methods can express the state transition directly.

Listing 1.1 Rust example

```
struct Draft {
    bytes: Vec<u8>,
}

struct Ready {
    bytes: Vec<u8>,
    checksum: u64,
}

impl Draft {
    fn new() -> Self {
        Self { bytes: Vec::new() }
    }

    fn append(&mut self, bytes: &[u8]) {
        self.bytes.extend_from_slice(bytes);
    }

    fn finish(self) -> Ready {
        let checksum = self.bytes.iter()
            .fold(0_u64, |sum, byte| sum + u64::from(*byte));
        Ready { bytes: self.bytes, checksum }
    }
}

impl Ready {
    fn submit(self) -> (usize, u64) {
        (self.bytes.len(), self.checksum)
    }
}

fn main() {
    let mut draft = Draft::new();
    draft.append(b"request");
    let ready = draft.finish();
    assert_eq!(ready.submit(), (7, 777));
}
```

The interesting detail is not the checksum algorithm. `finish` consumes the authority to modify a `Draft`, and produces authority to submit a `Ready`. There is no remaining draft handle through which the checksum could be invalidated. The compiler does not understand checksums; it understands that a consumed owner cannot be used again. We connect that general rule to the domain invariant through the API.

This technique also changes the error path. If finishing can fail, a return type such as `Result<Ready, (Draft, Error)>` can return the permission to edit. Returning only `Error` deliberately destroys the draft on failure. Neither policy is universally correct. The signature should make the product decision visible: whether rejected work is recoverable, whether recovery is cheap, and which component owns it after rejection.

Consumption does not imply successful completion. If `submit` panics before completion, its fields are still subject to ordinary unwinding behavior. Ownership proves availability and cleanup relationships; it does not prove that a remote operation occurred. Distributed side effects require a separate protocol, perhaps an idempotency key or a durable acknowledgment record.

Shared access and exclusive access are capabilities

The familiar distinction between `&T` and `&mut T` is best understood through permitted operations. A shared reference permits operations that preserve the relevant shared access guarantees. An exclusive reference permits operations that require temporary exclusivity. The word mutable is useful, but it can obscure the stronger fact that an exclusive borrow can be needed even when a method performs no write.

A method may require `&mut self` because its contract reserves the right to rearrange internal storage, lazily initialize an index, or make two operations atomic with respect to safe clients. Conversely, a method taking `&self` may update a `Cell`, `RefCell`, atomic, or mutex protected value. Interior mutability changes how certain writes are authorized; it does not abolish the need for authorization.

Listing 1.2 Rust example

```
struct Catalog {
    names: Vec<String>,
}

impl Catalog {
    fn first(&self) -> Option<&str> {
        self.names.first().map(String::as_str)
    }

    fn add(&mut self, name: String) {
        self.names.push(name);
    }
}

fn main() {
    let mut catalog = Catalog {
        names: vec![String::from("alpha")],
    };
    let name = catalog.first().unwrap();
    println!("{name}");
    catalog.add(String::from("beta"));
}
```

The borrow used by `name` is no longer needed after its last use. The subsequent insertion can therefore obtain exclusive access. If the print moved after the insertion, the proposed accesses would conflict. A vector may reallocate when it grows, so preserving a reference into it while allowing growth would permit the reference to become invalid. Rust does not need to predict whether this particular push reallocates. The method's contract permits reallocation, so the checker reasons from that contract.

This explains a common design surprise: reserving enough capacity does not generally make a conflicting safe borrow legal. Capacity is a runtime fact about the current vector. The type of `push` still requires exclusive access to the vector. To expose a narrower promise, an API needs a different representation or a specialized safe abstraction with a carefully stated invariant.

Ownership boundaries are architectural boundaries

Suppose a request processor receives a byte buffer, parses it, and produces an answer. There are at least three useful designs. It can borrow the input and return data tied to that borrow. It can consume the input and produce an owned result. Or it can copy selected fields and release the large input promptly. The right choice depends on the lifetime and memory behavior of the entire pipeline.

Borrowing minimizes copying when the consumer finishes before the input owner needs to mutate or release the storage. Consuming can simplify handoffs between independently scheduled operations. Selective copying can reduce retained memory: keeping a ten byte field borrowed from a ten megabyte message retains the entire message allocation. A zero copy interface can therefore increase peak memory despite reducing byte movement.

Listing 1.3 Rust example

```
#[derive(Debug, PartialEq)]
struct Header<'a> {
    key: &'a str,
    value: &'a str,
}

fn parse_header(input: &str) -> Option<Header<'_>> {
    let (key, value) = input.split_once(':')?;
    Some(Header { key: key.trim(), value: value.trim() })
}

fn owned_key(input: &str) -> Option<String> {
    Some(parse_header(input)?.key.to_owned())
}

fn main() {
    let message = String::from("mode: strict");
    let header = parse_header(&message).unwrap();
    assert_eq!(header.value, "strict");
    let key = owned_key(&message).unwrap();
    drop(message);
    assert_eq!(key, "mode");
}
```

The borrowed result carries a relationship with the input owner. The owned key has intentionally crossed that boundary. Both functions can be efficient for different workloads. Rather than asking whether cloning is

idiomatic, ask which component should own the bytes at each stage and how long the largest allocation remains reachable.

The same analysis applies to channels and tasks. Moving a message into a channel is a clear transfer. Sharing an `Arc<Message>` keeps independent operations coupled through the allocation's lifetime. Borrowing from a parent can be efficient when a scoped concurrency API guarantees all children finish before the parent releases its data. These are different lifetime architectures, not interchangeable spellings of the same design.

Copy is a semantic promise

An ordinary move prevents further use of the old place until it is reinitialized. For a `Copy` type, an operation that would move instead copies the value and leaves the original usable. The promise is stronger than cheapness: implicit duplication must not require custom ownership bookkeeping. An integer is straightforward. A borrowed reference can be copied because it does not own the pointee. An exclusive reference is not `Copy`, because duplicating it would duplicate a capability that requires exclusivity.

Smallness does not determine whether a type should implement `Copy`. A small handle might represent a unique right to close a resource, release a reservation, or consume a token. Automatically duplicating that handle would change the protocol. A much larger fixed array of copyable values can still be `Copy`, although copying it may be expensive. Performance and permission semantics are separate questions.

Listing 1.4 Rust example

```
#[derive(Clone, Copy)]
struct Position {
    byte: usize,
    line: usize,
}

struct Reservation {
    id: u64,
}

fn commit(reservation: Reservation) -> u64 {
    reservation.id
}
```

Listing 1.4 continued

```
fn main() {
    let start = Position { byte: 0, line: 1 };
    let checkpoint = start;
    assert_eq!(start.byte, checkpoint.byte);
    assert_eq!(start.line, 1);
    let ticket = Reservation { id: 42 };
    assert_eq!(commit(ticket), 42);
}
```

`Reservation` has no destructor in this small example, but withholding `Copy` still encodes a useful restriction. Unique logical rights do not need an operating system resource to justify ownership. When a public type implements `Copy`, clients may begin depending on implicit duplication. Removing that implementation later can break source compatibility. Treat it as an API commitment.

`Clone` is an explicit operation and can perform arbitrary work. Cloning a `String` duplicates its bytes; cloning an `Arc` creates another handle to the same allocation. A method accepting `T: Clone` cannot assume independent deep state. It only knows that the type offers the documented duplication behavior. This matters when designing snapshots, isolation boundaries, and caches.

Read signatures as a permission ledger

For each parameter and result, ask what the callee receives and what the caller gets back. `fn inspect(&Record)` temporarily permits inspection. `fn edit(&mut Record)` reserves exclusive access. `fn archive(Record)` transfers ownership. `fn detach(&mut Record) -> Payload` leaves the record usable but returns an independently owned payload; its implementation must replace whatever state it removes. `fn view(&Record) -> &Payload` creates a result whose usable lifetime is tied to the borrowed record.

This ledger exposes design mistakes before implementation. If a function takes ownership but only reads a value, it may unnecessarily prevent reuse. If it borrows a value but stores a result beyond the borrow, it needs ownership, a longer lived owner, or an explicit copying step. If a method returns a mutable reference to internal state, the owner remains exclusively borrowed for the result's use, so subsequent operations on that owner may be unavailable.

Do not mechanically choose the least permissive signature. A consuming method can express an important transition even if its current body only reads. A mutable receiver can reserve space for an invariant that future implementations must preserve. The choice should follow the domain contract rather than the smallest body that happens to compile today.

Ownership becomes easier when every long lived value has a clear owner and every temporary access has a clear end. When neither is clear, the borrow checker may be exposing an architectural ambiguity. Fixing the shape of ownership often removes several local lifetime problems at once.

Training Encounters

1. A message parser returns three borrowed strings. A queue must retain those strings after the original network buffer is reused. Describe two ownership designs and the memory cost each implies.

2. An authorization token contains only a `u64`. Should it implement `Copy` if calling `redeem` must make a second redemption impossible through the same token value? Explain what the type system can and cannot guarantee.

3. A method `fn current(&self) -> &Entry` is followed by a call to `fn refresh(&mut self)`. Explain why the second call can compile when the result of `current` is no longer used, and why preallocating storage does not generally change the answer when it is used afterward.

Training Debriefs

1. Copy the selected strings into an owned queue message, allowing immediate buffer reuse at a cost proportional to retained fields. Alternatively, transfer ownership of the complete buffer into a queued message and store byte ranges rather than self references. That avoids field copies but retains the full buffer. An `Arc` owned buffer can support several consumers, with the same retention consideration and reference counting overhead.

2. Withhold `Copy` and make `redeem` consume the token. This prevents reuse of that Rust value in safe code after transfer. It does not prevent a user from constructing another token with the same identifier if constructors

allow it, replaying a serialized identifier, or racing separate copies created elsewhere. Private construction and external redemption state may be required for the complete protocol.

3. The first result carries shared access to the entry through the owner. Once its last relevant use has passed, exclusive access can begin. If the result is used after refresh, the accesses overlap. Capacity is not the relevant contract: `refresh` requires exclusive access and may perform operations that invalidate or change the entry. A specialized API would need to express a narrower relationship.

Further Reading

• The Rust Reference on expressions defines places and value expressions used throughout ownership reasoning.

• The Rust standard library Clone trait documents explicit duplication contracts.

• The Rust standard library Copy trait explains the language level duplication promise.

The Double Delivery

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Build a small in-memory loading dock model. A capsule owns a byte buffer and a destination. Inspection may read both, and an editable capsule may change its payload before submission. The dock either accepts ownership and returns a receipt or rejects the submission and returns the same capsule with a reason. A rejected capsule must remain available for correction and resubmission. An accepted capsule must not remain accessible through a second editable handle. Describe the public signatures, implement the acceptance branch with an ordinary collection, and write a caller that recovers a rejected capsule. Then create one intentionally rejected caller that tries to inspect the old capsule after success. Explain which guarantee the compiler supplies and which delivery claims belong only to your model.

Encounter Constraints

• Do not implement Clone or Copy for the capsule, and do not clone its buffer.

• Use a consuming submission method whose error preserves the rejected value.

• Treat acceptance as an in-memory transfer, without claiming network or durable delivery.

Hint one / Hint two / Reveal debrief

Side Quest

Add a read-only audit record that may outlive the capsule. Compare copying a small summary with sharing the entire payload through an owned reference-counted handle. Measure what each representation keeps alive. Your answer should explain why a cheap handle clone can retain a large allocation and why an audit trail containing a destination and length may need no payload ownership at all.

Capability unlocked Cargo Authority - signatures that reveal where responsibility goes

Choose Your Exit

Follow the borrow bridges / Level 3
Learn how temporary access can be split without duplicating ownership.

Investigate the cleanup airlocks / Level 6
Discover what happens when a handoff is interrupted or abandoned.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 02

Three Crates One Exit

A sorting chamber in the Borrowed Citadel contains three labeled crates: header, payload, and annotation. Its old controller treats the whole chamber as either untouched or gone, even when one crate has already left. That crude model has made recovery reports unreliable. You must track initialized places individually and distinguish a value moving to a new owner from the storage it leaves behind. Every route through the chamber must end with a precise inventory of which values still exist and who will destroy them.

Mission Explain and implement branch-dependent moves while preserving valid cleanup and an honest recovery report.

Suggested loadout Level 1. These are guidance, not locked gates.

Mastery Targets

- Trace initialized and moved fields separately on at least three control-flow paths.
- Return all surviving resources through an explicit result representation.
- Explain why adding a custom Drop implementation changes field-move permissions.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 2

Moves Places Initialization and Drop Flags

A move changes what can be used

A Rust move is a semantic event: a value is transferred out of a place, and that place is no longer available as an initialized value until an appropriate reinitialization occurs. Whether generated machine code copies bytes, reuses registers, or eliminates the operation is a separate question. Thinking of every move as a heap transfer or a mandatory memory copy makes both correctness and performance harder to understand.

A place is an expression denoting a storage location, such as a local variable, a field, an indexed element, or a dereferenced pointer. The compiler tracks which places are initialized and which accesses are legal. This tracking can be more precise than treating a struct as one indivisible box. Moving one field may leave other fields available. Moving a complete value prevents access through all its fields until it is restored.

Listing 2.1 Rust example

```
struct Job {
    label: String,
    data: Vec<u8>,
}

fn main() {
    let job = Job {
        label: String::from("export"),
        data: vec![1, 2, 3],
    };
    let label = job.label;
    assert_eq!(job.data.len(), 3);
    assert_eq!(label, "export");
}
```

The value of `job.label` moves into `label`. The vector remains initialized and can be inspected. The complete `job` is partially moved, so passing `job` by value would be invalid. At the end of the scope, Rust drops the vector in the original place and drops the string through its new owner. It does not attempt to destroy the same string twice.

This distinction helps explain error messages containing phrases such as partially moved value or use of moved value. The question is which place lost initialization, not whether the source code still contains the variable's name. Names remain in scope even when the places they name cannot currently be used.

Initialization follows control flow

Rust permits a local variable to be declared before it is initialized, provided every path to a read has established a value. This is a static control flow property. There is no default zero value silently inserted to make the variable exist.

Listing 2.2 Rust example

```
fn choose_name(use_cache: bool) -> String {
    let name;
    if use_cache {
        name = String::from("cached");
    } else {
        name = String::from("fresh");
    }
    name
}

fn main() {
    assert_eq!(choose_name(true), "cached");
    assert_eq!(choose_name(false), "fresh");
}
```

The uninitialized period does not contain a valid but unspecified `String`. It contains no initialized `String` at all. Reading it would be rejected. The distinction becomes critical in unsafe code, where manufacturing an invalid `String` and intending to fill it later is already a violation. Safe delayed initialization does not require a value to exist before the assignment.

After a move, reinitialization can make a place usable again. A mutable binding allows assignment; a binding without `mut` can still receive its first initialization, but cannot be assigned a second value after it has already been initialized under the usual assignment rules. Keep first initialization separate from mutation of an established value.

The analysis is conservative at control flow joins. If one branch moves a value and another does not, a later unconditional use cannot assume the value remains initialized. You can often make the intended state explicit with an enum or `Option`, or restructure the branches so that ownership rejoins as a returned value.

Drop flags are an explanatory model

When initialization depends on runtime control flow, generated cleanup must know which destructors are appropriate. A useful conceptual model associates a flag with a place: initialized values must be cleaned up, and moved out values must not be cleaned up through their old places. The compiler can often resolve this statically and omit any runtime flag. Do not infer a specific memory representation from this model.

Imagine a function that creates two resources, transfers one conditionally, then returns early. Cleanup is determined by which resources still belong to the function on that path. The source need not contain explicit cleanup branches because the compiler elaborates destruction according to ownership and initialization.

Listing 2.3 Rust example

```
struct Trace(&'static str);

impl Drop for Trace {
    fn drop(&mut self) {
        println!("release {}", self.0);
    }
}

fn consume(value: Trace) {
    println!("consume {}", value.0);
}

fn run(send_first: bool) {
    let first = Trace("first");
    let second = Trace("second");
    if send_first {
        consume(first);
    }
    println!("still have {}", second.0);
}

fn main() {
    run(true);
    run(false);
}
```

When `send_first` is true, `consume` owns and destroys `first`; the caller destroys only `second` at its end. When false, both remain in the caller, and local destruction occurs in reverse declaration order. A compiler implementation might use a flag, duplicate cleanup paths, or optimize the structure differently. The language level obligation is the same.

This model is especially useful when reading generated intermediate representation or debugging panic paths. The fact that a destructor call appears in more than one control flow block does not imply double destruction. Each call may be guarded by different paths or initialization conditions.

Replacement preserves the destination invariant

Moving a value out through `&mut T` would leave the referent uninitialized while the caller still expects a valid `T`. A plain field move is therefore restricted. `std::mem::replace` solves a different, well defined operation: put a new value in the place and return the old one. `std::mem::take` does the same using `Default`. `Option::take` replaces `Some` with `None`, making absence an explicit valid state.

Listing 2.4 Rust example

```
struct Batch {
    pending: Vec<String>,
    retries: usize,
}

impl Batch {
    fn drain_for_send(&mut self) -> Vec<String> {
        self.retries += 1;
        std::mem::take(&mut self.pending)
    }

    fn restore(&mut self, unsent: Vec<String>) {
        assert!(self.pending.is_empty());
        self.pending = unsent;
    }
}

fn main() {
    let mut batch = Batch {
        pending: vec![String::from("record")],
        retries: 0,
    };
    let outgoing = batch.drain_for_send();
    assert!(batch.pending.is_empty());
    batch.restore(outgoing);
    assert_eq!(batch.pending.len(), 1);
}
```

The batch remains structurally valid even if the caller never restores the outgoing vector. Whether an empty pending list is a valid domain state is a separate question. If the protocol requires eventual restoration, ordinary safe ownership does not prove that the caller will perform it. A guard can improve normal control flow behavior, but safe code can leak guards, and abrupt process termination can bypass destructors.

Replacement can also influence failure semantics. Suppose you take a collection out of a field, then invoke user code that panics. The field remains empty during unwinding. If other fields or a custom destructor assume it always contains records, the representation may have become logically inconsistent. A sound safe design must remain memory safe in this state, and a robust domain design should document whether failed operations preserve or discard the original data.

Moving fields from a type with Drop

A type implementing `Drop` receives a mutable reference to the complete value when its destructor runs. The destructor may inspect its fields. Allowing safe code to move out one field while retaining an obligation to run that destructor would make the destructor observe a partially invalid object. Rust therefore restricts moving fields out of a type that implements `Drop`.

Listing 2.5 Rejected example

```
struct Envelope {
    payload: String,
}

impl Drop for Envelope {
    fn drop(&mut self) {
        println!("dropping {} bytes", self.payload.len());
    }
}

fn main() {
    let envelope = Envelope {
        payload: String::from("data"),
    };
    let payload = envelope.payload;
    println!("{payload}");
}
```

The restriction is a design signal. If the type needs an operation that transfers its payload, use a representation where the destructor can tolerate an emptied field, such as `Option<String>`, and take that option. If custom destruction is unnecessary, remove the `Drop` implementation and let fields clean themselves up. Implementing `Drop` merely to drop owned fields is usually redundant and reduces flexibility.

Specialized unsafe implementations sometimes use `ManuallyDrop` and pointer reads to control destruction. That is a different contract, with obligations about double drops, initialization, and panic behavior. It should not be the first response to a safe API design that can express its state with an enum.

Pattern matching is an ownership operation

Patterns can move, copy, or borrow parts of their input. A pattern is therefore more than a test of shape. In code involving owned strings or collections, ask what each binding receives. Match ergonomics can insert borrowing behavior based on the scrutinee, but the resulting binding modes still determine permissions.

Listing 2.6 Rust example

```
enum Event {
    Data { topic: String, bytes: Vec<u8> },
    Closed,
}

fn inspect(event: &Event) -> usize {
    match event {
        Event::Data { topic, bytes } => topic.len() + bytes.len(),
        Event::Closed => 0,
    }
}

fn extract(event: Event) -> Option<Vec<u8>> {
    match event {
        Event::Data { topic: _, bytes } => Some(bytes),
        Event::Closed => None,
    }
}

fn main() {
    let event = Event::Data {
        topic: String::from("updates"),
        bytes: vec![1, 2],
    };
    assert_eq!(inspect(&event), 9);
    assert_eq!(extract(event), Some(vec![1, 2]));
}
```

`inspect` matches a shared reference and obtains borrowed access to its fields. `extract` matches an owned value and transfers the vector out. The

ignored topic is destroyed as part of consuming the event. A wildcard does not create an owned local binding in the way a named binding does, so seemingly small pattern changes can affect move and drop behavior.

When a pattern becomes difficult to reason about, write the ownership intention explicitly. Introduce a short borrow, use `as_ref` or `as_mut` on an `Option`, or destructure an owned value at the point where ownership should actually transfer. Clever pattern compactness is a poor trade when it hides a significant resource transition.

Moves do not promise address stability

Code that stores a pointer to one of its own fields has a relationship with the containing value's address. An ordinary move does not preserve that relationship automatically. Rust's move semantics make no general promise that a value will stay at the same address after being returned from a function, inserted into a collection, swapped, or assigned.

Heap allocation alone is not a complete explanation either. Moving a `Box<T>` moves the box handle while normally leaving its allocation in place, but safe operations may still move the `T` out when the API permits it. Address sensitive values need an API that restricts those operations, such as the pinning contracts discussed later. A raw pointer to an element of a growable vector similarly depends on allocation behavior that the owner must control.

For most application code, avoiding stored self pointers is simpler. Store byte offsets into an owned buffer, indices into an arena, or separate owners and borrowed views. Those representations make moves harmless because their relationships are expressed relative to an owner rather than as assumptions about one temporary address.

The central habit is to track initialization and permissions through each operation. Once you can explain which place is initialized, which owner will destroy it, and whether any pointer depends on its address, many confusing move errors become straightforward consequences of the proposed state transition.

Training Encounters

1. A struct contains two `String` fields and has no custom destructor. One field moves into a function. Which remaining operations on the original variable are legal, and what will cleanup destroy?

2. A method takes a vector out of `&mut self` using `mem::take`, then calls a fallible encoder. The encoder returns an error. Propose two distinct error contracts and explain where the old vector belongs in each.

3. Why is a locally declared but not yet initialized `String` different from an unsafe zero initialized `String` that will supposedly be repaired before use?

Training Debriefs

1. The unmoved field can still be accessed or moved independently. The entire struct cannot be passed by value while partially moved. Cleanup destroys initialized fields remaining in the original place; the moved field is destroyed by its new owner. Adding a `Drop` implementation changes the permitted field moves because the destructor expects a complete value.

2. For a retry preserving contract, restore the old vector before returning the error or return it as part of the error result. For a consuming contract, allow the encoder or local owner to destroy it and document that failed encoding consumes the batch. In either case `self` must remain in a valid representation state while empty, including during a panic.

3. Delayed initialization means no `String` value exists in that local place before assignment, and the compiler prevents reads or destruction as a string. Zero initialization attempts to create an actual value with an invalid representation. The later repair does not retroactively make that creation valid. Unsafe initialization must use storage types whose validity rules permit uninitialized bytes.

Further Reading

- The Rust Reference on destructors specifies cleanup for initialized and partially initialized places.
- The Rust Reference on patterns specifies binding modes and pattern behavior.
- The Rust standard library mem module documents replacement and explicit ownership operations.

The Partial Inventory

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Model a shipment containing three owned strings. An inspection mode consumes only the annotation and returns the untouched header and payload in a recovery package. A delivery mode consumes header and payload into a delivered record and returns the unused annotation. A rejected mode returns the entire shipment unchanged. Implement the branching function without cloning and without assuming the original aggregate remains usable after a partial move. Add a trace-only resource wrapper or equivalent explanation that identifies cleanup responsibility for every branch. Finally, consider a second version where Shipment implements Drop and needs to inspect all three fields. Show why the earlier implementation can no longer move those fields directly, and propose a representation or consuming boundary that preserves the intended behavior.

Encounter Constraints

- Use safe Rust and initialized values rather than zeroed or uninitialized stand-ins.
- Represent distinct surviving resource sets explicitly in enum variants or dedicated structs.
- Do not use ManuallyDrop merely to bypass the compiler's objection.

Hint one / Hint two / Reveal debrief

Side Quest

Add a branch that deliberately panics after moving the annotation into a local wrapper but before constructing its result. Predict the destruction log before running the example. Check that remaining initialized fields and the moved annotation each have one cleanup path. Discuss how aborting panics differ from unwinding and why the test does not establish that destructors run after every possible process exit.

Capability unlocked Inventory Vision - tracking values independently from their old places

Choose Your Exit

Enter the airlock ledger / Level 6
Extend initialization reasoning into panic safety and resource cleanup.

Descend to unfinished storage / Level 32
See how the same inventory argument becomes explicit for MaybeUninit.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 03

The Bridge of Disjoint Regions

Two repair crews need exclusive access to different sections of the same conduit. The bridge controller can see the conduit, but the requested regions arrive as runtime ranges, and a label saying different does not prove they cannot overlap. Your job is to turn checked geometry into two access capabilities. The repair should be small enough to audit, should reject conflicting requests clearly, and should let a caller work on both accepted regions without granting access to the rest of the conduit.

Mission Construct two mutable region borrows from validated runtime ranges without unsafe code.

Suggested loadout Level 1, Level 2. These are guidance, not locked gates.

Mastery Targets

• Reject out-of-bounds, reversed, or overlapping nonempty ranges under a stated empty-range policy.

• Preserve the caller's requested region order even when their positions are reversed.

• Explain how splitting makes the nonoverlap argument available to safe Rust.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 3

Borrow Scopes Reborrowing and Splitting Access

A borrow has a useful region

The lexical scope of a variable answers where its name can be written. The region for which a borrow must remain valid answers where a particular access promise is needed. These are related but different. Modern Rust can often end a borrow before the enclosing block ends, because later code no longer uses the borrowed result. This is commonly called nonlexical lifetimes.

The practical consequence is that local reasoning should follow uses rather than counting braces. A reference created early in a function does not necessarily reserve access until the function returns. Conversely, a reference stored in another object or captured by a closure may keep access reserved longer than the nearby statements suggest. The checker follows the relationships between values through control flow.

Listing 3.1 Rust example

```
fn main() {
    let mut names = vec![String::from("one")];
    let first = &names[0];
    assert_eq!(first.len(), 3);
    names.push(String::from("two"));
    assert_eq!(names.len(), 2);
}
```

Here the shared borrow is needed for the assertion involving `first`. It is not needed for the later push. Adding another use of `first` after the push changes the constraint graph: now the reference must remain valid across an operation that requires exclusive access and may relocate the vector's allocation.

Dropping a reference leaves its referent in place. At the reference's last use, the earlier borrowing promise no longer needs to authorize later accesses. Distinguish liveness for borrow analysis from destruction of owned resources, especially when a guard type has a destructor.

Reborrowing lends an existing permission

An exclusive reference is not copyable, but using it does not always permanently move it. Many operations implicitly create a shorter reborrow of the referent. During that reborrow, the original reference cannot be used in a conflicting way; afterward it can regain usefulness. This is how functions can repeatedly receive the same mutable reference.

Listing 3.2 Rust example

```
fn add_marker(bytes: &mut Vec<u8>) {
    bytes.push(0xff);
}

fn main() {
    let mut bytes = vec![1, 2];
    let access = &mut bytes;
```

Listing 3.2 continued

```
    add_marker(access);
    add_marker(access);
    assert_eq!(access.as_slice(), &[1, 2, 0xff, 0xff]);
}
```

Each call lends the existing exclusive capability for the duration required by the callee. It does not create two simultaneous independent mutable references to the vector. The function signature bounds the nested use, and the caller may continue once that use ends.

An explicit `&mut *access` makes the reborrow visible. It can be useful when generic inference otherwise treats a reference as an owned generic argument and moves the reference value. A generic `fn identity<T>(value: T) -> T` sees an exclusive reference as one possible `T`; passing it can transfer that reference. A function whose parameter is explicitly `&mut U` has a clearer reborrowing context. If a generic call unexpectedly consumes a mutable reference, inspect the inferred parameter type before adding clones or changing ownership.

Reborrowing also permits a temporary shared view through an exclusive reference. While that shared view remains in use, mutation through the original exclusive handle must pause. The original handle's stronger authority does not override the promise made by its nested shared access.

The checker understands structural separation

A struct's fields are distinct places, so Rust can often borrow them separately. This supports useful APIs without runtime checking. For example, a processor may read configuration while mutating a buffer, even though both live inside one owner.

Listing 3.3 Rust example

```
struct Encoder {
    prefix: String,
    output: Vec<u8>,
    records: usize,
}

impl Encoder {
    fn write_record(&mut self, record: &[u8]) {
        let prefix = &self.prefix;
        let output = &mut self.output;
```

Listing 3.3 continued

```
        output.extend_from_slice(prefix.as_bytes());
        output.extend_from_slice(record);
        output.push(b'\n');
        self.records += 1;
    }

    fn split(&mut self) -> (&str, &mut Vec<u8>) {
        (&self.prefix, &mut self.output)
    }
}

fn main() {
    let mut encoder = Encoder {
        prefix: String::from("item="),
        output: Vec::new(),
        records: 0,
    };
    encoder.write_record(b"42");
    let (prefix, output) = encoder.split();
    assert_eq!(prefix, "item=");
    output.extend_from_slice(b"done\n");
}
```

The second method returns the separation as part of its contract. A caller can inspect the prefix and mutate the output without needing access to the whole encoder. However, a helper method taking `&self` generally borrows the whole receiver according to its signature, even if its implementation currently reads only `prefix`. The caller is checked against the signature rather than interprocedural knowledge of that body.

When helper methods create conflicts, consider exposing a smaller helper type or passing the particular fields to a free function. The goal is to express the real access boundary. Repeatedly adding `RefCell` to compensate for a method that borrows too much can turn a simple static separation into a runtime failure problem.

Index inequality needs an abstraction

For arbitrary indexing, the compiler does not generally prove that two runtime indices refer to different elements. Even if the programmer knows `left != right`, two separate indexing operations into the same mutable slice can appear to request overlapping exclusive access. A library operation can establish the separation once and return references with the correct relationship.

Listing 3.4 Rust example

```
fn two_mut<T>(
    values: &mut [T],
    first: usize,
    second: usize,
) -> Option<(&mut T, &mut T)> {
    if first == second || first >= values.len() {
        return None;
    }
    if second >= values.len() {
        return None;
    }
    if first < second {
        let (left, right) = values.split_at_mut(second);
        Some((&mut left[first], &mut right[0]))
    } else {
        let (left, right) = values.split_at_mut(first);
        Some((&mut right[0], &mut left[second]))
    }
}

fn main() {
    let mut balances = [100_i64, 40, 75];
    let (source, target) = two_mut(&mut balances, 0, 2).unwrap();
    *source -= 10;
    *target += 10;
    assert_eq!(balances, [90, 40, 85]);
}
```

The ordering branches put each requested index into a different slice. `split_at_mut` supplies the proof that those slices do not overlap. The returned pair preserves argument order, so callers need not know which branch was selected. Equal indices are rejected because there is no sound way to return two ordinary exclusive references to the same element for simultaneous use.

The function chooses `Option` for invalid indices. In application code, `values.get_disjoint_mut([first, second])` performs bounds and overlap checks and returns a `Result` in argument order. The explicit split above shows where the disjointness proof comes from.

Two phase borrowing explains selected nested calls

Code such as `values.push(values.len())` works even though push needs exclusive access and len uses shared access. For selected implicit mutable borrows, the compiler supports a reservation phase before activation. During argument evaluation, the reserved borrow permits certain shared reads; at the call, it becomes the exclusive borrow required by the method.

Listing 3.5 Rust example

```
fn main() {
    let mut values = vec![10_usize, 20];
    values.push(values.len());
    assert_eq!(values, [10, 20, 2]);
}
```

This is a limited compiler mechanism, not a general rule that every exclusive borrow can coexist with shared access until the first write. An explicit `&mut` in source code does not receive the same two phase treatment. Refactoring a method call into a free function call with an explicit mutable argument can therefore change borrow checking even if the runtime behavior appears equivalent.

Listing 3.6 Rejected example

```
fn append_length(values: &mut Vec<usize>, length: usize) {
    values.push(length);
}

fn main() {
    let mut values = vec![1, 2];
    append_length(&mut values, values.len());
}
```

The straightforward repair is to compute the length before creating the explicit exclusive borrow. That makes evaluation order and permission order agree. Avoid designing an API around subtle dependence on two phase borrowing when a clear local variable expresses the same intent.

The reservation cannot protect a reference that must survive activation. A push of a reference into the same vector, or a callback that keeps an alias alive through mutation, introduces a different conflict. Two phase borrowing helps evaluate inputs; it does not make invalidation safe.

Returning a reference extends the relevant relationship

The most significant borrow conflicts often arise when methods return references. A temporary mutable operation that returns a number may release its borrow immediately after the call. The same operation returning `&mut Entry` transfers an ongoing access capability to the caller. That capability remains relevant for every later use of the entry.

Suppose a cache method takes `&mut self` and returns a reference to a cached item. While the result remains in use, the caller cannot freely invoke another mutable cache operation. This remains true if the second operation seems unrelated. The method contract exposes a relationship with the receiver as a whole. Some designs can provide structural separation; others need independent owned handles, immutable snapshots, or a callback that confines the use.

A callback is especially useful when a temporary view should not escape. Its return type can be independent of the borrowed view, allowing the owner to resume mutation after the callback completes. A callback that returns a reference tied to the view deliberately preserves the relationship and therefore does not solve the same problem.

When a borrow appears to last too long, trace the result rather than only the input. Does it live in a collection? Does an iterator retain the owner borrow? Does a closure capture it? Does a guard's destructor need the reference? These paths often explain why introducing an apparently harmless local binding changes the error.

Refactoring around permission phases

A useful pattern is to divide an operation into an observation phase, a decision phase, and a mutation phase. First read enough state to compute an owned decision: an index, enum, key, or small summary. End the borrowed observation. Then acquire exclusive access to perform the update. This is not a requirement to copy all data; it is a way to make the actual dependency small.

For example, finding the position of a record and then modifying that position can be easier than keeping a reference from a search alive while also trying to change the collection. The index must be used before an intervening structural change invalidates its meaning. If other operations can remove or reorder records between phases, use a generation checked identifier or a synchronization boundary. Borrowing correctness does not automatically prove that a remembered index still denotes the intended record.

Another pattern is to move invariant preserving behavior onto a smaller owner. If a method needs only a mutable output buffer and a shared schema, make those its inputs. A large receiver can accidentally couple independent resources and make its clients appear more sequential than the underlying problem requires.

Borrow checker resistance can reveal either a missing local abstraction or a real overlapping dependency. The correct response is to identify which. Shortening a borrow is valuable when the earlier access has truly ended. Hiding it with unsafe code while the dependency remains creates a correctness obligation that the original error was accurately describing.

Training Encounters

1. A method returns `&mut Item` from a vector. Its caller wants to append another item and then inspect the returned item. Explain the conflict and propose two representations that avoid keeping a vector element reference across the append.

2. Why can two distinct struct fields often be borrowed separately while two arbitrary indices in a slice need a splitting operation?

3. Rewrite the rejected `append_length` example so it preserves the same result without depending on two phase borrowing. Explain why storing an ordinary `usize` differs from storing a reference into the vector.

Training Debriefs

1. Appending requires exclusive access to the vector and may invalidate an element reference by reallocating. Store an index and reacquire a reference after append, provided ordering and deletion rules preserve its meaning. Alternatively store independently allocated items behind owned handles, or use an arena with stable identifiers. The owner still must prevent deletion during a borrowed use.

2. Field projections expose fixed nonoverlapping storage relationships to the checker. Runtime index expressions can be equal, and Rust does not generally solve arbitrary index arithmetic. A safe splitter checks or constructs the disjointness condition and encodes the result as distinct borrowed slices or elements.

3. Compute `let length = values.len();` and then call `append_length(&mut values, length);`. The number has no borrowing relationship with vector storage. A stored reference would still reserve access to some part of the vector for its later use, so merely placing that reference in a separate local would not remove the conflict.

Further Reading

• The Rust Compiler Development Guide on two phase borrows describes the selected compiler cases.

• The Rust standard library slice documentation documents safe splitting and element access APIs.

• The Rustonomicon on splitting borrows develops the structural separation principle.

The Two Crews

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Write a helper that accepts one mutable slice and two half-open ranges, then grants a callback mutable access to the two selected regions. The ranges may arrive in either spatial order. Define whether empty ranges are accepted, and ensure the callback arguments still correspond to the original first and second requests. The callback must not receive access to the gap or the remaining conduit. Test adjacent ranges, reversed ordering, an overlap, a reversed range, and an endpoint beyond the slice. Explain why comparing two starting indices is insufficient for region disjointness, and identify the point at which the original whole-slice access is temporarily narrowed into separate capabilities. Keep the implementation's error policy separate from its memory-safety argument.

Encounter Constraints

- Use slice splitting and bounds checks rather than raw pointers.
- Do not copy the conduit or repair a detached copy instead of its original storage.
- Document empty-range behavior and keep callback execution after all validation.

Hint one / Hint two / Reveal debrief

Side Quest

Generalize the problem to a batch of nonoverlapping repair ranges. Decide whether sorting requests is allowed to change callback order and whether you want to allocate a list of region views. Compare this with repeated one-at-a-time edits. Explain why a handle-based plan can be prepared before borrowing, while retaining mutable references during later planning would constrain the available operations. State their boundaries.

Capability unlocked Bridge Splitter - converting checked disjointness into useful borrows

Choose Your Exit

Survey the lifetime observatory / Level 4
Learn to describe how long a granted view may remain usable.

Visit the lending library / Level 13
Explore interfaces that grant one temporary region at a time.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 04

The Observatory That Keeps No Guests

The Citadel's observatory decodes temporary manifests so visiting analysts can inspect them. The decoded records belong to the observatory and disappear when each inspection ends. Previous visitors tried to carry borrowed pages away; others were unnecessarily forbidden from returning any useful result. You must design an invitation that allows arbitrary owned summaries while preventing references into the temporary records from escaping. The challenge is to describe a relationship precisely enough that a caller's unrelated long-lived data remains usable without extending the observatory's storage.

Mission Design a scoped inspection API using lifetime quantification rather than leaked storage or blanket static bounds.

Suggested loadout Level 1, Level 3. These are guidance, not locked gates.

Mastery Targets

- Allow a callback to return an owned summary of locally decoded records.
- Reject returning a reference into those temporary records.
- Explain why an unrelated caller-owned reference may still be a valid result.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 4

Lifetimes as Constraints Between Operations

Lifetimes relate uses rather than schedule destruction

A lifetime annotation is a statement about relationships that must hold for references to be used safely. It does not extend an allocation, prevent a resource from being destroyed by magic, or tell the runtime when to free memory. The compiler uses the annotation while checking that the actual owners and uses satisfy the relationship.

For a function returning a borrowed result, the central question is which input owner must remain available for that result. A signature such as `fn field<'a>(input: &'a str) -> &'a str` states that the result is usable within the region authorized by the input borrow. It does not promise that the result uses the complete input, nor that it must remain in use for that entire region. A caller can use it briefly and release the dependency.

Lifetime parameters are generic parameters. They describe a family of valid calls rather than one global duration attached to a declaration. The caller supplies relationships consistent with its local owners; the callee must work for those relationships under its declared bounds. This is why adding the same lifetime name everywhere usually makes a signature less flexible instead of more correct.

Separate independent sources

Consider a function that looks for a key in a text record. The returned value is a slice of the record. The key is needed only during the lookup. Tying the result to both borrows would impose a dependency the implementation does not need.

Listing 4.1 Rust example

```
fn value_for<'record>(
    record: &'record str,
    wanted: &str,
) -> Option<&'record str> {
    for field in record.split(';') {
        let (key, value) = field.split_once('=')?;
        if key.trim() == wanted {
            return Some(value.trim());
```

Listing 4.1 continued

```
        }
    }
    None
}

fn main() {
    let record = String::from("mode=strict;limit=20");
    let result;
    {
        let key = String::from("mode");
        result = value_for(&record, &key).unwrap();
    }
    assert_eq!(result, "strict");
}
```

The short lived key can disappear before the result is used because the output lifetime names only the record relationship. If the signature used one lifetime for both arguments and the result, a call would be checked under a common usable region constrained by both inputs. Even if the implementation always returned record data, the caller would not be allowed to rely on that hidden fact.

This is a form of information design. A good signature exposes exactly the dependencies clients need to preserve. An overly broad dependency can force clients to retain unrelated data, avoid mutations, or clone unnecessarily. A missing dependency would permit dangling access and is rejected in safe code.

The example's parsing policy is intentionally strict: a field without an equals sign ends the search with `None`. A different parser might skip malformed fields or return a structured error. Lifetime correctness does not select parsing semantics. Review both independently when adapting the code.

Receiver lifetimes and stored lifetimes differ

A type containing a borrow usually has a lifetime parameter representing the owner it refers to. A method borrowing that type introduces another relationship: how long the caller borrows the wrapper itself. These lifetimes should not be tied together without a reason.

Listing 4.2 Rust example

```
struct Cursor<'input> {
    remaining: &'input str,
}

impl<'input> Cursor<'input> {
    fn new(input: &'input str) -> Self {
        Self { remaining: input }
    }

    fn next_word(&mut self) -> Option<&'input str> {
        self.remaining = self.remaining.trim_start();
        if self.remaining.is_empty() {
            return None;
        }
        let end = self.remaining.find(char::is_whitespace)
            .unwrap_or(self.remaining.len());
        let word = &self.remaining[..end];
        self.remaining = &self.remaining[end..];
        Some(word)
    }
}

fn main() {
    let text = String::from("alpha beta");
    let mut cursor = Cursor::new(&text);
    let first = cursor.next_word().unwrap();
    let second = cursor.next_word().unwrap();
    assert_eq!((first, second), ("alpha", "beta"));
}
```

The cursor mutates its own remaining slice, but each returned word borrows the immutable original input. It does not borrow mutable cursor storage. The temporary exclusive borrow of the cursor can end after each call while earlier words remain usable.

Writing `fn next_word(&'input mut self)` would demand borrowing the cursor exclusively for the input's lifetime. That can make repeated calls impossible while previous results remain in use, and may prevent the cursor itself from satisfying the requested duration. The receiver needs a short access borrow; the result needs the stored input borrow. They happen to participate in the same operation, but they represent different owners.

This pattern works because the cursor stores immutable slices into external data. A cursor that reuses an internal scratch buffer cannot

generally return results with an unrelated external lifetime. Its output must be tied to the receiver borrow, or must own its bytes. The correct signature follows where the bytes actually live.

Elision is a deterministic shorthand

Lifetime elision removes common annotations according to language rules. It is not an arbitrary whole program analysis that guesses what the function body intends. With one input lifetime, an elided output lifetime can normally refer to that input. In a method with a borrowed receiver, the receiver's lifetime guides elided borrowed outputs. Multiple independent borrowed inputs often require an explicit choice.

A useful review exercise is to expand elision mentally. `fn get(&self) -> &str` returns a string tied to the receiver borrow. For a wrapper holding a longer lived external string, that may be more restrictive than `fn get(&self) -> &'input str`. The first contract reserves the wrapper relationship for use of the result; the second can allow the wrapper to be modified or destroyed while the external owner remains alive.

Listing 4.3 Rust example

```
struct Label<'source> {
    text: &'source str,
}

impl<'source> Label<'source> {
    fn text(&self) -> &'source str {
        self.text
    }
}

fn main() {
    let source = String::from("persistent source");
    let text;
    {
        let label = Label { text: &source };
        text = label.text();
    }
    assert_eq!(text, "persistent source");
}
```

A shared reference can be copied out of the wrapper with its original lifetime relationship. An exclusive reference requires more care because it cannot simply be duplicated. Some methods must reborrow an exclusive

referent for the receiver's shorter borrow; a consuming method can instead transfer the original exclusive reference. Treating these cases identically is a frequent source of lifetime confusion.

Higher ranked bounds put the choice inside the callee

A generic lifetime parameter on a function is chosen in the context of a call. A higher ranked bound such as `for<'a> Fn(&'a Record)` requires a callback that works for any suitable lifetime the caller of that callback supplies. This distinction matters when a library creates a temporary value and wants to lend it to user code without allowing a borrow to escape.

Listing 4.4 Rust example

```
struct Sample {
    value: i64,
}

fn with_sample<F, R>(value: i64, inspect: F) -> R
where
    F: for<'a> FnOnce(&'a Sample) -> R,
{
    let sample = Sample { value };
    inspect(&sample)
}

fn main() {
    let doubled = with_sample(21, |sample| sample.value * 2);
    assert_eq!(doubled, 42);
}
```

The output `R` is outside the lifetime binder. It cannot be an arbitrary reference to the temporary sample whose lifetime is chosen inside the function. The callback can return an owned number, an owned string, or some value borrowing unrelated data it already owns permission to access. The bound confines this specific input borrow.

This is useful for scoped transactions, temporary decoded views, and APIs that borrow internal state while running a user operation. It does not make callbacks harmless. They can panic, reenter other components, allocate, or change externally shared state. The bound controls one escape route; the API still needs a policy for all those behaviors.

Higher ranked bounds also occur in function pointers and trait implementations. Read the position of the quantifier carefully. A callback

accepting only `&'static Record` is not a callback accepting arbitrary short borrows. A callback able to accept any short borrow can also accept an appropriately usable static borrow. The more general input contract supports more callers.

Outlives bounds describe contained references

The bound `T: 'a` means that references contained in `T` satisfy the required outlives relationship. It does not require a value of `T` to remain allocated until the end of `'a`. An owned `String` can satisfy `T: 'static` even when the particular string is dropped at the end of a short function. It contains no nonstatic borrowed references that would prevent moving it into a longer lived owner.

Similarly, `'long: 'short` says that the longer relationship can support uses required by the shorter one. It is a constraint, not a command to lengthen either owner. If a local allocation disappears too early, naming a longer lifetime cannot rescue it.

This distinction explains many task spawning errors. An API requiring an owned closure or future with a `'static` bound generally requires it not to contain unsupported short borrowed references. It does not mean the task must run forever. Moving owned data into the task often satisfies the bound because ownership travels with the task. Moving a reference only moves the reference value; it does not move the original owner.

Be precise about trait objects as well. A bound on the erased object's contained borrows and the lifetime of a reference pointing to that object are separate layers. `&'short (dyn Trait + 'long)` can lend a long compatible object for a short operation. Object lifetime defaults use rules distinct from ordinary function elision, so explicit `+ '_` or a named bound can clarify an intended borrowed object API.

Solving lifetime errors as a graph problem

Start with ownership nodes: where is each allocation or resource owned? Add edges for references stored in values and returned by functions. Then mark operations that need exclusive access or destroy an owner. A lifetime error usually says one edge would need to cross an incompatible mutation or destruction point.

There are four common repairs. Shorten the use of a reference when it is no longer needed. Return owned data when the result must outlive the source. Move the source owner into a longer lived component. Or revise the signature to remove an accidental relationship, as with the lookup key and cursor receiver above. Adding annotations is appropriate only for the last category or for exposing a real relationship that elision cannot express.

Avoid turning every error into a request for `'static`. Leaking an allocation can manufacture a static reference, but it changes resource behavior and usually conceals the missing ownership decision. Reference counting can be a legitimate owner transfer strategy, but it also changes retention, sharing, and synchronization choices. Pick those designs deliberately.

The most effective advanced lifetime reasoning remains concrete: identify the bytes, identify their owner, identify which use requires them, and give each independent relationship its own parameter. The notation becomes manageable when every name has a specific job.

Training Encounters

1. A lookup function takes a database slice and a temporary query string, and returns a reference into the database. Write its lifetime relationship in words and explain why the query should normally have an independent input lifetime.

2. A lexer returns slices of its external immutable source while advancing an internal index. Why can several returned tokens coexist? How does the answer change if it returns slices of one reusable internal buffer?

3. A task API requires `T: 'static`. Does passing an owned `String` require leaking it? Explain why moving a reference to a local string is a different operation.

Training Debriefs

1. The result depends on the database owner for every use, while the query is needed only during comparison. A signature can name the database lifetime in the result and leave the query's input lifetime independent. Tying them unnecessarily would constrain the result to the common usable region and could reject a result used after the query disappears.

2. Each token borrows unchanged source storage, so advancing an index does not invalidate earlier tokens. The lexer borrow can be short and separate from the source lifetime. A reusable buffer is owned and mutated by the lexer; returned slices must prevent conflicting reuse while they remain in use, or tokens must copy or own their bytes.

3. No. `String` owns its allocation and contains no short lived borrow that prevents the type from satisfying the bound. The task can own it and destroy it normally. Moving `&String` transfers only a borrowed handle, whose referent is still owned by the local scope and may disappear before the task finishes.

Further Reading

- The Rust Reference on lifetime elision specifies the shorthand rules and object lifetime defaults.
- The Rust Reference on trait and lifetime bounds specifies outlives and higher ranked relationships.
- The Rust standard library thread scope API is a concrete example of bounded borrowed work.

The Smuggled Page

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Implement an inspection function that owns a temporary vector of decoded records, invokes a supplied callback with borrowed access, and returns the callback's result. The function must release its temporary records before returning. Write one caller that returns an owned statistics structure and one intentionally rejected caller that tries to return a record's borrowed text. Then show a legal callback that returns a reference to unrelated caller-owned configuration. Explain why the last case does not violate the inspection boundary. Your design should make the callback work for the temporary borrow lifetime selected by the function, without forcing all captured data or all return values to be static. Describe how your signature differs from promising to return a borrowed view directly from a local vector.

Encounter Constraints

• Do not leak, globally store, or reference-count the temporary records just to extend their lifetime.

• Keep the result type outside the callback's quantified temporary lifetime.

• Treat parsing failure as an ordinary separate result path rather than hiding it in lifetime annotations.

Hint one / Hint two / Reveal debrief

Side Quest

Offer a second API that accepts a caller-owned manifest slice and returns a selected borrowed record directly. Compare its signature with the scoped inspector. Explain why neither is universally stronger: one preserves a stable external owner's data, while the other deliberately hides temporary storage and lets callers extract only results that survive independently. Include a caller that needs each version for a concrete reason.

Capability unlocked Scope Seal - granting temporary inspection without accidental escape

Choose Your Exit

Enter the mirror portals / Level 5
Study which lifetime substitutions preserve an interface's promises.

Rebuild the borrowing parser / Level 37
Apply owner and result-lifetime relationships to a complete parser.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 05

The Three Mirror Portals

The Citadel's mirror chamber translates permissions between districts. One portal only produces borrowed messages, another only accepts them, and a third both accepts and returns the same lifetime-shaped view. An old operator shortened every visible lifetime by habit and created promises that could not all remain true. Your repair is a substitution audit. You will use concrete witnesses to discover which direction each portal permits, and you will separate the lifetime of an outer borrow from the lifetime embedded in its contents.

Mission Derive covariance, contravariance, and invariance from the operations a type exposes.

Suggested loadout Level 3, Level 4. These are guidance, not locked gates.

Mastery Targets

• Predict substitutions through shared outputs, function inputs, and a combined input-output position.

• Demonstrate at least one accepted substitution and one rejected substitution with a reason.

• Keep an outer short borrow distinct from changing a stored lifetime parameter.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 5

Variance Subtyping and Phantom Types

Why a longer lived reference can fit a shorter use

Suppose a function needs a string for one operation. A string literal, available for the program's lifetime, should be acceptable. The function does not need to exploit the literal's full availability. Rust permits a reference with a sufficiently long validity relationship to be used where a shorter one is required. This is lifetime subtyping.

Variance answers whether that substitution remains legal after the reference is placed inside another type. A shared collection of long lived references can often be viewed through shorter lived references. A mutable slot containing a long lived reference cannot freely be treated as a slot for a shorter lived reference, because a caller might write the shorter reference into it. The outer type's operations determine whether the substitution would preserve its promises.

This is not object oriented subclassing. Rust's ordinary lifetime subtyping and its higher ranked relationships are much narrower than a general hierarchy of user defined data types. A `Dog` is not automatically a subtype of an `Animal` struct. Trait implementation supplies behavior through different mechanisms. Keeping these ideas separate prevents misleading analogies.

Listing 5.1 Rust example

```
fn shorten<'a>(text: &'static str) -> &'a str {
    text
}

fn main() {
    let local = String::from("local");
    let always = shorten("always available");
    let pair = [local.as_str(), always];
    assert_eq!(pair[1], "always available");
}
```

The function does not shorten the actual allocation's lifetime. It supplies a reference that clients are entitled to use under a shorter contract. The underlying literal still exists. Losing permission to rely on a longer duration is harmless for a reader; it is a restriction on what can be assumed, not a mutation of the referent.

Covariance preserves the direction

A generic type is covariant in a parameter when a subtype substitution in that parameter passes through the containing type. Shared references are covariant in their lifetime and in their referent type. Many owning collections are covariant in their element type as well. Their public mutation APIs remain safe because mutating the collection requires an exclusive borrow of the particular instantiated type.

Listing 5.2 Rust example

```
fn borrow_words<'a>(words: Vec<&'static str>) -> Vec<&'a str> {
    words
}

fn main() {
    let words = vec!["red", "green", "blue"];
    let shortened = borrow_words(words);
    assert_eq!(shortened.len(), 3);
}
```

This moves the complete vector into a type whose references are usable under a potentially shorter contract. It does not allow a caller retaining a vector of static references to observe arbitrary short references written into that same vector. Ownership of the vector itself transferred. If a shared reference to a vector is shortened, its shared access does not permit inserting a new element.

The distinction between changing an owned value's apparent lifetime and changing a mutable alias's referent type is the heart of variance reasoning. Read operations can safely forget some information. Write operations may need to preserve the original information for other users of the storage.

Variance is inferred for ordinary structs and enums from their fields. You do not annotate a user type as covariant in the way some languages allow. The compiler composes the variance of each field's type, and conflicting uses can make a parameter invariant. Therefore a private field can affect which public conversions compile even if clients never inspect that field.

Invariance prevents replacing a strong promise with a weak one

An exclusive reference is covariant in its own borrow lifetime but invariant in the type of the value it points to. These are separate parameters. A short reborrow of a mutable integer is ordinary. Reinterpreting a mutable slot that stores `&'static str` as one storing `&'short str` would allow a lasting promise to be overwritten with a temporary one.

Listing 5.3 Rejected example

```
fn overwrite<'a>(slot: &mut &'static str, incoming: &'a str) {
    *slot = incoming;
}

fn main() {
    let mut slot: &'static str = "permanent";
    let temporary = String::from("temporary");
    overwrite(&mut slot, &temporary);
}
```

If this were accepted, the caller could keep using `slot` as a static reference after `temporary` had been destroyed. The assignment would hide a short relationship behind a type that promises a long one. Invariance blocks the proposed substitution.

Interior mutable storage has the same issue even when the outer access is shared. `Cell<T>` and `UnsafeCell<T>` are invariant in `T`, because writes can occur through a shared reference to the container. A shared reference to an ordinary immutable slot is a reader; a shared reference to an interior mutable slot can authorize replacement through the container's API.

You can borrow an invariant container for a short operation. Rust 1.98 also permits shortening the borrow lifetime during certain unsizing coercions, such as `Cell<&'long mut i32>` to `Cell<&'short mut dyn Send>`. That coercion leaves the rule for subtyping through `Cell<T>` intact.

An advanced review should identify which parameter is varying before making a claim about a type. Saying mutable references are invariant is incomplete: they are invariant in the referent type and covariant in the reference's own lifetime. That detail is exactly what makes reborrowing useful without allowing invalid replacement.

Function inputs reverse the direction

A function able to process any short lived string can certainly process a static string. A function accepting only static strings cannot necessarily process a short lived one: it may store its argument somewhere that requires static validity. Function input types therefore introduce contravariant positions. Function results introduce covariant positions.

Listing 5.4 Rust example

```
fn length(text: &str) -> usize {
    text.len()
}

fn accepts_static_only(operation: fn(&'static str) -> usize) -> usize {
    operation("permanent input")
}

fn main() {
    assert_eq!(accepts_static_only(length), 15);
}
```

The `length` function is sufficiently general for this restricted caller. Reading function types as sets of accepted calls can make the reversed direction intuitive: an implementation that accepts more kinds of input can satisfy a client that will provide fewer kinds.

Higher ranked function types add another layer. A `for<'a> fn(&'a str) -> &'a str` must preserve the input and output relationship for each possible chosen lifetime. A function returning only a static string can sometimes satisfy borrowed output uses through shortening, but a function requiring a static input does not become general merely because its output is static. Input requirements and output guarantees must each be checked.

Variance can compose. A function taking a function argument introduces another reversal. Rather than memorizing nested cases, work from the operation: what may the client supply, what may the implementation store, and what promise will another component rely on afterward? This behavioral derivation scales better than counting symbols without understanding them.

PhantomData records a relationship without storing a value

Some types logically depend on a parameter even when their runtime representation does not contain that parameter directly. An iterator may hold raw pointers into borrowed storage. A typed identifier may contain only an integer but should not mix identifiers from different domains. `PhantomData` lets the type express a relationship that affects checking without allocating a value of that parameter.

Listing 5.5 Rust example

```
use std::marker::PhantomData;

struct Id<T> {
    index: usize,
    kind: PhantomData<fn() -> T>,
}

impl<T> Id<T> {
    fn new(index: usize) -> Self {
        Self { index, kind: PhantomData }
    }

    fn index(&self) -> usize {
        self.index
    }
}

enum User {}
enum Project {}

fn main() {
    let user = Id::<User>::new(3);
    let project = Id::<Project>::new(3);
    assert_eq!(user.index(), project.index());
}
```

The marker makes `Id<User>` and `Id<Project>` different types. It does not prove the index is in bounds, prevent an identifier from one user arena being passed to another user arena, or guarantee the referenced record still exists. Those are separate invariants. A generation, arena identity, or validated access operation may be required.

Why use `PhantomData<fn() -> T>` rather than `PhantomData<T>`? The latter models an ownership relationship with `T` for relevant analyses, including automatic thread safety traits. A pure numeric identifier does not own a `T`; requiring `T` to be thread safe merely to transfer its index would often be unnecessary. A function return marker expresses covariance in `T` without pretending to store or own a `T`. The choice should follow the actual logical relationship, not a recipe copied from another crate.

Phantom lifetimes and borrowing contracts

For a raw pointer based borrowed view, a marker such as `PhantomData<&'a T>` can express that the view is constrained by a shared borrow of `T` for `'a`. It contributes lifetime, variance, and automatic trait relationships. It does not validate the pointer. The constructor must still prove that the pointer refers to an appropriate live allocation, that the length is correct, and that no forbidden mutation occurs while the view is usable.

An invariant lifetime marker can be written using a parameter in both an input and an output position, for example `PhantomData<fn(&'a ()) -> &'a ()>`. This is useful in specialized branded APIs where accidental lifetime shortening would break a type level identity scheme. It is not a universal improvement over covariance. Making a lifetime invariant can unnecessarily reject useful clients if no invariant requires it.

Think of a phantom marker as part of the type's proof vocabulary. It tells the compiler which relationships matter; it does not establish them from runtime facts. A public constructor accepting any raw pointer and any lifetime chosen by the caller remains unsound if it returns a safe reference without requiring proof of validity. The marker does not repair that missing precondition.

The same caution applies to thread safety. Adding a marker that causes a type to lose `Send` or `Sync` can correctly preserve a thread affinity invariant. Choosing a marker that happens to produce convenient automatic traits without understanding why can hide a data race or an unnecessarily restrictive API. Start with the desired ownership and access model, then choose the representation that matches it.

Variance and destruction

Destruction creates uses that may be less obvious than explicit reads in a function body. A destructor might inspect a stored borrowed value. Rust's drop checking therefore considers whether generic relationships remain valid when destruction occurs. A marker can influence this analysis when a type logically owns values that are represented indirectly.

For ordinary application code, the safest approach is to use real owning fields where possible. A `Vec<T>` already encodes ownership, destruction,

variance, and allocation behavior. Replacing it with a raw pointer and a length requires reconstructing all those relationships. `PhantomData<T>` is one necessary piece in some designs, but allocation and cleanup invariants remain the implementation's responsibility.

Avoid assuming every subtle drop checking rule can be summarized as a single phantom table. Compiler rules and unstable escape hatches have details that should be checked against current documentation when implementing low level containers. Stable safe abstractions let most code avoid those details entirely. The advanced skill is knowing which guarantee the existing abstraction already provides and which guarantee a new unsafe layer must reestablish.

When diagnosing an unexpected lifetime error, draw the nested types and mark reading and writing positions. Shared references generally preserve the direction; exclusive referents and interior mutable storage often make it invariant; function inputs reverse it. Then connect the derived rule to a concrete operation that would become possible if the substitution were allowed. That concrete counterexample is the most reliable explanation.

Training Encounters

1. Explain why `&mut &'static str` cannot be treated as a mutable slot for an arbitrary local string, even when the outer mutable borrow itself is short.

2. A typed numeric handle does not own, borrow, or access its parameter `T`. What unintended restriction might `PhantomData<T>` impose compared with a function return marker? What guarantees does neither marker provide?

3. A wrapper contains `Cell<&'a str>`. Why is the wrapper invariant in `'a`, and why does borrowing the wrapper for a short time remain possible?

Training Debriefs

1. The outer borrow's duration controls how long the slot is exclusively accessed. The inner static lifetime controls what may remain stored in the slot after that access ends. Allowing a local reference to be written would violate the promise the caller retains about the slot's content. A short write can create a long lasting invalid state.

2. An ownership marker participates in automatic trait and ownership analyses as though the type owns a `T`, potentially making the numeric handle unavailable across threads when `T` is not thread safe. A function return marker models a different relationship. Neither checks index validity, record identity, generation freshness, or arena membership.

3. `Cell` can replace its content through shared access, so shortening the inner lifetime would permit writing a temporary reference into storage otherwise expected to hold a longer reference. Invariance prevents changing that embedded relationship. A short borrow of the wrapper does not change the type of its stored reference and is therefore a different, ordinary operation.

Further Reading

• The Rust Reference on subtyping and variance gives the language's variance relationships.

• The Rust standard library PhantomData documentation explains marker effects and intended uses.

• The Rustonomicon on drop checking discusses destruction related constraints for unsafe abstractions.

The Portal That Both Takes and Gives

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design three tiny wrapper types around lifetime-parameterized string access: a shared view, a function pointer that accepts a borrowed string, and a function pointer that both accepts and returns a string using one wrapper lifetime parameter. For each wrapper, predict whether that parameter is covariant, contravariant, or invariant. Write small substitution probes instead of relying on the type's name. Your combined wrapper should use a lifetime parameter in both the argument and result positions, rather than a higher-ranked function pointer that independently quantifies each call. Explain the difference. Finally, borrow an invariant wrapper briefly without changing its embedded lifetime and explain why that operation is still ordinary borrowing. The boss is cleared by a consistent argument, not by adding static until every example stops moving.

Encounter Constraints

• Use stable safe Rust and function pointer types for the substitution probes.

• Do not claim that a lifetime annotation can extend the storage behind a local string.

• Keep the higher-ranked alternative separate from the wrapper's own lifetime parameter.

Hint one / Hint two / Reveal debrief

Side Quest

Replace the combined function pointer with a higher-ranked callback that works for every call lifetime and returns text tied to that call. Write down what callers gain and what implementations must prove. Then add a phantom marker to a numeric identifier and explain why choosing variance does not validate the identifier's arena, generation, or bounds. Keep the two proofs separate.

Capability unlocked Mirror Sight - predicting which substitutions preserve permission

Choose Your Exit

Read the trait charters / Level 9
Connect substitution reasoning to obligations expressed by interfaces.

Descend to raw pointer contracts / Level 30
See why erased lifetime information must return as explicit invariants.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 06

The Airlock Ledger

Every expedition into Ferrum passes through nested airlocks: reserve a workspace, prepare a replacement, publish the result, and release the reservation. The emergency lights reveal that ordinary return, reported failure, unwinding, and abandoned guards are different exits. You must make the cleanup ledger match those exits while preserving valid state. The airlock is a teaching model of in-memory resources, so a tidy destruction log will not be mistaken for a guarantee about durable files, remote services, or cleanup after process death.

Mission State and enforce a failure postcondition while distinguishing semantic rollback from destructor-based cleanup.

Suggested loadout Level 1, Level 2, Level 3. These are guidance, not locked gates.

Mastery Targets

- Keep the published value unchanged when preparation returns an ordinary error.
- Explain the observed destruction order for success and unwinding paths.
- Show that forgetting an optional guard cannot make later safe operations memory-unsafe.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 6

Resource Acquisition Destruction and Panic Safety

Destruction is part of control flow

Resource acquisition is initialization associates resource responsibility with a value. When that value is dropped, its implementation releases the resource. Files, heap allocations, lock guards, temporary directories, and subscription handles can all follow this pattern. The key advantage is compositional cleanup: every early return and ordinary unwind path need not repeat a hand written release sequence.

Destruction does not mean garbage collection, nor does it guarantee that a business operation completed. A lock guard releasing a mutex makes the mutex available; it does not automatically repair partially updated application state. A file destructor closes a handle, but cannot return a writeback error to the already departing caller. Distinguish releasing resources from establishing the success of an external effect.

For values implementing `Drop`, the custom destructor runs before the fields are recursively dropped. Struct fields are then dropped in declaration order. Local variables in a scope are normally dropped in reverse declaration order. These are different orderings, and relying on the wrong one can create surprising behavior when fields represent dependent resources.

Listing 6.1 Rust example

```
struct Trace(&'static str);

impl Drop for Trace {
    fn drop(&mut self) {
        println!("drop {}", self.0);
    }
}

struct Pair {
    first: Trace,
    second: Trace,
}

impl Drop for Pair {
```

Listing 6.1 continued

```
    fn drop(&mut self) {
        println!("drop pair");
    }
}

fn main() {
    let earlier = Trace("earlier local");
    let pair = Pair {
        first: Trace("first field"),
        second: Trace("second field"),
    };
    let later = Trace("later local");
    let _ = (&earlier, &pair.first, &pair.second, &later);
}
```

The later local is dropped first, then the pair's custom destructor runs, then its first and second fields, and finally the earlier local. The custom destructor should not manually drop ordinary owned fields, because generated field cleanup still follows it. Most structs need no custom destructor at all; their fields already express the resource structure.

Explicit completion can report failure

`Drop::drop` cannot return a `Result`. If completion can fail in a way the caller must handle, offer an explicit operation such as `finish`, `flush`, `commit`, or `close`. The destructor can provide best effort cleanup or abandonment behavior, while the explicit method represents the success protocol.

Listing 6.2 Rust example

```
use std::io::{self, Write};

struct RecordWriter<W: Write> {
    output: W,
    records: usize,
}

impl<W: Write> RecordWriter<W> {
    fn new(output: W) -> Self {
        Self { output, records: 0 }
    }

    fn write_record(&mut self, bytes: &[u8]) -> io::Result<()> {
        self.output.write_all(bytes)?;
        self.output.write_all(b"\n")?;
        self.records += 1;
```

Listing 6.2 continued

```
        Ok(())
    }

    fn finish(mut self) -> io::Result<(W, usize)> {
        self.output.flush()?;
        Ok((self.output, self.records))
    }
}

fn main() -> io::Result<()> {
    let mut writer = RecordWriter::new(Vec::<u8>::new());
    writer.write_record(b"alpha")?;
    let (bytes, count) = writer.finish()?;
    assert_eq!(count, 1);
    assert_eq!(bytes, b"alpha\n");
    Ok(())
}
```

This type has no custom `Drop`, allowing `finish` to move the output field into its result. On a flush error, the writer is consumed and its output is dropped. If retrying or inspecting the output is a requirement, the error type can return the writer or underlying stream along with the error. Error ownership is a design choice rather than an incidental consequence of using `?`.

Even a successful flush is not a universal durability guarantee. Different writers and operating systems attach different meanings to buffering and synchronization. A storage API should document the level of completion it promises rather than treating destructor execution as evidence of durable persistence.

Panic safety has multiple levels

Memory safety requires that a panic cannot leave safe code able to cause undefined behavior through an abstraction. Logical validity requires the value to remain in a state its ordinary methods can handle. A stronger operation guarantee preserves the previous logical value when an operation fails or panics. These are distinct obligations.

An operation can be memory safe while leaving a partially updated list. That may be an acceptable documented contract. A transaction editor may

instead need all or nothing behavior. The implementation strategy follows the required guarantee, not the presence of a destructor alone.

One effective strong guarantee is to build new state separately, invoking fallible or user supplied behavior before replacing the old state. The old representation remains established until commit.

Listing 6.3 Rust example

```
fn rebuild<T, E, F>(values: &mut Vec<T>, mut make: F) -> Result<(), E>
where
    F: FnMut(&T) -> Result<T, E>,
{
    let mut next = Vec::with_capacity(values.len());
    for value in values.iter() {
        next.push(make(value)?);
    }
    *values = next;
    Ok(())
}

fn main() {
    let mut values = vec![2, 4, 6];
    let result = rebuild(&mut values, |value| {
        if *value == 4 { Err("rejected") } else { Ok(value * 10) }
    });
    assert_eq!(result, Err("rejected"));
    assert_eq!(values, [2, 4, 6]);
}
```

The original vector remains in place if `make` returns an error or unwinds before the final assignment. Newly created elements are cleaned up through their temporary vector. This guarantee concerns the vector's owned contents under the operation's ordinary assumptions. A callback can still change external state, mutate interior mutable fields through `&T`, or perform irreversible I/O. Staging a vector is not a transaction over every effect the callback may produce.

The final assignment also drops the old vector's elements. If an element destructor panics, replacement and destruction have their own observable behavior. Strong guarantees must be worded carefully when arbitrary user destructors are involved. Many practical APIs promise preservation until commit, then treat destructor failure as outside ordinary recoverable operation semantics.

Guards make temporary obligations visible

A guard can restore state if an operation exits before committing. The guard owns the right to perform cleanup and usually borrows the state it protects, preventing unrelated access while the obligation is active.

Listing 6.4 Rust example

```
struct AppendTransaction<'a, T> {
    values: &'a mut Vec<T>,
    original_len: usize,
    committed: bool,
}

impl<'a, T> AppendTransaction<'a, T> {
    fn new(values: &'a mut Vec<T>) -> Self {
        let original_len = values.len();
        Self { values, original_len, committed: false }
    }

    fn push(&mut self, value: T) {
        self.values.push(value);
    }

    fn commit(mut self) {
        self.committed = true;
    }
}

impl<T> Drop for AppendTransaction<'_, T> {
    fn drop(&mut self) {
        if !self.committed {
            self.values.truncate(self.original_len);
        }
    }
}

fn main() {
    let mut values = vec![1, 2];
    {
        let mut transaction = AppendTransaction::new(&mut values);
        transaction.push(3);
    }
    assert_eq!(values, [1, 2]);
    {
        let mut transaction = AppendTransaction::new(&mut values);
        transaction.push(4);
        transaction.commit();
```

Listing 6.4 continued

```
    }
    assert_eq!(values, [1, 2, 4]);
}
```

The guard can only append through its API, so truncation restores the original prefix. If it offered arbitrary element mutation, the same rollback would no longer restore the original values. The shape of the available operations is part of the proof. A guard is not automatically transactional merely because it has a destructor.

The implementation is memory safe if the guard is forgotten: the vector may retain the appended values, but its representation stays valid. This is necessary because safe code can call `mem::forget`. The logical rollback guarantee applies when the guard is actually dropped; soundness cannot depend on a destructor inevitably running.

Truncation drops appended values. If one of their destructors panics while another panic is already unwinding, the process normally aborts. Destructors used in recovery paths should therefore avoid panicking. A guard can simplify control flow, but it cannot turn arbitrary panic behavior into reliable recovery.

Destructors are not guaranteed on every exit

Normal scope exit and unwinding run appropriate destructors. Process abort, certain explicit process termination paths, and a panic configured to abort do not provide the same cleanup behavior. Safe code can also deliberately leak a value with `mem::forget`, or create reference counted cycles that keep values alive indefinitely.

Therefore a type must remain sound if its destructor never runs. A leaked allocation is usually a resource leak, not undefined behavior. An API that temporarily gives safe code access to invalid memory and relies on a future destructor to repair the situation is unsound. The caller might forget the repair guard or terminate before it runs.

This principle is particularly important for scoped threads, iterators exposing temporary holes in collections, and synchronization guards implemented with unsafe internals. During every state observable by safe clients, the representation must satisfy the memory safety contract even if

no future cleanup occurs. It can leak or become permanently unavailable; it cannot permit invalid access.

Logical resources need a related operational design. If a lock file, remote lease, or database record must eventually be released despite process death, the external system needs timeout, recovery, or ownership transfer rules. Local destruction is a useful fast path, not a complete failure detector.

Catching a panic does not roll back state

`catch_unwind` can catch unwinding panics within its supported boundary. It does not catch an aborting panic, make undefined behavior recoverable, or undo changes already performed. Its unwind safety bounds are a warning about potentially observable interrupted invariants, not a full transactional proof.

Listing 6.5 Rust example

```
use std::panic::{catch_unwind, AssertUnwindSafe};

fn main() {
    let mut values = vec![1];
    let result = catch_unwind(AssertUnwindSafe(|| {
        values.push(2);
        panic!("interrupted after mutation");
    }));
    assert!(result.is_err());
    assert_eq!(values, [1, 2]);
}
```

The explicit assertion of unwind safety in this example is justified by the simple known vector operation and the decision to accept partial mutation. It would not be justified merely because the compiler requested it. Before using the wrapper around a complex operation, identify every invariant interrupted execution could expose and explain why later methods remain correct.

Catch boundaries are useful around plugin execution, worker jobs, or test harnesses when the program has a defined policy for abandoning affected state. Often the right recovery is to discard an entire request or rebuild a component rather than continue using a half changed object. The safe

language prevents many memory errors, but it does not decide which application state remains trustworthy.

Make dependency order structural

If one field's destructor needs another resource to remain available, model that dependency directly where possible. A borrowed field naturally constrains the owner relationship, although self referencing structs introduce additional difficulties. Often the simpler design is a small inner owner whose destructor performs the required ordering explicitly, or an explicit `shutdown` method followed by ordinary field cleanup.

Do not rely on the visual order in which local variables happen to be created across distant branches. A future refactor can alter that order without changing any method signatures. Keep interdependent resources together and document why their field order matters when declaration order is part of the contract.

Panic safety becomes tractable when resource ownership, temporary representation states, and commit points are visible. At each user callback, allocation, fallible I/O operation, and destructor, ask what would remain if control never reached the next line. That question finds many bugs that happy path tests miss.

Training Encounters

1. A writer destructor attempts to flush and ignores errors. Why might the API still need an explicit consuming `finish` method? What should happen to the writer on a finish error if retry is required?

2. Extend the append guard concept to allow editing existing elements. Explain why remembering only the original length is no longer a sufficient rollback record.

3. An unsafe collection temporarily marks initialized elements as absent and promises a guard destructor will restore them. What must remain true if the guard is deliberately forgotten?

Training Debriefs

1. The caller needs an operation that reports completion failure before relinquishing control. If retry is required, the error should return the writer or preserve it behind a mutable receiver with a documented state. Consuming and dropping the stream on error discards that recovery opportunity.

2. Truncation only removes the appended suffix. It cannot reconstruct overwritten prefix elements. A rollback capable editor must store old values, stage an entire replacement, or restrict edits to reversible operations with a complete log. External effects need their own compensation or transactional system.

3. Safe clients must still be unable to read uninitialized values, drop the same value twice, or violate aliasing. Forgetting may leave a leak or permanently reduce the visible collection, but it cannot make later safe operations cause undefined behavior. Restoration cannot be a prerequisite for soundness.

Further Reading

• The Rust Reference on destructors specifies drop order, scope behavior, and skipped destruction.

• The Rust standard library panic module documents catch boundaries and unwind safety.

• The Rust standard library mem forget function explains why unsafe code cannot assume destruction.

The Interrupted Replacement

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Create a small in-memory service holding a published configuration and a reservation counter. A repair operation obtains a reservation, prepares a replacement using a fallible callback, and publishes it only after preparation succeeds. Keep user callback execution before the publication step. Use trace resources to explain cleanup order when preparation succeeds, returns an error, or panics under unwinding. State what happens if the reservation guard is deliberately forgotten. You may allow a leaked reservation to block future repairs, but it must not expose invalid memory. Finally, consider a configuration whose destructor can panic during replacement. Revise any overly broad claim that every possible failure leaves the old value intact. Identify the narrow representation assumptions under which your ordinary-error guarantee actually holds.

Encounter Constraints

• Separate preparation from publication instead of mutating the published configuration incrementally.

• Do not depend on a guard destructor as the only thing preventing undefined behavior.

• Report completion explicitly; do not treat Drop as a fallible transaction-commit API.

Hint one / Hint two / Reveal debrief

Side Quest

Add an explicit finish method that returns a repair report while leaving ordinary Drop responsible only for local cleanup. Decide whether a finish failure returns the resource for retry or consumes it. Then explain why the same design would need a different protocol if publishing also changed a file and notified a remote process. A local guard cannot define that distributed agreement.

Capability unlocked Airlock Discipline - separating cleanup, rollback, and completion

Choose Your Exit

Open the echo chamber / Level 7
Examine mutation policies and callbacks that can reenter a component.

Follow the cancellation rift / Level 26
Apply interruption reasoning to suspended computations and dropped futures.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 07

The Document That Answers Back

Inside the Borrowed Citadel, a document terminal can invoke a visitor's transformation plugin. Some plugins inspect the same document; a mischievous one edits it before returning its proposal. Holding a dynamic borrow across that callback jams the terminal. Releasing the borrow avoids the jam but introduces a stale-proposal problem. Your mission is to solve both layers explicitly: keep temporary access short enough for reentrancy, then validate whether the state used to compute the proposal still matches the state being changed.

Mission Use short RefCell borrows and a generation check to distinguish access validity from proposal freshness.

Suggested loadout Level 3, Level 4, Level 6. These are guidance, not locked gates.

Mastery Targets

- Allow a callback to inspect or edit the document without a retained conflicting RefCell borrow.
- Reject a proposal computed from an obsolete generation before applying it.
- Check generation exhaustion before changing text or its generation.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 7

Interior Mutability as a Controlled Access Policy

Shared access can include an update protocol

A shared reference normally rules out mutation of ordinary bytes reachable through it while the relevant shared access remains active. Interior mutability provides a deliberate exception through types built around `UnsafeCell`. This permits useful contracts such as a cache updated during a read, a counter changed through a shared handle, or a dynamically checked borrow of a graph node.

The exception is local and structured. Wrapping one field in a cell does not make every neighboring field mutable through shared access. `UnsafeCell` relaxes the shared immutability rule for its contents; it does not make simultaneous conflicting exclusive references valid, make raw pointers automatically safe, or provide synchronization between threads.

The important design question is which policy replaces ordinary exclusive access. `Cell` moves values in and out without lending ordinary references to the stored value. `RefCell` tracks active borrowing at runtime and returns guards. A mutex supplies synchronized access across threads. Atomic types permit specific concurrent operations with memory ordering rules. These are different policies, not progressively stronger forms of a single universal tool.

Choose the narrowest policy that matches the invariant. A counter does not usually need a dynamically borrowed struct. A field that can be swapped as a whole may not need a guard. A component already accessed exclusively may need no interior mutability at all.

Cell avoids references to the changing value

For `Copy` values, `Cell::get` returns a copy and `set` replaces the stored value. Because clients do not receive a shared reference to the changing inner value, the API can safely replace it through a shared reference to the cell. For some noncopyable values, operations such as `take` and `replace` similarly transfer ownership rather than lend a reference.

Listing 7.1 Rust example

```
use std::cell::Cell;

struct Meter {
    reads: Cell<u64>,
    value: String,
}

impl Meter {
    fn value(&self) -> &str {
        let count = self.reads.get();
        self.reads.set(count.saturating_add(1));
        &self.value
    }

    fn reads(&self) -> u64 {
```

Listing 7.1 continued

```
        self.reads.get()
    }
}

fn main() {
    let meter = Meter {
        reads: Cell::new(0),
        value: String::from("payload"),
    };
    assert_eq!(meter.value(), "payload");
    assert_eq!(meter.value(), "payload");
    assert_eq!(meter.reads(), 2);
}
```

The returned string borrows `value`, whose bytes do not change. The counter's update is confined to another field's policy. Saturating arithmetic makes the counter's overflow behavior explicit; instrumentation should not unexpectedly panic because it exceeded an integer limit.

This component is not automatically shareable through `&Meter` across threads because `Cell` is not `Sync`. The compiler prevents using this unsynchronized counter as if it were an atomic. If cross thread observations are required, choose an atomic counter or synchronized owner and define whether counts need exact ordering with the payload operation.

For a noncopyable value, a cell can implement a temporary ownership slot. Taking an `Option<String>` leaves `None`, so the container always remains a valid value. This is useful when moving a callback or resource out before invoking other code. The absence state must still have a clear meaning, especially during reentrancy.

Listing 7.2 Rust example

```
use std::cell::Cell;

fn main() {
    let pending = Cell::new(Some(String::from("work")));
    let work = pending.take().unwrap();
    assert!(pending.take().is_none());
    pending.set(Some(work.to_uppercase()));
    assert_eq!(pending.take().as_deref(), Some("WORK"));
}
```

Nothing guarantees restoration if later code panics or abandons the operation. The valid `None` state prevents an invalid representation, but the domain may require a state enum distinguishing empty, running, failed, and completed work. Interior mutability does not remove the need to model interrupted transitions.

RefCell turns borrowing into a runtime protocol

`RefCell<T>` permits shared access to a container while dynamically enforcing compatible inner borrows. A shared borrow returns a `Ref<T>` guard, and an exclusive borrow returns `RefMut<T>`. The guards retain the runtime borrow state until they are dropped. An incompatible request through the panicking API causes a panic; the `try_borrow` variants report failure instead.

This is not a runtime version of data race prevention. `RefCell` is designed for single threaded sharing and is not `Sync`. It addresses access patterns the static checker cannot conveniently express, such as graph navigation where ownership and local mutation are separated. A mutex solves a different concurrency problem and also has different blocking and failure behavior.

Listing 7.3 Rust example

```
use std::cell::{Ref, RefCell};

struct Document {
    lines: RefCell<Vec<String>>,
}

impl Document {
    fn first(&self) -> Option<Ref<'_, str>> {
        let lines = self.lines.borrow();
        if lines.is_empty() {
            None
        } else {
            Some(Ref::map(lines, |lines| lines[0].as_str()))
        }
    }

    fn append(&self, line: String) -> Result<(), String> {
        match self.lines.try_borrow_mut() {
            Ok(mut lines) => {
                lines.push(line);
                Ok(())
```

Listing 7.3 continued

```
            }
            Err(_) => Err(line),
        }
    }
}

fn main() {
    let document = Document {
        lines: RefCell::new(vec![String::from("first")]),
    };
    let first = document.first().unwrap();
    assert!(document.append(String::from("second")).is_err());
    assert_eq!(&*first, "first");
    drop(first);
    assert!(document.append(String::from("second")).is_ok());
}
```

`Ref::map` narrows the exposed view while retaining the original borrow guard. Returning an ordinary `&str` from the same temporary guard would be wrong: the guard would release the dynamic borrow while the reference remained usable. A later mutation could then invalidate that reference. The guard is part of the result's access permission, not an implementation detail that can be discarded.

The append error returns the unsaved line, preserving ownership for a retry. A richer API might distinguish busy access from allocation or validation errors. Treating a dynamic borrow failure as an ordinary recoverable condition is sensible only if callers have a defined retry or abandonment policy. Otherwise the failure may reveal an unexpected reentrant operation that should be fixed structurally.

Reentrancy is the hidden source of many borrow panics

A function that holds `RefMut` while calling user code exposes itself to reentrancy. The callback may invoke another method on the same component and request an incompatible borrow. The program is single threaded, but the call stack can still revisit shared state before the outer operation has completed.

One useful policy is to snapshot owned callback handles, release the container borrow, and only then invoke callbacks. The snapshot defines which listeners participate in the current event. Changes made by

callbacks affect the next event, rather than mutating the collection currently being traversed.

Listing 7.4 Rust example

```
use std::cell::RefCell;
use std::rc::Rc;

type Listener = Rc<dyn Fn()>;

#[derive(Default)]
struct Bus {
    listeners: RefCell<Vec<Listener>>,
}

impl Bus {
    fn subscribe(&self, listener: Listener) {
        self.listeners.borrow_mut().push(listener);
    }

    fn emit(&self) {
        let snapshot = self.listeners.borrow().clone();
        for listener in snapshot {
            listener();
        }
    }
}

fn main() {
    let bus = Rc::new(Bus::default());
    let weak_bus = Rc::downgrade(&bus);
    bus.subscribe(Rc::new(move || {
        if let Some(bus) = weak_bus.upgrade() {
            bus.subscribe(Rc::new(|| println!("later listener")));
        }
    }));
    bus.emit();
    assert_eq!(bus.listeners.borrow().len(), 2);
}
```

The temporary borrow used to clone the vector ends before iteration begins. Each cloned `Rc` keeps a listener alive independently of collection changes. A weak link in the callback avoids making the bus own a listener that strongly owns the bus. This resolves one ownership cycle without changing listener invocation semantics.

Snapshotting has a cost proportional to the number of handles, and it means removed listeners can still run if they were captured by the current

snapshot. If immediate cancellation is required, each listener can have a separate active flag checked before invocation. That introduces another state policy. The correct design depends on whether mutation during dispatch should be deferred, immediately observed, rejected, or queued.

Allowing recursive `emit` can still cause unbounded recursion or unexpected ordering. Releasing `RefCell` guards prevents a borrow panic; it does not prove the event protocol is sensible. A dispatching flag plus an event queue may be needed to turn recursive emission into iterative processing.

Once initialization has a different invariant

Some shared updates happen at most once. A lazily computed index or parsed configuration can use a once initialization abstraction rather than unrestricted replacement. For single threaded code, `std::cell::OnceCell` provides a cell whose initialized value can be borrowed without keeping a runtime borrow guard.

The reason is structural: once initialized, the shared API does not permit arbitrary replacement of the value while shared references exist. Its invariant is stronger and narrower than `RefCell`. A cross thread counterpart provides the required synchronization for shared initialization.

Choose once initialization when the cached value is genuinely stable. If a schema can change and invalidate its cache, pretending initialization is permanent simply moves the inconsistency elsewhere. An invalidatable cache needs a generation relationship, a mutable owner, or a policy that returns independent snapshots.

Initialization callbacks can also panic or attempt recursive initialization. Read the chosen abstraction's contract rather than assuming every lazy container handles reentrancy the same way. In particular, a cache fill operation should not silently depend on invoking itself to supply its own missing value.

UnsafeCell is a primitive rather than an access policy

`UnsafeCell<T>` permits obtaining a raw pointer to its contents from shared access to the cell. The caller of unsafe pointer operations must still establish compatible access, initialization, alignment, and lifetime. It provides no borrow counter, no lock, no wakeup mechanism, and no automatic proof that a returned reference can coexist with other operations.

A wrapper that exposes `fn get_mut(&self) -> &mut T` for arbitrary shared callers is generally unsound. Two calls through shared aliases could return overlapping exclusive references. The wrapper needs a mechanism that prevents this situation, such as an exclusive receiver, a dynamic guard, an ownership token, or a properly synchronized protocol.

`UnsafeCell` also does not permit a type to be safely transmuted into every enclosing representation that contains it. Niche optimizations and layout choices of an outer enum can differ. Having the same size and alignment as `T` is not a general license to reinterpret `Option<T>` as `Option<UnsafeCell<T>>`. Representation guarantees must be checked at the actual containing type.

Most application code should use the standard safe cells. Writing a new cell abstraction is justified when its narrower protocol provides something concrete that those abstractions do not express. The proof should describe how every safe method preserves that protocol, including failure paths and reentrant callbacks.

Interior mutability changes the public meaning of a read

An API taking `&self` communicates that callers may share access, but it does not guarantee observational purity. A read may populate a cache, advance a cursor, record statistics, or consume a pending callback. Document effects that matter to clients, especially effects influencing latency, ordering, or repeated results.

At the same time, do not expose interior mutability merely to avoid threading `&mut` through a design. If one component owns the state and updates occur sequentially, ordinary exclusive access gives simpler invariants and removes runtime borrow failures. Interior mutability earns

its place when shared ownership or an external callback boundary makes a different access protocol useful.

The strongest designs confine it. Keep a cell field private, expose a small set of domain operations, and avoid returning broad guards when a copied result or narrow mapped view is sufficient. This reduces the number of states clients can hold across callbacks and makes both performance and correctness more predictable.

Training Encounters

1. Why cannot a method borrow a string from a `RefCell<String>`, drop the `Ref` guard, and return a plain reference to that string?

2. A notification bus holds a mutable borrow of its listener list while invoking every callback. Describe a reentrant failure and a snapshot based policy that avoids it. State one semantic cost of the snapshot.

3. A cache is initialized once and never invalidated. Compare the borrowing behavior of a once cell and a `RefCell<Option<T>>`. What additional problem appears when the cached source becomes mutable?

Training Debriefs

1. Releasing the guard removes the runtime reservation that prevents conflicting mutation. Another caller could replace or grow the string while the returned reference remained in use. Returning a mapped `Ref<str>` keeps the access policy active; returning an owned string removes the dependency at the cost of copying.

2. A callback that subscribes or unsubscribes requests another mutable borrow while the outer borrow is active, causing failure. Clone owned listener handles under a short borrow, release it, then invoke the snapshot. A removed listener can still run during the current dispatch, so immediate cancellation needs another explicit mechanism.

3. A once cell's shared API preserves an initialized value, allowing ordinary borrowed access without a dynamic borrow guard. A `RefCell` must track borrowing because it permits later replacement. If the source changes, cached results can become stale; initialization policy alone does not express a dependency between source generation and cached generation.

Further Reading

• The Rust standard library cell module compares the safe interior mutability policies.

• The Rust standard library UnsafeCell documentation states the primitive's remaining obligations.

• The Rust standard library Ref documentation documents mapped views that preserve runtime borrowing.

The Reentrant Revision

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Model one single-threaded document as a generation number and owned text behind a RefCell. A propose operation takes a snapshot, invokes a user callback outside the RefCell borrow, and attempts to install the proposed replacement. The callback may itself edit the original document through another safe access path. A proposal based on the old generation must then be rejected without overwriting that intervening edit. Return a structured stale-proposal result, and decide whether to preserve the candidate text for inspection or retry. Test a pure transformation, a reentrant read, and a callback that makes an intervening edit. Explain why successfully avoiding a borrow panic does not prove that the resulting document transition is semantically valid.

Encounter Constraints

• Do not hold a Ref or RefMut while invoking the supplied callback.

• Make every mutation path advance the generation using the same checked policy.

• State that callback side effects outside the document are not rolled back by rejecting its proposal.

Hint one / Hint two / Reveal debrief

Side Quest

Avoid copying a large document by storing immutable text in Arc<str> and cloning only the handle for each proposal. Explain which allocation remains alive during long callbacks and how a retained historical view affects memory. Then compare this single-threaded protocol with a mutex-protected version: changing the guard type does not remove the need for a freshness rule. Measure sustained load.

Capability unlocked Echo Control - safe reentrancy with an explicit freshness check

Choose Your Exit

Map the shifting arena / Level 8
Use explicit identity when objects move or storage is reused.

Challenge the versioned archive / Level 38
Extend snapshot validation into transaction-level consistency.

Mark your quest log / Return to the map

ZONE 1 / THE BORROWED CITADEL / LEVEL 08

The Shifting Street Atlas

Ferrum's street atlas stores junctions densely so the patrol can scan them quickly. Unfortunately, deleting one junction moves another into its place, and old street signs now point at the wrong destination. The Borrowed Citadel needs an atlas whose physical packing can change without rewriting every public identifier. Your expedition follows the gap between a record's current storage position and its logical identity. The repair must handle movement, deletion, and reuse separately, with no raw pointer offered as a shortcut through the maze.

Mission Design stable generational handles over movable dense storage using an explicit indirection table.

Suggested loadout Level 1, Level 2, Level 3. These are guidance, not locked gates.

Mastery Targets

- Preserve a live record's handle when swap removal moves its dense storage position.
- Reject deleted handles even after a slot is reused.
- State an arena identity and generation-exhaustion policy rather than silently assuming either away.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 8

Arenas Handles Graphs and Ownership Architecture

Object relationships need not be ownership relationships

A graph can contain cycles, multiple incoming edges, and references that outlive the operation that created them. Representing every edge as a Rust borrow often creates an impossible lifetime puzzle because the graph's logical topology is being asked to determine allocation ownership. Those are separate structures.

One owner can hold all nodes while edges store identifiers. Removing a node is then an operation on that owner, and following an edge requires a fresh checked lookup. Alternatively each node can be reference counted, and some edges can be weak so cycles do not retain the entire structure. Another design owns nodes in a region that is destroyed all at once. Each architecture selects a different deletion, sharing, and lifetime policy.

The first step is to answer how nodes should disappear. If the whole graph dies together, a central arena is simple. If individual records are deleted and identifiers can survive, identifiers need stale reference detection. If independent components genuinely share ownership, reference counting may be appropriate. If ownership is hierarchical but navigation is cyclic, separate strong parent to child ownership from weak child to parent navigation.

Borrowing then becomes a temporary operation on that architecture. A lookup lends a node for one use. A mutation lends it exclusively. The long lived graph relationships remain identifiers or owned handles rather than permanent language references into movable containers.

A central owner makes cycles ordinary data

An index based graph can store arbitrary cyclic relationships without cyclic ownership. Its vector owns the nodes. Each edge is an integer interpreted by that graph. Moving the graph or reallocating its vector does not invalidate the integers, provided the graph's indexing policy preserves their meaning.

Listing 8.1 Rust example

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct NodeId(usize);

struct Node {
    label: String,
    edges: Vec<NodeId>,
}

#[derive(Default)]
struct Graph {
    nodes: Vec<Node>,
}

impl Graph {
    fn add(&mut self, label: &str) -> NodeId {
        let id = NodeId(self.nodes.len());
        self.nodes.push(Node {
            label: label.to_owned(),
            edges: Vec::new(),
        });
        id
    }

    fn connect(&mut self, from: NodeId, to: NodeId) -> bool {
        if to.0 >= self.nodes.len() {
            return false;
        }
        match self.nodes.get_mut(from.0) {
            Some(node) => {
                node.edges.push(to);
                true
            }
            None => false,
        }
    }

    fn label(&self, id: NodeId) -> Option<&str> {
        self.nodes.get(id.0).map(|node| node.label.as_str())
    }
}

fn main() {
    let mut graph = Graph::default();
    let first = graph.add("first");
    let second = graph.add("second");
    assert!(graph.connect(first, second));
    assert!(graph.connect(second, first));
    assert_eq!(graph.label(first), Some("first"));
}
```

This graph is append only. It deliberately offers no deletion or reordering operation, because either would require defining how old identifiers behave. An index newtype prevents accidental confusion with unrelated numeric quantities but does not prevent a `NodeId` from one graph being passed to another. The runtime bounds check prevents memory unsafety; it does not prevent a logically incorrect cross graph lookup that happens to be in bounds.

There are several ways to strengthen identity. Include an owner identifier in the handle, use a lifetime brand supplied by a scoped API, or make graph handles private behind an operation that never mixes owners. Each adds constraints or bookkeeping. The important point is to name the desired guarantee precisely: in bounds, not deleted, same owner, and same logical record are distinct properties.

Deletion turns indices into a versioning problem

Suppose slot 7 once contains a user, is removed, and later contains another user. A raw index 7 can no longer distinguish those records. This is the same shape as the ABA problem: the visible identifier returns to the same value while the underlying identity has changed.

A generational handle stores both a slot and a generation. Removal advances the slot's generation. Lookup accepts a handle only when both match. A stale handle can remain in another node without becoming a reference to the replacement object.

Listing 8.2 Rust example

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct Handle {
    slot: usize,
    generation: u64,
}

struct Slot<T> {
    generation: u64,
    value: Option<T>,
}

struct Arena<T> {
    slots: Vec<Slot<T>>,
    free: Vec<usize>,
}
```

Listing 8.2 continued

```
impl<T> Arena<T> {
    fn new() -> Self {
        Self { slots: Vec::new(), free: Vec::new() }
    }

    fn insert(&mut self, value: T) -> Handle {
        if let Some(index) = self.free.pop() {
            let slot = &mut self.slots[index];
            slot.value = Some(value);
            Handle { slot: index, generation: slot.generation }
        } else {
            let index = self.slots.len();
            self.slots.push(Slot { generation: 0, value: Some(value) });
            Handle { slot: index, generation: 0 }
        }
    }

    fn get(&self, handle: Handle) -> Option<&T> {
        let slot = self.slots.get(handle.slot)?;
        if slot.generation != handle.generation {
            return None;
        }
        slot.value.as_ref()
    }

    fn remove(&mut self, handle: Handle) -> Option<T> {
        let slot = self.slots.get_mut(handle.slot)?;
        if slot.generation != handle.generation {
            return None;
        }
        let value = slot.value.take()?;
        if let Some(next) = slot.generation.checked_add(1) {
            slot.generation = next;
            self.free.push(handle.slot);
        }
        Some(value)
    }
}

fn main() {
    let mut arena = Arena::new();
    let old = arena.insert(String::from("old"));
    assert_eq!(arena.remove(old).as_deref(), Some("old"));
    let new = arena.insert(String::from("new"));
    assert_eq!(old.slot, new.slot);
    assert_ne!(old.generation, new.generation);
```

Listing 8.2 continued

```
    assert!(arena.get(old).is_none());
    assert_eq!(arena.get(new).map(String::as_str), Some("new"));
}
```

The generation increment is checked. On exhaustion, the empty slot is retired instead of being reused with a wrapped generation. Wrapping would eventually permit an ancient handle to match a new object. A 64 bit counter makes exhaustion unlikely, but an invariant should not silently depend on unlikely being impossible.

The arena still lacks owner identity. Two separate arenas can produce the same slot and generation pair. Applications needing cross arena safety must add that relationship. The code also does not reclaim its slot vector's peak allocation; deletion reuses slots but retains storage capacity. These are deliberate limits of this small design, not properties generations automatically solve.

Handles trade immediate guarantees for checked access

A live Rust reference guarantees appropriate access to its referent for its usable region. A handle is inert data. It may be stale, refer to another owner, or fail lookup. This makes handles easy to store in cyclic structures, serialize, queue, and transfer, but every access must pass through a policy.

That policy can be an `Option`, a structured error, a transaction, or a snapshot. A graph traversal should decide whether a dangling logical edge is expected after deletion, a signal to prune the edge, or a violation of a stronger graph invariant. Silently skipping missing nodes can be correct for best effort exploration and incorrect for accounting or dependency resolution.

Once lookup returns a borrowed reference, ordinary borrowing resumes. A handle does not let the caller remove the node while a reference to it remains in use. The central owner still requires exclusive access for deletion. Thus handles separate long lived identity from short lived memory access rather than replacing the language's safety rules.

For concurrent access, a handle also does not keep a node alive between two unsynchronized lookups. A successful validation followed by an unrelated later dereference can race with deletion unless the same lock,

epoch, reference count, or snapshot protects the entire operation. This is a time of check versus time of use issue. Generations detect stale identities; they do not by themselves provide a borrowing or reclamation mechanism across threads.

Reference counting follows distributed ownership

`Rc<T>` provides shared ownership within one thread. `Arc<T>` provides thread safe reference count management, subject to the inner type's access requirements. Cloning either handle keeps the same allocation alive. It does not clone the data or make mutable access automatically safe.

Reference counting is a good fit when no central component naturally owns all users of an object. It can make a poor fit when it is introduced merely to avoid deciding ownership. Strong cycles retain allocations because every member continues to have an owner. Weak references allow navigation without contributing to the strong count.

Listing 8.3 Rust example

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};

struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn main() {
    let root = Rc::new(Node {
        name: String::from("root"),
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(Vec::new()),
    });
    let child = Rc::new(Node {
        name: String::from("child"),
        parent: RefCell::new(Rc::downgrade(&root)),
        children: RefCell::new(Vec::new()),
    });
    root.children.borrow_mut().push(Rc::clone(&child));
    assert_eq!(child.name, "child");
    assert_eq!(child.parent.borrow().upgrade().unwrap().name, "root");
    drop(root);
    assert!(child.parent.borrow().upgrade().is_none());
}
```

The root owns the child, while the child only observes its parent. The external child handle keeps the child alive after root destruction. The weak parent link can then fail to upgrade, which is an explicit part of the navigation contract.

A general attachment API would need more rules than this fixed construction. Allowing arbitrary strong child edges could create cycles. Reparenting requires updating both relationships consistently. Callbacks during reparenting could observe an intermediate state. The ownership building blocks prevent memory errors, but the tree invariant still belongs to the implementation.

Region ownership can simplify bulk lifetimes

A region or arena allocator can allocate many objects that are released together. This can reduce individual allocation overhead and express the fact that a compiler syntax tree, request working set, or game level has one common lifetime. The relevant owner is the region; individual edges can be borrowed within its valid lifetime or represented as indices.

Bulk memory release and object destruction are different promises. Some arena designs run element destructors; others are intended for data whose teardown is unnecessary or managed separately. If nodes own files, reference counted values, or other resources, skipping their destructors can retain resources even when the backing memory is reclaimed. Read an allocator's contract rather than assuming all arenas behave like a vector.

Stable addresses can also matter. A vector of nodes can move its elements on reallocation. A vector of boxes moves handles while pointees remain allocated separately, but deletion and extraction still matter. An arena promising stable addresses must control every operation that could relocate or destroy an object while references remain valid. Stability of storage is one part of a borrowing contract, not a substitute for it.

Serialization reveals the value of explicit identity

Raw addresses and ordinary references make poor persistent identities. They belong to one process execution and one allocation history. Handles with explicit keys can be serialized, validated on load, and remapped during compaction. This is useful even when borrowing would be possible inside the process.

A persistent identifier should not reuse an in memory slot number without a policy. Slots can change after compaction, generations may restart after a process restart, and two databases can issue the same local identifier. A durable identity often needs a namespace or stable key separate from the efficient runtime handle. Keeping those layers separate permits a dense arena internally without coupling the file format to its representation.

The same separation helps testing. A small model can represent a graph as maps from stable keys to records, while the implementation uses generational slots. Tests can compare observable operations without assuming identical storage layout. This makes it easier to detect stale handle, deletion, and compaction bugs.

Choose ownership before choosing pointers

When a recursive data model causes lifetime difficulties, write down who deletes each object and what a surviving edge should do after deletion. If deletion is global, central ownership may be sufficient. If deletion is individual, use validated identity. If ownership is distributed, decide which edges are strong and which are observations. If access is concurrent, select a synchronization and reclamation strategy for the whole access interval.

Only then choose `Box`, references, `Rc`, `Arc`, weak links, indices, or raw pointers. These types implement different answers to those questions. Starting with raw pointers because a graph has cycles discards useful checking before establishing a replacement policy.

The goal is a small number of ownership rules that remain true through insertion, lookup, deletion, traversal, and failure. A good architecture makes each local borrow temporary and unsurprising because the long lived identity and reclamation decisions have already been settled.

Training Encounters

1. Why does a generation counter solve stale slot reuse but fail to distinguish two independent arenas? Propose one additional identity mechanism and state its cost.

2. A tree uses strong reference counted edges in both directions. What happens when all external handles disappear? Explain how weak parent links change ownership while preserving navigation.

3. A concurrent lookup validates a handle, releases a lock, and later uses a raw pointer to the node. Why is the generation check insufficient? Name two mechanisms that could protect the complete use interval.

Training Debriefs

1. Separate arenas can independently issue equal slot and generation pairs. Adding an owner identifier makes lookups verify the namespace too, at a cost in handle size and owner identifier management. A scoped type brand can provide a static relationship but constrains how handles escape or are serialized.

2. The cycle maintains positive strong counts, so the nodes remain allocated. A weak parent link does not keep the parent alive; children can attempt to upgrade it when navigating and handle absence. Arbitrary child attachment must still preserve the intended acyclic strong ownership structure.

3. Deletion can occur after validation and before use, leaving the pointer dangling. Keep an appropriate lock guard while borrowing the node, acquire independent shared ownership, or use a correctly implemented epoch or hazard based reclamation scheme. The mechanism must protect storage throughout the dereference, not merely certify an earlier identity match.

Further Reading

• The Rust standard library Rc documentation explains shared ownership and weak links within one thread.

• The Rust standard library Arc documentation distinguishes thread safe ownership counting from inner access.

• The Rust standard library collections module documents central ownership alternatives and their operation costs.

The Compaction Mimic

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Build an atlas with dense record storage and a separate slot table. Public handles identify slots and generations, while each live slot records its current dense position. Deleting a record may swap the last dense record into the vacancy. Repair the moved record's slot mapping without changing its public handle. A deleted handle must never resolve to the replacement record after slot reuse. Test deletion from the beginning, deletion of the final record, repeated reuse, and a surviving handle whose dense index changed. Describe how a handle from another atlas is rejected or explicitly outside the supported contract. Decide what happens when a generation cannot advance. Explain why returning a borrowed record after successful lookup still creates an ordinary access relationship that constrains mutation.

Encounter Constraints

• Use safe owned collections and numeric handles rather than retaining raw element pointers.

• Keep both slot-to-dense and dense-to-slot relationships consistent after compaction.

• Retire an exhausted slot or provide another explicit nonwrapping identity protocol.

Hint one / Hint two / Reveal debrief

Side Quest

Export part of an atlas into another atlas while preserving edges among exported records. Build an old-handle-to-new-handle remapping rather than assuming equal slot numbers mean equal records. Decide what an edge to an unexported node becomes. This turns serialization and import into identity translation instead of an accidental copy of process-local coordinates. Record the remapping as an explicit import artifact.

Capability unlocked Atlas Keeper - preserving identity while storage moves

Choose Your Exit

Explore the parser district / Level 37
Compare handle indirection with owned buffers and stored byte ranges.

Study the collection vaults / Level 34
Evaluate the cache and allocation costs of the storage representation.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 09

The Foundry Charter

The Trait Foundry manufactures repair plans from many kinds of sensor input. Its charter currently demands that every sensor, every plan, and every error be printable, sortable, cloneable, and interchangeable. Useful machinery has been rejected for obligations no actual operation needs. Your assignment is to rewrite the charter as a set of precise relationships. A machine should promise its real output type, a policy should name the input it can inspect, and optional reporting should pay for its own formatting requirements.

Mission Connect associated outputs to parameterized policies while placing extra bounds on the algorithms that actually need them.

Suggested loadout Level 1, Level 4. These are guidance, not locked gates.

Mastery Targets

- Express the equality between a planner's output and the policy input accepted by the pipeline.
- Allow a useful non-Display plan to pass through a nonreporting path.
- Explain when an input parameter permits multiple implementations and an associated output selects one relationship.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 9

Traits as Relationships and Obligations

A Trait Is a Promise the Compiler Can Use

Developers often approach a trait as an interface with unfamiliar syntax. That analogy explains method lookup, but it misses the part that shapes advanced Rust: a trait establishes facts about types. A generic function can use those facts without knowing the concrete representation. An implementation supplies evidence that one particular type satisfies the relationship. Associated types, supertraits, and bounds describe the evidence the function may request.

Consider a component that accepts jobs and returns receipts. The important question is not whether it has a method named submit. The important questions are whether each component chooses one receipt type, whether callers may choose the job representation, whether submission mutates internal state, and whether failure preserves the original job. A trait declaration fixes these relationships before there is an implementation.

The distinction matters when a design starts to resist extension. Adding another type parameter may make an individual call compile while allowing combinations the domain cannot support. Adding an associated type may simplify callers while preventing a type from implementing several independent relationships. Trait design is therefore partly an exercise in deciding who chooses each type and when that choice becomes fixed.

Do not begin with a list of methods copied from a struct. Begin with a sentence: for every implementation of this trait, these operations have these relationships. If that sentence cannot be stated clearly, the abstraction may be grouping convenient functions rather than expressing a stable capability.

Inputs Are Chosen by Callers and Outputs by Implementations

A trait type parameter is an input to the implementation relationship. A type can implement the trait for several values of that parameter, provided the implementations do not overlap. An associated type is an output determined by a particular implementation. Once the implementing type and trait parameters are known, the associated type is fixed.

The following complete example uses both choices deliberately:

Listing 9.1 Rust example

```
trait Encode<Input> {
    type Error;

    fn encode(&self, value: Input) -> Result<Vec<u8>, Self::Error>;
}

struct Decimal;

impl Encode<u32> for Decimal {
    type Error = std::convert::Infallible;

    fn encode(&self, value: u32) -> Result<Vec<u8>, Self::Error> {
        Ok(value.to_string().into_bytes())
    }
}

impl Encode<bool> for Decimal {
    type Error = std::convert::Infallible;

    fn encode(&self, value: bool) -> Result<Vec<u8>, Self::Error> {
        Ok(if value { b"1" } else { b"0" }.to_vec())
    }
}

fn main() {
    assert_eq!(Decimal.encode(42_u32).unwrap(), b"42");
    assert_eq!(Decimal.encode(true).unwrap(), b"1");
}
```

The implementation relationship is Decimal encodes u32, or Decimal encodes bool. Each relationship chooses an error type. If Error were a second trait parameter, downstream code could in principle implement several error policies for the same encoder and input. That may be useful

in a conversion framework. It would usually be confusing in a component whose error representation is a property of its implementation.

An associated type can be owned or borrowed. For example, `Iterator::Item` can use a lifetime from its implementing type, as in Level 13's `EveryOther<'data, T>`. A lending interface can use a generic associated type to tie each item to the receiver borrow for that call.

One useful test is to imagine writing two implementations for the same concrete type. If changing the candidate type means choosing a different operation, a trait parameter may fit. If changing it would contradict what the implementation fundamentally returns, an associated type usually communicates the stronger relationship.

Bounds Are Facts Available Inside a Function

A generic function body is checked using its declared bounds. The compiler does not assume that all current callers happen to use types with extra methods. This prevents a distant implementation detail from quietly becoming a requirement of the function.

Listing 9.2 Rust example

```
use std::fmt::Display;

trait Source {
    type Item;
    type Error;

    fn next_item(&mut self) -> Result<Option<Self::Item>, Self::Error>;
}

fn summarize<S>(source: &mut S) -> Result<String, S::Error>
where
    S: Source,
    S::Item: Display,
{
    let mut result = String::new();
    while let Some(item) = source.next_item()? {
        if !result.is_empty() {
            result.push_str(", ");
        }
        result.push_str(&item.to_string());
    }
    Ok(result)
}
```

The bound on S permits next_item. The bound on S::Item permits formatting. These are different obligations. Source does not promise that every item is printable; summarize requests that additional capability because its own algorithm needs it. Keeping the requirement at the narrowest useful layer allows nonprintable items to participate elsewhere.

This way of reading generic code scales well. Treat each where clause as a local fact table. When the compiler reports that a method is unavailable, ask which fact would justify that method call. The answer may be a missing bound, an unwanted requirement introduced by a derive, or an associated type that differs from the one the algorithm expects. Adding Clone reflexively often obscures the actual relationship.

A function that requires Iterator<Item = Record> is stronger and often easier to use than one that requires Iterator plus several conversion traits on its item. Equality constraints express that the algorithm has already chosen the representation. Conversion bounds express that the algorithm accepts a family of representations. Neither is universally preferable; the signature should reflect where normalization belongs.

Following an Associated Type Through a Pipeline

Here is a small pipeline with two distinct abstractions. A producer determines its item type. A consumer chooses an input through a trait parameter. The wiring function proves that the two sides agree.

Listing 9.3 Rust example

```
trait Producer {
    type Item;
    fn produce(&mut self) -> Option<Self::Item>;
}

trait Consumer<T> {
    fn consume(&mut self, item: T);
}

struct Countdown(u8);

impl Producer for Countdown {
    type Item = u8;

    fn produce(&mut self) -> Option<u8> {
        if self.0 == 0 {
            None
```

Listing 9.3 continued

```
        } else {
            self.0 -= 1;
            Some(self.0)
        }
    }
}

#[derive(Default)]
struct Sum(u64);

impl Consumer<u8> for Sum {
    fn consume(&mut self, item: u8) {
        self.0 += u64::from(item);
    }
}

fn drain<P, C>(producer: &mut P, consumer: &mut C)
where
    P: Producer,
    C: Consumer<P::Item>,
{
    while let Some(item) = producer.produce() {
        consumer.consume(item);
    }
}

fn main() {
    let mut producer = Countdown(4);
    let mut sum = Sum::default();
    drain(&mut producer, &mut sum);
    assert_eq!(sum.0, 6);
}
```

The generic function never names u8. It does not need to. Its proof obligation is relational: whatever P produces, C must consume. This is a stronger design tool than collecting capabilities independently. A bound such as C: Consumer<u8> would unnecessarily specialize the wiring. A conversion inside drain would add another policy: which conversion, whether it can fail, and who owns that failure.

The receiver choices also carry meaning. produce takes an exclusive borrow because iteration advances state. consume takes ownership of the item, allowing a consumer to retain it without cloning. drain borrows both components so the caller can inspect them afterward. These ownership choices are part of the abstraction, not implementation noise.

For a fallible pipeline, make the failure ownership policy explicit. If a consumer may reject a value that the caller should retry, returning only an error loses the item. A result shaped like Result<(), (Error, Item)> can preserve it. If the consumer has partially committed work, returning the item may falsely imply retry is safe. A type signature cannot replace a semantic contract, but it can prevent the contract from being accidentally impossible to honor.

Supertraits Add Obligations Without Adding Storage

A declaration such as trait Reliable: Send does not insert a Send field. It requires every implementing type to satisfy Send, making that fact available wherever Reliable is assumed. The same applies to ordinary supertraits such as Display or another domain trait.

Supertraits are appropriate when the extra fact is inseparable from the capability. They are expensive when added for the convenience of one consumer. Requiring Debug on a public trait excludes otherwise valid implementations and commits implementors to another public behavior. Requiring Send excludes thread-confined implementations even in single-threaded applications. Those restrictions may be justified, but they should follow from the abstraction rather than from the first application that uses it.

An associated type bound similarly establishes a general promise. If every Source item must be Clone, declare that relationship on Source. If only one caching adapter clones items, place the bound on that adapter. This distinction keeps the core vocabulary useful while allowing algorithms to state stronger requirements locally.

Trait methods with default implementations run under the same obligations as other generic code. A default method can use the trait's methods and declared bounds. It cannot assume that implementors store data in a particular form. A good default expresses a valid fallback algorithm; an unsuitable default can silently impose performance or consistency behavior that implementors did not expect.

Qualified Paths Resolve Relationships Explicitly

Method syntax is concise until several traits provide the same name. Fully qualified syntax identifies the relationship directly. The same technique makes associated type projections unambiguous in complex signatures.

Listing 9.4 Rust example

```
trait Text {
    fn describe(&self) -> &'static str;
}

trait Wire {
    fn describe(&self) -> &'static str;
}

struct Packet;

impl Text for Packet {
    fn describe(&self) -> &'static str { "human readable" }
}

impl Wire for Packet {
    fn describe(&self) -> &'static str { "binary frame" }
}

fn main() {
    let packet = Packet;
    assert_eq!(<Packet as Wire>::describe(&packet), "binary frame");
    assert_eq!(Text::describe(&packet), "human readable");
}
```

The long form is not merely a workaround for ambiguous names. It documents which abstraction an algorithm relies on. In a signature, <T as Trait>::Item says that Item belongs to that implementation relationship. This becomes useful when a type implements several traits with identically named associated types.

Do not mistake method lookup conveniences for proof that one trait implies another. Deref coercions, autoref, and traits in scope can make a call work on a concrete expression while a generic bound remains insufficient. Diagnose the bound from the trait relationship, then consider lookup rules separately.

Laws Live Beside Signatures

A trait can require Eq and Hash, but the compiler does not prove that equal values produce equal hashes. It can require a method named next_item, but it does not prove that the method eventually terminates. Such behavioral laws belong in documentation and tests. Generic algorithms rely on them even though they are outside ordinary type checking.

Separate safety requirements from ordinary semantic laws. Violating a safe trait's documented sorting convention must not, by itself, allow a sound unsafe implementation to cause undefined behavior. An unsafe trait is the mechanism for obligations whose violation can make safe consumers unsound, and implementing it requires an explicit unsafe act. Most application traits should remain safe and treat dishonest implementations as logic errors.

An effective trait review asks what an adversarial but type-correct implementation could do. Could it return inconsistent lengths? Could it panic? Could it report success without changing state? Could it call a callback more often than expected? If an algorithm depends on stronger behavior, document it, validate it, or redesign the interface so the needed fact is carried by a value rather than assumed.

Training Encounters and Debriefs

Exercise 1. A repository trait has one entity type for each repository implementation, but several repositories store the same entity. Should Entity be a trait parameter or an associated type?

Solution. An associated type communicates one entity choice per implementation. Multiple repository types can choose the same associated entity without conflict. Use a parameter only if one repository type should implement independent repository relationships for several entities. Consider whether such a repository needs distinct state or methods; separate wrapper types may make the design clearer.

Exercise 2. A function consumes a Producer and writes each item using Display. Should Display become a supertrait of Producer?

Solution. No. Display applies to the associated item, not necessarily to the producer, and only this algorithm needs it. Add P::Item: Display to the function. Moving that requirement into Producer would exclude useful producers of binary buffers, opaque handles, and other items that need no textual representation.

Exercise 3. Extend drain so a consumer can fail while returning the rejected item. Explain the retry ambiguity.

Solution. Define a fallible consumer with an associated Error and a return type Result<(), (Self::Error, T)>. The wiring function can stop and return the pair. The signature preserves ownership, but it does not prove that no external effect happened before failure. Document whether an error means no commit, partial commit, or unknown commit. Retry logic depends on that distinction more than on the existence of Result.

Further Reading

- Rust Reference on traits defines trait items, bounds, and supertraits.
- Rust Reference on associated items specifies associated type declarations and qualified paths.
- Standard library Iterator is a substantial example of an associated output type combined with conditional adapter methods.

The Charter of Excessive Demands

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a planner interface that produces one plan type and one error type for each implementation. Design a separate inspection policy that may support several plan types. Then write a generic approve function connecting a planner to a compatible policy. The ordinary function must not require a plan to be Display merely because an optional reporting function prints it. Provide two planner implementations whose plan types differ, and one policy type with independent implementations for both. Show that a mismatched planner-policy pair is rejected. Finally, explain what would change if one planner type genuinely needed to implement several independent planning relationships distinguished by an input kind. Your explanation should locate each choice in the trait relationship rather than attaching convenience bounds indiscriminately to stored data.

Encounter Constraints

• Avoid trait objects in this first implementation so the obligations remain visible in generic signatures.

• Keep formatting bounds on the reporting operation rather than on every planner.

• Do not use Clone to conceal a mismatch in associated output and policy input.

Hint one / Hint two / Reveal debrief

Side Quest

Add a policy rejection that returns the owned plan together with its reason. Explain which component owns a rejected plan and whether the pipeline can submit it again. A type relationship can preserve the value, but it cannot certify that a policy performed no external effects before rejection. Give that behavior a separate contract. The next component must be able to distinguish these cases.

Capability unlocked Charter Writer - placing obligations at their narrowest useful boundary

Choose Your Exit

Enter the hall of masks / Level 10
Preserve a smaller capability after hiding concrete implementation types.

Visit the lending archives / Level 13
Make an associated result depend on each call's borrow lifetime.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 10

The Hall of Useful Masks

The Foundry's tool hall admits repair tools from districts that compile independently. Each tool arrives wearing a different concrete type, yet the control desk needs a shared operation. Previous architects either exposed every implementation detail or erased so much that callers spent their time guessing and downcasting. You must decide what survives the mask: a useful command, a stable outcome, borrowing constraints, and only the extra capabilities the host truly needs. The mask is an interface choice, not an automatic request for heap allocation.

Mission Create a small dyn-compatible plugin boundary without erasing ownership, lifetime, or result semantics.

Suggested loadout Level 4, Level 9. These are guidance, not locked gates.

Mastery Targets

• Store heterogeneous tools behind one deliberately small dynamic interface.

• Retain a tool that borrows host configuration using an explicit registry lifetime.

• Separate concrete construction helpers from operations callable through the erased receiver.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 10

Dispatch and Type Erasure

Choose Where Concrete Types Stop Mattering

A generic function preserves the concrete type of its argument through type checking and, ordinarily, through monomorphization. A trait object deliberately forgets that identity and retains a selected interface. This is an architectural choice about how far implementation details travel through a program. Runtime cost is one consequence, but compile time, binary size, dependency boundaries, and ownership are equally relevant.

Imagine an application that loads several output destinations from configuration. One destination writes to a file, another buffers in memory, and another forwards to an existing service. The application needs to store them together and choose among them at runtime. A generic parameter cannot vary independently for every element of one homogeneous vector. An enum can represent a closed set of alternatives. A trait object can represent an open set whose members satisfy one dyn-compatible contract.

The open set is not dynamic linking by itself. A boxed trait object inside a program does not establish a stable plugin ABI, and Rust trait object layout is not a portable foreign interface. Type erasure solves a language-level representation problem: callers can own or borrow values without carrying their concrete Rust types in every surrounding signature.

Start by asking which operations remain meaningful after identity is forgotten. Construction, cloning into the same concrete type, and arbitrary generic operations often need information that the erased interface does not retain. Designing the erased boundary first prevents later surprises when a seemingly ordinary trait refuses to become dyn Trait.

Static Dispatch Preserves Optimization Opportunities

For a generic function, rustc can generate specialized machine code for the concrete instantiations that are needed. This makes implementation details available to optimization. Calls may inline, constants may propagate, and abstractions may disappear. None of these optimizations is guaranteed by writing a generic signature. Large generic bodies can also multiply generated code and increase compilation work.

Listing 10.1 Rust example

```
trait Metric {
    fn observe(&mut self, value: u64);
}

#[derive(Default)]
struct Total(u64);

impl Metric for Total {
    fn observe(&mut self, value: u64) {
        self.0 += value;
    }
}

fn record<M: Metric>(metric: &mut M, values: &[u64]) {
    for &value in values {
        metric.observe(value);
    }
}

fn main() {
    let mut metric = Total::default();
    record(&mut metric, &[3, 5, 8]);
    assert_eq!(metric.0, 16);
}
```

The loop can be compiled with knowledge of Total. If the abstraction is used in a hot inner loop, that visibility may matter. If the loop spends its time writing to a slow device, specialization may have no measurable effect. Performance reasoning should begin with the work performed per call, the frequency of calls, and the optimizer's ability to see surrounding code.

Argument-position impl Trait offers similar generic behavior with different naming ergonomics. A parameter written as impl Metric accepts one caller-chosen concrete type for that parameter. It does not create a

trait object. Return-position impl Trait instead hides one implementation-chosen concrete type behind declared bounds. Different branches must still agree on that hidden type unless the branches themselves construct a shared representation.

Dynamic Dispatch Retains a Method Contract

A reference to dyn Metric carries access to the underlying value and information needed to dispatch the trait's methods. It is useful to think of this as data plus a method table, while avoiding assumptions about a stable raw layout. Calling a method selects the implementation at runtime. The data may live on the stack, in a Box, or inside another owner; dyn does not itself imply heap allocation.

Listing 10.2 Rust example

```
trait Metric {
    fn observe(&mut self, value: u64);
    fn current(&self) -> u64;
}

struct Maximum(u64);

impl Metric for Maximum {
    fn observe(&mut self, value: u64) {
        self.0 = self.0.max(value);
    }

    fn current(&self) -> u64 { self.0 }
}

fn record(metric: &mut dyn Metric, values: &[u64]) {
    for &value in values {
        metric.observe(value);
    }
}

fn main() {
    let mut maximum = Maximum(0);
    record(&mut maximum, &[3, 5, 8]);
    let view: &dyn Metric = &maximum;
    assert_eq!(view.current(), 8);
}
```

This example performs no allocation for the trait object. The maximum remains a local variable. The reference erases its concrete type for the

duration of the call. Ownership stays with the caller, and the borrow checker continues to enforce access restrictions across the erased boundary.

Box<dyn Metric> adds owned indirection and usually an allocation for a nonzero-sized concrete value. It lets a container store heterogeneous implementations with one known outer size. The box owns its concrete allocation and invokes the appropriate destructor when dropped. Type erasure therefore need not erase destruction behavior, although it hides the concrete destructor from ordinary source-level callers.

Dyn Compatibility Follows from What a Caller Knows

A trait object caller knows the erased interface and any specified associated types. It does not know the concrete Self. A dispatchable method cannot require the caller to manufacture an arbitrary concrete Self or select from infinitely many type-specialized method instances. These constraints explain many dyn compatibility rules more usefully than memorizing an error list.

A method returning Self needs a concrete result layout. A method with a type parameter would require generic dispatch that ordinary trait object method tables do not provide. An associated constant is not an instance method selected through a receiver. An async method or a method returning impl Trait introduces an opaque return representation that the ordinary erased contract does not specify.

Some methods can remain on the trait while being excluded from the object interface with Self: Sized. The trait can then provide both an erased operational surface and concrete-only conveniences.

Listing 10.3 Rust example

```
trait Destination {
    fn write(&mut self, bytes: &[u8]);

    fn empty() -> Self
    where
        Self: Sized;

    fn write_value<T: AsRef<[u8]>>(&mut self, value: T)
    where
        Self: Sized,
    {
```

Listing 10.3 continued

```
        self.write(value.as_ref());
    }
}

struct Buffer(Vec<u8>);

impl Destination for Buffer {
    fn write(&mut self, bytes: &[u8]) {
        self.0.extend_from_slice(bytes);
    }

    fn empty() -> Self { Self(Vec::new()) }
}

fn main() {
    let mut buffer = Buffer::empty();
    buffer.write_value("hello");
    let destination: &mut dyn Destination = &mut buffer;
    destination.write(b" world");
    assert_eq!(buffer.0, b"hello world");
}
```

The erased interface includes write. It excludes empty and write_value. This is a practical design when generic convenience can delegate to a small, stable core operation. It also keeps the runtime interface understandable: callers passing through dyn pay for one operation that they could already express without knowing Self.

Associated types are compatible with many trait objects when their values are specified. A dyn Iterator<Item = u8> hides the iterator implementation while preserving the item representation. An unspecified associated type would leave the caller unable to know what next returns. Generic associated types impose additional restrictions on dyn compatibility; do not assume that replacing one associated type with a family of types preserves an existing trait object API.

A Complete Heterogeneous Routing Boundary

Suppose a router accepts a sequence of sinks selected from configuration. Each sink must receive the same byte message. The router should not become generic over the number or concrete types of sinks. Its boundary can own erased sinks while exposing concrete implementations for testing.

Listing 10.4 Rust example

```
trait Sink {
    fn send(&mut self, message: &[u8]) -> Result<(), String>;
}

struct MemorySink {
    messages: Vec<Vec<u8>>,
}

impl Sink for MemorySink {
    fn send(&mut self, message: &[u8]) -> Result<(), String> {
        self.messages.push(message.to_vec());
        Ok(())
    }
}

struct LimitedSink {
    remaining: usize,
}

impl Sink for LimitedSink {
    fn send(&mut self, message: &[u8]) -> Result<(), String> {
        if message.len() > self.remaining {
            return Err("capacity exhausted".into());
        }
        self.remaining -= message.len();
        Ok(())
    }
}

struct Router<'a> {
    sinks: Vec<Box<dyn Sink + 'a>>,
}

impl<'a> Router<'a> {
    fn new(sinks: Vec<Box<dyn Sink + 'a>>) -> Self {
        Self { sinks }
    }

    fn broadcast(&mut self, message: &[u8]) -> Vec<String> {
        let mut failures = Vec::new();
        for sink in &mut self.sinks {
            if let Err(error) = sink.send(message) {
                failures.push(error);
            }
        }
        failures
    }
}
```

Listing 10.4 continued

```
fn main() {
    let memory = MemorySink { messages: Vec::new() };
    let limited = LimitedSink { remaining: 3 };
    let mut router = Router::new(vec![
        Box::new(memory),
        Box::new(limited),
    ]);
    assert!(router.broadcast(b"abc").is_empty());
    assert_eq!(router.broadcast(b"x").len(), 1);
}
```

The lifetime parameter is deliberate. Box<dyn Sink> in an ordinary type position commonly implies a static object lifetime bound. That does not mean the box lives forever; it means the hidden value cannot contain nonstatic borrows. Router<'a> permits sink implementations that borrow configuration or an external buffer for at least 'a. The router cannot outlive those borrows.

The failure policy is also visible. Broadcasting continues after a failure and returns all failure strings. A fail-fast policy would produce different external effects because later sinks would not be called. Dynamic dispatch does not determine this behavior. Good abstraction design keeps dispatch choice separate from consistency and recovery policy.

Erasure Can Happen Once Rather Than Everywhere

A large generic pipeline may propagate dozens of concrete adapter types into high-level code. Erasing at a coarse boundary can reduce the amount of code that depends on those types. Conversely, erasing each tiny operation can introduce frequent indirect calls and lose opportunities to combine work. The useful boundary is often a batch operation: process a buffer, execute a request, or produce a block of results.

An enum is often preferable when the implementation set is intentionally closed. It permits direct matching, carries variant-specific data, and can make exhaustive handling part of the design. A trait object is attractive when independently written implementations should participate without editing the central enum. These choices express different extension models, not simply fast versus slow.

Type erasure also affects discoverability. Once a value is dyn Sink, methods unique to MemorySink are unavailable. Downcasting can recover concrete identity when an Any-based interface permits it, but routine downcasting suggests the erased interface may not match what callers actually need. Before adding a downcast, ask whether the operation belongs on the trait, whether the caller should retain a separate typed handle, or whether the set should be an enum.

Ownership and Thread Bounds Survive Erasure

A Box<dyn Sink + Send> promises that the hidden value can be transferred between threads. A Box<dyn Sink> makes no such promise. Erasure does not automatically retain every auto trait of the original concrete value. Include the capabilities that the boundary requires in the object type.

Similarly, Arc<dyn Sink + Send + Sync> can share ownership, but the methods still determine access. A send method taking &mut self cannot be called through an ordinary shared Arc without a synchronization or ownership strategy. Wrapping a value in Arc does not make exclusive mutation available. The receiver contract remains in force after both reference counting and type erasure.

When comparing implementations, measure the complete design. A boxed sink may allocate once during startup, making allocation irrelevant to steady-state throughput. An enum may increase every element's size to fit its largest variant, affecting cache behavior. A generic pipeline may create enough specialized code to pressure instruction caches. Dispatch is one variable in a representation problem.

Training Encounters and Debriefs

Exercise 1. A trait has fn duplicate(&self) -> Self. A library wants to store Box<dyn Trait>. What design choices are available?

Solution. Exclude duplicate from dynamic dispatch with a Self: Sized bound, or provide an erased operation returning Box<dyn Trait> with an explicit ownership and lifetime policy. Another choice is a separate cloning trait designed for erased values. Replacing Self with a box changes

semantics and allocation behavior, so it should be a deliberate API decision.

Exercise 2. Does changing a parameter from &mut T where T: Sink to &mut dyn Sink require heap allocation?

Solution. No. The reference can point to a concrete local value, as in the Maximum example. Allocation depends on the owner and storage representation. The change affects dispatch and the information available to optimization, while borrowing and the underlying value's location can remain the same.

Exercise 3. A heterogeneous registry repeatedly downcasts every object to one of four known types. Should it keep using trait objects?

Solution. A closed enum is likely clearer if all valid cases are centrally known and variant-specific behavior is essential. Trait objects still fit if external extensions matter and downcasting is occasional optional behavior. The deciding question is whether the primary contract is a shared capability or a closed classification.

Further Reading

• Rust Reference on dyn compatibility specifies which trait items participate in trait objects.

• Rust Reference on trait objects describes erased trait types and object lifetime bounds.

• Standard library dyn keyword introduces dispatch and representation tradeoffs.

The Shapeshifter Registry

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a registry of repair tools. Every tool performs one command against an in-memory work context and returns a shared outcome enum, but one tool owns its settings while another borrows immutable host configuration. The registry must hold both without claiming the borrowed configuration is static. A proposed trait also contains a generic configure method and a method returning Self. Decide which operations belong in the erased interface, which can be excluded from dynamic dispatch with a suitable bound, and which belong in a separate construction API. Demonstrate a borrowed dyn call without allocating a tool. Then explain which allocations come from the chosen registry storage and why a boxed trait object alone does not provide a stable foreign plugin ABI.

Encounter Constraints

- Do not downcast every tool to rediscover a closed set of concrete implementations.
- Do not add Send or Sync unless the registry's actual execution contract requires them.
- Keep the configuration owner alive for the registry's full borrowed use.

Hint one / Hint two / Reveal debrief

Side Quest

Replace the registry with an enum when the tool set is deliberately closed and every new tool requires editing the host. Compare exhaustive matching, inlining opportunities, variant size, and extension policy. Choose based on the expected evolution of the system, then document why repeated downcasting was evidence that the original boundary may have been misplaced. State the chosen extension policy in writing.

Capability unlocked Mask Maker - hiding concrete types while preserving useful promises

Choose Your Exit

Inspect occupied implementation space / Level 11
Discover how generic implementation choices constrain future adapters.

Inspect async tool contracts / Level 27
Extend erasure and trait-bound decisions to returned futures.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 11

The Anvil That Already Belongs to Someone

At the Trait Foundry's arithmetic anvil, a vector designer wants the same notation in both directions: vector times scalar and scalar times vector. The right to add an implementation turns out to depend on more than mathematical symmetry. Some traits and types belong to other crates, and an uncovered type parameter can appear before the first local type. Your mission is to find a legal public interface that respects those ownership boundaries without reaching for an overlapping blanket rule that blocks future work.

Mission Apply ordered orphan-rule reasoning and choose a legal adapter for symmetric generic arithmetic.

Suggested loadout Level 9, Level 10. These are guidance, not locked gates.

Mastery Targets

• Explain why a local vector self type and a generic scalar self type face different orphan-rule conditions.

• Provide a legal local wrapper or explicit supported-scalar strategy for the reversed operation.

• Distinguish permission under orphan rules from freedom from overlapping implementations.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 11

Generic Bounds Coherence and Implementation Space

Generic Code Is Checked for Every Permitted Type

The hardest generic errors often arise because a programmer reasons from examples while the compiler reasons from a set. If a function accepts every T satisfying a bound, its body must work for every such T. If two implementations overlap for any permitted instantiation, the compiler cannot select a unique relationship merely because today's application never constructs that instantiation.

This set-based view explains why a broad implementation can block a seemingly reasonable specialized one. It also explains why bounds belong in public API review. A bound narrows the set of accepted programs. An implementation occupies part of the set of trait relationships. Publishing either choice can constrain future extensions across several crates.

Rust calls the requirement for a consistent implementation choice coherence. Coherence is supported by overlap checks and orphan rules. These rules make trait resolution usable in a dependency graph whose crates evolve independently. They are not a statement that one implementation would be impossible to execute; they are restrictions that keep separately compiled reasoning consistent.

When a generic design becomes difficult, draw its sets. Which types may enter? Which implementations apply? Which combinations are intentionally unavailable? This often reveals that the abstraction needs another wrapper type or another trait, rather than another where clause.

Put Bounds Where Their Operations Are Needed

A generic struct can often store a T without imposing any capability on T. Its methods then introduce bounds independently. This preserves the widest useful representation while making each operation state its own requirements.

Listing 11.1 Rust example

```
use std::fmt::Display;

struct Slot<T> {
    value: Option<T>,
}

impl<T> Slot<T> {
    fn empty() -> Self { Self { value: None } }

    fn insert(&mut self, value: T) -> Option<T> {
        self.value.replace(value)
    }

    fn take(&mut self) -> Option<T> {
        self.value.take()
    }
}

impl<T: Display> Slot<T> {
    fn describe(&self) -> String {
        match &self.value {
            Some(value) => value.to_string(),
            None => "empty".into(),
        }
    }
}

fn main() {
    let mut slot = Slot::empty();
    slot.insert(42);
    assert_eq!(slot.describe(), "42");
    assert_eq!(slot.take(), Some(42));
}
```

Placing Display on Slot itself would make even empty storage require displayability. That is an unnecessary restriction. Some bounds do belong on a type declaration, especially when its well-formedness depends on an associated type or when the type's invariant fundamentally requires a capability. The aim is precision, not avoiding bounds mechanically.

Derive macros can also introduce bounds broader than a hand-written implementation needs. A wrapper containing Arc<T> can clone the Arc without cloning T, yet a derived implementation on a generic shape may impose a T: Clone requirement depending on the derive and fields. Inspect the generated obligations when a bound seems surprising. A short manual

implementation can be more accurate than adding an unrelated trait bound to callers.

Overlap Is About Possible Instantiations

Consider a trait with a fallback for every displayable type. A separate implementation for String is redundant because String already satisfies Display. The compiler rejects the overlap even if the method bodies happen to agree.

Listing 11.2 Rejected example

```
use std::fmt::Display;

trait Label {
    fn label(&self) -> String;
}

impl<T: Display> Label for T {
    fn label(&self) -> String { self.to_string() }
}

impl Label for String {
    fn label(&self) -> String { self.clone() }
}
```

Stable Rust does not offer a general-purpose mechanism for overriding this blanket implementation with a more specific one. Designing an API around an anticipated specialization feature makes the current public contract difficult to implement. Instead, make the policy a type parameter, provide an explicit wrapper, or separate the fallback into a method that callers choose.

A common mistaken repair is to add another marker trait that String does not currently implement. The overlap question can still involve what upstream crates are allowed to implement later. The compiler must preserve consistency under legal evolution, not merely inspect a snapshot of today's impl blocks. Locality and trait ownership influence which future implementations are possible.

This is why coherence errors sometimes mention upstream crates. The compiler is explaining that an apparent absence is not a stable negative fact. A trait bound says that an implementation exists. Ordinary stable

bounds generally do not let a library claim that some arbitrary foreign type will never implement a foreign trait.

A Newtype Creates a New Implementation Identity

A wrapper is more than syntactic ceremony. It creates a local type whose trait implementations can express a policy that should not apply globally to the underlying representation. It also gives documentation a place to explain the policy.

Listing 11.3 Rust example

```
use std::fmt::{self, Display};

struct Redacted<T>(T);

impl<T> Display for Redacted<T> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.write_str("[redacted]")
    }
}

struct CommaSeparated<'a>(&'a [u32]);

impl Display for CommaSeparated<'_> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        for (index, value) in self.0.iter().enumerate() {
            if index != 0 {
                f.write_str(",")?;
            }
            write!(f, "{value}")?;
        }
        Ok(())
    }
}

fn main() {
    let secret = Redacted("access token");
    assert_eq!(secret.to_string(), "[redacted]");
    assert_eq!(CommaSeparated(&[2, 3, 5]).to_string(), "2,3,5");
}
```

The wrapper does not require allocating or copying the underlying data. CommaSeparated stores a borrow. Its formatting policy is explicit at the use site and cannot conflict with another library's preferred display of slices. Redacted similarly chooses one presentation policy without changing String or every value of T.

Avoid implementing Deref automatically for every newtype. Deref makes the wrapper participate in method resolution and exposes much of the underlying API. That may be appropriate for a smart pointer, but it can undermine a policy wrapper whose purpose is to restrict operations. A validated identifier wrapper should not casually expose mutable string operations that can violate its invariant.

The Orphan Rules Establish Who Can Add Relationships

At a high level, a crate can implement its own trait for types, or implement a foreign trait when a sufficiently local type appears in the implementation relationship. The detailed rule also tracks uncovered generic parameters before the first local type, with special treatment for fundamental types. The simple slogan about owning either the trait or type is useful, but it is not the complete rule for generic trait parameters.

The practical consequence is that a crate cannot generally implement Display directly for Vec<u32>. Both the trait and the type constructor are foreign. A local element type inside a foreign container does not automatically make every such implementation local. A newtype around the container establishes a clear local identity.

Listing 11.4 Rejected example

```
use std::fmt::{self, Display};

impl Display for Vec<u32> {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{} values", self.len())
    }
}
```

Imagine if two application dependencies could both add that implementation. The final program would receive incompatible global formatting definitions merely by depending on both crates. Orphan rules prevent that conflict at the implementation site rather than leaving it for arbitrary downstream consumers to resolve.

Trait ownership is therefore an extension mechanism. If you own a trait, you can decide which foreign types implement your capability, subject to overlap rules. If you own a type, you can integrate it with many foreign

traits. If you own neither, an adapter type is often the appropriate integration point.

Blanket Implementations Spend Future Design Space

A blanket implementation such as impl<T: Display> Label for T is convenient because many types gain the capability automatically. It also occupies every matching relationship. Downstream crates cannot give their displayable local types a different Label implementation without overlap. A library should treat this as a substantial policy decision.

Extension traits commonly use blanket implementations safely because the provided methods are derived conveniences with one obvious meaning. For example, a trait can add a helper to every iterator satisfying particular bounds. A domain trait whose behavior varies by application is more likely to suffer from a universal fallback.

Before publishing a blanket implementation, ask whether a user could reasonably want a different implementation for a type covered by it. If the answer is yes, consider an explicit adapter. An adapter moves the policy choice into the type and leaves the original implementation space available. Another option is to split a broad trait into a minimal capability and a derived convenience trait.

Negative reasoning is especially dangerous in public designs. The fact that a type lacks Display today does not necessarily create an isolated region for a special implementation tomorrow. Favor positive, locally controlled distinctions: a wrapper type, an enum, a policy parameter, or a private marker implemented only by the library.

A Policy Parameter Can Replace Specialization

Here is a complete design where callers choose formatting policy explicitly. There is no overlap because the policy is part of the trait relationship.

Listing 11.5 Rust example

```
struct Compact;
struct Verbose;

trait Render<Policy> {
    fn render(&self) -> String;
}
```

Listing 11.5 continued

```
struct Task {
    id: u64,
    title: String,
}

impl Render<Compact> for Task {
    fn render(&self) -> String {
        self.id.to_string()
    }
}

impl Render<Verbose> for Task {
    fn render(&self) -> String {
        format!("task {} {}", self.id, self.title)
    }
}

fn render<T, P>(value: &T) -> String
where
    T: Render<P>,
{
    <T as Render<P>>::render(value)
}

fn main() {
    let task = Task { id: 7, title: "rebuild index".into() };
    assert_eq!(render::<_, Compact>(&task), "7");
    assert_eq!(render::<_, Verbose>(&task), "task 7 rebuild index");
}
```

The policy types carry no runtime data here. They distinguish compile-time relationships. The verbose implementation is not more specific than the compact implementation; it is a different relationship. This clarity is often preferable to hidden specialization because callers can see which behavior they requested.

There is a cost. Generic signatures acquire another parameter, type inference may need help, and too many policy axes can make an API cumbersome. If policies are selected at runtime, an enum or a policy object is more suitable. Type-level policy should correspond to a useful compile-time distinction, not merely relocate every configuration flag into angle brackets.

Generic Boundaries Affect Compilation Architecture

Large generic functions expose their bodies to downstream instantiation and can increase compile work for every concrete combination. One useful pattern is a small generic entry point that normalizes input and delegates to a nongeneric core. For example, a function accepting AsRef<Path> can immediately obtain &Path and call a substantial internal implementation that accepts that one representation.

This pattern is not a requirement to erase all types. It separates flexible input ergonomics from a large algorithm that gains no benefit from multiple instantiations. Measure before restructuring performance-sensitive code, but recognize that genericity has a maintenance and compilation dimension as well as a runtime dimension.

Bounds also reveal accidental coupling. If a storage abstraction needs Serialize, Debug, Send, Sync, Clone, Default, and a database trait on every T, examine which operations require each fact. Several independent capabilities may have been fused into one component. Splitting those operations can reduce both conceptual and compiler complexity while making tests easier to construct.

Training Encounters and Debriefs

Exercise 1. A crate publishes impl<T: Display> AuditName for T. A downstream crate wants a custom AuditName for its own displayable User type. Why is local ownership insufficient?

Solution. The existing blanket implementation already applies to User. Local ownership allows an implementation under the orphan rules but does not exempt it from overlap checks. The upstream crate has occupied that relationship. A wrapper around User or a policy parameter can create a distinct relationship.

Exercise 2. A struct stores a Vec<T>, and only one method sorts it. Where should T: Ord appear?

Solution. Put Ord on the sorting method or its impl block unless the type's invariant requires permanent ordering and construction itself depends on Ord. Mere storage, removal, and iteration do not require ordering. A narrow bound allows the same representation to serve more types.

Exercise 3. A library wants every displayable value to gain a convenience method to_string_with_prefix, but users may want domain-specific labels. Should one trait serve both purposes?

Solution. Separate them. An extension trait with a blanket implementation can provide the mechanically derived formatting helper. A domain Label trait should leave room for meaningful implementations or use explicit wrappers. Keeping the capabilities distinct prevents a convenience implementation from consuming domain policy space.

Further Reading

• Rust Reference on implementations defines overlap, orphan rules, and covered parameters.

• Rust Reference on trait bounds explains where generic obligations may be declared.

• Cargo SemVer compatibility discusses compatibility consequences of public bounds and implementations.

The Commutative Gate

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Your crate defines Vector<T> and wants multiplication by T in both orders using the standard Mul trait. Compare a generic implementation with Vector<T> as Self to one with unconstrained T as Self and Vector<T> as the right-hand operand. Explain why mentioning a local type somewhere is not enough to make the latter implementation legal. Design two alternatives: a local Scalar<T> wrapper for reversed multiplication, and explicit implementations for a deliberately supported set of primitive scalars. Describe the cost each places on callers and maintenance. Then audit whether adding a broad convenience implementation could collide with either design. You do not need to implement a full numeric library; one small coordinate type and focused compiler probes are enough to establish the implementation-space argument.

Encounter Constraints

• Use stable trait rules rather than assuming general specialization is available.

• Do not change standard-library types or claim mathematical commutativity overrides coherence.

• Keep the concrete arithmetic bounds separate from the ownership and overlap argument.

Hint one / Hint two / Reveal debrief

Side Quest

Decide whether Scalar<T> should implement Deref, Copy, or formatting traits automatically. For each choice, identify a real caller need and any compatibility promise it introduces. Then design a named multiply function that accepts an explicit policy type, allowing domain-specific behavior without relying on an increasingly broad operator implementation. Compare the resulting caller code with the operator form in one example.

Capability unlocked Anvil Rights - finding legal extension points without consuming all future space

Choose Your Exit

Audit public promises / Level 16
Treat implementation space and auto traits as compatibility commitments.

Build the release forge / Level 41
Carry legal generic design into an actual crate support matrix.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 12

The Planner With a Memory

A Foundry planner may be invoked again when its first proposal conflicts with a newer city plan. Its callback owns expensive settings and keeps a counter of attempted decisions. One mechanic demands Fn because retries should be safe; another demands FnOnce because the settings moved into the closure. Both have confused a calling capability with a domain promise. Your mission is to expose exactly how the callback uses its environment and to keep repeated decision-making separate from irreversible effects.

Mission Choose closure bounds from invocation behavior and specify retry semantics independently of those bounds.

Suggested loadout Level 1, Level 4, Level 9. These are guidance, not locked gates.

Mastery Targets

- Accept a mutable callback environment when the planner may invoke it repeatedly.
- Explain why owning captured settings does not by itself make invocation consume them.
- Separate a returned effect description from executing that effect during speculative retries.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 12

Closures as Owned Environments

A Callback Is Code Plus State

A closure is easiest to understand as an unnamed struct containing captured state, together with an implementation of one or more call traits. This model explains ownership transfer, thread bounds, lifetime errors, and why two syntactically identical closures still have different types. The compiler constructs the environment type; ordinary Rust rules then apply to its fields.

The environment may own a String, borrow a configuration value, or hold an exclusive borrow of a counter. Capturing a value and consuming it when called are separate questions. The move keyword determines how captures enter the environment. The body determines what kind of access is needed when the closure is invoked.

This separation matters when designing callback APIs. A callback invoked once can accept more closures than one invoked repeatedly. A callback stored indefinitely needs a different lifetime contract from one used only during a call. A callback executed on another thread needs the environment to satisfy the relevant transfer requirements. These are not incidental compiler demands; they describe what the API intends to do with somebody else's state.

Before choosing a callback bound, write its execution contract in ordinary language. How many times may it run? Can it mutate its own state? Does it escape the current call? On which thread can it execute? Can it return a borrow into its input? Each answer has a corresponding effect on the signature.

The Call Traits Describe Access to the Environment

FnOnce consumes self when called, permitting a body to move a non-Copy capture out of its environment. FnMut calls through an exclusive borrow and permits mutation of captures without consuming the closure. Fn calls through a shared borrow. Every closure implements FnOnce; closures that support the stronger access patterns also implement the corresponding traits.

The names describe the permitted receiver operation, not a runtime call counter. An FnOnce bound means the API needs permission for one consuming invocation. A particular closure satisfying it may also be callable many times if it implements FnMut or Fn. Conversely, a closure can be invoked once by application logic even if it implements Fn.

Listing 12.1 Rust example

```
fn use_once<F, T>(operation: F) -> T
where
    F: FnOnce() -> T,
{
    operation()
}

fn use_twice<F>(mut operation: F) -> (usize, usize)
where
    F: FnMut() -> usize,
{
    (operation(), operation())
}

fn main() {
    let message = String::from("owned result");
    let result = use_once(move || message);
    assert_eq!(result, "owned result");

    let mut count = 0;
    let values = use_twice(|| {
        count += 1;
        count
    });
    assert_eq!(values, (1, 2));
    assert_eq!(count, 2);
}
```

The first closure owns message and moves it out. After invocation, no complete environment remains to call again. The second closure borrows

count exclusively and updates it. Once use_twice returns, its closure is gone, so the exclusive borrow ends and main can read count.

Choosing Fn instead of FnMut for a repeatedly invoked callback is a real restriction. It excludes ordinary stateful closures even when exclusive access would be available. Choosing FnMut instead of FnOnce for a one-shot operation excludes callbacks that transfer captured resources. Use the weakest call trait that satisfies the API's behavior.

Move Capture Does Not Imply One Shot Invocation

A move closure owns its captures, but it can inspect them through shared access. Such a closure may implement Fn. This is common when configuration must travel with a worker while remaining reusable.

Listing 12.2 Rust example

```
fn main() {
    let prefix = String::from("job");
    let format_name = move |id: u64| format!("{prefix}-{id}");

    assert_eq!(format_name(3), "job-3");
    assert_eq!(format_name(4), "job-4");
}
```

The closure owns prefix because of move. Calling format! borrows the captured string for formatting rather than moving it out. The environment remains usable. If the body instead returned prefix by value, it would require FnOnce.

Move also does not turn borrowed data into owned data. If a captured variable is a reference, moving that variable copies or moves the reference into the closure. The referent's lifetime still limits how long the closure may be stored. To cross a boundary requiring static captured data, own the underlying resource or use a suitable shared owner; adding move alone cannot lengthen a borrow.

This distinction explains many thread-spawning errors. A thread may outlive the current stack frame, so a callback borrowing local data cannot be accepted by an unscoped spawn API. A scoped thread can express a shorter execution region and permit appropriate borrows. The lifetime contract of the executor determines the requirement; closures do not bypass it.

Captures Are Often More Precise Than Whole Variables

In modern editions, closure capture can track individual fields rather than always capturing an entire struct. This can allow one field to be moved into a closure while another remains usable. Capture precision has detailed rules involving dereferencing, packed representations, and other cases, so avoid building an unsafe argument on an oversimplified whole-variable model.

Listing 12.3 Rust example

```
struct Work {
    payload: String,
    retries: u32,
}

fn main() {
    let work = Work {
        payload: "rebuild".into(),
        retries: 3,
    };

    let consume_payload = move || work.payload;
    assert_eq!(work.retries, 3);
    assert_eq!(consume_payload(), "rebuild");
}
```

The closure moves payload. The remaining Copy field is still accessible. This precision can affect drop timing because captured fields and uncaptured fields may now belong to different owners. If an invariant depends on coordinated destruction of fields, encode that coordination in a type with appropriate methods and Drop behavior rather than relying on an informal expectation that a closure captures everything together.

Capture precision also affects auto traits. A struct might contain both a thread-safe field and a thread-confined field. Capturing only the thread-safe field can produce a transferable closure. A seemingly harmless edit that references the other field can change the closure's Send status. When a callback boundary fails to compile after a refactor, inspect the actual captured places, not just the original variable's name.

Callback Lifetimes Describe Whether State Escapes

A function that immediately invokes a callback can accept borrowed environments with short lifetimes. A struct storing a callback must express how long the captured borrows remain valid. A trait object introduces another explicit lifetime decision.

Listing 12.4 Rust example

```
struct Predicate<'a> {
    test: Box<dyn Fn(&str) -> bool + 'a>,
}

impl<'a> Predicate<'a> {
    fn new<F>(test: F) -> Self
    where
        F: Fn(&str) -> bool + 'a,
    {
        Self { test: Box::new(test) }
    }

    fn accepts(&self, value: &str) -> bool {
        (self.test)(value)
    }
}

fn main() {
    let prefix = String::from("build/");
    let predicate = Predicate::new(|value| value.starts_with(&prefix));
    assert!(predicate.accepts("build/linux"));
    assert!(!predicate.accepts("test/linux"));
}
```

The object lifetime 'a constrains the environment's captures. It does not require every input string to live for 'a. The input lifetime is independently chosen for each invocation. Confusing these two lifetimes can produce an API that unnecessarily ties temporary input data to the storage duration of the callback.

Higher-ranked bounds make per-call borrowing explicit when needed. A bound such as for<'a> Fn(&'a Record) -> &'a str requires the callback to preserve the relationship between each input borrow and its result. It cannot return an arbitrary short-lived reference, and it cannot insist on only one particular input lifetime. These contracts are useful for selectors and visitors, though closure inference sometimes benefits from using a named function when the relationship is complex.

A Complete Deferred Commit Queue

A queue of deferred actions demonstrates why FnOnce is a resource-management abstraction. Each action may own a value that will be consumed exactly when the action executes. The queue owns the environment until then.

Listing 12.5 Rust example

```
struct CommitQueue<'a> {
    actions: Vec<Box<dyn FnOnce() + 'a>>,
}

impl<'a> CommitQueue<'a> {
    fn new() -> Self { Self { actions: Vec::new() } }

    fn push<F>(&mut self, action: F)
    where
        F: FnOnce() + 'a,
    {
        self.actions.push(Box::new(action));
    }

    fn run(self) {
        for action in self.actions {
            action();
        }
    }
}

fn main() {
    let mut output = Vec::new();
    {
        let mut queue = CommitQueue::new();
        let message = String::from("committed");
        queue.push(|| output.push(message));
        queue.run();
    }
    assert_eq!(output, [String::from("committed")]);
}
```

The callback captures an exclusive borrow of output and owns message. The queue's lifetime cannot exceed that borrow. run consumes the queue and then consumes each boxed FnOnce. The type prevents running the same action again through this owner. If the queue is dropped without running, the captured values are dropped instead; owning a closure does not imply that its body executes during destruction.

The name CommitQueue is intentionally stronger than the implementation's guarantees. In a production transaction system, merely executing closures sequentially would not provide atomic commit. If an action panics, later actions may never run. If an action performs an external effect and then fails, there is no automatic rollback. This example implements deferred execution with one-shot ownership, not transactional durability. Names and documentation must make that boundary clear in real code.

Changing the queue to FnMut would complicate the model. The queue would need to preserve environments after calls and define repeated execution. Changing it to Fn would reject actions that move resources into their final destination. The call trait is therefore part of the queue's semantics.

Function Pointers Have No Captured Environment

A function pointer such as fn(u32) -> u32 identifies code without captured state. Noncapturing, non-async closures can coerce to matching Rust ABI function pointers. For a C ABI pointer, declare a function such as `extern "C" fn notify(_: i32) {}`. Capturing closures need an environment, which can be generic, boxed, or passed through a context pointer.

A generic callback parameter can accept function pointers and closures while preserving their concrete types. A boxed dyn Fn can hold heterogeneous environments at runtime. A function pointer can be copied cheaply and has a straightforward code-only model. Select among these based on storage and extension needs, rather than treating them as interchangeable syntax.

Foreign callback interfaces frequently separate a function pointer from an opaque context pointer. That pair manually recreates code plus environment. Rust must then ensure the context stays alive, the pointer has the correct type and alignment, calls obey synchronization requirements, and registration ends before destruction. Level 33 examines that boundary in more detail.

Reentrancy Is Separate From Repetition

Fn permits invocation through shared access, but it does not promise that calling the closure recursively or concurrently is semantically sensible. Captured interior mutability may reject reentrant borrows. A Mutex may deadlock if a callback calls back into an operation while a lock is held. The call trait expresses access to the closure environment; it does not establish a complete scheduling contract.

When invoking user callbacks, release internal locks and temporary borrows first whenever the algorithm permits it. Copy or move the event data into a stable representation, then call outward. If the callback can mutate the registry that owns it, define whether additions take effect during the current dispatch or the next one. These behavioral details matter as much as whether the bound says Fn or FnMut.

An event dispatcher that stores FnMut callbacks cannot ordinarily hold an exclusive borrow of one callback and simultaneously permit arbitrary mutable access to the whole dispatcher. A staged design can remove a callback temporarily, invoke it, and reinsert it, but then panic and reentrancy policies need careful treatment. Often a command queue returned by callbacks makes ownership much simpler.

Training Encounters and Debriefs

Exercise 1. A move closure captures a Vec and only reads its length. Must an API require FnOnce to accept it?

Solution. No. Owning the vector does not consume it during invocation. The closure can implement Fn because len requires only shared access. An FnOnce API can still accept it, but that is because Fn closures also support a consuming call.

Exercise 2. Why can adding move fail to satisfy a static callback requirement when the closure captures &Config?

Solution. The environment owns the reference value, not Config. The reference still carries its original lifetime. Store owned configuration, use an owner such as Arc when appropriate, or choose an API whose execution lifetime is bounded by the borrow. A keyword cannot extend the referent's lifetime.

Exercise 3. A one-shot callback must run even if its owner is dropped. Is Box<dyn FnOnce()> sufficient?

Solution. No. Dropping the box drops captures without invoking the body. A separate guard type could run an action in Drop, but it must define panic behavior and whether the action can be cancelled. If the action must perform fallible or asynchronous work, an explicit completion operation is usually more appropriate than relying solely on destruction.

Further Reading

- Rust Reference on closure types specifies capture modes, capture precision, and call traits.
- Standard library FnOnce documents the consuming call relationship.
- Rust Edition Guide on disjoint captures explains the edition change that permits field-level capture.

The Retrying Oracle

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a bounded retry helper for an in-memory versioned planning model. On each attempt it passes the current snapshot to a callback and receives an owned proposal. The callback may update an attempt counter and may own a large configuration value that it only reads. The helper can invoke it more than once, but must stop after a stated attempt limit. Choose the least restrictive appropriate closure bound. Then contrast a callback that consumes a captured one-shot resource during its first invocation. Explain why it cannot satisfy the same retry contract without changing the protocol. Finally, replace a callback that sends a notification immediately with one that returns an effect description to execute after acceptance. State why even Fn would not have proved purity or idempotence.

Encounter Constraints

- Do not clone a consumed closure environment to manufacture another attempt.

• Keep retries bounded and report exhaustion separately from proposal rejection.

• Do not claim that Fn or FnMut certifies the absence of externally visible effects.

Hint one / Hint two / Reveal debrief

Side Quest

Add a per-attempt budget token supplied by the helper rather than captured once by the callback. Decide whether each failed attempt consumes a token and how exhaustion is reported. Compare this explicit resource protocol with blindly rerunning a closure that happens to mutate hidden state. Explain what an API can make visible even when it cannot prove semantic idempotence. Record the outcome.

Capability unlocked Oracle Contract - choosing callback capabilities without inventing purity

Choose Your Exit

Challenge the versioned archive / Level 38
Connect retrying decision callbacks to snapshot validation and conflicts.

Explore suspended environments / Level 23
See how retained captures become state in asynchronous computations.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 13

The Library of Borrowed Pages

The Trait Foundry's archive has two reading rooms. One points into a permanent manuscript, while the other decompresses each page into a reusable scratch buffer. Visitors want one inspection algorithm for both rooms, but they also want to keep every page afterward. Those requests cannot share exactly the same storage promise. Your mission is to build the common temporary reading capability and preserve a separate route for callers who genuinely need retention. The archive rewards honest ownership boundaries, not lifetime annotations that claim a scratch page never changes.

Mission Design a lending abstraction shared by stable and reusable producers without promising simultaneous retention.

Suggested loadout Level 3, Level 4, Level 9. These are guidance, not locked gates.

Mastery Targets

- Tie each common borrowed item to the particular access used to request it.
- Process both producer implementations without retaining a scratch-backed item across the next request.
- Offer an explicit owned or stable-source path when a consumer needs to collect results.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 13

Iteration Lending and Generic Associated Types

An Iterator Fixes Its Item Type Before Each Call

Iterator is a powerful abstraction partly because its item type is independent of the temporary borrow used to call next. After a call returns, a caller can usually call next again while retaining the earlier item. A collection iterator can yield references into a backing collection because the implementing type already carries the collection's borrow lifetime. The result stays valid across calls that advance the iterator.

This distinction is easy to miss because both interfaces can look like a loop that returns references. An iterator over a slice hands out access to stable external storage. A streaming parser that reuses one internal buffer hands out access that must end before the parser changes that buffer. The first fits Iterator naturally. The second needs a lending relationship, an owned result, or a callback that limits the borrow's scope.

Generic associated types provide a vocabulary for a family of output types indexed by a lifetime. They do not eliminate borrowing restrictions. They let a trait express precisely the restriction that a returned value may borrow from the current exclusive receiver borrow.

The design question is therefore not whether GATs are more advanced than Iterator. It is whether callers should be allowed to retain several items simultaneously, and who owns the storage those items use. Start with that ownership decision before choosing an iteration trait.

Borrowed Iterators Can Return Long Lived Items

The following iterator stores a shared borrow of its backing slice. Its next call briefly borrows the iterator itself, but its result carries the backing lifetime.

Listing 13.1 Rust example

```
struct EveryOther<'data, T> {
    remaining: &'data [T],
}

impl<'data, T> Iterator for EveryOther<'data, T> {
```

Listing 13.1 continued

```
    type Item = &'data T;

    fn next(&mut self) -> Option<Self::Item> {
        let (first, rest) = self.remaining.split_first()?;
        self.remaining = if rest.is_empty() { rest } else { &rest[1..] };
        Some(first)
    }
}

fn main() {
    let values = [10, 20, 30, 40];
    let mut iter = EveryOther { remaining: &values };
    let first = iter.next().unwrap();
    let second = iter.next().unwrap();
    assert_eq!((*first, *second), (10, 30));
}
```

Keeping first while asking for second is valid because both are shared references into values. The iterator advances its own slice view; it does not modify the referenced elements. Its mutable receiver protects iteration state, while the shared backing borrow protects the collection data. These are separate resources with separate lifetimes.

The standard mutable slice iterator can similarly produce several mutable references because it partitions the backing slice into disjoint regions. Each yielded reference refers to a region removed from the iterator's future access. The implementation must preserve nonoverlap. This is why iter_mut can support collecting mutable references, while an iterator of overlapping mutable windows cannot use the same unrestricted item relationship.

Owning iteration is another distinct contract. IntoIterator for Vec<T> moves elements out, while iteration over &Vec<T> borrows elements and iteration over &mut Vec<T> lends disjoint mutable access through a standard iterator. The visible for syntax hides these different ownership paths. When a loop unexpectedly moves a collection, inspect the type of the expression passed to IntoIterator.

Overlapping Mutable Windows Need a Shorter Result Borrow

With a one-element step, adjacent windows overlap when their width exceeds one. Returning both as unrestricted mutable slice references would allow simultaneous exclusive access to the same element. A lending interface ties each window to the receiver borrow for that call.

Listing 13.2 Rust example

```
trait LendingIterator {
    type Item<'a>
    where
        Self: 'a;

    fn next<'a>(&'a mut self) -> Option<Self::Item<'a>>;
}

struct WindowsMut<'data, T> {
    data: &'data mut [T],
    width: usize,
    position: usize,
}

impl<'data, T> WindowsMut<'data, T> {
    fn new(data: &'data mut [T], width: usize) -> Self {
        assert!(width > 0);
        Self { data, width, position: 0 }
    }
}

impl<T> LendingIterator for WindowsMut<'_, T> {
    type Item<'a> = &'a mut [T] where Self: 'a;

    fn next<'a>(&'a mut self) -> Option<Self::Item<'a>> {
        if self.width > self.data.len().saturating_sub(self.position) {
            return None;
        }
        let start = self.position;
        self.position += 1;
        Some(&mut self.data[start..start + self.width])
    }
}

fn main() {
    let mut values = [1, 2, 3, 4];
    {
        let mut windows = WindowsMut::new(&mut values, 2);
        while let Some(window) = windows.next() {
```

Listing 13.2 continued

```
            window[0] += window[1];
        }
    }
    assert_eq!(values, [3, 5, 7, 4]);
}
```

The associated type is now a family. For each lifetime 'a, the implementation supplies a corresponding borrowed slice type. The where Self: 'a clause permits that family to borrow from a Self that remains valid for 'a. The method chooses 'a through its exclusive borrow of self. A caller holding the returned window therefore keeps that exclusive borrow active and cannot obtain the next window until the first is no longer used.

Notice what this interface does not prove. It does not declare that windows overlap, and it does not reason about the arithmetic in the implementation. The ordinary safe slice operation checks bounds, while the receiver lifetime enforces sequential access. GATs describe the output relationship; they do not replace invariant design.

The width assertion is also part of the contract. Zero-width windows could be supported, but their count and advancement policy would need a definition. Rejecting zero here avoids a distracting special case. Production interfaces should document that panic or return a validation error if width comes from untrusted configuration.

The Borrow Checker Enforces the Lending Contract

The following complete example deliberately fails. It uses a concrete lending-style method without a trait so the conflict is easy to isolate.

Listing 13.3 Rejected example

```
struct Buffer(Vec<u8>);

impl Buffer {
    fn view(&mut self) -> &mut [u8] {
        &mut self.0
    }
}

fn main() {
    let mut buffer = Buffer(vec![1, 2, 3]);
    let first = buffer.view();
```

Listing 13.3 continued

```
    let second = buffer.view();
    first[0] += second[0];
}
```

The second borrow conflicts because first is used afterward. If first were no longer used, nonlexical lifetime analysis could end its borrow before the next call. A lending loop works because each iteration finishes using the current item before advancing. The restriction concerns actual borrow use, not merely the fact that two variable names appear in one lexical block.

This is why standard Iterator adapters cannot automatically be copied to a lending trait. collect intends to retain items. zip may need to retain items from separate sources while advancing them. filter may repeatedly borrow the source while deciding which value to return. Some adapters are expressible, but their signatures and implementation limits deserve independent analysis. A trait family is not a drop-in replacement for a fixed associated type.

Avoid promising an extensive generic lending ecosystem merely because one concrete iterator compiles. Advanced associated-type projections and higher-ranked bounds can encounter compiler limitations or produce surprising diagnostics. A small concrete interface or callback can sometimes give users a more stable and comprehensible API than a deeply generic adapter stack.

Reusing a Parser Buffer

A line reader that reuses one String is a practical lending interface. Its output is valid until the next mutable operation on the reader. A normal Iterator returning owned String values would allocate or transfer a separate buffer for each retained line. That may be exactly what callers need, but it is a different storage policy.

Listing 13.4 Rust example

```
use std::io::{self, BufRead};

struct LineReader<R> {
    input: R,
    buffer: String,
}

impl<R: BufRead> LineReader<R> {
```

Listing 13.4 continued

```
    fn new(input: R) -> Self {
        Self { input, buffer: String::new() }
    }

    fn next_line(&mut self) -> io::Result<Option<&str>> {
        self.buffer.clear();
        if self.input.read_line(&mut self.buffer)? == 0 {
            return Ok(None);
        }
        let line = self.buffer.as_str();
        let body = line.strip_suffix("\r\n").or_else(|| line.strip_suffix('\n'));
        Ok(Some(body.unwrap_or(line)))
    }
}

fn main() -> io::Result<()> {
    let input = io::Cursor::new(b"alpha\r\nbeta\nend\r");
    let mut reader = LineReader::new(input);
    let mut lengths = Vec::new();
    while let Some(line) = reader.next_line()? {
        lengths.push(line.len());
    }
    assert_eq!(lengths, [5, 4, 4]);
    Ok(())
}
```

No GAT is required for this concrete inherent method. Lifetime elision already ties the returned reference to &mut self. Introduce a lending trait only when several implementations need to participate in generic algorithms. Advanced language machinery is valuable when it expresses a relationship shared across implementations, not simply because the relationship is sophisticated.

Like `BufRead::lines`, the reader strips LF or CRLF and preserves a lone final carriage return. It uses `read_line`, so invalid UTF-8 produces an I/O error. A binary protocol reader can use byte buffers and validate text at an explicit layer.

The error path also deserves attention. read_line can append data before reporting some errors. This method clears the buffer on the next call but makes no transactional guarantee about the underlying stream position. A parser that needs recoverable framing must define what input has been consumed when an error occurs. Borrowing efficiency does not provide recovery semantics automatically.

Callbacks Can Express a Temporary View Without a Trait Family

An alternative is to pass each borrowed item to a callback whose invocation finishes before the buffer is reused. The producer controls the loop, and the callback cannot retain a reference unless the signature supplies an appropriate longer lifetime. This can be easier to compose when the consumer only needs aggregate results.

A callback-based scanner can expose visit_lines taking FnMut(&str), read into one buffer, invoke the callback, and then clear the buffer. A consumer can count records, update a hash, or copy selected records. The important tradeoff is control flow. The consumer may need an explicit return enum to stop early, propagate an error, or request a pause. With an iterator, the consumer naturally owns advancement.

Choose among owned iteration, borrowing from external storage, lending from internal storage, and callbacks by considering retention and control. If callers need random access to all records later, owned storage or an arena may be appropriate. If callers process each record once and discard it, a lending view can avoid unnecessary allocations. If callers need a mixture, offer an explicit to_owned operation at the point where retention begins.

Laziness Does Not Mean Work Is Free

Iterator adapters usually build a description of future work. The terminal operation executes it. This can improve composition, but it can also hide repeated traversal, side effects, or allocation in an apparently small expression. Review an iterator chain as an algorithm: how many elements are visited, which closures run, where errors stop processing, and what is retained.

try_fold and collection into Result provide structured short-circuiting, but partial work may already have occurred. A map closure that updates external state is not rolled back when a later element fails. A filter that calls an expensive function may run that function every time the iterator is traversed. A size_hint is an optimization hint with documented constraints, not an authority to perform unchecked memory writes in arbitrary unsafe code.

The distinction between exact length and trusted initialization is especially important for custom collections. A safe Iterator implementation can be buggy. Unsafe consumers must not assume undocumented stronger guarantees from a safe trait. Optimization-specific unsafe contracts require an appropriate unsafe abstraction and careful proof.

Training Encounters and Debriefs

Exercise 1. Why can slice::Iter return two references that remain alive together while WindowsMut cannot return two overlapping mutable windows?

Solution. Shared slice references may overlap, and their lifetime comes from the stable backing slice. Overlapping mutable windows would grant simultaneous exclusive access to the same elements. The lending method ties each mutable window to the receiver borrow, forcing sequential use. Disjoint mutable iteration is possible because each yielded region is removed from future access.

Exercise 2. A caller wants to collect every line from LineReader into Vec<&str>. What should the API do?

Solution. The reusable buffer cannot support those retained references. The caller should collect owned strings, or the design should retain the entire input or use stable backing storage such as an arena. Trying to lengthen the result lifetime would misrepresent the storage's validity. Ownership must change if retention requirements change.

Exercise 3. Does adding a GAT to LineReader improve the concrete next_line method?

Solution. Not by itself. The inherent method already expresses a result borrowing from self. A GAT becomes useful when a trait must abstract over several implementations whose borrowed result types differ. The abstraction should solve a real generic composition requirement, and its adapters should be designed around the lending lifetime rather than copied mechanically from Iterator.

Further Reading

• Standard library Iterator specifies fixed item types, adapters, and size hints.

• Rust Reference on generic associated types specifies associated type families and required where clauses.

• Standard library BufRead documents buffered borrowing and consumption operations.

The Vanishing Collection

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a page inspection interface implemented by a stable-source reader and a scratch-buffer reader. Their borrowed view types may differ, so use a generic associated type only where that generic abstraction needs it. Write a consumer that computes owned page summaries while inspecting one view at a time. Then analyze a proposed collect-all adapter returning borrowed views from either producer. Explain why the scratch implementation cannot honor that retention contract. Offer two honest alternatives, one that copies retained data and one that exposes the stable producer's stronger source-lifetime relationship through a separate interface. Describe the relevant receiver borrow for every output. Your solution should explain why a common lending interface can deliberately expose less retention capability than one specific implementation could provide alone.

Encounter Constraints

• Do not lengthen scratch-backed output lifetimes with unsafe code or invented static bounds.

• Keep borrowed views scoped before asking a reusable producer for another item.

• Distinguish an inherent borrowing method from a GAT needed for generic composition.

Hint one / Hint two / Reveal debrief

Side Quest

Design a callback-style inspect-next operation that exposes a view only during one invocation, and compare it with the GAT interface. Consider whether consumers need to name the view type, compose adapters, or return an owned result. Describe the ergonomic tradeoffs without assuming that every useful abstraction must imitate the standard Iterator trait exactly. Try the consumer against both producer implementations.

Capability unlocked Page Lease - abstracting temporary views without promising impossible retention

Choose Your Exit

Build the borrowing parser / Level 37
Choose between borrowed output, owned storage, and retained ranges.

Inspect representation costs / Level 34
Measure the allocation and retention consequences of each reader policy.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 14

The Repair Report That Can Be Used

Ferrum's import desk reports every failure as a dramatic sentence. Operators can read the prose, but the repair system cannot distinguish invalid input, a denied policy, unavailable storage, or a conflict that needs a fresh decision. Worse, some failed imports leave a prefix applied while others leave nothing changed. Your mission is to make the error boundary carry useful structure and a clear postcondition. A good report should help the next component choose an action without parsing punctuation or guessing which resources still belong to it.

Mission Design structured import errors with explicit failure state, preserved causes, and owned recovery data where useful.

Suggested loadout Level 1, Level 6, Level 9. These are guidance, not locked gates.

Mastery Targets

• Distinguish validation, policy, storage, and conflict categories without matching error message strings.

• State whether each failed operation leaves the published in-memory import state unchanged.

• Preserve relevant underlying causes and identify who owns rejected input or prepared output.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 14

Error Architecture and Recovery

An Error Type Defines Decisions Callers Can Make

Result is an ownership boundary carrying one of two values. Its advanced design problem is not how to write a question mark. The problem is deciding which information crosses the boundary, which recovery decisions remain possible, and what has already happened when failure is reported.

A storage operation can fail before changing anything, after committing, or while its commit status is unknown. All three cases can use Result. A parser can return a precise byte offset or a string that discards it. A queue can return a rejected job or drop it while returning an error. The enum shape determines what the caller still owns; documentation determines how that ownership relates to external effects.

Begin error design with the caller's decisions. Can it retry immediately, retry with a delay, ask for different input, choose another backend, or stop? If several failures require the same response, they may share a public category while retaining their underlying source for diagnosis. If two failures require different responses, flattening them into one string forces callers to inspect text or give up on structured recovery.

Not every detail should become a public variant. Exposing internal dependency errors directly can couple the API to a particular library and version. A good boundary preserves useful meaning while limiting accidental implementation commitments. This requires judgment about which layer owns the recovery policy.

Separate Domain Categories From Diagnostic Causes

A domain error can classify an operation while preserving an underlying error through the standard Error trait's source method. Display provides a human-readable description. Neither should be treated as a stable machine protocol unless that is explicitly promised.

Listing 14.1 Rust example

```
use std::error::Error;
use std::fmt;
use std::io;
use std::path::{Path, PathBuf};

#[derive(Debug)]
enum LoadError {
    Read { path: PathBuf, source: io::Error },
    InvalidLimit { line: usize, value: String },
}

impl fmt::Display for LoadError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::Read { path, .. } => {
                write!(f, "cannot read configuration {}", path.display())
            }
            Self::InvalidLimit { line, value } => {
                write!(f, "invalid limit on line {line}: {value:?}")
            }
        }
    }
}

impl Error for LoadError {
    fn source(&self) -> Option<&(dyn Error + 'static)> {
        match self {
            Self::Read { source, .. } => Some(source),
            Self::InvalidLimit { .. } => None,
        }
    }
}

fn load_limit(path: &Path) -> Result<u32, LoadError> {
    let text = std::fs::read_to_string(path).map_err(|source| {
        LoadError::Read { path: path.to_owned(), source }
    })?;
    let value = text.lines().next().unwrap_or("").trim();
    value.parse().map_err(|_| LoadError::InvalidLimit {
        line: 1,
        value: value.to_owned(),
    })
}
```

The Read variant adds the path because io::Error alone does not identify which configuration was being loaded. The source remains available for details such as permission denial. InvalidLimit preserves the invalid value and line because the caller may present a targeted correction. The parse

error's precise implementation type is not needed for this domain decision.

This example stores a PathBuf and String in errors. That allocation is often acceptable because the error path needs durable context after local borrows end. In allocation-constrained code, use compact offsets, numeric categories, or borrowed errors with carefully scoped lifetimes. Do not assume every error must implement Error or own a heap allocation; those are interoperability choices rather than requirements of Result.

The Question Mark Converts but Does Not Invent Context

The question mark propagates an early return and can convert an error through the relevant conversion relationship. It does not know what operation was attempted or which domain object matters. A broad From<io::Error> implementation can be convenient, but it cannot attach a path that it never received.

Use local map_err when the context is available at that call site. Use From when the conversion is genuinely context-free and unambiguous. A large application may use contextual error reporting at orchestration boundaries while its reusable libraries expose structured enums. These strategies serve different callers and can coexist.

Avoid wrapping every layer with a sentence that repeats the previous layer. A chain saying failed to load, failed to read, read failed adds little. Useful context names the operation, resource, position, or invariant that the lower layer could not know. The final rendering can show the high-level operation followed by the cause chain.

Also avoid logging and returning the same error at every level. If a caller is responsible for handling or reporting the failure, intermediate logging may create several misleading incidents for one event. Log where the operation's outcome is decided or where important context would otherwise be lost. A library can preserve context in the error without emitting anything itself.

Preserve Ownership When Recovery Needs It

A bounded queue that rejects a job should often return the job. Otherwise the caller cannot retry without cloning it before submission. The failure type can carry both the reason and the original value.

Listing 14.2 Rust example

```
#[derive(Debug, PartialEq, Eq)]
enum RejectReason {
    Full,
    Closed,
}

#[derive(Debug)]
struct Rejected<T> {
    reason: RejectReason,
    value: T,
}

struct Queue<T> {
    values: Vec<T>,
    limit: usize,
    closed: bool,
}

impl<T> Queue<T> {
    fn try_push(&mut self, value: T) -> Result<(), Rejected<T>> {
        let reason = if self.closed {
            Some(RejectReason::Closed)
        } else if self.values.len() >= self.limit {
            Some(RejectReason::Full)
        } else {
            None
        };

        if let Some(reason) = reason {
            Err(Rejected { reason, value })
        } else {
            self.values.push(value);
            Ok(())
        }
    }
}

fn main() {
    let mut queue = Queue {
        values: Vec::new(), limit: 0, closed: false,
    };
    let rejected = queue.try_push(String::from("job")).unwrap_err();
```

Listing 14.2 continued

```
    assert_eq!(rejected.reason, RejectReason::Full);
    assert_eq!(rejected.value, "job");
}
```

The method validates before modifying the vector. A returned rejection therefore means the queue did not accept ownership. This is a useful semantic guarantee in addition to memory safety. If submission involved a remote broker, the same guarantee might be impossible after a timeout. The error model would then need a category such as unknown outcome, and retry would need an idempotency strategy.

Returning T can make the error generic and large. That is a reasonable tradeoff when recovery needs ownership. Another API can borrow input and return a status if the operation never retains it. Do not force cloning solely to keep an error enum small without measuring the actual cost and caller needs.

Recovery Is a State Transition

Treat each fallible operation as a transition with a failure postcondition. For an in-memory update, the postcondition might be unchanged state. For a stream parser, it might be input consumed through a delimiter. For a database transaction, it might be a durable commit or an explicit rollback. The error's meaning depends on this postcondition.

A common robust pattern is validate, prepare, then commit. Validation checks preconditions without mutating shared state. Preparation builds a new owned value that may fail independently. Commit performs a small, ideally infallible replacement. This minimizes the region where state can be partially updated.

Listing 14.3 Rust example

```
#[derive(Debug, PartialEq, Eq)]
enum RenameError {
    Empty,
    TooLong,
}

struct Account {
    name: String,
    revision: u64,
}
```

Listing 14.3 continued

```
impl Account {
    fn rename(&mut self, input: &str) -> Result<(), RenameError> {
        let normalized = input.trim();
        if normalized.is_empty() {
            return Err(RenameError::Empty);
        }
        if normalized.len() > 64 {
            return Err(RenameError::TooLong);
        }
        let next = normalized.to_owned();
        let revision = self.revision.checked_add(1)
            .expect("account revision exhausted");
        self.name = next;
        self.revision = revision;
        Ok(())
    }
}
```

The revision overflow is treated as an internal exhaustion condition rather than a user-input error. The important ordering is that checked arithmetic happens before modifying name. If a panic unwinds from expect, the account still holds its original pair. After preparation, assigning the new String and revision requires no application-level fallible step.

This does not promise survival under every process failure. Allocation failure may abort, and a process crash can occur between machine instructions. The guarantee is about ordinary returned errors and the considered panic path in memory. Durability across crashes requires a storage protocol, not merely careful Result handling.

Panic Is an Escape From the Normal Return Contract

A panic is useful for violated programming assumptions when continuing the current operation is inappropriate. It should not be the ordinary representation of invalid user input or expected remote failures. Callers can choose to turn a Result into a panic; a callee that panics first removes their recovery option.

Unwinding runs destructors along the unwound path, but it does not roll back arbitrary mutations or external effects. A panic in the middle of a vector update can leave a logically incomplete value even if memory safety remains intact. Standard synchronization poisoning can signal that

protected data may need repair, but poisoning is advisory behavior rather than a universal proof of validity.

catch_unwind is a boundary tool for isolating unwinding panics in selected contexts. It catches neither aborting panics nor all possible failures. UnwindSafe is a marker about exposing potentially inconsistent state across that boundary, not a guarantee that an application can safely resume its domain workflow. Adding AssertUnwindSafe suppresses a type-level concern; it does not repair the underlying invariant.

A useful review separates memory safety, local invariant preservation, and business consistency. Safe Rust provides strong memory guarantees, but a caught panic may still leave a payment half recorded or a callback registry temporarily altered. Decide whether to repair, discard, restart, or terminate the affected component based on its actual state model.

Error Erasure Belongs at a Deliberate Boundary

Box<dyn Error + Send + Sync> can combine unrelated failures into one owned type. It is useful in application entry points, orchestration code, and diagnostic pipelines where the primary action is reporting rather than fine-grained recovery. It also hides concrete categories unless callers downcast or inspect sources.

Erasure is less suitable when library callers must distinguish missing data from corruption or retryable rejection from permanent invalid input. An opaque library error can still expose stable query methods such as kind or is_retryable, allowing internal representation to evolve. The right design depends on which decisions the boundary promises to support.

Be careful with a universal is_retryable boolean. Retryability can depend on the operation's idempotency, elapsed time, authentication state, and whether an effect may already have committed. A transport timeout is not inherently safe to retry for every request. An error can classify what is known; the orchestration layer often owns the final retry decision.

Testing Failure Paths Means Testing Postconditions

Tests that merely assert is_err miss most error architecture. For a rejected job, assert that ownership is returned and the queue length is unchanged. For a failed rename, assert that both name and revision retain their previous values. For a parse failure, assert the offset and the documented consumption behavior. For a wrapped I/O error, assert that the source chain preserves the relevant cause.

Inject failure at meaningful boundaries. A writer that fails after a chosen number of bytes can expose partial-output behavior. A repository stub that reports unknown commit status can exercise idempotency logic. A callback that panics can reveal whether a registry restores its internal structure. Such tests validate the contract rather than merely mirror enum construction.

Keep public error messages useful but avoid using them as a substitute for structured data. Exact string assertions are appropriate for a promised CLI format or user-facing snapshot, while library recovery tests should match variants or stable query methods. This lets wording improve without silently breaking machine behavior.

Training Encounters and Debriefs

Exercise 1. A network request times out after sending a payment command. Should its error automatically be classified as retryable?

Solution. No. The server may have committed the payment before the response was lost. Preserve an unknown-outcome category or equivalent information. Retry policy must use an idempotency key, a query for prior completion, or a protocol that defines safe repetition. Transport failure alone does not establish the operation's postcondition.

Exercise 2. A library converts every error to String. What capabilities has it removed?

Solution. Callers lose structured classification, reliable cause traversal, and access to owned recovery data unless those are encoded separately. Text can still be excellent for final reporting, but it is a poor machine interface. Preserve domain categories at library boundaries and render them at the reporting boundary.

Exercise 3. A mutation method returns Err after changing two of three fields. Is this memory unsafe?

Solution. Not necessarily. Safe Rust can preserve memory safety while allowing a broken domain invariant. The method must document partial updates or be redesigned around validation, preparation, and a smaller commit step. Tests should assert the failure postcondition, not just the presence of Err.

Further Reading

- Standard library Error documents diagnostic descriptions and source chains.
- Standard library Result describes ownership-preserving combinators and propagation operations.
- Standard library catch_unwind specifies the limits of panic interception and unwind safety.

The Ambiguous Import

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design an import operation for an owned batch of records. Reading source material can fail with a standard I/O error; parsing and policy validation can reject particular records; final acceptance can conflict with a newer local state. Choose an all-or-nothing in-memory publication contract and identify the preparation boundary that supports it. Define an error representation that lets an application distinguish these categories, show a useful human message, and inspect a preserved underlying I/O cause. Decide which failures return the original batch and which return a prepared proposal that must be reconsidered before retry. Explain why returning recovery data is not equivalent to certifying that a retry is appropriate. Include tests for the failure postcondition rather than checking only that Err appears.

Encounter Constraints

- Do not convert every failure to String at the reusable library boundary.
- Keep user-controlled validation and preparation before the stated publication point.

• Describe the process-local contract without presenting it as durable transaction recovery.

Hint one / Hint two / Reveal debrief

Side Quest

Add a deliberately partial-import mode that commits valid records independently. Give it a different result contract reporting accepted, rejected, and unattempted records, rather than disguising partial success as the same all-or-nothing error. Explain how caller accounting and retry behavior change even though both implementations remain memory-safe Rust. Show that the accounting covers every original record exactly once after the operation ends.

Capability unlocked Recovery Compass - errors that reveal state and support a deliberate next move

Choose Your Exit

Forge valid protocol states / Level 15
Move some runtime error conditions into constructor and transition types.

Enter the review arena / Level 42
Audit invariants and evidence after the compiler has accepted the program.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 15

The Gate of False Credentials

The Trait Foundry issues dispatch tickets for Ferrum's evacuation trams. Its old console stores destination, approval, receipt, and failure as unrelated optional fields. A ticket can claim to be delivered without ever naming a destination. Your task is to replace that console's model before the next departure. The fiction gives you the problem; the evidence you earn comes from a small Rust API and examples of transitions its callers can and cannot express.

Mission Represent dispatch phases so their data and permitted operations agree, including a genuinely uncertain delivery outcome.

Suggested loadout Level 1, Level 4, Level 9, Level 14. These are guidance, not locked gates.

Mastery Targets

• Invalid destination and oversized payload are rejected before an approved ticket is constructed.

• Sending consumes approval, and an ordinary client cannot send the same approval value again.

• A failure before handoff returns reusable approval, while an unknown post-handoff outcome remains distinguishable.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 15

Encoding State and Validity in Types

Move Validation to the Boundary Where Evidence Appears

An application often carries facts in comments and conventions: this identifier has been validated, this request has been authenticated, this connection has completed its handshake. Rust types can carry some of those facts as values that are difficult to construct incorrectly. The goal is not to encode every business rule at compile time. The goal is to make important distinctions survive ordinary refactoring without repeated checks or undocumented assumptions.

There are several useful tools. An enum represents a closed set of runtime alternatives. A newtype distinguishes values with the same representation but different meaning. A private constructor protects an invariant. Typestate changes the available operations by changing a type parameter or concrete wrapper after a transition. Each tool answers a different question about what information is known and when.

Use a runtime enum when the program must store and inspect several states together. Use typestate when the control flow naturally establishes a state and then performs operations valid only there. Use a validated newtype when a property of one value should remain true across many operations. Combining them is often clearer than forcing all state into one generic machine.

The central discipline is evidence preservation. Once a value has been checked, represent the successful check with a value that cannot be forged through the public safe API. Then make operations require that value. If mutable access can invalidate the evidence, restrict that access or return to an unchecked representation.

Newtypes Distinguish Meaning Without Repeating Validation

Consider an account identifier that must be nonempty ASCII and contain only letters, digits, and underscores. A String alone does not express that condition. A private field and validating constructor can establish it once.

Listing 15.1 Rust example

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct AccountId(String);

#[derive(Debug, PartialEq, Eq)]
enum AccountIdError {
    Empty,
    InvalidByte { index: usize },
}

impl AccountId {
    fn parse(value: String) -> Result<Self, AccountIdError> {
        if value.is_empty() {
            return Err(AccountIdError::Empty);
        }
        if let Some(index) = value.bytes().position(|byte| {
            !(byte.is_ascii_alphanumeric() || byte == b'_')
        }) {
            return Err(AccountIdError::InvalidByte { index });
        }
        Ok(Self(value))
    }

    fn as_str(&self) -> &str { &self.0 }

    fn into_string(self) -> String { self.0 }
}

fn main() {
    let id = AccountId::parse("build_42".into()).unwrap();
    assert_eq!(id.as_str(), "build_42");
    assert_eq!(AccountId::parse("".into()), Err(AccountIdError::Empty));
}
```

Consumers of AccountId can rely on the validation rule if all constructors preserve it. Returning &str is safe because shared access cannot alter the bytes. Returning &mut String would let callers insert invalid characters and destroy the evidence. An unchecked constructor would need a carefully justified visibility or unsafe contract if memory safety depended on the property, though this application-level format rule ordinarily does not require unsafe.

The example returns an error without returning the original String. That is acceptable for a simple parser, but a recovery-focused API could return the value alongside the error. The invariant design and ownership-on-failure

design are independent. A sophisticated type does not excuse an inconvenient recovery contract.

Do not confuse a type alias with a newtype. A type alias for String remains the same type and does not prevent passing a project name where an account identifier is expected. A wrapper creates a distinct type identity and a place to enforce construction rules.

Enums Preserve Correlations Between Fields

A struct with several Options can represent combinations that the domain never permits. For example, a download may have a pending request, an active byte count, or a completed path. Independent optional fields allow completed-without-path and pending-with-byte-count states unless every method maintains a convention.

Listing 15.2 Rust example

```
use std::path::PathBuf;

enum Download {
    Pending { url: String },
    Active { url: String, received: u64 },
    Complete { path: PathBuf, bytes: u64 },
    Failed { url: String, message: String },
}

fn status(download: &Download) -> String {
    match download {
        Download::Pending { url } => format!("waiting for {url}"),
        Download::Active { url, received } => {
            format!("{url} received {received} bytes")
        }
        Download::Complete { path, bytes } => {
            format!("{} contains {bytes} bytes", path.display())
        }
        Download::Failed { url, message } => {
            format!("{url} failed: {message}")
        }
    }
}
```

Each variant carries exactly the data needed for that state. Exhaustive matching makes state additions visible to code that handles every case. This is valuable inside an application where the set is centrally controlled.

For public library enums intended to grow, non_exhaustive can reserve extension space, with the cost that downstream matches need a fallback.

Enums do not automatically enforce transition legality. Code with access to the variants can construct Active directly. Encapsulation and methods determine which transitions are available. If only certain operations should create states, keep the internal enum private and expose a controlled wrapper.

The representation can also affect storage. An enum must accommodate its variants according to Rust's layout rules, possibly with niche optimizations. Do not assume a particular size without a documented representation guarantee. If one variant owns a very large inline payload, boxing that payload may reduce the size of every enum value at the cost of an allocation and indirection.

Typestate Makes Operations Depend on a Proven Phase

A request builder is a natural typestate example. A draft request can be edited, but only a validated request can be dispatched. Validation consumes the draft and returns a distinct type, preventing accidental reuse of the old phase.

Listing 15.3 Rust example

```
struct Draft;
struct Ready;

struct Request<State> {
    destination: String,
    body: Vec<u8>,
    state: std::marker::PhantomData<State>,
}

#[derive(Debug)]
struct ValidationError;

impl Request<Draft> {
    fn new(destination: String) -> Self {
        Self {
            destination,
            body: Vec::new(),
            state: std::marker::PhantomData,
        }
    }
```

Listing 15.3 continued

```
    fn body(mut self, body: Vec<u8>) -> Self {
        self.body = body;
        self
    }

    fn validate(self) -> Result<Request<Ready>, ValidationError> {
        if self.destination.is_empty() || self.body.len() > 1024 {
            return Err(ValidationError);
        }
        Ok(Request {
            destination: self.destination,
            body: self.body,
            state: std::marker::PhantomData,
        })
    }
}

impl Request<Ready> {
    fn dispatch(self) -> (String, Vec<u8>) {
        (self.destination, self.body)
    }
}

fn main() {
    let request = Request::new("worker-7".into())
        .body(b"rebuild".to_vec())
        .validate()
        .unwrap();
    let (destination, body) = request.dispatch();
    assert_eq!(destination, "worker-7");
    assert_eq!(body, b"rebuild");
}
```

The PhantomData field records a type-level distinction without storing a runtime state value. The transition moves the existing String and Vec rather than cloning their contents. Only Request<Ready> has dispatch. The compiler therefore rejects dispatch on a draft before any runtime check could be forgotten.

The ready state proves only the validation actually performed. It does not prove that the destination exists, that a network connection will succeed, or that the request remains authorized forever. Types carry bounded evidence. Naming a state Authorized does not make a revocable authorization permanently valid. Time-sensitive facts often need a runtime check at the moment of use.

Consumption Prevents Accidental Reuse but Not Every Duplicate

A consuming transition prevents use of that particular owner afterward. If the state type or wrapper implements Clone, callers may still duplicate the capability before consuming it. If the resource is uniquely consumable, do not derive Clone casually. For a true external one-time action, the server or resource manager may also need to enforce uniqueness because process memory alone cannot prevent replay from another process.

Similarly, a typestate token can prove that an initialization path ran, but it cannot make unsafe aliases elsewhere obey the token. Encapsulation must ensure that the token actually controls access to the protected resource. A public zero-sized Ready type paired with a public Request field would allow construction without validation. The proof depends on the whole public construction surface.

Transitions that can fail should consider returning the original state. A handshake attempt that consumes a socket and returns only an error may make retry impossible. A result like Result<Connected, (HandshakeError, Unconnected)> preserves ownership when the failure postcondition permits another attempt. If failure has corrupted or closed the socket, returning an apparently usable Unconnected state would be misleading.

State Data Can Be More Useful Than Marker Types

Not every state should be a zero-sized marker. A connected state may contain a negotiated protocol version, a verified peer identity, or a session key. Keeping that data in the state parameter can ensure that operations requiring it cannot exist without it.

Listing 15.4 Rust example

```
struct Disconnected;

struct Connected {
    peer: String,
    sequence: u64,
}

struct Channel<S> {
    state: S,
}
```

Listing 15.4 continued

```
impl Channel<Disconnected> {
    fn new() -> Self { Self { state: Disconnected } }

    fn connect(self, peer: String) -> Channel<Connected> {
        Channel { state: Connected { peer, sequence: 0 } }
    }
}

impl Channel<Connected> {
    fn frame(&mut self, payload: &str) -> String {
        let sequence = self.state.sequence;
        self.state.sequence = sequence.checked_add(1)
            .expect("sequence exhausted");
        format!("{}:{sequence}:{payload}", self.state.peer)
    }

    fn disconnect(self) -> Channel<Disconnected> {
        Channel { state: Disconnected }
    }
}
```

This is a local state machine model, not a complete network channel. Its value is structural: Connected owns the data that connected operations need. There is no separate Option<String> that must be unwrapped under a comment saying connected channels always have a peer.

The pattern can become awkward when a collection must contain channels in different phases. Their concrete types differ. A runtime enum such as AnyChannel with Disconnected and Connected variants can erase the phase into a closed representation. Another approach is to keep phase-specific collections. The right choice follows the application's scheduling and storage requirements.

Avoid a Combinatorial Type Surface

Typestate can overfit an API. If five independent booleans become five type parameters, users may face dozens of state combinations and difficult diagnostics. If every operation merely changes one marker and returns another deeply nested type, the public surface may become harder to understand than a small runtime validator.

Encode distinctions that eliminate important misuse or simplify downstream code. Keep incidental configuration as values. A request's compression setting usually does not need a type parameter unless algorithms genuinely depend on compile-time specialization or compatibility. A borrowed buffer's access mode may justify a type distinction because it controls which operations are sound.

Consider how users name intermediate values, store them in structs, return them from functions, and branch between states. A type-level design that makes a simple conditional impossible to express without boxing may be too rigid for the workflow. Enums and private runtime state machines remain powerful, explicit tools.

Documentation should explain both the state graph and the operations available at each state. A diagram can help, but the type signatures should remain understandable without it. Name transitions after domain actions such as validate, connect, finish, and close. Names like into_state_b reveal implementation machinery without telling users what evidence has changed.

Validation Must Survive Every Escape Hatch

Serialization, database loading, FFI, and test helpers can bypass a carefully designed constructor if implemented casually. A custom deserializer for AccountId must call the validating path or perform equivalent checks. A public From<String> implementation implies infallible construction and would contradict a fallible format invariant. TryFrom is a more appropriate integration when input may be invalid.

Unsafe code should not assume an application invariant merely because a type name suggests it. If memory safety depends on the invariant, all safe constructors and mutators must enforce it, and any unchecked construction must have an explicit safety contract. If the invariant is only business correctness, keep the distinction clear so users do not confuse a logic error with undefined behavior.

Tests should attempt invalid construction through every public conversion path. They should also verify that valid values remain valid after transformations. A lowercase normalization method, for example, may preserve an ASCII identifier invariant, while arbitrary replacement may

not. The type's value comes from maintaining the evidence over time, not just checking it once in the most obvious constructor.

Training Encounters and Debriefs

Exercise 1. A validated EmailAddress exposes DerefMut<Target = String>. What is the problem?

Solution. Callers can mutate the string into an invalid address without passing through validation. The wrapper no longer carries reliable evidence. Expose a shared view, consuming extraction, and specific validated transformations. If editing is needed, return an unchecked draft that must be revalidated.

Exercise 2. A typestate authorization token is valid for ten minutes. Can its type alone prove a request is authorized when sent?

Solution. No. The type can prove that an authorization check happened and can carry its expiry data, but time can pass and permissions can be revoked. The send boundary must validate the time-sensitive conditions or rely on a protocol that enforces them. Static evidence and dynamic validity must be distinguished.

Exercise 3. An application stores thousands of connections in arbitrary phases and frequently branches on phase. Should every phase be a different collection element type?

Solution. A runtime enum is often simpler. It preserves state-specific payloads while permitting homogeneous storage and exhaustive dispatch. Typestate can still be used inside operations that temporarily extract one variant and perform a controlled transition. The two approaches are complementary.

Further Reading

- Rust Reference on enumerations defines variant payloads and enum behavior.
- Standard library PhantomData describes marker relationships and their effects.
- Standard library TryFrom provides a conventional fallible construction interface.

The Receipt That Never Happened

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a dispatch API for a payload, a nonzero destination identifier, and a receipt. Input begins unchecked. Validation produces an approved ticket that owns the payload. Starting dispatch consumes that ticket. A local capacity rejection occurs before handoff and must return the approved ticket intact. After handoff, a lost acknowledgement must produce an uncertain outcome carrying the operation identifier, rather than another approved ticket that can be sent blindly. A confirmed delivery produces a receipt. Choose runtime enums, state parameters, or a combination, and write client snippets for every outcome. Your design must also explain which constructors are public and what information a caller can inspect without regaining mutation that would invalidate approval.

Encounter Constraints

• Use private representation for validated evidence; no unsafe code is required.

• A consumed Rust value is not evidence that a remote side effect occurred exactly once.

• Preserve the payload on pre-handoff rejection without requiring a deep clone.

Hint one / Hint two / Reveal debrief

Side Quest

Add an operator console that stores tickets in all phases in one collection. Keep the phase-specific payloads inside an enum, and let a match expose the precise operation available for each variant. Then add expiry to approval. Decide where the current clock must be consulted and explain why a successful validation performed earlier cannot establish that approval is still current when dispatch finally begins.

Capability unlocked Phase Sight distinguishes validated evidence from a wished-for state.

Choose Your Exit

Forge a public contract / Level 16
Make these state types usable without freezing every internal choice.

Open the protocol dungeon / Level 40
Apply consuming transitions to a connection with partial external progress.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 16

The Museum of Broken Promises

A thousand tools depend on the Foundry's public registry crate. Its maintainers want to replace the storage engine, add plugin metadata, and improve diagnostics. The museum next door exhibits upgrades that compiled perfectly in their own repository and broke their clients. You receive four surviving client fragments and the proposed release. Your expedition is a compatibility investigation: identify what each client relies on before deciding which changes can leave the museum alive.

Mission Design an evolution plan around observable client capabilities, including auto traits and borrowing behavior.

Suggested loadout Level 4, Level 5, Level 9, Level 10, Level 11, Level 15. These are guidance, not locked gates.

Mastery Targets

- List construction, matching, trait implementation, borrowing, and thread-bound promises used by representative clients.
- Separate an internal representation change from a change to a public capability.
- Provide compile-time client checks for the promises the release intends to retain.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 16

Public APIs and the Cost of Commitments

A Public Signature Is Only Part of the Contract

A reusable Rust library exposes types, methods, implementations, lifetimes, auto traits, errors, feature behavior, and documented guarantees. Downstream code can depend on all of them. A change that looks internal may alter whether a type is Send, whether an iterator can outlive an input, or whether a trait remains dyn compatible. API design therefore includes deciding which facts should remain stable and which details should stay hidden.

The first useful distinction is between an application boundary and a library boundary. Inside one application, a compiler-guided refactor can update every caller together. A published crate may have callers the author cannot inspect. Its design needs room for evolution, but excessive abstraction can make the current API unpleasant. The aim is a small set of deliberate commitments, not maximum concealment.

Ask what downstream users should be able to construct, implement, match, name, and rely on. Public fields permit direct construction and pattern matching. Public traits permit external implementations unless sealed. Public enums permit exhaustive matching unless marked otherwise. Return-position impl Trait hides a concrete type but still exposes selected capabilities and captured generic relationships.

An API review should examine these operations explicitly. Looking only at the method list misses much of the surface that Rust's type system makes available to callers.

Private Fields Preserve Construction and Representation Freedom

A struct with all public fields permits callers to construct it with a literal. Adding another required field then breaks those callers. A private field prevents external literal construction and makes constructors the supported entry point. It also creates a place to maintain invariants.

Listing 16.1 Rust example

```
pub struct RetryPolicy {
    attempts: u32,
    base_delay_ms: u64,
}

#[derive(Debug, PartialEq, Eq)]
pub enum PolicyError {
    ZeroAttempts,
}

impl RetryPolicy {
    pub fn new(attempts: u32) -> Result<Self, PolicyError> {
        if attempts == 0 {
            return Err(PolicyError::ZeroAttempts);
        }
        Ok(Self { attempts, base_delay_ms: 100 })
    }

    pub fn with_base_delay_ms(mut self, delay: u64) -> Self {
        self.base_delay_ms = delay;
        self
    }

    pub fn attempts(&self) -> u32 { self.attempts }

    pub fn delay_for(&self, retry: u32) -> u64 {
        self.base_delay_ms.saturating_mul(u64::from(retry))
    }
}
```

The constructor establishes nonzero attempts. The builder-style method adjusts a value that has no additional validation here. The representation can later gain internal bookkeeping without requiring callers to provide it. This freedom does not mean every behavioral change is compatible. If callers rely on documented delay semantics, changing the formula can still be disruptive even though no signature changes.

Getters can also create commitments. Returning &Vec<T> exposes a specific container where &[T] may express the intended read-only sequence more narrowly. Returning an iterator can hide representation but introduces iterator capability and lifetime decisions. Choose the least specific return type that remains convenient for real use cases, then document meaningful guarantees such as ordering and allocation.

Trait Implementors Are API Users Too

Adding a required method to a public trait breaks downstream implementations. Adding a default method avoids that particular problem but can still introduce method-name ambiguity or change dyn compatibility. Tightening a bound can exclude implementations or callers that previously worked. A trait is an extension point with two audiences: consumers and implementors.

If external implementations are not part of the intended contract, seal the trait. A private supertrait can prevent outside crates from satisfying the required relationship while leaving the public trait usable as a bound.

Listing 16.2 Rust example

```
mod private {
    pub trait Sealed {}
}

pub trait WireInteger: private::Sealed + Copy {
    fn append_be(self, output: &mut Vec<u8>);
}

impl private::Sealed for u16 {}
impl private::Sealed for u32 {}

impl WireInteger for u16 {
    fn append_be(self, output: &mut Vec<u8>) {
        output.extend_from_slice(&self.to_be_bytes());
    }
}

impl WireInteger for u32 {
    fn append_be(self, output: &mut Vec<u8>) {
        output.extend_from_slice(&self.to_be_bytes());
    }
}

pub fn encode<T: WireInteger>(value: T) -> Vec<u8> {
    let mut output = Vec::new();
    value.append_be(&mut output);
    output
}
```

The private module is inaccessible to downstream crates, so they cannot name and implement the sealing trait. The public trait can still describe

the supported integer family. This is useful when the library needs generic convenience over a controlled set, not an open extension mechanism.

Sealing is a policy choice, not a default requirement. If users need to integrate custom backends, sealing the backend trait defeats the purpose. A smaller open core trait plus sealed convenience traits can provide a better balance. Document whether external implementation is supported, and test an external-style implementation in a separate crate when that promise matters.

Exhaustiveness Is a Choice About Future Growth

A public enum lets downstream users match every variant. That is useful when the set is intentionally complete. If the library expects to add variants in compatible releases, non_exhaustive requires external callers to include a fallback from the beginning.

Listing 16.3 Rust example

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[non_exhaustive]
pub enum CacheOutcome {
    Hit,
    Miss,
    Bypassed,
}

pub struct CacheReport {
    outcome: CacheOutcome,
    bytes: usize,
}

impl CacheReport {
    pub fn outcome(&self) -> CacheOutcome { self.outcome }
    pub fn bytes(&self) -> usize { self.bytes }
}
```

The example declares an extensible outcome set. A downstream match must decide what unknown future outcomes mean. That fallback is itself a semantic responsibility. If every new outcome requires a distinct response for correctness, claiming extension compatibility through a wildcard may merely hide necessary migration work. Nonexhaustiveness provides syntactic room; it does not make all future behavior compatible.

The same principle applies to structs and variants marked non_exhaustive. Consider the caller's construction and matching needs before applying the attribute. Adding it later can break code that previously relied on exhaustive construction or matching, so reserve extension space when the type first becomes public.

Auto Traits Are Observable Commitments

A type's Send and Sync behavior can matter even if the documentation never mentions threads. Downstream code may move the type into a worker or store it behind Arc. Adding an Rc field can remove Send and Sync and break those users. Replacing an internal pointer or marker can similarly alter auto-trait behavior.

For types intended to be transferable, add compile-time assertions in the library's meaningful validation suite. These assertions document an actual compatibility promise. For a type intended to remain thread-confined, encode that constraint deliberately rather than relying on an accidental current field.

Listing 16.4 Rust example

```
pub struct Snapshot {
    data: std::sync::Arc<[u8]>,
}

fn assert_send_sync<T: Send + Sync>() {}

fn verify_contract() {
    assert_send_sync::<Snapshot>();
}
```

The function body is checked even if it is not executed. This verifies the type relationship for the current implementation. It does not test runtime thread safety beyond the promises of the constituent abstractions. Unsafe implementations of auto traits require an independent soundness argument.

Other observable capabilities include Unpin, unwind-safety markers, and trait implementations such as Clone, Eq, or AsRef. Adding an implementation is often useful, but broad blanket relationships can interact with downstream code. Removing one is ordinarily a direct loss of

capability. Review implementations as part of the public surface, not as harmless decoration around a struct.

Opaque Returns Still Expose Lifetime Relationships

Return-position impl Trait hides one concrete return type while exposing declared bounds. Callers can depend on whether the value borrows an input, whether it is Send, and which generic parameters the opaque type captures. Edition 2024 capture rules make this especially important when designing lifetime-sensitive APIs.

A function that returns an iterator over an input slice naturally needs to capture that input lifetime. A function that returns an independent computed value may want to avoid unnecessarily tying the result to unrelated borrows. Precise capture syntax can express this, but it is a commitment: changing the set of captured lifetimes can change how long callers must keep inputs alive.

The following edition 2024 example explicitly captures only the lifetime used by the returned iterator:

Listing 16.5 Rust example

```
fn selected<'a, 'b>(
    values: &'a [u32],
    _label: &'b str,
) -> impl Iterator<Item = &'a u32> + use<'a> {
    values.iter().filter(|value| **value % 2 == 0)
}

fn main() {
    let values = [1, 2, 3, 4];
    let iter;
    {
        let label = String::from("even");
        iter = selected(&values, &label);
    }
    assert_eq!(iter.copied().collect::<Vec<_>>(), [2, 4]);
}
```

The result does not borrow label. The explicit capture list makes that relationship part of the interface. If the implementation later tries to retain label, it must change the contract rather than silently imposing a longer borrow. When type and const parameters are present, precise capture lists

have additional requirements; consult the current Reference rather than generalizing from this lifetime-only example.

Opaque return types also preserve their concrete auto traits. If an implementation changes from a Send iterator to a thread-confined one, callers can break even if the written return bounds look unchanged. A named wrapper type can sometimes provide a more deliberate compatibility boundary for substantial public iterators or futures.

Features Should Compose Under Dependency Unification

Cargo features are usually additive configuration. Several dependencies may enable different features of the same crate, and the final build can receive their union. Designing features as mutually exclusive modes can therefore create conflicts far from the library author. A default feature is not a global switch that an application can reliably disable if another dependency enables it.

Prefer features that add capabilities or integrations while preserving the meaning of existing APIs. If a feature changes an algorithm, ensure the combined configuration is meaningful and document observable behavior. If two backends truly cannot coexist, separate packages or an explicit runtime selection may communicate the constraint more honestly than hidden feature interactions.

Feature testing needs a model of supported combinations. Testing only all-features can miss a missing import in the minimal configuration. Testing only defaults can miss integration conflicts. A small matrix covering the minimal build, default build, important independent integrations, and their supported union often provides better evidence than an arbitrary collection of combinations.

The minimum supported Rust version is another compatibility dimension. Declare it in the manifest when maintaining such a policy, and test it. A dependency update can raise the effective minimum even when the library's own syntax is unchanged. The policy should match what users can reasonably depend on, rather than merely what the current developer workstation happens to compile.

Documentation Must Explain Costs and Failure Behavior

Rustdoc shows signatures well. It cannot infer ordering guarantees, lock behavior, allocation patterns, or the postcondition after an error. Describe these when they affect user decisions. A method named snapshot might clone all data, increment an Arc, or hold a lock-backed view. Those designs have very different latency and lifetime implications.

Examples should exercise the intended path through the API, including construction, operation, failure handling, and cleanup where relevant. Compileable documentation examples are especially useful because they validate that the public surface can express its advertised workflow without private helpers. They should not rely on hidden imports or impossible setup.

Avoid documenting incidental internals as guarantees unless users need them. Promising the exact growth factor of a vector or the exact number of allocations can make future optimization unnecessarily difficult. On the other hand, hiding a linear scan behind a method users expect to be constant time creates performance traps. State complexity and ownership behavior at the level that supports sound application design.

Training Encounters and Debriefs

Exercise 1. A public struct has two public fields and no private fields. Why can adding a third private field break callers?

Solution. External callers may currently construct it with a struct literal or use exhaustive patterns. Adding a private field prevents those operations. Constructors and an initially private representation would have preserved more evolution freedom. Compatibility includes construction and matching, not just method calls.

Exercise 2. A public trait has a default method added that is generic over T. Could this break Box<dyn Trait> users?

Solution. Yes, if the new method is not excluded from dispatch with an appropriate Self: Sized bound, it can make the trait non-dyn-compatible. A default body does not remove the compatibility restriction. Trait evolution must consider consumers using erased values as well as external implementors.

Exercise 3. A crate has features fast and accurate that choose incompatible definitions of the same function. Why is this fragile?

Solution. Different dependencies can enable both features through feature unification. The combined build must have a defined supported meaning, or users encounter conflicts they did not directly choose. Additive capabilities, separate packages, or explicit policy values usually compose more reliably.

Further Reading

• Cargo SemVer compatibility catalogs observable compatibility changes in Rust APIs.

• Cargo features explains additive feature behavior and unification.

• Rust Edition Guide on opaque type capture describes edition 2024 capture behavior.

• Rust Reference on non_exhaustive specifies downstream construction and matching restrictions.

The Semver Mimic

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

A registry exposes an opaque Registry type, a borrowed entries iterator, and a trait implemented by external plugins. One client moves Registry to a worker thread. Another retains an entry view while reading another entry. A third implements the plugin trait without importing your dependency types. A fourth stores plugins behind an erased interface. The proposed release adds thread-local scratch storage, changes the iterator's capture behavior, and adds optional metadata to the plugin model. Produce a release review with an explicit compatibility decision for each client. Propose a design that preserves the intended clients while allowing the storage engine to change. You may choose a major release for a genuine contract change, but you must name the exact capability that changes.

Encounter Constraints

• Do not equate unchanged method names with source compatibility.

• Keep the proposed stable client checks small enough to explain which promise each one protects.

• An opaque return type still exposes its bounds and lifetime relationships.

Hint one / Hint two / Reveal debrief

Side Quest

Prepare two release notes for the same internal refactor. In the first, all intended client capabilities survive; describe only the observable cost or behavior changes that matter. In the second, the iterator intentionally becomes lending and clients may no longer retain multiple views. Explain the migration in terms of storage reuse and retention, and show where an owned adapter can preserve the old workflow at an explicit copying cost.

Capability unlocked Contract Ledger reveals promises hidden outside the printed signature.

Choose Your Exit

Enter the release workshop / Level 41
Carry compatibility claims into feature matrices and packaged artifacts.

Inspect erased machinery / Level 10
Revisit which capabilities survive when a concrete implementation is hidden.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 17

The Workshop of Borrowed Spells

The Foundry's maintenance scripts repeat the same resource ritual: acquire a guard, perform a calculation, then release the resource on every ordinary scope exit. Someone offers a macro that removes the repetition. Its first demonstration looks wonderful; its second evaluates an expensive acquisition twice, and its third changes where return goes. You are asked to build a smaller spell whose expansion has a contract a reviewer can actually inspect.

Mission Design a resource-scoping macro whose evaluation, control flow, and cross-crate names are deliberate.

Suggested loadout Level 6, Level 11, Level 12, Level 16. These are guidance, not locked gates.

Mastery Targets

• The acquisition expression runs exactly once and may return a non-Copy guard.

• The macro documents how question-mark propagation and early return interact with the caller.

• A downstream test succeeds without depending on imports present only in the defining crate.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 17

Macros as Controlled Code Generation

Generate Repetition Without Hiding the Contract

A macro operates on Rust syntax and produces more Rust syntax. The generated program still has to satisfy name resolution, type checking, borrowing, and all ordinary validity requirements. Macros are useful when the repeated structure is syntactic: a family of implementations, a small domain notation, or a declaration that should generate several mutually consistent items. They are less useful when an ordinary function can express the same ownership and control-flow behavior clearly.

The strongest reason to introduce a macro is often consistency. A command table can generate both an enum and its numeric decoder from one list. A test matrix can generate named cases with the same setup. A tracing macro can capture source locations and format arguments without evaluating disabled arguments. These operations need access to syntax or call-site context that a normal function does not receive.

The cost is a second language at the API boundary. Users must understand the macro's grammar, expansion behavior, diagnostics, and dependencies. A macro that saves six lines but obscures a move or repeats a side effect may make code harder to review. Design the generated Rust first, then decide whether a macro makes that structure easier to maintain.

Treat the expansion as the real implementation. Review which expressions run, how many times they run, which names are introduced, which paths are resolved, what gets moved, and where control flow goes. A clean invocation cannot compensate for an unclear expansion.

Declarative Matching Recognizes Structured Fragments

macro_rules matches token patterns. Fragment specifiers such as expr, ty, ident, and tt determine what kind of syntax a metavariable accepts. Repetition describes a list of matched fragments and how that list is reproduced. The matcher does not perform arbitrary semantic analysis of the fragment's type.

Listing 17.1 Rust example

```
macro_rules! pairs {
    ($($key:expr => $value:expr),* $(,)?) => {{
        let mut result = Vec::new();
        $(result.push(($key, $value));)*
        result
    }};
}

fn main() {
    let mut next = 0;
    let values = pairs! {
        "first" => { next += 1; next },
        "second" => { next += 1; next },
    };
    assert_eq!(values, [("first", 1), ("second", 2)]);
    assert_eq!(next, 2);
}
```

The matcher accepts zero or more key-value pairs and an optional trailing comma. The expansion is a block expression whose last expression is the vector. Each key and value expression appears exactly once in the generated pushes. The execution order follows the generated statements, so the example makes the side effects visible.

This macro deliberately uses Vec::new rather than precomputing capacity by evaluating arguments. Counting tokens is possible, but an optimization should not accidentally evaluate user expressions an extra time. A capacity hint is less important than preserving semantics. If a macro promises a particular evaluation order, keep tests with side-effecting inputs so later rewrites preserve that promise.

Fragment choices affect grammar evolution. An expr fragment is parsed as an expression, not an arbitrary token sequence that another rule can inspect freely. A tt fragment is more flexible but transfers more parsing responsibility to the macro. Use a narrow fragment when the API expects ordinary Rust syntax; use token trees when the macro genuinely defines another notation.

Evaluation Count Is Part of the Public API

A naive macro can substitute an expression twice and thereby duplicate I/O, mutation, allocation, or ownership transfer. The problem is not specific to side effects. Repeating a non-Copy expression can cause a move error, and borrowing it twice can extend lifetimes unexpectedly.

Listing 17.2 Rust example

```
macro_rules! inspect_value {
    ($expression:expr) => {{
        let value = $expression;
        eprintln!("{} = {:?}", stringify!($expression), &value);
        value
    }};
}

fn main() {
    let mut calls = 0;
    let result = inspect_value!({
        calls += 1;
        String::from("payload")
    });
    assert_eq!(calls, 1);
    assert_eq!(result, "payload");
}
```

The temporary binding captures the result once. Formatting borrows that result, and the final expression moves it back to the caller. This is useful behavior, but it also imposes a Debug requirement. An inspection macro changes the set of accepted expressions even when it appears observational. It may also perform blocking output or trigger a formatting panic. Document and name such behavior honestly.

Temporary variables can influence drop timing. A macro that stores an intermediate in a local block may keep it alive until the block ends, whereas an inlined expression might drop temporaries at another boundary. This matters for lock guards, borrow guards, and resources whose destructors have observable effects. Review a macro invocation involving a guard by expanding its scope mentally or inspecting actual expansion output.

Control-flow tokens also belong to the caller's program. A return inside a captured block can return from the enclosing function rather than from an imagined macro function. The question mark propagates according to the

generated context. If a macro wraps user code in a closure, it changes these relationships. Such a wrapper is a semantic decision, not merely an implementation convenience.

Hygiene Prevents Some Collisions but Does Not Solve Every Name

Declarative macros use mixed-site hygiene. Local bindings introduced by a macro do not simply capture arbitrary caller variables with the same spelling. Other names and paths still need careful treatment. Exported macros must work when invoked from another crate whose imports differ from the author's test module.

Use $crate to refer to the defining crate, and use sufficiently explicit paths for standard items. The referred helper must still be visible to the downstream expansion; $crate does not bypass privacy. This creates an important public implementation surface for exported macros.

Listing 17.3 Rust example

```
#[doc(hidden)]
pub mod __macro_support {
    pub fn checked_sum(values: &[u64]) -> Option<u64> {
        values.iter().try_fold(0_u64, |sum, value| {
            sum.checked_add(*value)
        })
    }
}

#[macro_export]
macro_rules! checked_sum {
    ($($value:expr),* $(,)?) => {{
        $crate::__macro_support::checked_sum(&[$($value),*])
    }};
}

fn main() {
    assert_eq!(checked_sum!(2, 3, 5), Some(10));
    assert_eq!(checked_sum!(u64::MAX, 1), None);
}
```

The helper is public because external expansions need to call it. doc(hidden) keeps it out of ordinary generated documentation, but it does not make it private or remove compatibility considerations. A published macro and its helper paths must evolve together. Removing the helper can

break users of an older expansion source even if the ordinary public API appears unchanged.

The macro evaluates each argument once and borrows the array for the helper call. The helper's signature fixes the element type as `u64`. Rust 1.99.1 promotes both arrays in this example to static storage. Runtime arguments can need temporary storage. Measure large expansions to assess their stack cost.

Generate Correlated Declarations From One Table

A macro can prevent two manually maintained declarations from drifting apart. A command enum and its decoder should agree on numeric codes. The following complete example generates both from one table of fieldless variants.

Listing 17.4 Rust example

```
macro_rules! commands {
    ($name:ident { $($variant:ident = $code:literal),+ $(,)? }) => {
        #[derive(Debug, Clone, Copy, PartialEq, Eq)]
        #[repr(u8)]
        enum $name {
            $($variant = {
                let code: u128 = $code;
                ::core::assert!(code <= u8::MAX as u128);
                code as u8
            }),+
        }

        impl TryFrom<u8> for $name {
            type Error = u8;

            fn try_from(value: u8) -> Result<Self, Self::Error> {
                match value {
                    $(v if v == Self::$variant as u8 => Ok(Self::$variant),)+
                    unknown => Err(unknown),
                }
            }
        }
    };
}

commands! {
    Command {
        Ping = 1,
        Fetch = 2,
        Close = 9,
```

Listing 17.4 continued

```
    }
}

fn main() {
    assert_eq!(Command::try_from(2), Ok(Command::Fetch));
    assert_eq!(Command::try_from(7), Err(7));
    assert_eq!(Command::Close as u8, 9);
}
```

The generated enum's explicit representation makes the numeric cast meaningful for these fieldless variants. The decoder validates incoming bytes rather than transmuting them into an enum. Unknown discriminants remain ordinary errors. The macro does not turn an arbitrary byte into a valid enum value merely because the representation is u8.

The compiler rejects duplicate discriminants. The enum reads each code as `u128` and checks the byte range before narrowing it. The const assertion rejects `256` even if `overflowing_literals` is allowed. The decoder compares incoming bytes with the checked enum values.

Use fieldless variants with unsuffixed nonnegative integer codes from 0 to 255. The `u128` context also accepts an explicit `u128` suffix. Payload variants, expressions, attributes, visibility, and generics would complicate the contract. A small declaration language is easier to document, test, and maintain.

Procedural Macros Receive Tokens Rather Than Typed Programs

A procedural macro runs during compilation and transforms token streams. Derive macros inspect the annotated item's syntax and emit additional items. Attribute macros can replace or extend an item. Function-like procedural macros accept tokens in a macro invocation. These mechanisms provide more flexible parsing than macro_rules, but they do not generally receive the compiler's fully resolved type information.

A derive macro can see that a field is written as SomeType<T>. It cannot generally know that SomeType is an alias for a particular external type or prove which trait implementations T will have after name resolution.

Generate appropriate bounds and let type checking establish them. Trying to infer semantic identity from token spelling is fragile under imports, aliases, and renamed dependencies.

Procedural macros live in a proc-macro crate. That package arrangement is part of the build architecture. A library often reexports its derive macro from an ordinary runtime crate while sharing names and support paths. The generated code must account for crate renaming and optional integrations. Hard-coding a dependency's expected import name can fail when downstream users rename that dependency in Cargo.toml.

Parsing token streams by converting them to strings and splitting on punctuation is rarely robust. Rust syntax includes nested groups, attributes, generics, lifetimes, raw identifiers, and tokens whose spacing is significant. A parser library can provide an appropriate syntax tree, but it adds compile-time dependencies and versioning work. Use the smallest parsing strategy that correctly handles the documented input grammar.

Diagnostics Are a Major Part of Macro Usability

A compiler error inside generated code can be technically correct while leaving the user unable to identify the invalid input. Preserve useful spans when generating tokens and emit targeted errors for unsupported syntax. A derive macro that rejects unions should say that the derive supports structs and enums, rather than generating an invalid impl and letting a later type error obscure the reason.

Do not panic for ordinary user mistakes in macro input. A procedural macro panic reports a tool failure rather than a well-formed validation error. Emit compile_error or a structured diagnostic strategy supported by the chosen stable tooling. Reserve internal panics for genuinely unreachable macro implementation defects, and still prefer an actionable message when possible.

For declarative macros, a final catch-all arm can provide a concise grammar hint, but it should not accidentally swallow a more specific diagnostic. Match rule order matters. Broad token-tree rules placed before precise cases can accept input the author intended to reject. Test malformed invocations as carefully as successful ones.

Editor support is another practical cost. Generated methods may be discoverable through expansion-aware tooling, but a custom notation can weaken ordinary completion and navigation. A macro API should earn that cost by eliminating meaningful duplication or enabling a capability normal Rust syntax cannot express conveniently.

Test Expansion Semantics Across Crate Boundaries

Unit tests in the defining module do not adequately test an exported macro. They may accidentally provide imports, private visibility, or helper names that downstream crates lack. An integration test should invoke the macro from an external-style context with minimal imports and potentially conflicting local names.

Test ownership behavior with non-Copy inputs. Test evaluation count with a counter. Test empty input and trailing separators if supported. Test generic types and lifetimes when the grammar claims to support them. Compile-fail cases should verify rejected syntax and missing bounds. For a derive, include renamed dependencies or reexports if those are supported workflows.

Do not make every expansion detail part of the stable contract. A snapshot of generated tokens can help maintainers inspect changes, but callers usually care about generated items and behavior. Tests should distinguish internal formatting from public semantics. A harmless token formatting change should not distract from a changed ownership requirement.

Macro dependencies execute during the build and can increase compilation time significantly. Share heavy parsing work within one macro crate, avoid generating unnecessarily large token streams, and measure build impact for common use cases. A macro that generates hundreds of nearly identical implementations may be appropriate, but the cost should correspond to real user value.

Editions Affect Macro Grammars

Macro fragment behavior can evolve with editions. In edition 2024, expr matches additional expression forms, while expr_2021 preserves the older fragment behavior. The relevant edition is associated with the macro definition, not simply whichever edition the caller uses. A macro crate's edition migration can therefore change which rule matches a downstream invocation.

When migrating a macro crate, review rule precedence and ambiguous patterns rather than assuming ordinary source compatibility is enough. A new expression form accepted by an earlier rule can prevent a later specialized rule from matching. Keep tests containing syntax near the grammar boundary, especially if the macro uses tokens such as const as part of its own notation.

This is another reason to prefer simple, explicit grammars. A macro with familiar separators and narrow fragment expectations is easier to evolve than one whose meaning depends on subtle competition between several broad patterns. Language extension should reduce repetition while preserving the reader's ability to reason from the invocation to the generated program.

Training Encounters and Debriefs

Exercise 1. A macro expands check!(make_value()) into make_value().is_valid() followed by make_value().save(). What is wrong?

Solution. The expression runs twice, potentially creating different values or duplicating side effects. Bind it once inside the expansion, then operate on the binding with appropriate ownership. Decide whether validation borrows or consumes it, and test with a counter and a non-Copy value.

Exercise 2. An exported macro calls helper() successfully in unit tests but fails downstream. What should be checked?

Solution. The expansion may rely on the defining module's imports or private helper visibility. Use $crate with the correct helper path, ensure external expansions can access it, and test from an integration crate with minimal imports. Hygiene does not automatically qualify every item reference or bypass privacy.

Exercise 3. A procedural derive wants different output when a field's type is an alias for Vec<u8>. Can token spelling reliably establish that fact?

Solution. No. The macro sees syntax before full type resolution. Aliases, imports, and reexports can hide the semantic identity. Require an explicit attribute, generate a trait-based obligation, or redesign the derive around syntax it can actually recognize. Do not pretend token inspection is type introspection.

Further Reading

• Rust Reference on macros by example specifies matching, repetition, hygiene, and $crate.

• Rust Reference on procedural macros describes token streams, macro crate forms, and hygiene limitations.

• Rust Edition Guide on fragment specifiers explains edition 2024 expression matching changes.

The Mirror Expansion

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Specify and implement a declarative scoped_resource macro for a fallible guard acquisition and a user block. Its purpose is to name the acquired guard inside the block and return the block's result. Choose whether early return and question-mark propagation act in the surrounding function or in an introduced closure; either policy is acceptable if the syntax and documentation make it clear. Test a side-effecting acquisition, a guard that is not Copy, a body that returns an error, and a body that tries to return a reference to the local guard. Finally, use the macro from a separate test crate with no convenient helper imports. Present the expansion of one invocation so another developer can verify the ownership story.

Encounter Constraints

• Use macro_rules rather than a procedural macro for this challenge.

• Do not introduce an invisible second acquisition or require Clone to hide an ownership error.

• Releasing a local guard does not promise that remote asynchronous cleanup completed.

Hint one / Hint two / Reveal debrief

Side Quest

Make an expansion inspection checklist for a teammate's macro. Include argument evaluation order, temporary lifetime, introduced bindings, move behavior, propagation targets, helper visibility, and diagnostics for malformed syntax. Then find one convenience you would deliberately leave out of scoped_resource because supporting it would make the expansion harder to predict than the repeated source code it replaces. Explain that trade in a short API note.

Capability unlocked Expansion Lens exposes the program underneath a convenient invocation.

Choose Your Exit

Freeze part of the spell / Level 18
Compare token generation with actual compile-time execution.

Audit the contract / Level 42
Review generated ownership and failure behavior through explicit invariants.

Mark your quest log / Return to the map

ZONE 2 / THE TRAIT FOUNDRY / LEVEL 18

The Frozen Signal Vault

Ferrum's signal vault manufactures fixed-capacity frame cartridges. Capacity is selected when a device is built; the number of useful bytes changes with every message. An apprentice encoded both facts as the same const parameter and now needs a different type for every packet arriving from the street. You must reopen the vault with a representation that separates manufactured capacity, runtime content, and the wire format's fixed-width length field.

Mission Use const generics for a genuine type-level capacity while keeping dynamic length and checked encoding at runtime.

Suggested loadout Level 2, Level 11, Level 15. These are guidance, not locked gates.

Mastery Targets

• One Cartridge<CAP> value accepts several payload lengths up to its capacity without changing type.

• Construction or insertion preserves the initialized-byte and used-length invariant for all supported CAP values.

• Wire length checks do not depend on host pointer width or unsupported arithmetic in array types.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 18

Constant Evaluation and Const Generic Design

Compile Time Is an Execution Environment With Rules

Rust can evaluate selected expressions during compilation. A const context requires this evaluation; an ordinary runtime context may merely offer an optimization opportunity. Understanding that distinction prevents two common mistakes: assuming const fn always runs during compilation, and assuming an optimizer's ability to fold an expression makes it legal in a const context.

Constant evaluation uses the language's compile-time rules, with restrictions on operations, references, destruction, and generic expressions. It is not a general permission to run arbitrary application code during the build. Reading a configuration file, consulting a network service, or allocating an unrestricted runtime object graph belongs to another mechanism. The fact that a build script can execute those operations does not make them constant evaluation.

The useful applications are structural. Array lengths belong in types. Lookup tables can be derived from a small definition. Fixed protocol constants can be validated before a program is built. Const generics let one implementation work across a family of compile-time sizes. These capabilities can remove runtime checks or represent useful distinctions, but they can also multiply type instantiations and complicate APIs.

Ask which property genuinely needs to be known during compilation. A packet buffer's capacity may justify a const parameter in an embedded system with fixed storage. A user-selected batch size usually belongs in a runtime value. Moving configuration into a type changes where choices can be made and how values can be stored together.

Const Functions Can Also Run at Runtime

A const fn promises that its body is suitable for constant evaluation under the applicable rules. Calling it in an ordinary function does not require compile-time execution. The same function can validate a literal configuration during compilation and process dynamic input later.

Listing 18.1 Rust example

```
const fn parse_port(input: &[u8]) -> Option<u16> {
    if input.is_empty() {
        return None;
    }
    let mut value = 0_u16;
    let mut index = 0;
    while index < input.len() {
        let byte = input[index];
        if byte < b'0' || byte > b'9' {
            return None;
        }
        value = match value.checked_mul(10) {
            Some(value) => value,
            None => return None,
```

Listing 18.1 continued

```
        };
        value = match value.checked_add((byte - b'0') as u16) {
            Some(value) => value,
            None => return None,
        };
        index += 1;
    }
    Some(value)
}

const DEFAULT_PORT: u16 = match parse_port(b"8080") {
    Some(port) => port,
    None => panic!("invalid default port"),
};

fn main() {
    assert_eq!(DEFAULT_PORT, 8080);
    let dynamic = String::from("65535");
    assert_eq!(parse_port(dynamic.as_bytes()), Some(65535));
    assert_eq!(parse_port(b"65536"), None);
}
```

The parser uses checked arithmetic, so its failure behavior is explicit in both environments. DEFAULT_PORT must be evaluated during compilation. The call using dynamic input executes as part of the program unless optimization can prove otherwise. The const modifier does not require duplicating the algorithm or changing its error model.

The function avoids the question mark and generic parsing traits because its compile-time interface is intentionally simple. The set of const-capable operations evolves, so examples intended for a supported compiler range should use operations that range actually supports. A stable runtime method being available does not automatically mean it is callable in a const fn.

A failed assertion in a required constant is a build error. In an ordinary runtime call, a panic follows the runtime panic strategy. This difference is useful for validating fixed configuration, but it should not turn user input into a requirement to rebuild the application.

Constants Are Values and Statics Are Storage

A const item names a value expression that can be used in many places. It is not a promise of one unique address for that value. A static item names a storage location with program-wide duration and additional rules about its type and initialization. Choose a static when shared identity or a stable storage location is part of the design.

This distinction matters for caches and synchronization. A const containing a constructible synchronization object can produce separate instances at different use sites rather than one shared lock. A static synchronization object identifies shared state. Conversely, a small numeric constant normally needs no storage identity and is appropriately a const.

Both const and static initializers involve constant evaluation, but their reference and interior-mutability rules differ in important ways. Avoid reasoning from superficial syntax such as an ampersand in an initializer. A reference must still satisfy validity and lifetime requirements, and compile-time pointer operations have restrictions that do not mirror arbitrary runtime address arithmetic.

For address identity among statics, use distinct statics of nonzero size. An immutable static can share an address with a promoted or constant value. Two uses of a constant can also share storage. Comparing the addresses of promoted values is therefore unsuitable as an identity protocol. Use an explicit identifier if identity must survive the lifetime of an allocation or be compared across program runs.

Derive Tables From One Algorithm

A compile-time lookup table can make a runtime operation simple without requiring a manually maintained list. The table's derivation remains ordinary code that can be reviewed and tested.

Listing 18.2 Rust example

```
const fn hex_table() -> [u8; 256] {
    let mut table = [u8::MAX; 256];
    let mut digit = 0;
    while digit < 10 {
        table[b'0' as usize + digit] = digit as u8;
        digit += 1;
    }
```

Listing 18.2 continued

```
    let mut letter = 0;
    while letter < 6 {
        table[b'a' as usize + letter] = (10 + letter) as u8;
        table[b'A' as usize + letter] = (10 + letter) as u8;
        letter += 1;
    }
    table
}

const HEX: [u8; 256] = hex_table();

fn decode_hex(input: &[u8]) -> Option<Vec<u8>> {
    if input.len() % 2 != 0 {
        return None;
    }
    let mut output = Vec::with_capacity(input.len() / 2);
    for pair in input.chunks_exact(2) {
        let high = HEX[pair[0] as usize];
        let low = HEX[pair[1] as usize];
        if high == u8::MAX || low == u8::MAX {
            return None;
        }
        output.push((high << 4) | low);
    }
    Some(output)
}

fn main() {
    assert_eq!(decode_hex(b"0aFF"), Some(vec![10, 255]));
    assert_eq!(decode_hex(b"0x"), None);
    assert_eq!(decode_hex(b"a"), None);
}
```

The sentinel is outside the valid nibble range. Every input byte safely indexes the 256-entry table because converting u8 to usize produces a value from zero through 255. The table converts syntax classification into one lookup, but whether that is faster than arithmetic or matching depends on the workload and target. Compile-time generation guarantees consistency, not performance superiority.

The decoder allocates output at runtime because its length depends on runtime input. Its capacity calculation is bounded by the input length, and it rejects odd-length input before processing pairs. If invalid input appears late, an allocated partially filled vector is dropped. The API returns no

partial bytes, which is a useful failure postcondition for this small operation.

The same table can be generated by a build script or stored as a literal, but constant evaluation keeps the derivation close to its use without adding a host executable or file-generation step. For substantially larger generated data, build time, object size, and incremental rebuild behavior may change the best choice.

Const Parameters Make Values Part of Type Identity

Packet<32> and Packet<64> are different types. The compiler can use their sizes when laying out enclosing values and checking array accesses. A const parameter is not merely a constructor argument stored in a field.

Listing 18.3 Rust example

```
#[derive(Debug, PartialEq, Eq)]
struct WrongLength {
    expected: usize,
    actual: usize,
}

struct Packet<const N: usize> {
    bytes: [u8; N],
}

impl<const N: usize> Packet<N> {
    fn from_slice(input: &[u8]) -> Result<Self, WrongLength> {
        let bytes: [u8; N] = input.try_into().map_err(|_| WrongLength {
            expected: N,
            actual: input.len(),
        })?;
        Ok(Self { bytes })
    }

    fn as_slice(&self) -> &[u8] { &self.bytes }

    fn checksum(&self) -> u8 {
        self.bytes.iter().fold(0, |sum, byte| sum.wrapping_add(*byte))
    }
}

fn main() {
    let packet = Packet::<4>::from_slice(&[1, 2, 3, 4]).unwrap();
    assert_eq!(packet.checksum(), 10);
```

Listing 18.3 continued

```
    assert_eq!(packet.as_slice().len(), 4);
    assert!(Packet::<4>::from_slice(&[1, 2]).is_err());
}
```

The type carries exact length after construction. Conversion from a dynamic slice performs the one necessary runtime check. Methods can then rely on the array length without repeatedly validating the same property. This is a good example of moving a dynamic boundary into a statically represented invariant.

The packet does not allocate its byte array separately. It stores the array inline wherever the packet itself lives. A large N can therefore make stack frames or enclosing structures large. Box<Packet<N>> changes storage ownership but not the type's exact-size identity. Avoid presenting const generics as automatically cheaper than a Vec; representation and allocation tradeoffs depend on sizes and lifetimes.

Different N values also complicate heterogeneous storage. A vector cannot directly hold Packet<32> and Packet<64> as one element type. An enum, trait object, or erased byte representation can bridge that boundary. If sizes vary freely at runtime, a single dynamic type is often the simpler architecture.

Arithmetic in Values Is Not the Same as Arithmetic in Types

Stable const generics support many useful size parameters, but arbitrary expressions involving generic parameters in type-level array lengths have restrictions. A function may calculate N + 1 as an ordinary value while an array type [u8; N + 1] is rejected in the supported language subset used here.

Listing 18.4 Rejected example

```
fn extended<const N: usize>() -> [u8; N + 1] {
    [0; N + 1]
}
```

This is a language expressiveness limit, not evidence that the arithmetic is mathematically ambiguous. Supporting general generic constant expressions requires the compiler to track validity and equality of

expressions across generic instantiations. Overflow, evaluation requirements, and normalization all become part of type checking.

Do not work around the restriction by introducing unsafe layout tricks unless the representation independently justifies them. Often the clean design stores a header separately from [u8; N], accepts the total capacity as another const parameter, or uses a runtime Vec. A struct containing header: u8 and body: [u8; N] may express the domain better than an array whose extra element is hidden in arithmetic.

An associated constant can still calculate values from N when used in a permitted value context. That does not automatically make the associated constant usable as a generic array length. Keeping value computation and type formation separate helps interpret compiler diagnostics accurately.

Compile Time Assertions Have a Scope

An inline const block can check a property during the relevant compile-time evaluation. In generic code it can refer to generic parameters under its applicable rules, allowing checks such as a nonzero fixed capacity. The check is valuable when the API intentionally treats an invalid const argument as a programming error.

Listing 18.5 Rust example

```
struct NonEmpty<const N: usize> {
    values: [u8; N],
}

impl<const N: usize> NonEmpty<N> {
    fn new(values: [u8; N]) -> Self {
        const { assert!(N > 0, "capacity must be nonzero"); }
        Self { values }
    }

    fn first(&self) -> u8 { self.values[0] }
}

fn main() {
    let values = NonEmpty::new([7, 8, 9]);
    assert_eq!(values.first(), 7);
}
```

The constructor is the proof boundary. If the struct fields are private outside the implementation's module, callers must pass through the check.

The existence of a generic type name NonEmpty<0> alone is not necessarily rejected merely because its constructor would fail to instantiate. The assertion belongs to the evaluated code path, not to an imaginary universal theorem attached to the type's spelling.

If an API needs a globally valid nonzero capacity representation, consider whether a runtime NonZeroUsize or a different type structure provides a clearer contract. A compile-time assertion can improve diagnostics for fixed configurations, but it should not be described as an unrestricted const-generic where predicate if the language does not provide that predicate.

Also distinguish a checked precondition from an optimization assumption. Safe indexing in first still checks according to the language's rules unless optimization proves the bound. The constructor's assertion gives the programmer a useful invariant, but it does not justify unchecked access in unrelated code without a complete argument about all construction paths.

Build Scripts and Constant Evaluation Solve Different Problems

A build script executes on the host during the build and can produce files, compiler configuration, or link instructions. Constant evaluation computes Rust values under target-aware language rules. Cross compilation makes the distinction concrete: the machine executing the build script may differ from the machine that will run the program.

Do not infer a target's pointer width or byte order from the build script host's ordinary runtime types. Use Cargo's target configuration information and explicit target data when generating code. A generated binary layout that accidentally uses host usize can work locally and fail on another target.

Build scripts also introduce reproducibility concerns. Reading arbitrary environment variables or external files without correctly declaring rebuild dependencies can make stale output appear valid. Network-dependent generation can make builds nonreproducible or unavailable offline. Constant evaluation of a pure table avoids those particular dependencies, although it can still consume substantial compilation resources.

Choose the mechanism that matches the input. Pure fixed Rust data is often suitable for const evaluation. Native library discovery and foreign code generation belong to build tooling. User configuration belongs to runtime loading unless the product intentionally builds a separate binary for each configuration.

Compile Time Work Still Has a Cost

Moving computation into compilation does not remove it from the engineering system. Large tables can increase compile time and binary size. Many const-generic instantiations can increase generated code. A complex const parser may produce difficult build diagnostics. A small runtime calculation executed once at startup may be cheaper overall than a large compile-time framework.

Measure the dimension that matters. Embedded firmware may prioritize zero dynamic allocation and predictable storage. A frequently rebuilt developer tool may prioritize incremental compile time. A server may amortize a small startup cost over days of execution. The same const-generic design can be appropriate in one setting and burdensome in another.

The most durable use of const features is expressing a real structural invariant with a simple implementation. A fixed digest length, a protocol field width, or a small derived classification table communicates meaning and reduces duplicated checks. If the type-level machinery becomes harder to explain than the property, reconsider whether the property belongs in a runtime value.

Training Encounters and Debriefs

Exercise 1. A const fn computes a checksum from a runtime slice. Is the checksum necessarily computed during compilation?

Solution. No. const fn permits constant evaluation when used in a const context and may be optimized elsewhere. A runtime input generally requires runtime execution. The modifier describes eligibility, not a universal scheduling guarantee.

Exercise 2. Why might Packet<N> be a poor representation for messages whose lengths come from the network?

Solution. Each N is a distinct compile-time type, while arbitrary network lengths are runtime data. A dynamic buffer or a bounded buffer with a runtime used length usually matches that variability better. Const generics fit fixed protocol sizes or fixed capacity families, not every dynamic size.

Exercise 3. A build script uses std::mem::size_of::<usize>() to generate the target's file format. What can go wrong?

Solution. The build script runs for the host, whose pointer width may differ from the target. File formats should use explicitly specified widths, and target-dependent generation should consult target configuration rather than host assumptions. Constant evaluation and build-script execution have different environments.

Further Reading

- Rust Reference on constant evaluation specifies const contexts and permitted operations.
- Rust Reference on generic parameters describes const parameter types and expression restrictions.
- Cargo build scripts explains host-side build execution and rebuild instructions.

The Capacity Doppelganger

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design Cartridge<const CAP: usize> for initialized bytes plus a runtime used length. The cartridge must support replacing its payload from a slice, reject oversized input without changing the previous payload, and report the encoded size of a four-byte length prefix plus the used bytes. A payload length that cannot fit the wire's u32 field must be rejected even on a wider machine. Show where a const function is useful for a fixed cartridge configuration and where the same operation executes at runtime for incoming data. Include CAP equal to zero in your reasoning. Your stable design must not rely on an array type written as CAP plus four merely because that arithmetic is valid in an ordinary value expression.

Encounter Constraints

• Use initialized storage such as [u8; CAP]; this level does not require MaybeUninit.

• Perform all rejection checks before mutating an existing accepted payload.

• No build script may infer the wire format from size_of::<usize>().

Hint one / Hint two / Reveal debrief

Side Quest

Give the vault two cartridge capacities and a runtime request that may fit either. Design a dispatcher using an enum or borrowed slice interface so callers do not need a fresh monomorphized type for every incoming length. Compare the cost and public type surface of this approach with always allocating a vector. Record which capacity decisions remain compile-time configuration and which decisions legitimately belong to runtime policy.

Capability unlocked Capacity Compass separates type identity from a changing value.

Choose Your Exit

Descend into initialization / Level 32
Replace fully initialized bytes with a carefully tracked initialized prefix when that is justified.

Ship the cartridge crate / Level 41
Turn compile-time and target assumptions into tested support claims.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 19

The Surveyors Who Share a Horizon

Beyond the Foundry lies the Clockwork Wilds, where several surveyors search the same catalog of place names. The catalog belongs to their coordinator, and the report should borrow the matching names rather than copy every string. Ordinary unscoped workers demand a lifetime the coordinator cannot offer. You can either change who owns the horizon or choose a boundary that proves every surveyor returns before the horizon can disappear.

Mission Build a scoped parallel search that returns borrowed matches in input order without cloning the catalog's strings.

Suggested loadout Level 3, Level 4, Level 8, Level 12. These are guidance, not locked gates.

Mastery Targets

• The result borrows from the caller's catalog and remains usable after all scoped workers join.

• Worker-local results are combined in original input order despite completion-order differences.

• The design handles an empty input and states its policy for zero workers and worker panic.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 19

Thread Boundaries as Type Boundaries

Concurrency changes the meaning of ownership. Inside one thread, a value may contain assumptions about the thread that manipulates it, the references that alias it, or the mechanism that counts its owners. Moving that value to another thread can invalidate those assumptions even though the move itself copies no application data. Rust therefore asks two separate questions: may ownership cross a thread boundary, and may access through a shared reference cross that boundary? The answers are represented by `Send` and `Sync`.

These traits are useful beyond getting a thread example to compile. They let a library publish a concurrency contract without mentioning a particular scheduler. A type can be usable from a worker thread, shareable between workers, neither, or both. Understanding the distinction prevents the common mistake of adding `Arc` and `Mutex` until an error disappears without understanding what the resulting architecture permits.

Transfer and Shared Access

`T: Send` means transferring ownership of a `T` between threads is safe. It does not mean a particular value is already shared, and it does not promise that methods are lock free. A `String` is `Send`: one thread can finish using it, transfer it, and let another thread mutate or destroy it. Its allocation need not be copied. The ownership handoff ensures that both threads do not independently own the same unique buffer.

`T: Sync` means that a shared reference to `T` can be transferred safely between threads. An equivalent useful formulation is that `&T` is `Send` when `T` is `Sync`. This describes what shared access allows. A shared reference is not automatically a promise of physical immutability because interior mutability exists. It is a promise that the operations available through that reference uphold Rust's safety requirements, including under concurrent use.

The traits are automatically derived from a type's structure where appropriate. A wrapper containing only thread safe components will usually receive the corresponding implementations without annotations.

Conversely, adding a field such as `Rc<T>` can remove an auto trait from an otherwise ordinary wrapper. This structural behavior makes private representation relevant to public API compatibility: downstream code may rely on an auto trait even when no explicit implementation appears in source.

Here is a complete program that separates transferring an owned buffer from sharing an immutable one.

Listing 19.1 Rust example

```
use std::thread;

fn main() {
    let owned = String::from("move this allocation");
    let length = thread::spawn(move || owned.len())
        .join()
        .unwrap();
    assert_eq!(length, 20);

    let values = vec![2_u64, 4, 6, 8];
    thread::scope(|scope| {
        let left = scope.spawn(|| values[..2].iter().sum::<u64>());
        let right = scope.spawn(|| values[2..].iter().sum::<u64>());
        assert_eq!(left.join().unwrap() + right.join().unwrap(), 20);
    });
    assert_eq!(values.len(), 4);
}
```

The first closure owns its string. The scoped closures borrow the vector and may read it concurrently. Their bodies finish before the scope returns, so those borrows cannot escape. Automatic joining by the scope may return while thread-local destructors are still running.

Why Static Is Often Misread

`thread::spawn` requires its closure and result to satisfy `'static` bounds. This does not mean the closure must run forever or that every captured allocation must be leaked. It means the captured value must not contain a borrowed reference whose validity expires too early. An owned `String` ordinarily satisfies `'static` because the string manages its own allocation; dropping it tomorrow or a microsecond later is compatible with that bound.

An owned wrapper can still fail the bound if one of its fields borrows short lived data. Moving `View<'a>` moves the wrapper, not the storage referred to by `'a`. The compiler cannot turn a borrowed resource into an owned resource merely because the closure is marked `move`.

The next example deliberately fails. It is valuable because the closure uses `move` and still cannot satisfy the lifetime requirement.

Listing 19.2 Rejected example

```
use std::thread;

fn main() {
    let text = String::from("borrowed");
    let view = text.as_str();
    let handle = thread::spawn(move || view.len());
    assert_eq!(handle.join().unwrap(), 8);
}
```

Although this program joins immediately, the signature of `spawn` does not encode a borrowing scope tied to that join. `JoinHandle` can be dropped without joining. A scoped API expresses the stronger guarantee in its types and implementation. Prefer that API when work is naturally nested in a caller's stack frame. Prefer ownership transfer when the job must outlive the call that submits it.

Send and Sync Are Independent Questions

`Cell<T>` is a useful example of a type that may be moved between threads but cannot be freely shared between them. Its ordinary operations perform unsynchronized interior mutation. If one thread owns a `Cell<u32>`, it can safely use it and later transfer the entire cell to another thread. Sharing `&Cell<u32>` would expose mutation to multiple threads without synchronization, so `Cell<u32>` is not `Sync`.

`Rc<T>` is different. Cloning and dropping its owners update a non atomic reference count. Moving one `Rc` to another thread while another owner remains in the original thread would let both threads modify that count concurrently. Consequently, `Rc` does not become suitable for thread transfer merely because the payload is immutable.

`Arc<T>` makes ownership bookkeeping atomic. It does not automatically make every possible payload safe for concurrent access. `Arc<Cell<u32>>`

does not turn cell operations into atomic operations. The allocation is shared and retained correctly, but its contents still have their own concurrency contract. This distinction is a precise instance of a broader rule: a wrapper can protect one invariant without protecting all invariants of the wrapped value.

The following compile time assertions record useful contracts without executing any concurrency.

Listing 19.3 Rust example

```
use std::cell::Cell;
use std::sync::{Arc, Mutex};

fn require_send<T: Send>() {}
fn require_sync<T: Sync>() {}

fn main() {
    require_send::<Cell<u32>>();
    require_send::<String>();
    require_sync::<String>();
    require_send::<Arc<Mutex<Vec<u8>>>>();
    require_sync::<Arc<Mutex<Vec<u8>>>>();
}
```

Such assertions are useful in a library's tests when a public type is expected to remain sendable or shareable. They catch accidental changes to the structural contract. They do not establish that application protocols are correct: a perfectly thread safe queue can still deliver work to the wrong account or process a request twice.

Scoped Parallel Mutation Without Locks

Concurrency does not require shared mutable state. Disjoint mutable regions can be processed concurrently because the exclusivity requirement is satisfied separately for each region. The important operation is proving disjointness before starting the threads.

Listing 19.4 Rust example

```
use std::thread;

fn scale(values: &mut [i64], factor: i64) {
    let middle = values.len() / 2;
    let (left, right) = values.split_at_mut(middle);
    thread::scope(|scope| {
```

Listing 19.4 continued

```
        scope.spawn(move || {
            for value in left {
                *value *= factor;
            }
        });
        scope.spawn(move || {
            for value in right {
                *value *= factor;
            }
        });
    });
}

fn main() {
    let mut values = [1, 2, 3, 4, 5];
    scale(&mut values, 3);
    assert_eq!(values, [3, 6, 9, 12, 15]);
}
```

`split_at_mut` establishes that the slices do not overlap. Each closure receives its own exclusive reference. The parent cannot access the original slice while these borrows remain active, and the scope waits for the children. No mutex is required because no memory location is accessed by both workers.

This is also an architectural pattern. Partition data into independent ownership domains before selecting synchronization primitives. A batch of records can be divided by index range; independent connections can be owned by independent tasks; a tree operation can split at subtrees. Some divisions introduce load imbalance, but that is a scheduling tradeoff rather than a reason to abandon ownership.

The example creates threads for two tiny slices, which would be a poor performance choice in real computation. Thread creation and joining have costs. The conceptual result is the absence of conflicting access, not a promise of speedup. A reusable worker pool or parallel iterator library may implement the same ownership pattern with better scheduling.

Unsafe Implementations Are Global Claims

`Send` and `Sync` are unsafe traits because other safe code may rely on their truth. An incorrect implementation can make an otherwise safe program exhibit undefined behavior. The proof must cover all safe operations on the type, including destruction, cloning, callbacks, and error paths.

Imagine a wrapper around a foreign handle. The foreign library says a handle may be used only by its creating thread. The wrapper might contain just an integer identifier, yet that integer's Rust representation does not capture the foreign rule. Automatically receiving `Send` would be wrong for the intended abstraction. Conversely, a raw pointer wrapper might legitimately represent uniquely owned storage that can move between threads, but a manual implementation needs to establish the ownership and destruction rules rather than reason only about the pointer's bit pattern.

Do not implement `Sync` because methods currently take `&self`. That is the very surface through which sharing becomes possible. Inspect interior mutation, foreign operations, retained aliases, and operations performed by `Drop`. Also inspect whether callbacks can reenter the wrapper or expose references beyond a protected interval. A local lock around one method is insufficient when another safe method bypasses that lock.

The proof should be written next to the unsafe implementation in concrete terms. It should identify who owns the resource, how concurrent access is serialized or forbidden, whether thread affinity exists, and why destruction may run on the receiving thread. If these facts are difficult to state, redesigning the wrapper is often more productive than strengthening the comment.

From Trait Errors to Architecture

A failed `Send` bound is often a compressed design review. Read the chain of causes until reaching the field or borrow that supplies the missing requirement. Ask whether the value actually needs to cross threads. A single threaded executor may be appropriate for thread affine state. A worker may own a resource and accept messages. A copied summary may cross the boundary while the original object stays local.

Adding a mutex changes the access model. Cloning an owned value changes identity and update semantics. Adding `Arc` changes ownership duration and sharing. These are legitimate tools, but they implement different designs. A compiler error is resolved correctly when the chosen design matches the application's required behavior.

For example, a parser with `Rc` nodes may remain on one thread while it produces an owned, compact result for workers. Converting the entire graph to `Arc` would permit a different sharing topology and introduce atomic bookkeeping throughout traversal. Neither choice is universally superior. The important decision is where the ownership boundary belongs.

Training Encounters

1. A job contains `String` and `&'a [u8]`. Explain why moving the job into `thread::spawn` may fail even though `String` is `Send`. Give two redesigns with different ownership costs.

2. A cache uses `Arc<RefCell<Vec<u8>>>`. The author argues that `Arc` makes the cache safe for all threads. Identify the invariant protected by `Arc` and the invariant left unprotected. Propose a replacement when concurrent mutation is required.

3. A transformation independently rewrites every element in a large slice. Describe an ownership partition that avoids a lock. State what extra problem remains if some elements take much longer than others.

Training Debriefs

1. `Send` and lifetime validity are separate requirements. The borrowed slice may expire while a spawned thread remains alive. Copying the bytes into `Vec<u8>` gives the job ownership and allocation cost. Using `thread::scope` permits borrowing while requiring every worker body to finish within the scope. A third option is retaining shared ownership of immutable bytes through an appropriate `Arc` allocation.

2. `Arc` protects the shared ownership count and allocation lifetime. `RefCell` performs dynamic borrow checking suitable for a single thread; it does not serialize concurrent payload access. `Arc<Mutex<Vec<u8>>>` provides

exclusive access through a guard, although lock duration and panic behavior still need design. If mutation can be isolated to one owner, a channel based cache actor is another design with different latency and failure behavior.

3. Split the slice into disjoint mutable chunks and give each worker one chunk. The borrow checker can verify the partition when it is expressed through safe slice APIs. No worker accesses another worker's elements. Unequal work introduces load imbalance: one worker can dominate total latency. Smaller chunks and dynamic job scheduling can improve balance while preserving exclusive ownership of each chunk.

Further Reading

- Send defines the transfer contract and lists important implementations.
- Sync defines the relationship between shared references and thread transfer.
- Scoped threads documents the lifetime and joining guarantees used by the partitioning examples.

The Unreturning Survey Team

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Implement a function that searches an immutable slice of owned strings in parallel and returns references to the strings containing a chosen ASCII marker. The input and marker are caller-owned. Divide the catalog into contiguous ranges, let scoped workers produce their own vectors of borrowed matches, and combine those vectors in range order. Your result must not borrow from any worker-local vector or temporary copy of a name. State how many workers you create for small inputs, what zero means for the requested worker count, and what your function does when a worker panics. Include a test where a later range finishes earlier while the final match order remains the catalog's order.

Encounter Constraints

- Do not leak the catalog, require static input, or clone its strings.
- Use standard-library scoped threads; no external executor is needed.

• Worker completion order must not accidentally define report order.

Hint one / Hint two / Reveal debrief

Side Quest

Change the search predicate from a fixed marker to a callback shared by all workers. Derive the callback bounds from how it is captured and called rather than adding every familiar trait. Then compare a shared immutable callback with one distinct callback owned by each worker. Explain how captured state can change Send or Sync requirements even when the callback's argument and return types stay identical.

Capability unlocked Scoped Banner permits borrowed expeditions with an enforced return boundary.

Choose Your Exit

Study the lock gates / Level 20
Handle state that cannot be partitioned into independent worker ownership.

Build the worker expedition / Level 39
Add bounded queues, accounting, and explicit shutdown to a larger pipeline.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 20

The Observatory of Sleeping Readers

The Wilds' observatory publishes numbered weather snapshots. Readers sleep until a requested revision exists, but an old console treats every bell as proof that a new snapshot is ready. Some wake to stale weather; others sleep through shutdown. The observatory needs a protocol in which the protected predicate is authoritative and notifications merely invite another look. Your repair must also prevent a reader's slow processing from occupying the observatory's only lock.

Mission Coordinate versioned snapshots and waiting readers with a mutex-protected predicate and a condition variable.

Suggested loadout Level 6, Level 7, Level 19. These are guidance, not locked gates.

Mastery Targets

• A waiter returns a snapshot meeting its requested revision or a documented stopped outcome.

• Spurious or coalesced notifications do not create false readiness or lose a completed publication.

• Readers process independently owned snapshot handles after releasing the state lock.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 20

Locks Protect Invariants Over Time

A mutex is often introduced as permission to mutate shared data. That description misses the hardest part of using it well. A mutex protects a relationship between operations over an interval. The interval begins when a guard is acquired and ends when the guard is dropped. During that interval, intermediate states may exist that other threads must not observe. Good lock design starts by naming those states and the invariant that must hold whenever the guard becomes available to another thread.

Suppose an account has an available balance and a reserved balance. Protecting each number with a separate mutex makes each update individually race free, but a reservation changes both values. A reader that observes the fields separately can see an impossible combination. Thread safety at the field level does not imply consistency at the operation level. The correct synchronization boundary is usually the smallest boundary containing the entire invariant.

Guards Define the Critical Section

Rust makes lock ownership visible as a value. A `MutexGuard` gives access to the protected data and releases the lock when dropped. This makes lexical blocks useful for expressing critical sections, but a guard can also remain alive longer than expected because it is stored in a local or retained inside a larger expression.

Consider a queue whose lock is needed only to remove one item. The following complete example deliberately moves expensive processing outside the critical section.

Listing 20.1 Rust example

```
use std::collections::VecDeque;
use std::sync::Mutex;

fn take_job(queue: &Mutex<VecDeque<String>>) -> Option<String> {
    let mut jobs = queue.lock().unwrap();
    jobs.pop_front()
}

fn process(job: String) -> usize {
```

Listing 20.1 continued

```
    job.chars().filter(|c| !c.is_whitespace()).count()
}

fn main() {
    let queue = Mutex::new(VecDeque::from([
        String::from("parse one"),
        String::from("parse two"),
    ]));
    let job = take_job(&queue).unwrap();
    assert_eq!(process(job), 8);
    assert_eq!(queue.lock().unwrap().len(), 1);
}
```

The helper returns owned data. It does not return a reference into the queue, so it can release the guard before processing begins. This is more than a performance adjustment: callbacks or downstream work cannot accidentally reenter the queue while its lock is held. The API's ownership decision enforces a useful concurrency boundary.

Returning a guard is sometimes appropriate, but then the caller controls the critical section. That exposes lock ordering and contention to code outside the abstraction. A library should make that tradeoff consciously. Methods such as `with_state` also expose a risk: user supplied closures can block, panic, recursively call the library, or invoke foreign code while the guard is held. A closure API does not make the critical section inherently short.

Atomic Operations at the Domain Level

The following reservation model uses a single lock to protect a two field invariant. The calculation validates failure conditions before mutating state.

Listing 20.2 Rust example

```
use std::sync::Mutex;

#[derive(Debug, PartialEq)]
struct Balance {
    available: u64,
    reserved: u64,
}

struct Account {
```

Listing 20.2 continued

```
    balance: Mutex<Balance>,
}

impl Account {
    fn reserve(&self, amount: u64) -> Result<(), &'static str> {
        let mut state = self.balance.lock().unwrap();
        let available = state.available.checked_sub(amount)
            .ok_or("insufficient balance")?;
        let reserved = state.reserved.checked_add(amount)
            .ok_or("reserved balance overflow")?;
        state.available = available;
        state.reserved = reserved;
        Ok(())
    }

    fn snapshot(&self) -> (u64, u64) {
        let state = self.balance.lock().unwrap();
        (state.available, state.reserved)
    }
}

fn main() {
    let account = Account {
        balance: Mutex::new(Balance {
            available: 100,
            reserved: 0,
        }),
    };
    account.reserve(30).unwrap();
    assert_eq!(account.snapshot(), (70, 30));
    assert!(account.reserve(80).is_err());
    assert_eq!(account.snapshot(), (70, 30));
}
```

There are two forms of atomicity here. The mutex prevents another lock respecting participant from observing the update halfway through. The validation order ensures that a reported ordinary error leaves the old state unchanged. The lock alone does not provide the second guarantee. If the function subtracted first and then discovered overflow, returning an error could expose a damaged account after guard release.

The code uses `unwrap` on lock acquisition, choosing to propagate poisoning as a panic. A production application might instead expose a domain failure, restore state, or terminate the affected component. The right choice depends on whether the protected state can be reconstructed and what callers may safely assume after an interrupted operation.

Poisoning Is Evidence of Possible Damage

The standard mutex usually becomes poisoned when a thread panics while holding its guard. Poisoning signals that the invariant may have been interrupted. It is advisory: the error contains access to the guard, and code can choose to inspect or repair the data. It is not a memory safety mechanism, and unsafe code must not rely on poisoning as the sole barrier against invalid access. There are cases in which panic detection does not behave like a simple universal flag.

An important distinction is between a valid Rust value and a valid domain state. A vector can remain memory safe while its entries violate application rules. Recovering a poisoned guard is memory safe through the standard API, but using its contents without checking may be logically wrong. Conversely, a panic might occur after all state changes have completed, leaving the invariant intact. Poisoning conservatively reports uncertainty rather than diagnosing the exact stage of failure.

Here is a complete recovery example for a reconstructible cache. The recovery policy discards cached values rather than pretending an interrupted mutation was completed.

Listing 20.3 Rust example

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let cache = Arc::new(Mutex::new(vec![1_u32, 2, 3]));
    let other = Arc::clone(&cache);
    let result = thread::spawn(move || {
        let mut values = other.lock().unwrap();
        values.push(99);
        panic!("cache rebuild interrupted");
    }).join();
    assert!(result.is_err());

    let mut values = match cache.lock() {
        Ok(guard) => guard,
        Err(poisoned) => {
            let mut guard = poisoned.into_inner();
            guard.clear();
            cache.clear_poison();
            guard
        }
    };
```

Listing 20.3 continued

```
    values.push(7);
    assert_eq!(&*values, &[7]);
}
```

Clearing the poison flag is justified by a concrete repair operation. It should not be a habitual response added to every lock error. An authoritative financial ledger cannot be repaired by clearing its vector. A cache can often be invalidated because its source of truth lives elsewhere. This relationship between ownership, authority, and reconstruction determines the recovery strategy.

Lock Ordering and Reentrancy

Deadlock is a progress failure, not necessarily a data race. Two threads may each hold one valid guard and wait indefinitely for the other's lock. Rust's type system prevents many invalid references but does not infer an application's global lock order.

Assign an order to locks that may be acquired together. For account transfers, one common rule is to lock accounts by a stable account identifier. The identifier must be unique within the synchronization domain; an ordering that compares equal for distinct locks needs a defined tie breaker. The same object case must be handled before trying to lock the same mutex twice.

Reentrant acquisition can arise without an obvious second `lock` in the same function. A logging callback can consult shared state. A destructor can access a registry. A notification hook can call back into the object that invoked it. Therefore, avoiding user code under a lock is a useful design rule when feasible. Build an owned notification while holding the guard, release the guard, then deliver it.

Releasing the guard before a callback introduces a new semantic question: the state may change again before the callback runs. Include a version or event identifier if observers need to relate the notification to a particular transition. A short critical section is beneficial only when its resulting observation model is clear.

`try_lock` does not solve lock ordering automatically. Repeated failed attempts can create livelock or starvation. A retry loop needs a scheduling

policy and a reason it will eventually make progress. Adding arbitrary sleeps can hide contention during development while leaving the protocol unspecified.

Reader Writer Locks Have a Workload Contract

An `RwLock` permits multiple readers or one writer. That can improve throughput when read operations are substantial, mutation is uncommon, and the implementation's behavior fits the workload. It can also be slower than a mutex when reads are tiny or contention patterns make bookkeeping dominant. The standard library does not promise a universal reader or writer priority policy; relying on one across platforms is a portability error.

Read access must actually be read access to the protected invariant. A read guard may expose a payload containing other interior mutability, but that payload needs its own synchronization rules. A cache lookup that updates recency counters might not fit the simple many readers model unless those counters are independently synchronized or the design tolerates approximate accounting.

Upgrading a read lock to a write lock is another source of subtle bugs. Dropping the read guard and acquiring a write guard creates an interval in which another thread can change the state. Recheck the condition after obtaining exclusive access. Keeping the read guard and trying to acquire a write guard can deadlock; do not assume implicit upgrade support.

Snapshotting is an alternative. A reader briefly clones an `Arc` to an immutable state, then traverses that state without holding a central lock. Writers build a new version and replace the current one. This moves cost to allocation and retained versions. It is attractive when reads are long and stable snapshots are useful, but old snapshots can retain substantial memory. The synchronization primitive is only one part of the resource model.

Condition Variables Protect Predicates

A condition variable waits for a predicate protected by a mutex. The notification itself is not the state. A consumer should loop while the queue is empty and the service remains open, because wakeups may be spurious and another consumer may remove the item first. The predicate must be checked while holding the same mutex that protects the corresponding state transition.

The usual sequence is: acquire the guard, inspect the predicate, wait if necessary, and inspect again after waking. Waiting atomically releases the mutex and later reacquires it. This connects the decision to sleep with the producer's change to the protected state. Checking a separate boolean outside the lock and then waiting creates a lost notification window.

Shutdown belongs in the predicate. A waiter that asks only whether a queue is nonempty can remain blocked forever after the last producer exits. A useful predicate is often `queue is nonempty OR service is closed`. A notification wakes the consumer so that it can discover either reason to stop waiting.

Training Encounters

1. Two counters represent free slots and occupied slots in a fixed capacity pool. Each counter has its own mutex. Explain why a snapshot can violate the capacity invariant without any data race, and redesign the state.

2. A method locks state, updates it, and invokes an observer closure before releasing the guard. Identify two progress or correctness problems. Propose a notification design that makes the timing explicit.

3. A read lock finds a missing cache entry. The code releases the read guard, acquires a write guard, and inserts an expensive value unconditionally. Explain the race and a reasonable repair.

Training Debriefs

1. A reservation updates both counters, but separate lock acquisitions allow a reader to combine one old value with one new value. Put the counters in one struct protected by one mutex and take snapshots under a single guard. Alternatively derive one counter from capacity and the other counter, reducing the number of independent facts that must remain synchronized.

2. The observer can reenter the object and deadlock, or perform slow work that blocks unrelated users. It can also panic while the invariant is in a state not designed for interruption. Construct an owned event containing the transition's version while locked, release the guard, then invoke the observer. Document that later state may already exist when the event is delivered, and include ordering machinery if observers require stronger delivery guarantees.

3. Another writer can populate the entry during the gap. Recheck under the write guard and use the existing entry if present. If constructing the value is expensive, decide whether duplicate speculative construction is acceptable or whether a per entry initialization protocol is needed. Holding the global write guard during construction avoids duplication but can serialize unrelated keys.

Further Reading

- Mutex documents guard behavior and the limits of poisoning.
- RwLock describes reader and writer access and platform dependent priority.
- Condvar documents predicate loops and spurious wakeups.

The Bell Without a Forecast

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design an Observatory holding the current revision, an immutable shared snapshot, and an open or stopped phase. A reader asks for a revision and may wait until that revision is available. A publisher prepares a replacement snapshot outside the lock, installs it under the lock with an advancing revision, and wakes relevant waiters. Shutdown must release readers whose targets will never be reached. Implement or precisely sketch the wait loop, publication, and stop transitions using Mutex, Condvar, and Arc. Specify revision exhaustion and poisoning policy. Demonstrate a notification before a reader begins waiting, a wake with no relevant state change, and shutdown while a reader waits for a future revision. User processing must happen after access to protected state ends.

Encounter Constraints

• Every decision to wait or return must inspect the predicate under the same mutex.

• Never call an arbitrary reader callback while holding the observatory lock.

• A notification is not a stored event count or a substitute for the revision field.

Hint one / Hint two / Reveal debrief

Side Quest

Allow readers to request the next revision after the one they last observed. Decide whether skipping intermediate revisions is allowed. If every revision must be processed, the latest-snapshot interface is insufficient; design an event log with retention and subscriber progress instead. Explain what happens when a slow reader exceeds retention, and keep that semantic failure distinct from a lost condition-variable notification.

Capability unlocked Predicate Key opens waits by state rather than by superstition about notifications.

Choose Your Exit

Carry events by channel / Level 21
Compare persistent state predicates with messages that transfer ownership.

Track service shutdown / Level 28
Coordinate admission, draining, and observers at service scale.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 21

The Courier With Two Destinations

A relay in the Wilds must deliver each announcement to the east and west towers. Both routes use bounded queues. The old relay reports one Boolean result, hiding the case where east accepted the announcement and west disconnected. Operators then retry both routes and duplicate east's work. Your mission is to give ownership transfer a precise vocabulary and make partial fanout visible instead of compressing several facts into an optimistic success flag.

Mission Design a bounded two-recipient fanout protocol that reports partial queue acceptance honestly.

Suggested loadout Level 1, Level 14, Level 19, Level 20. These are guidance, not locked gates.

Mastery Targets

- Each announcement has one stable identifier and explicit per-recipient queue-acceptance outcomes.
- A failed send recovers the undelivered value, while earlier successful sends remain recorded.
- The protocol distinguishes queue acceptance from recipient processing and from durable delivery.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 21

Channels Define Ownership Protocols

A channel does more than transport values. It defines when a producer relinquishes ownership, how a consumer learns that work exists, and what happens when one side disappears. A message boundary can turn shared mutation into sequential ownership: one component owns state, and other components request operations by sending commands. This often makes invariants easier to inspect because there is one place where they change.

The simplification has limits. A message can be accepted without being processed. A reply can be lost after an operation commits. A queue can retain more memory than the application can afford. Channels remove some forms of aliasing, but they introduce a temporal protocol. This level treats capacity, acknowledgement, disconnection, and cancellation as parts of that protocol rather than incidental API details.

Sending Transfers a Value

Sending an owned message transfers the value. A buffered channel can queue it for later receipt, so the producer and consumer may progress independently. With `sync_channel(0)`, a successful `send` means a receiver has already taken the value. In either case, the receiver can process or drop the value without borrowing the sender.

When sending fails because the receiver is disconnected, the standard channel returns the unsent message inside the error. That matters for recovery: the application can retry elsewhere, persist the message, or return it to a caller. Treating every send error as an opaque string throws away an ownership opportunity.

Listing 21.1 Rust example

```
use std::sync::mpsc;

fn main() {
    let (sender, receiver) = mpsc::channel::<String>();
    drop(receiver);
    let message = String::from("retain on failure");
    let recovered = sender.send(message).unwrap_err().0;
    assert_eq!(recovered, "retain on failure");
}
```

The recovered string is the original owned value. No compensating clone was required. This pattern generalizes to queues that reject submissions under overload: return the rejected job when the caller can usefully choose another policy. The error type should preserve the information needed for that decision.

Cloning a sender creates another producer capability. It also affects termination. A receiver normally learns that a standard channel is disconnected only after all senders are dropped and queued messages have been drained. A forgotten sender in a supervisor or test fixture can therefore keep a consumer waiting forever. Ownership of channel endpoints is part of shutdown design.

Capacity Is a Resource Policy

An unbounded channel decouples producer speed from consumer speed by storing the difference. If the long term arrival rate exceeds the service rate, retained work grows. Each message may own large buffers, file handles, or shared allocations, so queue length alone may understate resource use. A queue with ten messages is not small when each message retains a gigabyte.

A bounded channel makes capacity visible. Blocking send applies backpressure to the producer; a nonblocking attempt lets the producer reject, drop, coalesce, or redirect work. Neither policy is inherently correct. A user facing service may reject quickly to preserve latency. A batch pipeline may intentionally block upstream stages. A telemetry stream may discard obsolete intermediate updates.

The next complete example demonstrates explicit rejection while preserving the rejected job.

Listing 21.2 Rust example

```
use std::sync::mpsc::{self, TrySendError};

fn main() {
    let (sender, receiver) = mpsc::sync_channel(1);
    sender.send(String::from("first")).unwrap();
    let rejected = match sender.try_send(String::from("second")) {
        Ok(()) => panic!("the queue should be full"),
        Err(TrySendError::Full(job)) => job,
        Err(TrySendError::Disconnected(_)) => panic!("no receiver"),
```

Listing 21.2 continued

```
    };
    assert_eq!(rejected, "second");
    assert_eq!(receiver.recv().unwrap(), "first");
    sender.send(rejected).unwrap();
    assert_eq!(receiver.recv().unwrap(), "second");
}
```

Bounded item count is useful only when the message size distribution is understood. A byte budget can be enforced separately by transferring permits with jobs or by rejecting oversized messages before enqueueing. If a permit is returned when a job is dropped, the ownership system can help account for resources across success, rejection, and cancellation paths.

A zero capacity synchronous channel is a rendezvous. A successful send establishes receipt, while processing needs its own acknowledgement. A producer holding a lock during the handoff can deadlock with a consumer that needs that lock before receiving. Include both channel operations and mutex acquisition in the wait dependency analysis.

An Actor With Explicit Replies

An actor owns state and receives commands. The command type is the public protocol. Each variant should make the requested transition and the response path clear. This complete example implements a small counter owner with checked updates and a snapshot request.

Listing 21.3 Rust example

```
use std::sync::mpsc::{self, Sender};
use std::thread;

enum Command {
    Add {
        amount: u64,
        reply: Sender<Result<u64, &'static str>>,
    },
    Read(Sender<u64>),
}

fn run(receiver: mpsc::Receiver<Command>) -> u64 {
    let mut value = 0_u64;
    for command in receiver {
        match command {
            Command::Add { amount, reply } => {
                let result = match value.checked_add(amount) {
```

Listing 21.3 continued

```
                    Some(next) => {
                        value = next;
                        Ok(next)
                    }
                    None => Err("counter overflow"),
                };
                let _ = reply.send(result);
            }
            Command::Read(reply) => {
                let _ = reply.send(value);
            }
        }
    }
    value
}

fn main() {
    let (sender, receiver) = mpsc::channel();
    let owner = thread::spawn(move || run(receiver));

    let (reply, response) = mpsc::channel();
    sender.send(Command::Add { amount: 7, reply }).unwrap();
    assert_eq!(response.recv().unwrap(), Ok(7));

    let (reply, response) = mpsc::channel();
    sender.send(Command::Read(reply)).unwrap();
    assert_eq!(response.recv().unwrap(), 7);
    drop(sender);
    assert_eq!(owner.join().unwrap(), 7);
}
```

The state is ordinary `u64`, with no mutex, because only the actor thread accesses it. The actor's serialization is the invariant boundary. Checked addition validates before commit, and the reply reports the resulting value. Ignoring a failed reply send is intentional: a caller that abandoned its receiver does not reverse the committed addition.

That last point deserves attention. The request can succeed in the actor even when the caller never receives confirmation. If a client retries after losing the reply, the addition may happen twice. The channel alone cannot provide exactly once business semantics. A request identifier and deduplication state may be required, and the lifetime or persistence of that state must match the failure model.

The actor also uses an unbounded request channel for brevity. A real service must choose its capacity policy. Making the request queue bounded does not bound reply queues if callers retain an arbitrary number of outstanding requests. Resource accounting follows the whole request path, including queued commands, active work, replies, and abandoned requests.

Acceptance Completion and Durability

Name the point represented by an acknowledgement. Enqueued means the channel accepted a value. Started means an owner began work. Committed means the domain transition took effect. Durable means the effect survives a defined class of failures. These are different claims, and a single boolean named `success` obscures the difference.

For an in memory actor, a successful send usually says nothing about process survival. If the actor panics after receiving a command but before replying, the caller may know only that the reply channel disconnected. It cannot infer whether the operation changed state. Returning a more descriptive error does not recover information that the protocol never recorded.

One design makes requests idempotent by describing desired state rather than an increment. Another associates a unique operation identifier with each change and stores the result for repeated requests. A third accepts at least once delivery and makes duplicate effects harmless. The correct design follows the business operation, not a universal preference for one channel library.

Within a process, command ordering also needs careful wording. A channel can define an order for messages it delivers, but independent producers may race to send. The application should not infer a wall clock ordering between unrelated threads merely from when their logs were printed. If commands from different producers need causal ordering, include a sequence, version precondition, or explicit dependency.

Closing Is a State Transition

Dropping all senders is a useful shutdown signal when producers naturally own the right to keep a service alive. The actor drains queued messages and then exits. This is elegant for finite pipelines, but less explicit for services that retain sender clones in many places.

A shutdown command provides a named transition, yet it introduces ordering questions. Does the actor stop immediately, drain earlier commands, or reject later commands? A message named `Stop` does not answer those questions by itself. If multiple producers remain active, commands can race with the shutdown request. An admission gate outside the queue or a state flag inside the owner may be needed to define which requests are accepted.

There is also a distinction between closing admission and cancelling active work. A service can stop accepting new jobs while finishing existing ones. It can discard queued jobs while allowing the active one to finish. It can ask the active operation to stop at a cooperative boundary. Each policy has different guarantees for resource release and caller responses.

The final owner of a reply capability determines when a caller can conclude that no response will arrive. Accidentally retaining a reply sender in a registry can keep a caller waiting after the actor has otherwise completed. This is the same endpoint ownership issue as retaining a request sender, now in the reverse direction.

Actor Boundaries and Long Running Work

An actor that performs a slow operation inside its receive loop cannot process unrelated commands during that operation. This may be desirable serialization, or it may turn a small state owner into a latency bottleneck. Splitting work requires a new protocol: the actor reserves state, delegates owned work, and later receives a completion message.

That protocol must handle stale completions. A request may have been cancelled or superseded while a worker ran. Include a generation or operation identifier so that the actor can decide whether the completion still applies. Do not let workers mutate actor owned state through a side

channel, because that bypasses the serialization property that motivated the design.

Likewise, a worker waiting for the actor while the actor waits for the worker creates a cycle even without shared memory. Draw a wait graph for synchronous request reply calls. Channel based code can deadlock just as lock based code can. Ownership clarifies who holds data; it does not automatically guarantee progress.

Training Encounters

1. A receiver drains all visible messages and then waits forever despite every producer task having returned. Identify an ownership cause and a debugging technique.

2. A bounded queue allows 100 jobs, but memory still grows without limit. Give two message or pipeline designs that explain the growth and propose corresponding bounds.

3. A payment actor commits a charge, then fails to send its reply because the caller timed out. The caller retries. Explain why successful channel operations alone cannot prevent a duplicate charge.

Training Debriefs

1. A sender clone remains alive outside the producer tasks, perhaps in the supervisor, a test variable, or a registry. The receiver has no evidence that future messages are impossible. Track endpoint ownership explicitly and drop the supervisor's original sender after spawning producers. A scoped owner or wrapper that records producer identities can make such leaks easier to find.

2. Jobs may own arbitrarily large buffers, requiring a byte budget in addition to an item limit. Alternatively, consumers may dequeue rapidly and spawn unlimited active work, so the queue is bounded but in flight jobs are not. Bound active operations with permits retained until completion. Reply buffers, retained snapshots, and retries can require separate accounting as well.

3. The commit and acknowledgement are separate events. Losing the acknowledgement leaves the caller uncertain about the effect. Give the

operation a stable identifier and deduplicate at the authoritative owner, preferably with persistence appropriate to the failure model. An idempotent domain operation is another possibility. Retrying a raw increment or charge cannot be made safe by choosing a different queue capacity.

Further Reading

- Standard channels describes endpoint ownership and disconnection.
- Synchronous channels documents bounded buffering and rendezvous behavior.
- TrySendError preserves rejected messages and distinguishes full from disconnected channels.

The Half Delivered Decree

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Build or sketch a relay that fans an owned announcement out to two bounded standard-library channels. The payload is immutable and can be shared through Arc; its operation identifier must remain stable. One tower may disconnect after the other has accepted its copy. Return a report that describes each recipient separately and gives the caller enough information to retry only destinations whose queue transfer failed. Explain what happens if the relay itself stops after the first successful send but before returning its report. Choose a blocking or nonblocking admission policy and state the consequences of a slow tower. You are not required to create atomic broadcast, but you must identify precisely why two successful or failed send operations do not create it.

Encounter Constraints

- Do not report all-or-nothing delivery unless an additional protocol establishes it.
- Do not hold a shared application-state lock while blocking on a full recipient queue.
- A bounded message count does not bound arbitrary payload bytes or external Arc retention.

Hint one / Hint two / Reveal debrief

Side Quest

Make the east tower process announcements in order while the west tower may batch them. Decide whether the relay's contract requires the two towers to observe the same prefix at every instant or merely eventually receive the same accepted announcements. Add a recovery log or a documented weaker guarantee. Explain how retaining undelivered shared payloads affects memory even when both live channel buffers have fixed capacities.

Capability unlocked Delivery Prism separates accepted, processed, durable, and unknown outcomes.

Choose Your Exit

Prove an atomic transition / Level 22
Study a smaller publication problem with an explicit ordering argument.

Assemble the bounded pipeline / Level 39
Extend transfer accounting through workers and result collection.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 22

The Observatory Between Two Ticks

A tiny instrument in the Wilds publishes two related counters. Readers must see one coherent reading, but the instrument offers only atomic fields and no room for a mutex in this exercise. Someone made every operation sequentially consistent and declared victory. You find readings assembled from opposite sides of an update. The repair is a protocol that detects interference, not a stronger adjective attached to two unrelated loads.

Mission Design a single-writer atomic snapshot with a generation protocol, a stated ordering proof, and bounded reader retries.

Suggested loadout Level 19, Level 20, Level 21. These are guidance, not locked gates.

Mastery Targets

- Every accepted pair comes from one complete publication under the stated single-writer assumptions.
- Readers return Busy after a defined retry budget rather than claiming guaranteed completion.
- Generation exhaustion cannot silently reuse an old represented history.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 22

Atomics Require an Ordering Argument

An atomic operation makes a particular memory access indivisible according to the atomic model. It does not make a surrounding algorithm indivisible. Two atomic increments do not form one transaction, and an atomic pointer does not keep its pointee alive. The difficult work is identifying which observations must be ordered and which resources must remain valid while another thread can observe them.

Start an atomic design with a proof written in ordinary language. Identify the state, the thread allowed to change each part, the event that publishes a change, and the observation that authorizes another thread to use it. Then choose orderings that establish those relationships. Beginning with a collection of `Relaxed` operations and adding stronger orderings until tests pass reverses the reasoning process.

Atomicity and Invariants

Consider counting completed requests. If the count is only a statistic, each request can increment an atomic counter with relaxed ordering. The count must not lose increments, but reading it need not grant access to other request state. Relaxed ordering expresses that limited role.

Listing 22.1 Rust example

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

fn main() {
    let completed = AtomicUsize::new(0);
    thread::scope(|scope| {
        for _ in 0..4 {
            scope.spawn(|| {
                for _ in 0..1_000 {
                    completed.fetch_add(1, Ordering::Relaxed);
                }
            });
        }
    });
    assert_eq!(completed.load(Ordering::Relaxed), 4_000);
}
```

The scope joins the workers before the final assertion. The counter's increments are atomic, and completion of the scoped threads provides the necessary lifetime and synchronization context for the final observation. The counter is not being used as a publication flag for some separate buffer.

If the counter instead means that request records in another allocation are ready to read, the design changes. The reader needs a relationship between the record writes and its subsequent reads. Atomicity of the count alone does not establish that relationship. An ordering choice is meaningful only in relation to the rest of the accesses.

All accesses must still satisfy Rust's memory model. Mixing conflicting atomic and non atomic accesses without the required synchronization is not a clever optimization. Atomic operations of incompatible sizes over overlapping memory also have constraints. A raw pointer obtained from an atomic does not suspend these rules.

Publication Through Release and Acquire

A release operation can publish preceding work to an acquire operation that observes the appropriate value through the relevant synchronization relationship. The critical phrase is observes: an acquire load that still sees the old value has not thereby acquired the new publication. In a one shot flag, the reader waits until it sees the published state.

This complete program uses atomic payload storage so that even a weakened version would remain memory safe, while the ordering proof remains visible.

Listing 22.2 Rust example

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let payload = AtomicUsize::new(0);
    let ready = AtomicBool::new(false);
    thread::scope(|scope| {
        scope.spawn(|| {
            payload.store(42, Ordering::Relaxed);
            ready.store(true, Ordering::Release);
        });
        scope.spawn(|| {
            while !ready.load(Ordering::Acquire) {
                std::hint::spin_loop();
            }
            assert_eq!(payload.load(Ordering::Relaxed), 42);
        });
    });
}
```

There is one writer and one publication. The payload store is sequenced before the release store. The reader exits its loop only after an acquire load reads the published `true`. The payload read follows that acquisition. These relationships order the payload write before the read. The payload load can be relaxed because the flag carries the inter thread ordering.

If both accesses to `ready` were relaxed, the flag would still be atomic, but the algorithm would lack the required publication edge for the payload observation. It is not enough to say that one source line appears before another in the writer. Source order within one thread and visibility constraints across threads are different pieces of the proof.

The loop is educational, not a general waiting strategy. It consumes CPU while waiting and can perform badly when a producer is descheduled. A condition variable, channel, or appropriate runtime notification primitive is often preferable. Correct ordering does not imply acceptable scheduling behavior.

Sequential Consistency Has a Specific Scope

`SeqCst` adds a single total order for sequentially consistent operations, consistent with the model's other constraints. It is a useful conservative choice while designing an algorithm, especially when reasoning about multiple atomic locations. It is not a transaction over all memory, and it does not repair dangling pointers, missing ownership, or a logically incomplete state machine.

For example, storing `x = 1` and then `y = 1` as two sequentially consistent operations does not prevent a reader from running between them. A reader can legitimately see a partial domain transition if the algorithm permits that interleaving. To present an all or nothing pair, encode the pair in one suitably supported atomic representation, use a versioned protocol with a correct read algorithm, or use a lock.

Stronger orderings can simplify one part of a proof but should not hide other obligations. A reference count may use carefully chosen orderings for increments and final destruction; replacing them all with `SeqCst` can preserve correctness but does not by itself establish when reclamation is safe. A hand written reference counter also needs to handle overflow, weak references, allocation layout, and races around the last owner.

Orderings are not portable performance estimates. Their cost depends on architecture, compiler lowering, contention, and the surrounding instructions. Measure the workload after correctness is established. A lock can outperform a complicated atomic retry loop under contention while being easier to audit.

Compare Exchange Is a Conditional Transition

Compare exchange attempts to replace a value only if it still equals an expected value. Failure returns the observed value so that the caller can decide whether to retry. The weak form may fail spuriously, which is convenient for loops that already tolerate retry. The computation inside a retry loop must be safe to repeat.

Here is a bounded permit counter. It illustrates a domain precondition that must be checked against the value actually being changed.

Listing 22.3 Rust example

```
use std::sync::atomic::{AtomicUsize, Ordering};

fn try_take(available: &AtomicUsize) -> bool {
    let mut current = available.load(Ordering::Relaxed);
    loop {
        if current == 0 {
            return false;
        }
        match available.compare_exchange_weak(
            current,
            current - 1,
            Ordering::Relaxed,
            Ordering::Relaxed,
        ) {
            Ok(_) => return true,
            Err(observed) => current = observed,
        }
    }
}

fn main() {
    let available = AtomicUsize::new(2);
    assert!(try_take(&available));
    assert!(try_take(&available));
    assert!(!try_take(&available));
    assert_eq!(available.load(Ordering::Relaxed), 0);
}
```

Relaxed ordering is sufficient for this counter's stated purpose: deciding whether a numeric permit was obtained. This example does not publish a resource stored elsewhere. If returning a permit also hands off access to a buffer, then the buffer ownership and visibility protocol require additional reasoning. A counter named `available` does not automatically encode that protocol.

Do not perform an external side effect while computing the next value in a retry loop. If a failed compare exchange repeats the computation, it could send duplicate messages, charge twice, or allocate repeatedly. Compute a pure candidate first; perform the committed side effect only after a successful transition, and still consider failure between those two events.

The success and failure orderings describe different paths. A failed compare exchange performs a load rather than the successful replacement, so it cannot have release ordering on failure. The required failure ordering depends on whether the newly observed value authorizes reading published data. Avoid copying a pair of orderings from another algorithm without copying its proof.

The ABA Problem Is About Identity

A compare exchange compares the represented value. If a location changes from A to B and back to A, a thread holding an old expectation of A may succeed even though an important history occurred. When A is a pointer, the address may even refer to a newly allocated object after the old object was removed and freed. Equal bits need not mean the same logical state.

Generation counters can distinguish some histories, but a finite counter can wrap. A proof needs either a bound that makes wrapping impossible while an old observation remains relevant or a protocol that tolerates wrapping. Calling a large counter practically infinite is a risk assumption, not a mathematical invariant.

Memory reclamation is a separate difficulty. An atomic pointer load does not retain ownership of the allocation. Another thread may remove and free the object before the first thread dereferences it. Acquire ordering can order writes to an object that remains valid; it cannot make freed storage valid again. Hazard pointers, epoch based reclamation, reference counting, and guarded ownership schemes address this problem with different costs and constraints.

This is why a short lock free stack implementation can be much more dangerous than its line count suggests. The pointer update may be correct while allocation lifetime is wrong. Use an established abstraction when possible, and inspect its reclamation model as part of evaluating the API.

Progress Guarantees Need Their Own Proof

The words nonblocking, lock free, and wait free describe different progress properties. An operation that does not use a mutex may still spin indefinitely. A compare exchange loop can starve one participant while others repeatedly succeed. An allocator or destructor called by an otherwise atomic algorithm may block internally.

State the level at which a progress claim applies. Does at least one thread make progress under contention, or does every operation finish within a bounded number of its own steps? What assumptions are made about scheduling, memory allocation, and stalled participants? Without those details, saying an algorithm is lock free often communicates less than saying exactly what it does when contended.

Contention also affects cache coherence. Many threads incrementing one atomic can force ownership of the same cache line to move repeatedly. Sharding counters and combining them later may improve throughput when exact instantaneous totals are unnecessary. That is a semantic relaxation: readers receive an aggregate snapshot with defined approximation or consistency properties. Performance follows from changing the required behavior, not simply from selecting a weaker ordering.

Training Encounters

1. A program writes a payload, sets a relaxed ready flag, and reads the payload after seeing the flag. Explain the missing relationship. Distinguish a payload stored atomically from ordinary unsynchronized memory.

2. A compare exchange retry loop invokes a callback to compute its next value. The callback sends a notification. Explain why a successful final update can still produce several notifications.

3. A shared atomic pointer is loaded with acquire ordering. Another thread removes and frees the allocation. Explain why acquire does not authorize dereferencing the old pointer and name two families of solutions.

Training Debriefs

1. The ready flag lacks a release acquire publication relationship. With atomic payload storage, the program can remain memory safe while observing a result the intended protocol did not allow. With conflicting ordinary memory accesses and no other synchronization, the design can contain a data race and exhibit undefined behavior. Establish ownership and synchronization together rather than treating a flag as an informal promise.

2. The comparison can fail because another thread changed the value, or the weak operation can fail spuriously. The callback can therefore run repeatedly before one update succeeds. Keep retry computation free of externally visible effects. After success, emit an event tied to that committed transition, while accounting for the possibility of interruption before event delivery.

3. Ordering relates accesses; it does not extend allocation lifetime. Reclamation can occur after the load and before dereference. A lock or ownership guard can retain validity, while hazard pointers or epoch based schemes can defer reclamation until readers are no longer exposed. Reference counting is another approach when acquiring a reference is itself made safe by the surrounding protocol.

Further Reading

- Atomic memory model defines constraints on atomic and conflicting accesses.
- Ordering specifies the supported memory orderings.
- AtomicUsize documents compare exchange success and failure behavior.

The Counter Hydra

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Model a single writer publishing two AtomicUsize payload fields with an AtomicUsize generation. The writer marks the generation odd, stores both payloads, then publishes the next even generation. A reader loads an even generation, reads both payloads, and accepts them only if a second generation load matches. For this challenge, use SeqCst operations throughout so your reasoning can use their total order; do not weaken them as an optimization exercise. Give readers a finite retry budget. Check generation advancement before starting a write and define exhaustion as refusal to publish further updates. Explain why matching generations establish a coherent pair only when your assumptions exclude concurrent writers and generation reuse. All payload accesses remain atomic, including unsuccessful reads.

Encounter Constraints

• Exactly one writer may execute a publication at a time; make that assumption enforceable or explicit.

• No ordinary shared mutable payload storage or raw-pointer reclamation is needed.

• Do not claim wait freedom or successful completion merely because readers avoid a mutex.

Hint one / Hint two / Reveal debrief

Side Quest

Replace the numeric payload with an atomic pointer to a heap-allocated reading, but do not dereference it yet. List the additional lifetime and reclamation obligations that the numeric design did not have. Explain why acquire ordering or a matching generation cannot alone prevent another thread from freeing the pointed-to allocation. Compare retaining a guarded owner with an established reclamation strategy before proposing any unsafe implementation.

Capability unlocked Ordering Compass locates the exact events that justify an observation.

Choose Your Exit

Follow suspended time / Level 23
Move from cross-thread observation to state retained between polls.

Descend to pointer provenance / Level 30
Separate ordering evidence from the right to access an allocation.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 23

The Engine That Yields No Ground

Ferrum's checksum mill has been moved into an async service. It no longer blocks on a socket, yet other work still freezes while one large buffer is examined. The mill contains await syntax, so its keeper insists it must be cooperative. Your mission is to open the state machine and impose an actual work budget. A poll should make measurable progress, preserve unfinished state, and return control while more bytes remain.

Mission Implement a finite, budgeted computation future and explain exactly what survives each Pending result.

Suggested loadout Level 2, Level 6, Level 12, Level 19. These are guidance, not locked gates.

Mastery Targets

• Each poll examines at most a chosen fixed number of bytes before returning or completing.

• Pending with remaining immediately available work schedules another poll instead of sleeping forever.

• Dropping an unpolled or partially polled future releases its owned input without executing a promised final statement.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 23

Futures Are Suspended State Machines

Calling an async function constructs a value representing work. Polling that value executes the function until it completes or reaches a point where it cannot currently proceed. The distinction between construction and execution explains many surprising observations: a log at the beginning of an async body may never appear, a borrowed argument remains borrowed while the future exists, and dropping an unpolled future can release resources without running the body.

Model the future as a state machine. It retains values needed after suspension, including those whose destructors still have to run. A local whose scope ends before the await can be dropped first. The future also keeps the awaited operation and the control state needed to resume. These are obligations the future must preserve, rather than a list of fields at fixed offsets. The compiler chooses the concrete layout and may represent different suspension points differently.

Construction Captures Arguments

An async function's body does not run merely because the function was called. Its arguments are captured into the returned future. That can transfer ownership immediately even though the body has not yet inspected the value. Dropping the future then drops those captured arguments according to the language's rules.

The following complete program observes the difference through a drop counter and polls a separate immediately ready future without an executor.

Listing 23.1 Rust example

```
use std::cell::Cell;
use std::future::Future;
use std::pin::pin;
use std::rc::Rc;
use std::task::{Context, Poll, Waker};

struct Token(Rc<Cell<usize>>);

impl Drop for Token {
    fn drop(&mut self) {
        self.0.set(self.0.get() + 1);
    }
}

async fn use_token(_token: Token) -> u32 {
    7
}

fn main() {
    let drops = Rc::new(Cell::new(0));
    let future = use_token(Token(Rc::clone(&drops)));
    assert_eq!(drops.get(), 0);
    drop(future);
    assert_eq!(drops.get(), 1);
```

Listing 23.1 continued

```
    let mut ready = pin!(async { 12_u32 });
    let mut context = Context::from_waker(Waker::noop());
    assert_eq!(ready.as_mut().poll(&mut context), Poll::Ready(12));
}
```

A no operation waker is appropriate here because the polled future completes immediately. It would be inappropriate for a future whose progress depends on receiving notifications that the executor must act on. Polling is easy to demonstrate manually; waiting correctly is a separate executor responsibility.

Compare an `async fn` with a normal function returning an async block. The normal function can perform synchronous validation before constructing its return value. This can deliberately split eager work from deferred work. The distinction affects error timing and cancellation: eager work has already occurred even if the returned future is never polled.

What Await Actually Delegates

An await expression polls a child future as part of polling its parent. If the child is ready, the parent receives its output and continues. If the child is pending, the parent normally becomes pending as well, retaining the child and the state needed to resume. Await does not inherently create a thread, spawn an independent task, or introduce parallelism.

Consequently, two awaits written in sequence are sequential with respect to the surrounding control flow. The second child is not necessarily even constructed until the first completes. To make independent operations progress concurrently, some enclosing combinator or task arrangement must poll both. This is concurrency through interleaved progress, which can occur on a single thread.

The following library example separates eager construction from deferred execution. Each function is complete and may be compiled as library code.

Listing 23.2 Rust example

```
use std::future::{self, Future};

fn prepare(
    text: String,
```

Listing 23.2 continued

```
) -> Result<impl Future<Output = usize>, &'static str> {
    if text.is_empty() {
        return Err("empty input");
    }
    Ok(async move {
        future::ready(()).await;
        text.len()
    })
}

async fn deferred_check(text: String) -> Result<usize, &'static str> {
    if text.is_empty() {
        return Err("empty input");
    }
    Ok(text.len())
}
```

`prepare` can return an error before the caller obtains a future. `deferred_check` returns a future whose output is a result; its check happens only when polled. Neither form is universally better. An API should choose based on what it means to accept an operation and which resources should be acquired before scheduling.

The `ready` await in this example does not suspend. An await point is a place where suspension is possible, not a promise that the current poll returns to the executor. A loop containing awaits can still run for a long time if every awaited operation is immediately ready. Cooperative scheduling may require a runtime specific yield mechanism or a deliberate work budget.

Implementing a Small Future

A future's `poll` method receives a pinned mutable reference and a context containing the current task's waker. It returns `Ready(output)` when complete or `Pending` when the task must wait. If it returns pending and progress can later occur, the design must arrange a wakeup. Returning pending without a wakeup path can strand a task forever.

This deliberately simple future yields once by scheduling its own task and then returning a value on the next poll.

Listing 23.3 Rust example

```
use std::future::Future;
use std::pin::Pin;
use std::sync::Arc;
use std::sync::atomic::{AtomicUsize, Ordering};
use std::task::{Context, Poll, Wake, Waker};

struct YieldOnce {
    yielded: bool,
}

impl Future for YieldOnce {
    type Output = u32;

    fn poll(
        mut self: Pin<&mut Self>,
        context: &mut Context<'_>,
    ) -> Poll<u32> {
        if self.yielded {
            Poll::Ready(99)
        } else {
            self.yielded = true;
            context.waker().wake_by_ref();
            Poll::Pending
        }
    }
}

struct CountWake(AtomicUsize);

impl Wake for CountWake {
    fn wake(self: Arc<Self>) {
        self.0.fetch_add(1, Ordering::Relaxed);
    }
}

fn main() {
    let notifications = Arc::new(CountWake(AtomicUsize::new(0)));
    let waker = Waker::from(Arc::clone(&notifications));
    let mut context = Context::from_waker(&waker);
    let mut future = Box::pin(YieldOnce { yielded: false });
    assert_eq!(future.as_mut().poll(&mut context), Poll::Pending);
    assert_eq!(notifications.0.load(Ordering::Relaxed), 1);
    assert_eq!(future.as_mut().poll(&mut context), Poll::Ready(99));
}
```

The `bool` state does not depend on a stable address, so this type is `Unpin`. That is why mutating its field through the pinned receiver is

straightforward. A wrapper containing an address sensitive child future needs a projection strategy discussed in the next level.

The notification counter is not an executor. It records that a wake occurred, and the test manually performs the next poll. A real executor must associate the notification with a runnable task and ensure the task is polled without concurrent mutable access to its future.

Suspension Determines Stored Resources

Every value retained across suspension affects the future's size, destruction behavior, and auto traits. Retaining a large array can make a future unexpectedly large. Retaining a non `Send` value can prevent migration between executor threads. Retaining a guard can keep a lock or permit held while unrelated work waits.

A useful review technique is to stop at each await and list the live resources. Include references, owned buffers, transaction handles, lock guards, and child futures. Ask which of them must remain alive. A smaller lexical scope can end a temporary's lifetime before the await, making both the intent and the generated state clearer.

The next complete library example demonstrates a future that remains sendable because a non sendable temporary is confined to a synchronous block.

Listing 23.4 Rust example

```
use std::future;
use std::rc::Rc;

fn require_send<T: Send>(_: T) {}

async fn summarize() -> usize {
    let length = {
        let local = Rc::new(String::from("temporary"));
        local.len()
    };
    future::ready(()).await;
    length
}

fn main() {
    require_send(summarize());
}
```

This pattern is useful when a synchronous library uses local reference counting internally but produces an owned or copied result suitable for asynchronous work. It avoids changing the entire representation solely to satisfy a scheduler's boundary. The scope is also a communication device: future maintainers can see that the local object is intentionally absent from suspended state.

Do not infer the exact future layout from source structure. Compiler optimizations and lowering details are not an ABI contract. Measure size when it matters, and treat size observations as properties of a concrete toolchain and build. The semantic obligations concern lifetimes and behavior, not a promised enum representation.

Completion and Repeated Polling

After a future returns ready, callers should stop polling it unless its particular contract says otherwise. The general `Future` trait does not require repeated polls after completion to return the same value. A future may panic or otherwise behave unhelpfully, although safe polling must still obey Rust's memory safety rules.

A combinator that polls several children must therefore track which children completed. Accidentally polling a completed child on the next invocation is a state machine bug. The parent may store each result and remove or mark the corresponding child before returning pending for the remaining work.

Similarly, the output is transferred out of the future. If it owns a buffer, that buffer is now the caller's resource. The future cannot casually return the same owned buffer twice. This ownership fact explains why a completed future often has a distinct terminal state rather than behaving like a reusable query.

Fusing a future means adapting it to a defined post completion behavior, commonly remaining pending. That can simplify loops that combine multiple optional operations, but it is an additional abstraction rather than a universal property of all futures.

Async Does Not Remove Blocking

Ordinary blocking operations remain blocking inside an async function. A filesystem call, synchronous channel receive, CPU intensive loop, or blocking mutex acquisition can occupy the executor thread until it returns. The `async` keyword does not transform arbitrary system calls into readiness driven operations.

This matters most on a single threaded executor, where one blocking operation can stop every other task. On a multithreaded executor, enough blocked tasks can occupy the entire worker pool. Moving blocking work to an appropriate dedicated facility changes scheduling, but the offloaded work's cancellation and shutdown behavior then need their own contract.

The future abstraction is intentionally small. It describes resumable computation and notification. It does not choose a reactor, thread pool, timer service, admission policy, or supervision tree. Keeping those responsibilities distinct helps diagnose whether a failure comes from the future's state machine, the executor's scheduling, or the application's resource protocol.

Training Encounters

1. An async function owns a file handle argument, but the caller immediately drops the returned future without polling it. Explain which code runs and what happens to ownership of the handle.

2. A loop awaits an operation that is always immediately ready. Explain why the loop can starve other tasks despite containing an await on every iteration.

3. A future is unexpectedly non `Send`. A diagnostic points to an `Rc` created before an await and used only for a synchronous calculation. Propose a source change that expresses the intended lifetime clearly.

Training Debriefs

1. Constructing the future captures the argument without running the async body. Dropping the future drops the captured handle, so its destructor can release the resource. Statements in the body, including logging or explicit cleanup, never execute. Cleanup that must happen even for unpolled operations belongs in ownership based destruction or in an eager construction stage with a clear contract.

2. Await continues in the same poll when its child is ready. The loop can therefore consume an unbounded amount of work without returning control to the scheduler. Introduce a runtime appropriate yield or process a bounded amount of work before arranging another poll. The scheduling requirement is cooperative progress, not merely the presence of syntax.

3. Put the `Rc` and its synchronous uses in an inner block that produces an owned or copied summary. Await only after that block ends. This makes the non sendable temporary absent from suspended state. If the object actually must remain alive, choose a local executor or redesign its ownership rather than erasing the requirement.

Further Reading

- Future specifies polling, wakeups, and completion behavior.
- Async functions explains construction and argument capture.
- Async blocks documents capture and the unspecified generated representation.

The Endless First Poll

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Build a future that owns a Vec<u8> and computes a deliberately noncryptographic wrapping byte sum. Store a cursor and accumulator in the future. Each poll may process at most sixty-four bytes. If work remains, arrange another poll and return Pending; if the buffer is exhausted, return the result. Decide and document your post-completion polling policy within the Future contract. Test empty input, a buffer smaller than one budget, and a buffer spanning several budgets using a controlled waker or a minimal polling harness. Record the cursor after each Pending to demonstrate bounded progress. Explain why moving this future before or between polls is harmless for this particular representation and why that does not generalize to every compiler-generated future.

Encounter Constraints

• No external executor is required to inspect the future's poll behavior.

• Use owned data and indices rather than references into the future's own Vec.

• A wrapping sum is an intentional arithmetic policy, not a cryptographic integrity claim.

Hint one / Hint two / Reveal debrief

Side Quest

Run two budgeted sums under a tiny round-robin harness and record the maximum bytes processed before the other receives an opportunity to poll. Then make the work budget configurable at construction. Compare latency and polling overhead with measurements rather than asserting that smaller budgets are always faster. Explain why an await on an immediately ready child would not, by itself, create the same explicit scheduling boundary.

Capability unlocked Poll Budget makes cooperative progress visible in stored state.

Choose Your Exit

Enter the moving house / Level 24
Understand futures whose representation really does depend on address stability.

Trace the scheduler bells / Level 25
See how a request for another poll becomes an executor action.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 24

The House Whose Rooms Must Stay Still

Two scouts race to find an open route through the Wilds. Their suspended equipment may contain address-sensitive state, yet the race controller itself must be easy to move between ordinary containers. A previous controller demanded that every scout be Unpin and excluded the very operations it was meant to coordinate. Your repair chooses a stable home for each child and keeps a clear distinction between moving a handle and moving what it owns.

Mission Compose two potentially non-Unpin futures using pinned owned handles and an explicit terminal policy.

Suggested loadout Level 2, Level 6, Level 10, Level 23. These are guidance, not locked gates.

Mastery Targets

• The combinator accepts child futures without requiring the child types to be Unpin.

• A completed result consumes or retires both child handles without moving a pinned pointee out of storage.

• The review identifies exactly which value is pinned, who owns it, and how it is destroyed.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 24

Pinning Is an Address Stability Contract

Most Rust values can move whenever ownership moves. A compiler may implement a move by copying bytes, but the semantic point is that the old location no longer contains the owned value and the new location does. References usually prevent invalid moves because a borrowed owner cannot be freely moved while the borrow remains in use. Some abstractions need a stronger, longer lived promise: once a value enters an address sensitive state, it must remain at its location until its destruction protocol finishes.

`Pin` represents that promise through a pointer like owner or reference. It does not make all memory immovable, and it does not change assignment throughout the language. Instead, it restricts safe access through the pinned handle so that code cannot obtain the unrestricted mutable access that would permit moving an address sensitive pointee out. Understanding what is pinned, and which fields inherit that promise, is the heart of using the abstraction correctly.

Pointer Movement and Pointee Movement

Moving a `Box<T>` changes where the box handle is stored, but normally leaves its heap allocation at the same address. This physical fact is useful but insufficient as a complete contract: ordinary `Box<T>` still permits moving `T` out of the box. `Pin<Box<T>>` closes that safe escape route for a pointee that is not `Unpin`.

This complete example records the observed address of a pinned value and then moves the owning handles through a vector.

Listing 24.1 Rust example

```
use std::cell::Cell;
use std::marker::PhantomPinned;
use std::pin::Pin;

struct Resident {
    address: Cell<Option<usize>>,
    _pinned: PhantomPinned,
}
```

Listing 24.1 continued

```
impl Resident {
    fn observe(self: Pin<&Self>) -> usize {
        let current = self.get_ref() as *const Self as usize;
        if let Some(previous) = self.address.get() {
            assert_eq!(previous, current);
        } else {
            self.address.set(Some(current));
        }
        current
    }
}

fn main() {
    let resident = Box::pin(Resident {
        address: Cell::new(None),
        _pinned: PhantomPinned,
    });
    let before = resident.as_ref().observe();
    let mut owners = Vec::new();
    owners.push(resident);
    for _ in 0..100 {
        owners.reserve(1);
    }
    let resident = owners.pop().unwrap();
    assert_eq!(resident.as_ref().observe(), before);
}
```

The stored address is only diagnostic data; the example never reconstructs a pointer or dereferences it. The vector can move its handle values while the box allocation remains pinned. `PhantomPinned` prevents the automatic `Unpin` implementation, ensuring that safe APIs treat the pointee as potentially address sensitive.

Adding `PhantomPinned` does not itself establish a self reference or initialize a pinning protocol. Before a value is actually pinned, it can still be moved normally. The marker changes which safe operations are available after pinning. A real address sensitive type must define when it establishes internal pointers and must maintain their validity through every later operation.

Unpin Opts Out of the Restriction

`Unpin` says that a type does not need the pinning restriction for soundness. Most types are `Unpin`. For such a pointee, pinned access may safely become ordinary mutable access. This makes APIs such as `Future::poll` usable by both address sensitive futures and simple movable state machines.

The distinction concerns the pointee type. A pointer wrapper can be movable even when its pointee must remain fixed. `Pin<Box<F>>` is itself an ordinary movable handle; moving it does not move `F` out of its allocation. This is why boxing is a convenient boundary for generic future combinators.

Listing 24.2 Rust example

```
use std::pin::Pin;

fn reset(mut value: Pin<&mut u32>) {
    *value.as_mut().get_mut() = 0;
}

fn main() {
    let mut value = 9_u32;
    reset(Pin::new(&mut value));
    assert_eq!(value, 0);
}
```

The pinned reference here adds no meaningful restriction because `u32` is `Unpin`. The same `get_mut` operation is not generally available for a `PhantomPinned` containing type. Treating all pinned values as ordinary mutable references would erase the promise that address sensitive implementations rely on.

The following deliberately rejected example demonstrates that restriction without unsafe code.

Listing 24.3 Rejected example

```
use std::marker::PhantomPinned;

struct Fixed {
    _pinned: PhantomPinned,
}

fn main() {
    let mut value = Box::pin(Fixed { _pinned: PhantomPinned });
    let ordinary: &mut Fixed = value.as_mut().get_mut();
```

Listing 24.3 continued

```
    let replacement = Fixed { _pinned: PhantomPinned };
    let _old = std::mem::replace(ordinary, replacement);
}
```

The problem is not that replacing a marker field would be physically difficult. The type opted into a contract under which generic code must preserve location. Safe access cannot assume that this particular representation happens to be harmless, because a later version may depend on the promise in an internal invariant.

Pinning a Borrow Is Also a Promise

Heap allocation is not required for pinning. The `pin!` macro can pin a value in local storage, with the generated ownership structure ensuring that safe code cannot move it out afterward. This is useful when a future is awaited within a bounded scope and does not need a separately allocated owner.

Pinning through a mutable reference is especially important to reason about over time. An unsafe constructor that creates `Pin<&mut T>` promises more than keeping the value still until that particular reference is dropped. The pointee must satisfy the pinning contract for the rest of its relevant lifetime, including after the temporary pinned reference goes away. Returning to an old ordinary mutable reference and moving the value later can break that promise.

This is why casually wrapping an arbitrary local with an unchecked pin constructor is dangerous. A future might store an internal reference during its first poll, and subsequent movement would invalidate it even if the original `Pin<&mut F>` no longer exists. Safe constructors and macros arrange the ownership restrictions that make the long term promise credible.

The promise also includes destruction. Address sensitive code may rely on the value remaining valid at its address until its destructor runs. Reusing storage or deallocating it without following the required destruction behavior can invalidate other components' cleanup assumptions. Pinning is therefore not merely a rule about calling `mem::swap`.

Structural Projection Is a Design Decision

A pinned struct does not automatically imply that every field is structurally pinned. An implementation must decide which fields carry the address stability promise and expose projections consistent with that decision. A structurally pinned child future needs `Pin<&mut Child>` for polling. An ordinary bookkeeping counter might be exposed as `&mut usize` if moving that field cannot violate the enclosing invariant.

The difficulty is that safe code cannot generally turn `Pin<&mut Parent>` into unrestricted `&mut Parent` when `Parent` is not `Unpin`. Doing so would allow moving all fields, including the ones promised stable. Projection libraries generate carefully audited code for common patterns. Hand written unsafe projection must account for field movement, `Drop`, manual `Unpin` implementations, and every method that can reach the fields.

One simple alternative is to pin children in their own allocations. The parent then contains movable pinned handles, so ordinary mutable access to the parent can be safe when its other fields permit it. The following complete combinator uses that approach and requires its mapping closure to be `Unpin`.

Listing 24.4 Rust example

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll};

struct Map<F, G> {
    inner: Pin<Box<F>>,
    map: Option<G>,
}

impl<F, G> Map<F, G> {
    fn new(inner: F, map: G) -> Self {
        Self {
            inner: Box::pin(inner),
            map: Some(map),
        }
    }
}

impl<F, G, U> Future for Map<F, G>
where
    F: Future,
    G: FnOnce(F::Output) -> U + Unpin,
```

Listing 24.4 continued

```
{
    type Output = U;

    fn poll(
        self: Pin<&mut Self>,
        context: &mut Context<'_>,
    ) -> Poll<U> {
        let this = self.get_mut();
        match this.inner.as_mut().poll(context) {
            Poll::Pending => Poll::Pending,
            Poll::Ready(value) => {
                let map = this.map.take().expect("polled after completion");
                Poll::Ready(map(value))
            }
        }
    }
}

fn main() {
    let future = Map::new(async { 4_u32 }, |value| value * 3);
    let mut future = Box::pin(future);
    let mut context = Context::from_waker(std::task::Waker::noop());
    assert_eq!(future.as_mut().poll(&mut context), Poll::Ready(12));
}
```

The child `F` can be non `Unpin`; its `Pin<Box<F>>` handle remains movable while the allocation is stable. The closure is stored in an `Option` so it can be consumed exactly once after readiness. The `Unpin` bound on `G` lets the parent be safely accessed through `get_mut` without an unsafe projection. This trades one allocation for a simpler implementation and a visible bound.

The combinator should not be polled after completion. Its `expect` makes one terminal misuse obvious, but it first polls the completed child, whose own contract may already reject that action. A reusable production adapter can track a terminal flag before touching the child if it promises a specific post completion behavior. The general future contract does not require that extra behavior.

Drop and Replacement Need Careful Distinctions

Pinning does not forbid destroying the pointee. It requires the destruction protocol to respect the address sensitive state. A pinned value can be dropped in place, after which storage can be reused according to the relevant API. Operations such as replacing a pinned pointee are sound only when they perform the appropriate destruction first and uphold the contract of the pointer type and stored value.

For an intrusive list node, destruction may unlink the node using its stable address. Moving the node before that unlinking would leave neighboring nodes pointing to the old location. A destructor therefore participates in the invariant, not just resource reclamation. Forgetting the node may leak it; freeing or reusing its storage while links remain is a different and potentially unsafe operation.

An enclosing type's destructor must not move out structurally pinned fields. Likewise, a manual `Unpin` implementation must not claim movability when any projected field depends on the parent's pinning. These obligations connect seemingly separate implementation choices. A safe accessor added months later can invalidate an earlier unsafe projection if it exposes a way to replace the pinned field.

This is a good reason to keep the pinning boundary small and document it near the type. List structurally pinned fields, describe the address sensitive state, and explain how destruction ends it. Review changes to field visibility and mutation methods against that list.

Choosing Where to Pay Complexity

An allocation is sometimes cheaper than an unsafe proof. Boxing a child future can simplify storage, type erasure, and pin projection at once. Avoiding the allocation may matter in a hot path, but it should be justified by measurement and accompanied by a sound projection strategy.

At the API level, accept `Pin<&mut F>` when the caller has already established pinning and the operation genuinely needs it. Accept an owned `F` when the implementation can establish its own storage. Requiring `F: Unpin` can simplify an adapter, but it excludes common compiler generated

futures; document the reason instead of treating the bound as harmless decoration.

The central question is always which value's location matters and who guarantees that location through destruction. Once those two facts are explicit, the variety of `Pin<Box<T>>`, `Pin<&mut T>`, `Unpin`, and projections becomes a set of ownership arrangements around one contract.

Training Encounters

1. Explain why moving `Pin<Box<F>>` into a vector can be sound even when `F` is not `Unpin`. Identify the move that would be forbidden.

2. A wrapper projects a pinned field as `Pin<&mut Child>`. A safe method uses `mem::replace` to return the old child by value. Why can this break pinning?

3. An adapter can avoid unsafe projection by boxing its child future, but this adds an allocation. Describe the evidence needed before replacing the boxed implementation with a more complex inline one.

Training Debriefs

1. The vector moves the owning pointer handle, while the heap allocation holding `F` remains at its address. Moving `F` out of that allocation before its pinning obligation ends would break the contract. The handle and pointee are different values with different movement consequences.

2. The projection promises structural pinning. Returning the old child by value moves it before destruction. Require `Child: Unpin` to allow that move. To replace a child that needs pinning, use `Pin::set` on the valid projection. It runs the old child's destructor in place before overwriting the storage.

3. Measure allocation frequency and end to end cost under a representative workload. Confirm that allocation is material rather than hidden by I/O or other work. Then audit projection, destruction, `Unpin`, and mutation methods together, and test the resulting abstraction with relevant dynamic tools. A benchmark improvement does not replace the soundness argument.

Further Reading

- Pin module describes the address stability and destruction contracts.
- Pin documents safe and unsafe constructors and projections.
- Unpin explains when pinning restrictions can be relaxed.

The Twin Room Collapse

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a RaceTwo future that owns two child futures and returns either the left output or the right output, tagged by an enum. Put each child in its own Pin<Box<F>>-style storage so the parent can use safe access to its movable handles. Poll the children according to an explicit order, and explain the bias when both are ready. When one completes, drop the losing child's owned handle and enter a terminal state. Do not promise that dropping the loser reverses any external effect it already performed. Write a representation walkthrough that covers parent movement, child polling, taking an Option containing a pinned handle, and destruction. Then state how a poll after completion behaves without making an unsupported guarantee about the child.

Encounter Constraints

- Use no unsafe projection in the primary solution.
- Do not add Unpin bounds on child futures merely to obtain mutable parent access.
- Keep completed status in the parent so its terminal policy does not repoll a completed child accidentally.

Hint one / Hint two / Reveal debrief

Side Quest

Sketch an allocation-free version with inline children, but stop before writing unchecked projection. List the obligations that a projection implementation would need to maintain across every method, destructor, and manual Unpin implementation. Compare that proof burden with a well-maintained projection helper and with the boxed design. The side quest is complete when you can justify where complexity is paid, rather than when you have removed an allocation at any cost.

Capability unlocked Anchor Sigil distinguishes a movable owner from an address-sensitive pointee.

Choose Your Exit

Hunt a lost wakeup / Level 25
Make child readiness reliably reach the next polling opportunity.

Audit an unsafe boundary / Level 31
Practice whole-abstraction reasoning before implementing manual projection.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 25

The Bell Thief's Last Message

A one-use courier leaves a value in a locked chest and rings a scheduler bell. Some couriers leave before ringing; some readers change which task is waiting; some chests are abandoned. The town records bells instead of chest state, so messages and sleepers drift apart. Your mission is to design a tiny one-shot channel future in which the value or closure state persists independently of whether a notification is coalesced.

Mission Specify a single-sender, single-receiver future protocol with no lost completion or closure wakeup.

Suggested loadout Level 20, Level 21, Level 23, Level 24. These are guidance, not locked gates.

Mastery Targets

- Polling registers interest and checks readiness under a race-free protocol.
- Dropping an unresolved sender makes a waiting receiver observe closure rather than remain Pending forever.
- Completion wakes outside the protected critical section, and a later poll can replace an outdated waker.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 25

Wakeups Connect Resources to Schedulers

A pending future has two obligations that belong to different components. The resource side must arrange a notification when progress becomes possible. The executor side must turn that notification into another opportunity to poll the task. A `Waker` is the capability connecting those components. It identifies a task to the scheduling machinery without requiring the resource to know the task's concrete future type.

This separation makes async composition efficient, but it also creates a protocol that can fail in subtle ways. A notification delivered just before a

task goes to sleep must not be lost. Repeated notifications need not cause repeated simultaneous polls. A future moved between scheduling contexts must not retain only an obsolete waker. These are state transition problems, not merely questions of calling a method named `wake`.

Wake Means Poll Again

A wakeup does not mean the requested operation is complete. A socket may become readable but still contain too few bytes for a frame. Another consumer may take the available item first. A timer notification may race with cancellation. The future must recheck its own readiness predicate whenever polled.

Wakeups can also be coalesced. If a task is already queued, five more notifications may be represented by the same runnable state. The executor owes an opportunity to make progress, not necessarily one poll per notification. A resource that counts events only by counting wake calls is therefore using the wrong storage. Store events or readiness in resource state, and use the waker only to prompt observation.

`Waker` is cloneable and usable across threads. A resource can retain a clone while the original `Context` exists only during one poll call. The resource should update its registration when the current task changes. The most recently supplied context is the relevant one for future wakeups, so blindly keeping the first waker is incorrect.

The safe `Wake` trait lets a program construct a waker from an `Arc` backed type. It avoids the manual pointer and vtable invariants required by `RawWaker`. Use the safe route when its ownership model fits. A custom raw implementation is a low level optimization with obligations around cloning, consumption, thread safety, and destruction.

A Small Executor With a Predicate

The following complete executor runs one future on the current thread. A mutex protected boolean records whether a notification has happened. The condition variable waits for that predicate rather than treating a notification as durable state.

Listing 25.1 Rust example

```
use std::future::Future;
use std::sync::{Arc, Condvar, Mutex};
use std::task::{Context, Poll, Wake, Waker};

struct Signal {
    notified: Mutex<bool>,
    changed: Condvar,
}

impl Signal {
    fn notify(&self) {
        *self.notified.lock().unwrap() = true;
        self.changed.notify_one();
    }

    fn wait(&self) {
        let mut notified = self.notified.lock().unwrap();
        while !*notified {
            notified = self.changed.wait(notified).unwrap();
        }
        *notified = false;
    }
}

impl Wake for Signal {
    fn wake(self: Arc<Self>) {
        self.notify();
    }

    fn wake_by_ref(self: &Arc<Self>) {
        self.notify();
    }
}

fn run<F: Future>(future: F) -> F::Output {
    let signal = Arc::new(Signal {
        notified: Mutex::new(false),
        changed: Condvar::new(),
    });
    let waker = Waker::from(Arc::clone(&signal));
    let mut context = Context::from_waker(&waker);
    let mut future = Box::pin(future);
    loop {
        match future.as_mut().poll(&mut context) {
            Poll::Ready(output) => return output,
            Poll::Pending => signal.wait(),
        }
    }
}
```

Listing 25.1 continued

```
fn main() {
    assert_eq!(run(async { 6 * 7 }), 42);
}
```

Analyze the important race. A future can call `wake_by_ref` during its poll and then return pending. The boolean is already true when `wait` acquires the mutex, so the executor does not sleep. If the notification occurs while `wait` is about to block, the condition variable's release and wait operation coordinates with the same mutex protected predicate. Spurious wakeups are harmless because the predicate is checked in a loop.

There is no queue of tasks here, no timer wheel, no I/O reactor, and no worker stealing. The implementation is intentionally narrow: one pinned future, one notification state, one waiting thread. It demonstrates why the readiness predicate must survive the timing of the notification. A production executor adds many responsibilities without changing that basic requirement.

This executor must not be used to wait on an operation that assumes a different runtime is driving its reactor. A future's trait implementation does not imply independence from runtime services. A runtime specific timer may need registration with that runtime, and an I/O object may require its reactor. The generic polling interface is shared; the surrounding environment can still have requirements.

Registering a Waker Without Losing Readiness

The resource side needs a complementary protocol. A naive implementation checks whether a result exists, then stores a waker, then returns pending. If another thread produces the result between the check and registration, it may find no waker to notify. The consumer then sleeps despite the result already being available.

One straightforward solution protects readiness and registration with the same mutex. This library example defines a one shot completion pair. It focuses on successful completion; a production version would also define sender disappearance and receiver cancellation.

Listing 25.2 Rust example

```
use std::future::Future;
use std::pin::Pin;
use std::sync::{Arc, Mutex};
use std::task::{Context, Poll, Waker};

struct Shared<T> {
    value: Option<T>,
    waiter: Option<Waker>,
}

struct Completion<T>(Arc<Mutex<Shared<T>>>);
struct Promise<T>(Arc<Mutex<Shared<T>>>);

fn completion_pair<T>() -> (Completion<T>, Promise<T>) {
    let shared = Arc::new(Mutex::new(Shared {
        value: None,
        waiter: None,
    }));
    (Completion(Arc::clone(&shared)), Promise(shared))
}

impl<T> Completion<T> {
    fn complete(self, value: T) {
        let waiter = {
            let mut shared = self.0.lock().unwrap();
            shared.value = Some(value);
            shared.waiter.take()
        };
        if let Some(waiter) = waiter {
            waiter.wake();
        }
    }
}

impl<T> Future for Promise<T> {
    type Output = T;

    fn poll(
        self: Pin<&mut Self>,
        context: &mut Context<'_>,
    ) -> Poll<T> {
        let next = context.waker().clone();
        let mut shared = self.0.lock().unwrap();
        if let Some(value) = shared.value.take() {
            return Poll::Ready(value);
        }
        let previous = shared.waiter.replace(next);
        drop(shared);
        drop(previous);
```

Listing 25.2 continued

```
            Poll::Pending
        }
    }
```

If completion obtains the lock first, the next poll observes the value. If polling obtains the lock first, it registers the waker before completion can store the value and take that waker. There is no interval in which readiness can be published without either being observed or having a registered notification path.

The producer wakes after unlocking. The consumer clones before locking and drops the displaced waker after unlocking. Clone and drop may run custom waker code as well. On the ready path, Rust drops the lock guard before the unused clone because the guard was declared later.

This example intentionally leaves an important failure case visible: if `Completion` is dropped without completing, `Promise` can remain pending forever. Adding a `closed` flag and waking on sender drop would let the future return a result describing cancellation. The future's output type would then change to represent both completion and disconnection. Failure semantics belong in the state machine from the beginning.

Readiness Must Outlive Notifications

A readiness flag is often level triggered: while the condition remains true, polling can observe it. A notification is a transient scheduling event. Combining a persistent predicate with notifications makes coalescing safe. If three items arrive while a task is already queued, a single later poll can inspect the queue and process some or all of them.

Edge triggered systems require especially careful draining and rearming rules. If code consumes only part of the available work and then waits for a new edge that never occurs, it can stall with work still present. The abstraction must define when readiness is cleared and how another notification becomes guaranteed.

Even a level triggered design can lose progress if it clears readiness after checking the resource in the wrong order. The exact order depends on the primitive. Mutex based code can combine state and registration directly. Lock free registration often uses a double check or a specialized atomic

waker abstraction. Copying only one atomic operation from such an algorithm is unsafe reasoning because the protocol spans several transitions.

The resource should also avoid retaining stale wakers indefinitely. A waker may own an executor task or scheduling object. If a cancelled future leaves its registration in a long lived resource, memory can remain retained after the application believes the task has ended. Registration cleanup and ownership cycles are resource management issues, even when no use after free is possible.

Fair Polling Requires Work Budgets

A task can wake itself when it has more immediately available work than it should process in one poll. Returning pending after arranging another wake gives the executor a chance to schedule peers. This is a cooperative yield, not an operating system preemption guarantee.

The following library future limits how many arithmetic steps it performs per poll. It demonstrates the scheduling pattern without depending on a runtime.

Listing 25.3 Rust example

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll};

struct SumRange {
    next: u64,
    end: u64,
    sum: u64,
}

impl Future for SumRange {
    type Output = u64;

    fn poll(
        mut self: Pin<&mut Self>,
        context: &mut Context<'_>,
    ) -> Poll<u64> {
        for _ in 0..128 {
            if self.next >= self.end {
                return Poll::Ready(self.sum);
            }
            self.sum = self.sum.wrapping_add(self.next);
```

Listing 25.3 continued

```
            self.next += 1;
        }
        context.waker().wake_by_ref();
        Poll::Pending
    }
}
```

The wrapping addition makes overflow behavior explicit. The progress budget is arbitrary and would need measurement in a real workload. A budget measured in items is not a reliable time bound if item costs vary dramatically. The important property is that the future retains partial progress, arranges another poll, and returns control after bounded local work.

An executor can still provide unfair scheduling despite a cooperative future. For example, repeatedly putting a self waking task at the front of a queue can starve peers. Fairness is a relationship between task behavior and scheduling policy. A library can avoid monopolizing a poll, while the executor remains responsible for how runnable tasks are selected.

Why Concurrent Polling Is Forbidden by Ownership

`poll` takes `Pin<&mut Self>`, which represents exclusive access to the future for that invocation. An executor must not use unsafe machinery to poll the same future concurrently on two workers. A task can receive a wake while already running; that should update its scheduling state, not create a second simultaneous mutable access.

A common task state machine distinguishes idle, queued, running, notified while running, and completed. The details vary, but the obligation is stable: preserve a pending notification without violating exclusive polling. Completing a task must also prevent stale wakeups from resurrecting a destroyed future. The waker may remain alive after completion, so its scheduling object needs a harmless terminal behavior.

These executor internals are a strong reason to use established runtimes for applications. Understanding them is still valuable: it explains why a future can hang despite correct business logic, why a wake storm need not produce one poll per event, and why blocking in a poll affects unrelated work.

Training Encounters

1. A future checks an empty queue, releases a lock, stores its waker, and returns pending. Construct an interleaving that strands it while the queue contains an item.

2. A resource calls a waker ten times, but the executor polls once. Explain why this can be correct and what state must preserve the ten logical events.

3. A task is woken while one worker is polling it. Explain why immediately polling it on another worker is invalid and describe the scheduling state needed instead.

Training Debriefs

1. The producer inserts an item after the empty check but before registration. It sees no registered waker and sends no useful notification. The consumer then registers and sleeps without rechecking. Protect readiness and registration together, or use a proven registration protocol that rechecks after registration and handles the race.

2. Wakeups are scheduling prompts and may coalesce. Store the events in a queue, counter, or other resource state that a poll can inspect. The executor need only ensure an opportunity to observe the ready state. Using wake calls as the sole event count confuses notification with data storage.

3. Concurrent polling would require overlapping exclusive access to the same future. Record that another poll is needed, and enqueue or repoll only after the current invocation returns. A running plus notified state prevents the wake from being lost while preserving one poll at a time.

Further Reading

- Waker defines notification and current registration behavior.
- Wake provides safe construction of `Arc` backed wakers.
- Context supplies the current task's waker during polling.

The Unrung Completion

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a one-shot pair using Arc, Mutex, and safe Waker operations. One noncloneable sender can resolve one owned value or drop without sending. The receiver's waker may change across polls. Model waiting, ready value, sender gone, and receiver gone as states or equivalent data. Explain send, poll, and the drop of each endpoint. A sender cannot overwrite a result. A surviving sender must retain neither a payload nor a waker after the receiver abandons the operation. Test completion racing with registration. Use stored state as the authority and wake only after releasing the lock.

Encounter Constraints

- The sender is not cloneable; multiple-producer reference accounting is outside this boss.
- Do not call wake while holding the state mutex.
- Do not treat the number of wake calls as the number of stored results.

Hint one / Hint two / Reveal debrief

Side Quest

Use a test waker to count scheduling requests and track its lifetime. Test send before first poll, sender drop after Pending, receiver drop after Pending while the sender stays alive, and a poll with a new waker. Check that receiver drop releases the registered waker. Explain which outcomes are required, then identify the sender-count bookkeeping needed to make the sender cloneable.

Capability unlocked Wake Lantern reveals the persistent predicate behind a scheduling signal.

Choose Your Exit

Cross the cancellation boundary / Level 26
Treat endpoint destruction as part of a larger operation's observable behavior.

Study service ownership / Level 28
Assign an observer to every task and outstanding result.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 26

The Bridge With a Missing Middle

Ferrum's export bridge writes a frame in small pieces. Operators may cancel a wait at any time and later resume, but the current implementation stores the byte offset only inside the temporary future. After cancellation, the connection still looks ready while its next write begins in the middle of an old frame. The bytes are memory-safe; the conversation is ruined. Your repair must leave every interrupted operation in a state another method can understand.

Mission Keep resumable partial-write progress in its resource owner across future cancellation.

Suggested loadout Level 6, Level 15, Level 23, Level 24, Level 25. These are guidance, not locked gates.

Mastery Targets

• Dropping the operation future cannot discard the frame buffer or its acknowledged local write offset.

• A new frame cannot interleave with an unfinished frame through the safe public API.

• Tests cancel at controlled Pending boundaries and resume without skipping or duplicating locally accepted bytes.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 26

Cancellation Is Destruction at a Boundary

An async operation can stop being polled because its owner drops it. A timeout, a losing branch in a race, a disconnected caller, or service shutdown can all lead to that event. The future's stored values are then destroyed, but statements that would have run after the current suspension point do not run. Cancellation therefore connects control flow directly to ownership and destruction.

The difficult question is what has already happened. Dropping a future can release a local reservation, but it cannot erase a packet already sent or a database commit already accepted. A good cancellation contract identifies the boundary between reversible local preparation and externally visible effects. It also defines what the caller may conclude when that boundary was crossed but no result was received.

Cleanup Must Exist in Stored State

Consider code that acquires a permit, awaits work, and then explicitly returns the permit. If the future is dropped during the await, the final return statement is skipped. Resource release must instead be represented by an owned guard whose destructor runs when suspended state is destroyed.

This complete program reserves capacity and then suspends forever. Dropping the future restores capacity through the guard.

Listing 26.1 Rust example

```
use std::cell::Cell;
use std::future::{self, Future};
use std::rc::Rc;
use std::task::{Context, Poll, Waker};

struct Pool {
    free: Rc<Cell<usize>>,
}

struct Permit {
    free: Rc<Cell<usize>>,
}

impl Pool {
    fn take(&self) -> Option<Permit> {
        let available = self.free.get();
        if available == 0 {
            return None;
        }
        self.free.set(available - 1);
        Some(Permit { free: Rc::clone(&self.free) })
    }
}

impl Drop for Permit {
    fn drop(&mut self) {
        self.free.set(self.free.get() + 1);
```

Listing 26.1 continued

```
    }
}

async fn hold(pool: &Pool) {
    let _permit = pool.take().expect("capacity available");
    future::pending::<()>().await;
}

fn main() {
    let pool = Pool { free: Rc::new(Cell::new(1)) };
    let mut operation = Box::pin(hold(&pool));
    assert_eq!(pool.free.get(), 1);
    let mut context = Context::from_waker(Waker::noop());
    assert_eq!(operation.as_mut().poll(&mut context), Poll::Pending);
    assert_eq!(pool.free.get(), 0);
    drop(operation);
    assert_eq!(pool.free.get(), 1);
}
```

Before polling, the body has not acquired a permit. After the first poll, the guard is retained in suspended state. Dropping the boxed future destroys that state and returns the permit. The example uses `Rc<Cell<_>>` because all operations occur on one thread; a concurrent pool would use a synchronization design appropriate to its accounting.

The destructor must remain simple and reliable. It cannot perform an ordinary async wait, and panicking during cleanup can create additional failure paths. Returning an in memory permit is a good fit. Completing a remote rollback that may block indefinitely is not. Ownership based cleanup solves resource lifetime locally; it does not turn a network protocol into a synchronous destructor.

Cancellation Safety Is Operation Specific

One useful question is whether dropping and recreating an operation loses progress or changes meaning. A receive future may be designed so that cancellation before completion leaves the next message available. A multi step parser may instead consume some bytes into its own temporary buffer and lose them when dropped. Both can be memory safe, yet only one is restartable under that particular calling pattern.

The answer depends on where partial progress lives. If consumed bytes live in a connection object's persistent buffer, cancelling one read attempt

can leave the connection ready for the next attempt. If they live only inside the future, destruction may discard them. Designing persistent state outside the cancellable operation can make retries safe, at the cost of a more explicit state machine.

A write operation has a different problem: some prefix may already have reached the peer. Restarting the whole write can duplicate bytes. Even when an API documents that a particular primitive is cancellation safe, a larger composition may not be. A sequence of individually restartable steps can still have an externally visible effect between them.

Read cancellation documentation at the granularity of the actual operation. A runtime's selection macro can describe how branches are dropped, but it cannot promise that every arbitrary future placed in a branch is safe to abandon. The future's own resource protocol supplies that part of the contract.

Separate Preparation From Commit

A useful design makes preparation an owned value and commit an explicit transition. Preparation can allocate buffers, validate input, and reserve local capacity. Commit then transfers the prepared value into the authoritative owner. Whether commit itself can suspend is an important API choice.

The following library example keeps the domain transition synchronous. It is a small model of a larger design, not a claim that durable storage can always commit without waiting.

Listing 26.2 Rust example

```
#[derive(Debug, PartialEq)]
struct Receipt {
    index: usize,
}

struct Prepared {
    record: String,
}

struct Ledger {
    records: Vec<String>,
}
```

Listing 26.2 continued

```
impl Prepared {
    fn validate(record: String) -> Result<Self, &'static str> {
        if record.trim().is_empty() {
            return Err("empty record");
        }
        Ok(Self { record })
    }

    fn commit(self, ledger: &mut Ledger) -> Receipt {
        let index = ledger.records.len();
        ledger.records.push(self.record);
        Receipt { index }
    }
}

async fn prepare(record: String) -> Result<Prepared, &'static str> {
    std::future::ready(()).await;
    Prepared::validate(record)
}
```

Dropping `Prepared` before commit discards an uncommitted local record. Once `commit` returns, the record belongs to the ledger. There is no await inside the state transition, so task cancellation cannot intervene through an async suspension in the middle of this function. Ordinary panics and allocation failures still require their own analysis; absence of await is not a universal transaction guarantee.

For a remote system, commit usually involves uncertainty. The server can accept a request while the client times out before receiving a reply. A stable operation identifier lets the client query or retry without inventing a second operation. The durable authority must define deduplication and retention, because a local cancellation guard cannot know the remote outcome.

Timeouts Bound Waiting Under Assumptions

A timeout commonly races an operation against a timer. If the timer wins, the wrapper drops or stops polling the operation and returns a timeout result. This bounds the caller's wait only if the executor can poll the timer and the operation yields control. A blocking poll or long synchronous computation can prevent the timeout branch from running promptly.

A timeout also does not necessarily stop independently spawned work. A future awaiting a worker's reply may be dropped while the worker continues. The result receiver disappears, but the job's owner is elsewhere. This can be correct if the service deliberately completes accepted work, or wasteful if the caller expected cancellation to propagate.

Define cancellation propagation explicitly. Does dropping a request signal the worker? Does the worker check that signal between stages? Can the worker stop after an external effect begins? Does it still record the final outcome for later lookup? These are protocol questions that cannot be answered solely by the lifetime of a local future.

Deadlines often compose better than repeated relative timeouts. Passing one absolute deadline through several layers prevents each layer from consuming the full budget independently. The clock and deadline representation should match the runtime's timer system. Retries should consume the same overall budget unless the API intentionally grants a new one.

Cooperative Cancellation Has Checkpoints

An independent worker can observe a cancellation token. The token asks the worker to stop; it does not preempt arbitrary instructions. The worker chooses safe checkpoints at which its invariants permit exit.

This complete threaded example makes that distinction visible without an async runtime.

Listing 26.3 Rust example

```
use std::sync::Arc;
use std::sync::atomic::{AtomicBool, Ordering};
use std::thread;

struct CancelOnDrop(Arc<AtomicBool>);

impl Drop for CancelOnDrop {
    fn drop(&mut self) {
        self.0.store(true, Ordering::Release);
    }
}

fn main() {
    let cancelled = Arc::new(AtomicBool::new(false));
    let owner = CancelOnDrop(Arc::clone(&cancelled));
```

Listing 26.3 continued

```
    let worker = thread::spawn(move || {
        let mut batches = 0_u64;
        while !cancelled.load(Ordering::Acquire) {
            batches = batches.wrapping_add(1);
            thread::yield_now();
        }
        batches
    });
    drop(owner);
    let _completed_batches = worker.join().unwrap();
}
```

The worker may perform zero or many batches before observing cancellation. Joining establishes that it actually stopped. Dropping the guard alone only changes the flag. A real worker should place checkpoints between bounded units of work and define which partial results are retained or discarded.

If a checkpoint occurs while a local invariant is temporarily broken, stopping there can expose damaged state. A good decomposition makes each unit complete a coherent transition before checking cancellation again. Long blocking calls may not offer such checkpoints, so cancellation latency includes their duration unless the underlying API provides its own interruption mechanism.

Dropping a Handle and Stopping Work Differ

A task handle is an ownership relationship with runtime specific semantics. Dropping it might detach the task, request cancellation, or perform some other documented action. Never infer the behavior from the generic word handle. Likewise, requesting an abort may require later scheduling before the task's future is actually destroyed.

The distinction matters during shutdown. A service that drops every handle may leave background work alive. A service that requests cancellation but never waits may exit before cleanup finishes. A structured owner should retain the capabilities needed to observe termination, collect failures, and release resources in a defined order.

Cancellation can also race with completion. A caller may receive a cancellation result even though an effect completed just before the signal

was processed. The API should avoid promising that cancellation means nothing happened unless it has a protocol that truly establishes that fact. Often the honest result is that the caller stopped waiting and must consult an operation identifier for the final outcome.

Design Around Observable Outcomes

Write a cancellation table for each meaningful phase: not started, locally prepared, submitted externally, committed, and acknowledged. For each phase, state who owns resources, what dropping the future does, and what the caller can safely retry. This table is an engineering tool rather than a public ritual; it exposes missing cases before they become production incidents.

Keep local cleanup and domain compensation distinct. Releasing a buffer is cleanup. Sending a reversing transaction is a new domain operation that can fail independently and may need an audit trail. A destructor should not silently pretend that compensation restores history.

When cancellation cannot safely interrupt an operation, isolate that operation behind an owner that continues to completion and records the result. The caller can abandon its wait while the owner preserves consistency. This can increase background work, so admission control must account for accepted operations that no longer have waiting callers.

Training Encounters

1. A future takes a permit, awaits a network response, and returns the permit with an explicit final statement. Explain the cancellation leak and redesign ownership.

2. A timeout returns an error while a charge request may already have reached a server. State what the timeout proves and what additional protocol is needed for a safe retry.

3. A worker checks a cancellation flag after removing an item from a queue but before recording where it will go. Explain the state loss risk and propose a checkpoint boundary.

Training Debriefs

1. Dropping suspended state skips the final statement. Store an owned permit guard across the await and release capacity in its destructor. Ensure the destructor performs bounded local cleanup and cannot require an async wait. If release itself is remote, an owning service must manage that protocol separately.

2. The caller's waiting policy expired; the remote effect may be absent, in progress, or committed. Use a stable operation identifier and an authoritative deduplication or status lookup protocol. Retrying the same logical operation must not create a new charge merely because the previous acknowledgement was lost.

3. The item has left durable or persistent queue state but has not entered another accounted ownership state. Cancellation can make it disappear from the intended workflow. Retain it in an owned in flight record with a defined drop policy, or check cancellation before removal and again only after a coherent handoff. The correct boundary is where every item still has an accountable owner.

Further Reading

- Destructors defines scope based destruction behavior.
- Tokio select cancellation safety gives operation specific cancellation examples.
- Tokio task cancellation documents the distinction between task handles and cancellation behavior.

The Vanishing Cursor

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design an Exporter that owns a pending encoded frame and a byte offset. A temporary future borrows it to advance a mock sink. Define the sink contract explicitly: Ready with a count means that many bytes were accepted, while Pending accepts no bytes for that invocation. The future may be dropped after any returned Pending. Progress must remain available for a later resume operation, and ordinary send-new-frame must refuse while a frame is unfinished. Build a deterministic mock that sometimes accepts a short prefix and sometimes returns Pending. Describe or implement the transition order that updates the offset after Ready without adding a suspension point in between. Separately explain why this local byte-progress contract does not prove whether a remote peer committed the logical request.

Encounter Constraints

• Keep indispensable progress in Exporter rather than only in the borrowed future.

• No asynchronous work may be assumed to run from Drop.

• The primary mock contract must not hide external effects behind a Pending result.

Hint one / Hint two / Reveal debrief

Side Quest

Replace the local mock sink with a remote request protocol that can lose its acknowledgement. Write three distinct outcomes: no attempt accepted locally, partial or complete local transmission, and peer-confirmed completion. Decide which outcomes permit a fresh identifier and which require consulting the same identifier. The task is to preserve uncertainty honestly; a timeout or dropped future is not an undo command sent backward through the network.

Capability unlocked Resume Rune keeps the operation's history outside a disposable wait.

Choose Your Exit

Publish the future contract / Level 27
Expose borrowing and cancellation behavior through a reusable async interface.

Navigate the protocol fortress / Level 40
Integrate partial progress into typed and runtime connection states.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 27

The Guild of Unnamed Futures

The Wilds' inspection guild accepts scanners from different workshops. Each scanner borrows the inspected bytes and may suspend, while the guild wants to place returned work on a multithreaded scheduler. A scanner being Send and Sync has been mistaken for evidence about every future it creates. Your mission is to publish the operation's actual bounds, preserve useful borrowing, and choose one clear location where concrete implementations become erased plugins.

Mission Design static and erased async scanner interfaces with deliberate Send and lifetime contracts.

Suggested loadout Level 4, Level 9, Level 10, Level 19, Level 23, Level 24. These are guidance, not locked gates.

Mastery Targets

- The static interface explicitly guarantees the returned future's Send property where generic callers need it.
- The erased interface permits borrowing scanner and input without accidentally requiring static data.
- A separate owned boundary explains what an unscoped scheduler's static requirement would demand.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 27

Async Traits Publish Future Contracts

A synchronous trait method publishes the types of its arguments and result. An async trait method additionally publishes a family of suspended computations. Those computations can borrow the receiver, retain arguments, depend on a runtime, and have auto traits that determine where they may be scheduled. Designing the trait therefore requires thinking about both the service object and the future returned by each call.

The distinction is easy to miss. A service can be `Send + Sync` while a particular returned future is not `Send`. Conversely, a non shareable owner may produce an owned future that is safe to transfer. Bounds on the receiver and bounds on the operation answer different questions. This level makes those questions explicit and compares several stable ways to represent the resulting contracts.

Native Async Methods and Static Dispatch

An async method in a trait describes an implementation supplied future type. Under static dispatch, the compiler can work with the concrete implementation and its generated future. Callers get ordinary await syntax without every implementation allocating a boxed future.

Listing 27.1 Rust example

```
trait Lookup {
    async fn find(&self, key: &str) -> Option<String>;
}

struct PrefixLookup {
    prefix: String,
}

impl Lookup for PrefixLookup {
    async fn find(&self, key: &str) -> Option<String> {
        key.strip_prefix(&self.prefix).map(str::to_owned)
    }
}

async fn query<L: Lookup>(lookup: &L) -> Option<String> {
    lookup.find("item:42").await
}
```

This is complete library code. It establishes no runtime dependency because the implementation is immediately ready, though real implementations could await I/O. The trait's async syntax alone does not guarantee that every returned future is sendable to a multithreaded scheduler.

A public trait should decide whether that omission is intentional. Local applications can often let concrete types determine auto traits at call sites. A generic public abstraction may need to promise `Send` so that downstream generic callers can spawn the operation. Adding the promise later can

exclude implementations, while omitting it can limit callers. This is an API compatibility decision.

Native async trait methods are not generally callable through a trait object merely because the trait can be named. The vtable needs a uniform calling convention for the erased method, while each implementation's hidden future can have a different representation. An explicit erased return type supplies a common representation when dynamic dispatch is required.

The following complete program is deliberately rejected at the attempt to form a trait object.

Listing 27.2 Rejected example

```
trait Lookup {
    async fn find(&self, key: &str) -> Option<String>;
}

fn use_erased(_lookup: &dyn Lookup) {}

fn main() {}
```

The limitation concerns dyn compatibility, not whether async methods are useful. Static generic code can still call them. If only a subset of methods needs erasure, splitting the trait can keep the static interface expressive and provide a separate object compatible boundary.

Promising Send on the Returned Future

A method returning `impl Future + Send` exposes the operation's scheduling property explicitly. The implementation can still use an async block or async function style internally. The trait contract now requires every implementation to produce a sendable future for that method.

Listing 27.3 Rust example

```
use std::future::Future;

trait Fetch {
    fn fetch(&self) -> impl Future<Output = Vec<u8>> + Send;
}

struct Memory {
    bytes: Vec<u8>,
}
```

Listing 27.3 continued

```
impl Fetch for Memory {
    fn fetch(&self) -> impl Future<Output = Vec<u8>> + Send {
        async move { self.bytes.clone() }
    }
}

fn require_send<T: Send>(_: T) {}

fn main() {
    let source = Memory { bytes: vec![1, 2, 3] };
    require_send(source.fetch());
}
```

The future captures `&Memory`, so its sendability depends on `Memory` being shareable in the required way. If an implementation retained a shared reference to a `RefCell` across suspension, it would fail this contract. The error would correctly identify a conflict between implementation storage and the trait's promised scheduling model.

`Send` is still not `'static`. The future above borrows `source`; a spawn API requiring an owned `'static` future cannot accept that borrow merely because the future is sendable. One solution moves an owned shared service handle into a task and calls the borrowing method inside the task. Another designs an operation that owns the required input and returns a future independent of a short lived borrow.

Avoid adding `Sync` to every future by habit. Executors commonly need to move a future between worker threads while preserving exclusive polling, which calls for `Send`, not shared concurrent polling through `&Future`. Extra bounds restrict implementations and should correspond to an actual caller requirement.

Boxing at a Dynamic Boundary

Dynamic dispatch needs a uniform returned type. A pinned boxed trait object is a common representation: the box owns storage of unknown concrete size, `dyn Future` erases the implementation type, and `Pin` supports polling address sensitive futures.

Listing 27.4 Rust example

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};

type Operation<'a, T> =
    Pin<Box<dyn Future<Output = T> + Send + 'a>>;

trait Reader: Send + Sync {
    fn length(&self) -> Operation<'_, usize>;
}

struct MemoryReader {
    bytes: Vec<u8>,
}

impl Reader for MemoryReader {
    fn length(&self) -> Operation<'_, usize> {
        Box::pin(async move { self.bytes.len() })
    }
}

fn main() {
    let reader: Box<dyn Reader> = Box::new(MemoryReader {
        bytes: vec![4, 5, 6],
    });
    let mut operation = reader.length();
    let mut context = Context::from_waker(Waker::noop());
    assert_eq!(operation.as_mut().poll(&mut context), Poll::Ready(3));
}
```

The lifetime on `Operation` says that the erased future may borrow from the receiver. Omitting it and accidentally demanding `'static` would exclude this straightforward implementation. Erasure does not erase lifetime constraints; it packages a concrete value behind a uniform interface while retaining the required bounds.

There can be two dynamic dispatch steps: one to choose the service implementation and another whenever polling the erased future. There is also a heap allocation for the boxed future in this design. These costs may be negligible for network operations and significant for tiny in memory calls. The benefit is a stable representation for heterogeneous implementations and a smaller amount of monomorphized caller code.

Box only where dynamic behavior is needed. A generic internal pipeline can keep concrete future types and erase them at a plugin registry or

configuration boundary. Conversely, a system with many implementation combinations may benefit from erasure earlier to control code size and build time. Measure the workload rather than treating static dispatch as a universal quality ranking.

Associated Future Families

Generic associated types can name a family of future types indexed by the receiver borrow. This makes the relationship explicit and can allow callers to express bounds on the associated future family. Stable implementations must still provide a nameable concrete type; using an unnamed async block as an associated type is not generally solved by simply writing `impl Future` in the associated type definition.

This complete library example uses `Ready`, whose type can be named directly.

Listing 27.5 Rust example

```
use std::collections::HashMap;
use std::future::{self, Future, Ready};

trait Store {
    type Read<'a>: Future<Output = Option<&'a str>> + 'a
    where
        Self: 'a;

    fn read<'a>(&'a self, key: &'a str) -> Self::Read<'a>;
}

struct MemoryStore {
    entries: HashMap<String, String>,
}

impl Store for MemoryStore {
    type Read<'a> = Ready<Option<&'a str>>;

    fn read<'a>(&'a self, key: &'a str) -> Self::Read<'a> {
        future::ready(self.entries.get(key).map(String::as_str))
    }
}
```

The returned reference borrows the store, so the store cannot be destroyed while the output is used. The future itself also has a lifetime relationship to the call. This model resembles lending interfaces: each call creates an

operation tied to a particular borrow rather than one universal operation type independent of all borrows.

The associated type approach is verbose, but the verbosity can be useful when a library needs callers to reason about the exact future family. For simple application traits, native async methods may be clearer. For heterogeneous registries, boxed futures may be simpler. Choose the representation based on the boundaries that callers actually cross.

Mutable Receivers Serialize Borrowing

An async method taking `&mut self` retains exclusive access for as long as the returned future requires it. Two concurrent calls on the same owner are therefore not automatically possible. This can be a feature: a protocol connection may permit only one in flight operation because its framing state is sequential.

Changing the receiver to `&self` merely to allow concurrent calls requires an implementation strategy. A mutex can serialize access dynamically. Splitting the connection into independent read and write halves can express genuine independence. An actor can own the connection and schedule requests. A multiplexed protocol can assign stream identifiers and maintain separate operation state. The receiver type should reflect the protocol rather than serve as a workaround for a borrow error.

Borrowed buffers introduce similar choices. Polling a read future to `Poll::Ready` leaves the future in place, where it can still hold its `&mut [u8]` borrow. Drop the underlying future or let it go out of scope to release a retained borrow. An owned buffer operation can return the allocation with the result, which may suit detached tasks or a buffer pool.

A borrowed output can keep the buffer borrowed even after the future is gone. Check how long that result's borrow stays live before modifying the buffer. If callers need to keep several results while continuing reads, return owned records or shared immutable chunks. One mutable buffer cannot hold several versions of the same bytes at once.

Runtime and Cancellation Are Part of the Interface

The `Future` bound alone does not state which runtime services an implementation requires. A trait intended to be runtime neutral should avoid constructing runtime specific timers or I/O objects behind an undocumented assumption. If a runtime is required, make the requirement clear in construction or documentation.

Cancellation also belongs in the method contract. Does dropping the future leave the service reusable? Can partial bytes remain in an internal buffer? Is an accepted request still processed after the caller stops waiting? A trait that permits several implementations should require a common minimum behavior or expose differences explicitly. Otherwise generic code can accidentally rely on behavior provided only by the in memory test implementation.

Error types should reflect the operation boundary. An error can distinguish rejection before submission from uncertainty after submission. Returning only a generic transport failure may prevent callers from choosing a safe retry policy. Associated error types can preserve implementation specificity, while an application level enum can provide a stable set of recovery categories.

Training Encounters

1. A service type is `Send + Sync`, but a generic caller cannot prove that its async method's future is `Send`. Explain why the service bounds do not settle the future bound and propose an explicit trait signature.

2. A boxed future alias omits a lifetime and defaults to a requirement that excludes borrowing the receiver. Explain the resulting design pressure and how to express a borrowing erased operation.

3. A connection method takes `&mut self`, and callers want two simultaneous operations. Give three architectural alternatives and explain why simply changing the receiver to `&self` is incomplete.

Training Debriefs

1. The method can create and retain non sendable locals across suspension even when the service itself is shareable. A signature returning `impl Future<Output = T> + Send` publishes the needed operation guarantee. A boxed `dyn Future + Send` boundary is another option when erasure is also needed. Receiver and operation bounds should be reviewed separately.

2. An unnecessarily static erased future can force cloning or moving data that could safely be borrowed. Use a lifetime indexed alias such as `Pin<Box<dyn Future<Output = T> + Send + 'a>>` and tie `'a` to the receiver borrow. This preserves the lifetime contract while erasing the concrete future representation.

3. Split truly independent resources, serialize commands through an owner, or implement protocol level multiplexing. A lock is another explicit serialization option. Changing the signature alone does not establish how simultaneous operations update framing state, allocate request identifiers, or handle cancellation. The implementation needs a concurrency protocol consistent with the new access surface.

Further Reading

- Trait dyn compatibility describes which methods can participate in trait objects.
- Async functions in traits explains stable syntax and public `Send` design choices.
- Associated types documents generic associated type families.

The Contract Hydra

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a scanner operation that borrows both its receiver and an input byte slice, suspends, and returns an owned inspection report. First give generic callers an interface whose returned future is explicitly Send. Then design an object-safe adapter returning a pinned boxed future with the appropriate borrowing lifetime and Send bound, so heterogeneous scanners can share one registry. Demonstrate the intended contracts with compile-time helper functions rather than requiring an actual runtime: one helper accepts Send futures, another accepts Send plus static futures. Show why a call borrowing a local input may satisfy the first and fail the second. Provide an owned request wrapper or another explicit handoff design for the unscoped case, and account for any copies or shared ownership it introduces.

Encounter Constraints

- Do not infer the future's Send property solely from scanner Send or Sync bounds.
- Keep the inspection report owned so its lifetime does not introduce another hidden output borrow.
- Do not use native async trait syntax as if it automatically supplied an erased dyn interface.

Hint one / Hint two / Reveal debrief

Side Quest

Add a scanner whose synchronous preparation uses Rc, then drops that temporary before its first suspension. Explain how it may still produce a Send future if no non-Send state survives the relevant boundary. Contrast it with a scanner that retains Rc across suspension and is valid only for local execution. Decide whether the registry should reject that implementation or offer a separate local adapter with an intentionally weaker operation contract.

Capability unlocked Future Seal publishes thread and lifetime obligations at the operation boundary.

Choose Your Exit

Supervise the guild / Level 28
Own accepted operations, their failures, and their shutdown outcomes.

Return to erasure / Level 10
Reassess which concrete facts a dynamic interface must preserve.

Mark your quest log / Return to the map

ZONE 3 / THE CLOCKWORK WILDS / LEVEL 28

The Last Light in the Wilds

The district is closing its gates for repair. Some jobs are queued, some workers are running, one has failed, and another may have changed a remote system before disappearing. The old supervisor reports clean shutdown as soon as it sends cancellation requests. Your final mission in the Clockwork Wilds is to build a report that earns its certainty. Every accepted operation must end somewhere on the ledger, including the places where certainty is impossible.

Mission Design an accountable shutdown protocol with bounded admission, observed worker termination, and honest uncertain outcomes.

Suggested loadout Level 6, Level 14, Level 19, Level 20, Level 21, Level 26, Level 27. These are guidance, not locked gates.

Mastery Targets

- Admission closure and accepted-work registration cannot race into an unaccounted operation.
- Every accepted identifier appears in exactly one terminal report category after the defined shutdown procedure.
- Clean drain, local cancellation, worker failure, and uncertain external effect are distinguishable outcomes.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 28

Services Need Owners for Work and Failure

A service is a collection of operations that remain meaningful while callers arrive, disappear, retry, and fail. Its hardest bugs often live between individually correct components: an accept loop can keep creating jobs after shutdown begins, a bounded queue can feed unlimited active tasks, or a supervisor can drop the only handles capable of reporting failures. Rust's ownership model helps when the service assigns explicit owners to admission, active work, results, and termination.

Start with four questions. Who may accept new work? Who owns a request after acceptance? Who observes the operation's final outcome? Who waits for every owned activity to stop? The answers should survive cancellation of any one caller. A service with no answer to the last question tends to accumulate detached tasks and uncertain shutdown behavior. A service with several incompatible answers to the first question tends to admit work during transitions that were intended to close it.

The goal is not a universal framework. A small batch program and a long lived network server need different machinery. The transferable idea is that lifecycle transitions must be represented somewhere authoritative, just as data invariants must be represented in an owning type. A boolean named `running` is rarely enough when the service distinguishes accepting, draining, cancelling, failed, and terminated states.

Admission Is a Linearization Boundary

An admission decision has a moment at which the service accepts responsibility for a request. Before that moment, the caller owns the unaccepted request and may retry elsewhere. After that moment, the service owes a documented outcome for accepted work. Closing admission must coordinate with that same decision so that every concurrent request falls on one defined side of the boundary.

Checking an atomic `closing` flag and then incrementing a separate active counter can leave a race: shutdown sees zero active work and finishes while a caller that already read the old flag proceeds to register. The fields may all be atomic, yet the combined protocol is incomplete. One simple solution protects admission state and active accounting with one mutex. More elaborate atomic designs need an equally explicit proof of the combined transition.

This complete example models admission and draining. A lease accounts for one accepted operation until it is dropped.

Listing 28.1 Rust example

```
use std::sync::{Arc, Mutex};

#[derive(Clone, Copy, Debug, PartialEq)]
enum Phase {
    Accepting,
```

Listing 28.1 continued

```
    Draining,
    Stopped,
}

struct State {
    phase: Phase,
    active: usize,
    limit: usize,
}

#[derive(Clone)]
struct Admission(Arc<Mutex<State>>);

struct Lease(Arc<Mutex<State>>);

impl Admission {
    fn new(limit: usize) -> Self {
        Self(Arc::new(Mutex::new(State {
            phase: Phase::Accepting,
            active: 0,
            limit,
        })))
    }

    fn try_enter(&self) -> Option<Lease> {
        let mut state = self.0.lock().unwrap();
        if state.phase != Phase::Accepting || state.active == state.limit {
            return None;
        }
        state.active += 1;
        Some(Lease(Arc::clone(&self.0)))
    }

    fn close(&self) {
        let mut state = self.0.lock().unwrap();
        state.phase = if state.active == 0 {
            Phase::Stopped
        } else {
            Phase::Draining
        };
    }

    fn snapshot(&self) -> (Phase, usize) {
        let state = self.0.lock().unwrap();
        (state.phase, state.active)
    }
}

impl Drop for Lease {
```

Listing 28.1 continued

```
    fn drop(&mut self) {
        let mut state = self.0.lock().unwrap();
        state.active -= 1;
        if state.phase == Phase::Draining && state.active == 0 {
            state.phase = Phase::Stopped;
        }
    }
}

fn main() {
    let admission = Admission::new(2);
    let first = admission.try_enter().unwrap();
    let second = admission.try_enter().unwrap();
    assert!(admission.try_enter().is_none());
    admission.close();
    assert_eq!(admission.snapshot(), (Phase::Draining, 2));
    assert!(admission.try_enter().is_none());
    drop(first);
    drop(second);
    assert_eq!(admission.snapshot(), (Phase::Stopped, 0));
}
```

The lease is not cloneable. Acceptance increments the count, and dropping the lease decrements it. Forgetting a lease leaves its capacity charged and can prevent draining. A task should retain the lease with the work even if its caller drops the response receiver, so the count tracks work rather than waiting callers.

This model deliberately does not include a wait notification. An asynchronous service would connect the transition to zero with a suitable notification primitive, using a protocol that cannot lose wakeups. It also chooses to propagate mutex poisoning through panic. A production owner needs a defined failure policy for its state. The important invariant is visible: admission phase and active count change under one guard, and stopping cannot race with an unregistered accepted operation.

Bound Every Stage That Can Retain Work

A request can occupy several different stages: accepted but not queued, queued, active, waiting on a downstream service, waiting to return a result, or retained for retry. Bounding one queue does not bound all of those stages. If consumers dequeue and spawn immediately, the queue may remain nearly empty while active work grows without limit. If each request creates several downstream operations, fanout can multiply the resource demand again.

An admission lease can represent the overall request budget, while narrower permits represent scarce subresources such as database connections or large buffers. Retain each permit for the interval in which the corresponding resource is actually committed. Releasing an admission permit immediately after dequeueing gives a misleading measure if processing continues elsewhere. Likewise, acquiring a scarce connection before waiting in a long queue can waste capacity that active requests need.

Resource bounds can interact through wait cycles. Suppose every request holds one buffer while waiting for a connection, and a connection owner needs another buffer to produce its reply. If the buffer pool is exhausted, the system can deadlock despite every individual pool having a sensible limit. Order acquisitions consistently, avoid holding resources while waiting for unrelated scarce resources when possible, or reserve the complete resource bundle needed for a coherent operation.

Overload policy should preserve useful work. Rejecting at admission is often cheaper than accepting a request that will time out deep in the pipeline. Coalescing updates may be correct for current state notifications but wrong for an audit log. Dropping the oldest job may improve freshness while violating ordering guarantees. State the policy in domain terms so that capacity choices do not silently alter business semantics.

A Supervisor Owns Observation

Spawning a task transfers execution, but it should not discard accountability. A supervisor retains the capability to observe completion, classify failure, and decide what the rest of the service should do. A child can finish normally with a domain error, panic, or be cancelled by the supervisor. These outcomes deserve separate treatment because they imply different recovery possibilities.

The following library code demonstrates classification for ordinary worker threads. Async task handles use runtime specific error types, but the ownership principle is the same.

Listing 28.2 Rust example

```
use std::{any::Any, thread::JoinHandle};

#[derive(Debug)]
enum WorkerFailure {
    InvalidInput,
    DependencyUnavailable,
}

#[derive(Default, Debug, PartialEq)]
struct Summary {
    completed: usize,
    domain_failures: usize,
    panics: usize,
}

fn collect(
    workers: Vec<JoinHandle<Result<(), WorkerFailure>>>,
) -> (Summary, Vec<Box<dyn Any + Send>>) {
    let mut summary = Summary::default();
    let mut payloads = Vec::new();
    for worker in workers {
        match worker.join() {
            Ok(Ok(())) => summary.completed += 1,
            Ok(Err(_)) => summary.domain_failures += 1,
            Err(payload) => payloads.push(payload),
        }
    }
    summary.panics = payloads.len();
    (summary, payloads)
}
```

The caller receives panic payloads after every join. Dropping a payload can itself panic, so the caller chooses how to inspect, retain, or propagate it

after collection. Joining in creation order can delay failure observation. A supervisor that needs to react sooner can listen for completion events.

A panic is evidence that an assumed invariant may have failed. Automatically restarting a child can be appropriate for isolated, reconstructible state, but it can also repeat a deterministic bug or duplicate an externally committed operation. Restart policy needs a state recovery story and a retry budget. Backoff alone slows a failure loop; it does not make an operation safe to repeat.

The supervisor also needs its own owner. If a request handler spawns a supervisor and immediately loses its handle, the accountability problem has merely moved one level up. Organize ownership so that a top level lifecycle object can eventually enumerate or await all activities it accepted responsibility for. Detached work should be a deliberate exception with an independent owner, not the default consequence of dropping a handle.

Shutdown Has Several Distinct Transitions

Graceful shutdown usually begins by closing admission. It then decides what to do with queued work, lets selected active operations finish, and eventually escalates if the deadline expires. Only after observing termination should it release shared infrastructure needed by those operations. Closing a database pool before workers finish using it turns orderly draining into a cascade of avoidable failures.

The word graceful needs a precise promise. It might mean every accepted operation is completed, every active operation reaches a safe checkpoint, or every client receives a final response. These promises can conflict with a finite deadline. A service should define which guarantee survives escalation and which work may remain uncertain. The caller of shutdown should receive enough information to distinguish a complete drain from a forced stop.

Queued work needs an explicit disposition. Draining it preserves accepted work but can make shutdown slow under a large backlog. Cancelling it requires returning or recording a terminal outcome for each accepted request. Persisting it for another process requires a handoff protocol and durable ownership. Simply dropping the queue may release memory safely while violating the service's accepted work contract.

Active work may be cooperative or effectively uninterruptible. A cancellation token can request that a task exit at a checkpoint, while a blocking foreign call may not offer one. A forced abort of the future may release local guards but cannot reverse external effects. The shutdown result should preserve uncertainty about those effects rather than reporting a fictional clean cancellation.

Deadlines Should Travel With Work

A request that spends most of its budget waiting in a queue should not receive a fresh full timeout at every later layer. Carry one deadline through admission, queueing, dependency calls, and retries. Each stage can compute the remaining budget and refuse to start work that can no longer produce a useful result.

This complete library helper uses a monotonic deadline and makes expiration explicit.

Listing 28.3 Rust example

```
use std::time::{Duration, Instant};

#[derive(Clone, Copy)]
struct Budget {
    deadline: Instant,
}

impl Budget {
    fn from_now(duration: Duration) -> Option<Self> {
        Instant::now().checked_add(duration)
            .map(|deadline| Self { deadline })
    }

    fn remaining(self) -> Option<Duration> {
        self.deadline.checked_duration_since(Instant::now())
            .filter(|duration| !duration.is_zero())
    }
}
```

The helper does not stop anything; it supplies policy information. An async runtime's timer may use a related but distinct instant type, so integrate with the runtime's clock consistently, especially in tests that control time. An overflow in deadline construction is represented as failure rather than silently converting an unreasonable duration into a different policy.

Deadline propagation also affects retries. A retry should consume the original budget and the original operation identity when it represents the same logical request. Creating a new deadline and a new identifier on every attempt can turn a bounded request into unbounded background load and defeat deduplication. A retry policy needs both temporal and semantic continuity.

Some work remains valuable after a caller's deadline, such as finishing an already committed record or recording an authoritative outcome. Keep that work under the service's owner even when the response path is gone. Distinguish caller usefulness from consistency obligations. Otherwise a cancellation optimization can leave the system without the record needed to explain what actually happened.

Failure Domains Should Match State Authority

An isolated worker that owns a temporary scratch buffer can often fail without stopping the service. A state owner that maintains the only in memory copy of an authoritative index may require broader shutdown or reconstruction. Draw failure boundaries around what can be restored independently, not merely around convenient task spawn points.

Shared mutable state expands the failure domain. If several workers can modify one complex structure, a panic in any worker may leave everyone uncertain about that structure's invariant. Immutable snapshots, actor ownership, and transactional updates can narrow the scope of uncertainty. This is a reliability benefit of ownership architecture that does not appear in simple throughput benchmarks.

Failure reporting should preserve the identity of the operation and the component that owned it. A message saying only that a task failed forces operators to reconstruct ownership from timestamps. Record the request identifier, phase, relevant version, and whether an external effect may have occurred. Avoid logging the same failure at every layer without adding information; duplicate records can obscure the authoritative outcome.

Supervision policy should also consider dependencies. If a storage owner fails, continuing to accept dependent requests may only build a backlog of doomed work. Closing or degrading admission can be a better response

than letting every request independently discover the same failure. Recovery can reopen admission after the dependency's state is known to be usable, with a new generation identifier if old completions might still arrive.

A Lifecycle Review Before Implementation

Walk one request through the service and identify the owner at every step. Include rejection, queue cancellation, worker panic, reply receiver disappearance, dependency timeout, and shutdown escalation. At each handoff, ask whether ownership transfer is atomic with the corresponding accounting update. A request that exists in neither owner's accounting can disappear; one counted by both can be processed twice.

Then walk shutdown through the same structure in reverse. Close capabilities that create new work before waiting for the work they created. Keep shared dependencies alive until their users finish. Retain observation handles until outcomes have been collected. If the process must terminate with outstanding uncertainty, report that state explicitly to the shutdown caller or durable log.

This review often produces simpler code than starting with a large framework. A small admission owner, bounded queue, explicit worker set, and documented shutdown phases can cover a substantial service. Additional machinery should answer a concrete lifecycle problem: dynamic children, hierarchical cancellation, deadline scheduling, or restart isolation. Complexity is justified when it preserves an invariant that would otherwise be scattered across unrelated tasks.

Training Encounters

1. A service checks `closing == false`, then increments a separate active counter. Shutdown sets `closing = true`, sees zero active work, and releases shared infrastructure. Construct the race and identify the combined transition that needs synchronization.

2. A bounded request queue feeds a loop that spawns one task per dequeued item. Explain why memory and dependency concurrency can still grow, and state where a permit must remain owned.

3. A supervisor requests cancellation of every child and immediately reports a clean shutdown. Identify two facts it has not established and propose a more informative result.

Training Debriefs

1. A caller reads the old open state, pauses, and registers after shutdown's zero check. Admission and registration must form one authoritative transition coordinated with closure. A mutex around phase and count is a simple implementation. An atomic alternative needs a state representation and proof that no accepted operation can appear after stopping was established.

2. Dequeueing removes the queue's bound while spawned tasks continue retaining messages, buffers, and dependency calls. Acquire an active work permit before admitting or starting the operation, according to the desired policy, and keep it with the operation until completion or defined cancellation cleanup. Bound fanout and large payload bytes separately when necessary.

3. A cancellation request does not prove that children stopped, and stopping locally does not prove that external effects were absent. Await or join termination, collect failures, and distinguish a completed drain from deadline escalation. Report outstanding operations or uncertain outcomes when the service cannot establish a stronger conclusion.

Further Reading

• Tokio graceful shutdown discusses detecting shutdown, notifying tasks, and waiting for completion.

• Thread JoinHandle documents joins. The panic payload notes explain the risk when dropping payloads.

• Instant documents monotonic time calculations used by deadline budgets.

The Supervisor That Declared Victory

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Specify a finite service with a bounded input queue, a fixed worker limit, and a supervisor that owns all worker completion observations. Every accepted job has an identifier and one ledger entry. Shutdown first closes admission, then attempts a graceful drain, then may escalate at a deadline. Build a deterministic failure script containing a queued job, an ordinary job error, a worker panic before reporting, and a job whose remote effect may have occurred before local interruption. Define report categories and an accounting equation that includes uncertainty rather than calling it success. State where accepted-work registration becomes authoritative, who keeps active-work capacity owned, how abandoned queued work is recorded, and what evidence the supervisor gathers before claiming every local worker stopped.

Encounter Constraints

- A cancellation request alone cannot count as observed termination.
- Queue capacity does not replace an active-worker bound or a variable-payload byte budget.
- No terminal category may assert absence of a remote effect without a protocol that establishes it.

Hint one / Hint two / Reveal debrief

Side Quest

Design a rolling restart instead of total shutdown. Decide which accepted jobs drain in the old generation, which may transfer to the new generation, and which require persistent status or deduplication before retry. Put a generation on local worker identity without confusing it with the operation's stable external identifier. Explain the recovery authority if both generations briefly exist and one loses its final acknowledgement.

Capability unlocked Last Light Ledger accounts for every accepted operation without inventing certainty.

Choose Your Exit

Build the endgame pipeline / Level 39
Implement concrete queue ownership and join ordering for a bounded batch.

Enter the versioned archive / Level 38
Give decisions an explicit consistency and retry model.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 29

The Shape Shifter

Beneath Ferrum, a relay translates messages between machines built decades apart. Its engineers saved bandwidth by sending the memory image of a Rust structure. The relay works in their demonstration and corrupts reports elsewhere. Your map shows a clear route into the Unsafe Depths: separate the message people mean from the arrangement one compiler happens to choose. The broken relay is a design case, not an executable puzzle hidden in this page.

Mission Specify and implement a portable byte format without treating an in-memory Rust value as its wire representation.

Suggested loadout Level 2, Level 18. These are guidance, not locked gates.

Mastery Targets

• Produce explicit encoder and decoder functions with documented field widths and byte order.

• Reject unknown tags and malformed lengths before constructing a validated message.

• Explain which layout observations are target-specific and which format guarantees your code establishes.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 29

Layout Alignment Representation and ABI

A type is more than its visible fields

When Rust code remains within ordinary safe operations, a struct's layout can usually remain an implementation detail. The compiler locates its fields, constructs valid values, and applies the calling convention. Low level interfaces change the question. A foreign function, memory mapped record, device register, or custom allocator may require a precise account of size, alignment, field placement, and which bit patterns represent valid values.

These properties must be considered separately. Two types can have equal size and different alignment. Two types can have equal size and alignment but different field order. Two equally laid out types can have different valid bit patterns or calling conventions. A successful `size_of` assertion proves only the property it actually checks.

The default Rust representation deliberately leaves substantial freedom to the compiler. Field order in memory is not a portable serialization contract. A compiler can choose layouts that reduce padding or support enum optimizations. Observing one layout in a debugger does not convert it into a stable promise for another compiler, target, or instantiation.

The advanced habit is to distinguish a guaranteed property from a measured property. A benchmark may legitimately report the size of a particular build. An unsafe implementation needs guarantees sufficient to justify every access it makes. A file format needs a specification independent of the process that writes it.

Size and alignment describe different obligations

Size determines the storage occupied by a value, including padding needed for arrays. Alignment constrains the addresses at which values of that type can be accessed through ordinary typed operations. The size of a sized type is a multiple of its alignment, allowing consecutive array elements to remain aligned.

Listing 29.1 Rust example

```
use std::mem::{align_of, offset_of, size_of};

#[repr(C)]
struct Header {
    kind: u8,
    length: u32,
    flags: u16,
}

fn main() {
    assert_eq!(offset_of!(Header, kind), 0);
    assert_eq!(offset_of!(Header, length) % align_of::<u32>(), 0);
    assert_eq!(offset_of!(Header, flags) % align_of::<u16>(), 0);
    assert_eq!(size_of::<Header>() % align_of::<Header>(), 0);
    println!("size {} alignment {}", size_of::<Header>(),
        align_of::<Header>());
}
```

The example checks relationships without pretending every target has the same numeric layout. `repr(C)` follows the target's C layout rules for the represented struct. It does not declare that `u32` has one universal alignment across all platforms or that a foreign compiler was invoked with compatible packing settings.

Padding can occur between fields and at the end of a struct. It is not an extra logical field. Typed moves and assignments need not preserve initialized padding bytes. Reading an entire object representation as a byte slice can therefore encounter uninitialized padding even if every actual field is valid. This is one reason raw struct bytes are a poor general serialization format.

Increasing alignment can improve a particular hardware or vectorization use case, but it can also increase object size and allocation cost. Cache line alignment used to reduce false sharing should be justified by measurements on the relevant architecture. A larger alignment is a representation choice with consequences for arrays and allocators, not a free optimization hint.

C representation is a boundary contract

`repr(C)` is useful for structs and unions crossing a compatible C boundary, but it does not recursively replace the representation of every nested field. If a C represented struct contains a Rust represented inner struct, the inner layout remains governed by its own declaration. Every field participating in a foreign contract needs an appropriate representation and compatible meaning.

Similarly, `repr(C)` does not turn `String`, `Vec<T>`, or an arbitrary Rust enum into a C language data structure that foreign code can safely manipulate. Their ownership and validity contracts remain Rust specific unless an interface explicitly defines how they are represented and used. Common foreign interfaces instead pass pointers, lengths, capacities when appropriate, and exported functions that preserve allocation ownership.

Layout also does not settle the function calling convention. An `extern "C"` function uses a specified ABI, while an ordinary Rust function uses the Rust ABI. A structure's representation and a function's ABI must both agree with the other side. Transmuting function pointers between incompatible calling conventions is not justified by equal pointer sizes.

Enums require particular care. A primitive representation can fix the representation of a fieldless enum's discriminant, but arbitrary integer values do not thereby become valid variants. A foreign library that may return unknown numeric tags should generally expose an integer at the boundary and validate it before constructing a Rust enum. For data carrying enums, tag and payload representation has additional rules; do not infer them from a small fieldless example.

Transparent wrappers preserve one representation

A transparent wrapper is useful when a type should retain a field's representation and ABI while giving it a distinct Rust identity. The representation has precise field restrictions; it is not a general instruction to flatten an arbitrary struct.

Listing 29.2 Rust example

```
use std::mem::{align_of, size_of};
use std::num::NonZeroU32;

#[repr(transparent)]
#[derive(Clone, Copy)]
struct RequestId(NonZeroU32);

impl RequestId {
    fn new(raw: u32) -> Option<Self> {
        NonZeroU32::new(raw).map(Self)
    }

    fn get(self) -> u32 {
        self.0.get()
    }
}

fn main() {
    assert_eq!(size_of::<RequestId>(), size_of::<NonZeroU32>());
    assert_eq!(align_of::<RequestId>(), align_of::<NonZeroU32>());
    assert_eq!(RequestId::new(9).unwrap().get(), 9);
    assert!(RequestId::new(0).is_none());
}
```

The wrapper's invariant remains that the value is nonzero. Transparent representation does not permit constructing it from zero or bypassing private constructors without proof. It preserves a representation relationship, while the type's semantic contract still restricts valid values.

Niche optimizations use representations that a type cannot otherwise hold to encode another state, such as the absence of certain pointer or nonzero values. Some `Option<T>` layout relationships are explicitly guaranteed by the standard library. Other observed enum sizes are implementation details. Before relying on a niche in unsafe code or an ABI, identify the documented guarantee for that exact type rather than generalizing from `Option<&T>`.

A wrapper intended for public foreign use should document which field determines its representation. Keeping that field private can preserve construction invariants, but clients still need a stable contract for legitimate boundary operations. Type identity and representation compatibility serve different purposes and can coexist.

Packed fields may not support references

Packing can reduce or remove padding by lowering alignment requirements of a containing representation. The resulting fields may not be aligned for their own types. Ordinary references require suitable alignment, even if the hardware can perform an unaligned load. Creating an unaligned reference is not repaired by immediately converting it to a raw pointer.

Listing 29.3 Rust example

```
#[repr(C, packed)]
struct PackedHeader {
    kind: u8,
    length: u32,
}

fn main() {
    let header = PackedHeader { kind: 1, length: 0x0102_0304 };
    let pointer = &raw const header.length;
    // SAFETY: the field is initialized and readable; alignment is
    // handled by read_unaligned rather than by a reference.
    let length = unsafe { pointer.read_unaligned() };
    assert_eq!(length, 0x0102_0304);
}
```

The raw address expression avoids creating a reference to the packed field. `read_unaligned` addresses the alignment issue for the read. It does not waive other obligations such as initialization, valid representation, sufficient live storage, or access compatibility. For a noncopyable type, reading a value out would also introduce ownership and destruction concerns. The example uses a copyable integer to keep those separate.

Formatting a packed field can accidentally create a reference through macro expansion. Copying the field into an ordinary aligned local before formatting can be appropriate for copyable values. When building a reusable packed abstraction, expose methods that perform the correct access rather than asking every caller to remember these details.

Packing is often unnecessary for protocol parsing. A parser can read individual byte ranges and construct ordinary aligned Rust values. That avoids coupling protocol layout to host alignment and permits validation before typed values are created.

External bytes should be decoded deliberately

A wire record needs explicit endianness, field widths, bounds behavior, and versioning. These decisions exist independently of Rust's in memory representation. A small parser can make them visible without unsafe code.

Listing 29.4 Rust example

```
#[derive(Debug, PartialEq)]
struct WireHeader {
    kind: u8,
    length: u32,
    flags: u16,
}

fn decode_header(bytes: &[u8]) -> Option<WireHeader> {
    let bytes = bytes.get(..7)?;
    let length = u32::from_le_bytes(bytes[1..5].try_into().ok()?);
    let flags = u16::from_le_bytes(bytes[5..7].try_into().ok()?);
    Some(WireHeader { kind: bytes[0], length, flags })
}

fn main() {
    let header = decode_header(&[2, 4, 0, 0, 0, 1, 0]).unwrap();
    assert_eq!(header, WireHeader { kind: 2, length: 4, flags: 1 });
    assert!(decode_header(&[2, 4]).is_none());
}
```

The parser does not depend on the size of `WireHeader`. It handles short input before indexing and states little endian interpretation directly. A production parser may also validate the tag, reject reserved flag bits, limit length, and distinguish malformed input from incomplete input. Those checks establish semantic invariants before later code relies on them.

Zero copy parsing can still be worthwhile, but its proof is broader than alignment. The input bytes must form valid values of the target type, contain no unsupported padding assumptions, use the expected byte order, and remain alive without conflicting mutation. A byte order aware view type may be a better interface than pretending network bytes are an ordinary native integer struct.

If the format evolves, explicit decoding provides a natural version boundary. Old fields can receive defaults, unknown tags can remain numeric, and length limits can be enforced before allocation. Direct memory reinterpreting often makes those policy decisions harder to see.

Dynamically sized values require metadata

A slice has a runtime element count, and a trait object has runtime information needed for erased operations. Pointers to dynamically sized types therefore carry information beyond the data address under the language's model. The exact representation details should not be assumed beyond documented guarantees.

For a slice, the length participates in the validity of access to the complete range. A valid first element pointer plus an arbitrary large length is not a valid slice. The total range must fit the allocation and the API's size restrictions, and an empty slice still has pointer requirements when created as a reference through raw slice constructors.

A trait object's metadata is associated with its concrete erased type. Combining one object's data pointer with unrelated metadata can cause methods or destruction to interpret storage as the wrong type. A vtable is not a stable plugin ABI merely because a debugger displays one. Use a defined foreign interface when independently compiled components need a durable binary contract.

This distinction affects size queries. `size_of::<&[u8]>()` measures the reference. `size_of_val(slice)` measures the elements. Rust 1.99 stabilized `Layout::for_value_raw` for layout queries without making a reference. For a dynamically sized pointee, follow the function's metadata and size safety requirements.

Zero sized types still have semantics

A type can occupy zero bytes while having meaningful construction, destruction, trait, and ownership behavior. A zero sized token can represent a capability. An array of zero sized elements still has a logical length. Dropping its elements can still matter when the element type has a destructor.

Addresses of zero sized values need not distinguish logical identity. Algorithms that infer uniqueness from pointer inequality can therefore fail for such types. Generic collections must handle element counts and destruction separately from byte counts. Allocator APIs may also impose

nonzero size requirements even though a `Layout` for a zero sized type can exist.

These cases explain why a hand written generic allocation wrapper is substantially harder than a wrapper specialized to one ordinary nonzero sized type. Correct generic code needs a coherent policy for zero size, alignment, panic paths, and element ownership. Tests using only `u64` do not exercise those obligations.

Layout knowledge should remain local

The more code depends on a representation detail, the more difficult it becomes to evolve. Keep unsafe layout dependent operations in a small module, expose validated safe values, and document the specific guarantee each operation uses. A representation test is useful for detecting accidental changes to an intended contract, but cannot create a language guarantee where none exists.

For public protocols, define the bytes. For foreign functions, define both layout and calling convention. For internal performance work, measure a concrete target and avoid turning a temporary optimization into an undocumented external format. These separate decisions let the compiler optimize ordinary Rust values while boundary code remains explicit and reviewable.

Training Encounters

1. A team writes a Rust struct directly to disk because its size matches a C struct in one build. Identify four independent properties that still need a specification.

2. Why does `repr(C)` on an outer struct fail to establish a C layout for an inner Rust represented struct? What should an FFI review inspect recursively?

3. A packed record contains a `u32`. Explain why converting `&record.field` to a raw pointer is too late to solve alignment, and how a raw address expression plus an unaligned read differs.

Training Debriefs

1. At least field offsets and padding, integer byte order, valid tags or bit patterns, and format versioning need definition. Target widths and alignment, padding initialization, ownership of embedded pointers, and compatibility across compilers are additional concerns. Equal total size addresses none of these by itself.

2. Representation attributes belong to type declarations. The outer layout places an inner field according to that field's existing size and alignment; it does not redefine the inner field order. Inspect every boundary field's representation, validity, ownership meaning, and ABI suitability.

3. Creating the reference already asserts suitable alignment. A later cast cannot undo that invalid creation. A raw address expression obtains the field pointer without that intermediate reference, and `read_unaligned` performs a read designed for insufficient alignment while retaining the remaining pointer safety requirements.

Further Reading

• The Rust Reference on type layout specifies representation guarantees and their scope.

• The Rust standard library pointer documentation documents alignment and raw access obligations.

• The Rust standard library Option representation section lists guaranteed option layout relationships.

The Mimic Packet

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

The relay carries a one-byte kind, a four-byte sequence, a two-byte payload length, and payload bytes. Kinds 1 and 2 are valid. All integers use little-endian encoding, and extra bytes belong to the next packet. Implement a decoder that distinguishes incomplete input from invalid input and returns the consumed count only after one complete packet. Implement the matching encoder. Then show an independently written byte fixture for each kind; do not generate every fixture through your encoder. A colleague proposes replacing both functions with a cast from a repr(C) structure because its measured size looks convenient. Write a precise rejection of that proposal, including padding and validity concerns.

Encounter Constraints

- Use safe byte operations for the codec.
- Check length arithmetic and reject payloads longer than the length field permits.
- Do not rely on native endianness, pointer width, or measured structure size.

Hint one / Hint two / Reveal debrief

Side Quest

Add a version byte and an optional extension region. Decide whether an older decoder can skip an unfamiliar extension or must reject it. Give the region an explicit length so skipping does not depend on a Rust enum layout. Design two compatibility fixtures: a known message with an unknown optional extension and an unknown required extension. Explain why versioning is a protocol policy rather than a representation attribute.

Capability unlocked Wire Cartographer

Choose Your Exit

Follow the pointer trail / Level 30
Separate a byte address from the permission to access its allocation.

Take the interoperability bridge / Level 33
Apply representation decisions to a real ownership boundary with foreign code.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 30

The Window With No Owner

Ferrum exports image fragments to inspection tools. A fragment is described by a pointer and a length, but the old interface forgot to say who keeps the image alive. One tool holds a fragment after the renderer reuses its buffer. You have reached a door that cannot be opened by checking for null alone. The repair is to connect access to an owner, then make any remaining unsafe step justify its exact range.

Mission Design a mutable byte-window API whose lifetime and splitting rules remain connected to valid backing storage.

Suggested loadout Level 3, Level 4, Level 29. These are guidance, not locked gates.

Mastery Targets

• Construct windows from a borrowed mutable byte slice rather than an arbitrary safe raw-pointer constructor.

• Split one window into disjoint children with no simultaneous overlapping mutable access.

• Write an access argument covering empty windows, range limits, lifetime, and ownership.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 30

Raw Pointers Provenance Aliasing and Validity

A raw pointer postpones obligations

A raw pointer can represent an address without carrying the safe access guarantees of a reference. It can be null, unaligned, stale, or point to storage that does not currently contain an initialized value of its nominal pointee type. Merely having such a pointer is not the same as having permission to dereference it. The obligations become concrete when an operation reads, writes, forms a reference, performs constrained pointer arithmetic, or otherwise relies on a stronger contract.

This flexibility is useful for implementing allocators, foreign interfaces, collections, and device access. It is also why raw pointers do not solve a borrowing problem by themselves. They remove static enforcement from part of the operation and require the implementation to preserve the same underlying memory safety conditions through another argument.

Separate four questions. Does the pointer identify an appropriate live allocation? Is the addressed range properly aligned and in bounds for this operation? Does the storage contain valid initialized values when the operation requires them? Is the access compatible with other active references, pointers, and threads? A positive answer to one question does not imply the others.

`NonNull<T>` answers only a narrow representation question: the pointer is nonnull, with the type's documented variance and other properties. It does not establish ownership, alignment for dereference, initialization, allocation identity, or freedom from alias conflicts. A nonnull dangling pointer can still be useful as a sentinel in a correctly designed empty container.

References assert a stronger contract immediately

Creating `&T` or `&mut T` is itself a meaningful event. It asserts appropriate validity, alignment, lifetime, and access relationships. It is not safe to create an invalid reference temporarily on the theory that no load will occur before it is repaired. The compiler may optimize based on the reference's contract.

Raw address expressions are useful when an implementation needs an address without first making those assertions. This includes addresses of uninitialized fields and packed fields. The raw pointer is not a license to access the field incorrectly; it avoids prematurely asserting a reference contract.

Listing 30.1 Rust example

```
use std::mem::MaybeUninit;

struct Pair {
    left: u32,
    right: u32,
}
```

Listing 30.1 continued

```
fn main() {
    let mut storage = MaybeUninit::<Pair>::uninit();
    let pointer = storage.as_mut_ptr();
    // SAFETY: these raw field addresses are within allocated storage;
    // write initializes each field without reading an old value.
    unsafe {
        (&raw mut (*pointer).left).write(10);
        (&raw mut (*pointer).right).write(20);
    }
    // SAFETY: both fields now contain valid initialized u32 values.
    let pair = unsafe { storage.assume_init() };
    assert_eq!(pair.left + pair.right, 30);
}
```

The code never creates a reference to the uninitialized `Pair`. A field projection used to obtain a raw address still needs to stay within the appropriate allocated storage. Forming arbitrary out of bounds projections is not made valid merely by avoiding a load.

The example's fields are copyable integers, so partial initialization cannot leak owned resources. A field by field constructor involving strings needs a cleanup strategy for a panic after the first field is initialized. That is an ownership problem in addition to the validity problem addressed here.

Provenance tracks the origin of access

A pointer is not adequately modeled as just an integer address. Its ability to access storage also depends on its relationship with an allocation and the history by which it was derived. This family of relationships is discussed as pointer provenance. Two pointers displaying the same address do not thereby establish that every operation through them is interchangeable.

Rust's precise aliasing and provenance semantics continue to be developed. Avoid presenting one experimental model, such as a particular Miri borrowing mode, as a complete final language specification. At the same time, the absence of a complete formal model is not permission to ignore documented pointer requirements. Standard library safety contracts and the Reference still impose concrete obligations.

Strict provenance APIs make some intended operations explicit. `addr` extracts the address component. `with_addr` creates a pointer with a changed address while retaining the source pointer's provenance under its contract. These operations help express address manipulation without assuming an integer alone reconstructs the needed access authority.

Listing 30.2 Rust example

```
use std::mem::size_of;

fn main() {
    let values = [10_u32, 20, 30];
    let base = values.as_ptr();
    let address = base.addr() + size_of::<u32>();
    let second = base.with_addr(address);
    // SAFETY: second is derived from this live allocation, points
    // to its initialized second element, and only reads shared data.
    assert_eq!(unsafe { second.read() }, 20);
}
```

Ordinary `base.add(1)` would express this particular operation more directly. The example isolates the role of address adjustment. `with_addr` does not prove the new address is in bounds, aligned, or initialized. The subsequent read still needs those facts.

Pointer tagging techniques can reserve unused address bits and later restore the original usable address, but they need a proof about alignment, masks, allocation provenance, and every dereference. Strict provenance operations make that proof easier to state; they do not supply the proof automatically. If pointer tagging merely saves a small field with no demonstrated benefit, a separate enum or integer tag may be a better engineering choice.

Arithmetic has operation specific requirements

Different pointer arithmetic methods have different contracts. In bounds arithmetic carries stronger requirements than wrapping address arithmetic. A one past the end pointer can be useful for iteration or range representation, but it cannot be dereferenced as an element. Going outside an allocation and later returning is not something to justify casually with hardware behavior; read the exact method's requirements.

Byte offsets and element offsets also differ. `add(1)` on `*const u64` advances by one element, while byte based methods express byte movement. Generic code must account for zero sized types, where logical iteration length changes while an element pointer's address may not. Counting bytes is therefore not a substitute for counting initialized elements.

An implementation should derive pointers from the owner or borrowed slice at a point where the owner is known to remain alive and stable. Retaining raw pointers across vector growth, buffer replacement, or object movement creates additional invalidation rules. A raw pointer does not keep an allocation alive and does not freeze its address.

These rules matter even when tests happen to reuse the same allocation address. An allocator can return the same address for a later allocation, but an old pointer's history is not automatically renewed. Treat deallocation as ending the old access relationship. Reacquire pointers from the new owner rather than reviving stale ones based on numeric equality.

Reads and writes also affect ownership

`ptr::read` creates a value by reading storage without treating the source place as moved out in the compiler's ordinary safe analysis. For a copyable integer, copying can be straightforward. For an owning `String`, an uncoordinated read can duplicate destruction responsibility for the same allocation. The implementation must ensure the old location is no longer read or dropped as that owning value, or use an operation that replaces it coherently.

`ptr::write` writes a value without dropping a previous value at the destination. It is appropriate for uninitialized storage, but overwriting an initialized owner can leak the old resource. Ordinary assignment performs different work because it accounts for the old initialized value.

Listing 30.3 Rust example

```
fn replace_string(slot: &mut String, incoming: String) -> String {
    let pointer: *mut String = slot;
    // SAFETY: the exclusive borrow supplies one initialized String
    // place. replace leaves incoming there and returns the old owner.
    unsafe { pointer.replace(incoming) }
}

fn main() {
```

Listing 30.3 continued

```
    let mut value = String::from("before");
    let old = replace_string(&mut value, String::from("after"));
    assert_eq!(old, "before");
    assert_eq!(value, "after");
}
```

Safe `mem::replace` is preferable for this ordinary case. The raw version makes the ownership obligation visible for implementations already operating through pointers. The important property is that the destination remains initialized and the old owner has exactly one returned destruction responsibility.

Bulk copies require the same reasoning. Copying the representation of owning values does not automatically clone their resources. A vector implementation may move elements with byte copies only because it also updates which locations are considered initialized and will be dropped. The byte operation and the ownership bookkeeping form one proof.

Range constructors need a complete lifetime contract

`slice::from_raw_parts` can turn a pointer and a length into a borrowed slice. Its apparent convenience can hide several obligations: suitable alignment and nonnull pointer requirements, valid initialized elements, a range within one allocation, size constraints, and a valid shared access relationship for the entire returned lifetime.

Listing 30.4 Rust example

```
/// Borrows bytes supplied by an external owner.
///
/// # Safety
/// For the complete returned lifetime, pointer must be nonnull and
/// valid for reads of length initialized bytes in one allocation.
/// The range must fit isize::MAX and must not be mutated except as
/// permitted by Rust's shared reference rules.
unsafe fn borrowed_bytes<'a>(
    pointer: *const u8,
    length: usize,
) -> &'a [u8] {
    // SAFETY: these are precisely the caller's obligations above.
    unsafe { std::slice::from_raw_parts(pointer, length) }
}

fn main() {
```

Listing 30.4 continued

```
    let bytes = [1_u8, 2, 3];
    // SAFETY: bytes remains alive and unchanged through this use;
    // its pointer and length describe one initialized allocation.
    let view = unsafe { borrowed_bytes(bytes.as_ptr(), bytes.len()) };
    assert_eq!(view, &[1, 2, 3]);
}
```

The unconstrained output lifetime makes this unsafe helper easy to misuse if its precondition is not understood. A safer public design often takes a borrowed owner or returns a view tied to a guard, making the lifetime relationship explicit in safe types. Merely checking for null and checking the length cannot verify that arbitrary foreign memory is allocated, live, or immutable for the desired duration.

For a foreign API that represents an empty buffer with a null pointer, normalize the empty case before constructing a slice, for example by returning an ordinary empty slice. Do not assume a zero length waives every requirement of a reference constructor. The exact contract of the called function determines what must hold.

Aliasing is about active access promises

The phrase one mutable reference or many shared references is a useful first approximation, but advanced unsafe reasoning needs to consider derivation, reborrowing, access liveness, and interior mutability. A raw pointer derived from an exclusive reference can participate in legitimate internal operations, but it cannot be used in ways that contradict references handed to safe clients.

For example, a collection cannot return a shared slice and then mutate its ordinary element bytes through a stored raw pointer while the slice is actively used. The raw pointer may predate the slice, but it does not override the later shared contract. Likewise, `UnsafeCell` only changes the relevant shared immutability rule for its contents; it does not allow duplicate exclusive references.

When implementing an unsafe container, write down when raw access is permitted relative to every safe result. If a method returns `&mut T`, all other paths that could access that same value must be incompatible or

suspended for the result's usable region. A dynamic guard, an exclusive owner borrow, or a private state machine can enforce that relationship.

Do not base a proof solely on the statement that the program is single threaded. Aliasing violations can occur within one call stack. Callbacks and destructors can reenter a container, and saved references can overlap later accesses even without concurrency.

Safe inputs can establish useful pointer facts

A function that receives ordinary borrowed slices starts with many facts already established by safe typing. An exclusive destination slice and a shared source slice usable at the same time cannot overlap in a way that violates their reference contracts. Their lengths establish accessible ranges. Their constructors establish initialization and alignment for the element type.

Listing 30.5 Rust example

```
fn copy_prefix(destination: &mut [u8], source: &[u8]) -> bool {
    if source.len() > destination.len() {
        return false;
    }
    // SAFETY: safe slice inputs establish live initialized ranges.
    // The exclusive destination cannot conflict with source, and
    // the checked destination range is large enough for this copy.
    unsafe {
        std::ptr::copy_nonoverlapping(
            source.as_ptr(),
            destination.as_mut_ptr(),
            source.len(),
        );
    }
    true
}

fn main() {
    let mut bytes = [0_u8; 4];
    assert!(copy_prefix(&mut bytes, &[7, 8]));
    assert_eq!(bytes, [7, 8, 0, 0]);
}
```

This example has a straightforward safe equivalent using `copy_from_slice` on a checked prefix. The unsafe version is educational, not a performance recommendation. Compilers can optimize the safe version to the same underlying operation. Use unsafe code when it provides a concrete

capability or measured improvement, and preserve the safe baseline for comparison.

Review pointer code by obligations rather than syntax

For each unsafe operation, identify the allocation owner, range, alignment, initialization state, ownership effect, alias compatibility, and concurrency policy. Then identify which constructor, bound, check, or invariant supplies each fact. A comment saying the pointer is valid compresses too many independent assumptions to be useful in a serious review.

Miri and sanitizers can find important errors in tested executions, including some that ordinary optimized runs appear to tolerate. They do not prove correctness for all clients, targets, and schedules. Use them to challenge a written invariant, and revise the invariant when a failure reveals a missing condition.

The strongest pointer code keeps raw access intervals short and converts back to safe types only after every stronger contract has been established. That local discipline allows the rest of the program to reason with ownership and borrowing instead of repeating the same pointer proof at every call site.

Training Encounters

1. A raw pointer is nonnull and has the expected numeric address. Name four additional facts needed before reading a `T` through it.

2. A helper returns a slice with any lifetime selected by its unsafe caller. Why cannot null and length checks make that helper safe for arbitrary pointers?

3. A custom container uses `ptr::read` to move a string out of an element slot. What bookkeeping must change before normal destruction of the container resumes?

Training Debriefs

1. The storage must belong to an appropriate live allocation, cover the complete range, have suitable alignment for the operation, contain a valid initialized `T`, and permit access under aliasing and concurrency rules. Nonnull and address equality establish none of those by themselves.

2. Runtime checks cannot generally establish allocation lifetime, one allocation range membership, initialization, or the absence of conflicting mutation throughout an arbitrary future lifetime. A borrowed owner, guard, or unsafe caller contract must supply those relationships.

3. The slot must no longer be treated as containing the moved owning string, or it must receive a replacement value before any operation reads or drops it as initialized. Otherwise cleanup may free the same allocation through both the returned string and the old slot. Panic paths must preserve the same accounting.

Further Reading

• The Rust standard library pointer module documents pointer access and provenance APIs.

• The Rust Reference on undefined behavior states current language obligations and their limits.

• The Rust standard library slice from raw parts function provides the complete slice constructor contract.

The Overlapping Observatory

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Build a PixelWindow abstraction supporting length inspection, byte replacement, and splitting at a checked offset. Its caller owns the backing slice, and successful splitting transfers the original window into two child windows. Decide whether raw pointers are useful at all; a safe slice representation is a valid victory. If you choose an internal raw pointer representation, explain how construction, zero-length windows, subranges, and returned references preserve the original borrow. Show a rejected usage that tries to mutate the original slice while a child window is later used. Your solution must also reject an out-of-bounds split without inventing an arbitrary returned lifetime or exposing two overlapping mutable regions.

Encounter Constraints

- The public construction path must be safe and borrow existing storage.
- A failed split must have an explicit ownership policy.
- No pointer-to-integer-to-pointer round trip is needed.

Hint one / Hint two / Reveal debrief

Side Quest

Add a read-only inspection mode that can create several shared child views. Explain why switching an exclusive window into shared views should consume or restrict the exclusive capability until those views expire. Then compare retaining a borrowed view with returning a copied thumbnail. Include the effect on a large backing image that otherwise could have been released or reused immediately.

Capability unlocked Allocation Witness

Choose Your Exit

Audit the sealed chamber / Level 31
Turn the window proof into an audit of every safe entry point.

Return to the ownership maze / Level 8
Use checked handles when stable identity matters more than direct borrowed access.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 31

The Trusted Label

The Unsafe Depths contain a display service with a tiny optimization: every station label is exposed as text through an unchecked conversion. Its constructor accepts only ASCII, so the first inspection looks reassuring. A later convenience method replaces any byte. Ferrum's review board wants the whole abstraction examined, including methods added after the original safety comment. Your enemy is an invariant that exists only in the constructor's author's memory.

Mission Audit a text wrapper by tracing its representation invariant through every safe operation and failure path.

Suggested loadout Level 7, Level 29, Level 30. These are guidance, not locked gates.

Mastery Targets

- State a representation invariant sufficient for every returned text view.
- Demonstrate a safe call sequence that breaks the original byte-replacement design.
- Repair all mutation entry points and explain whether unsafe conversion remains justified.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 31

Designing and Auditing Unsafe Abstractions

Soundness quantifies over safe clients

An unsafe operation is justified by a set of preconditions. A safe abstraction is sound when every client using only its safe interface is prevented from violating those preconditions. This is stronger than saying the current application happens to call it correctly. A public safe method must remain memory safe under surprising inputs, unusual call orders, panics, forgotten values, and every legal implementation of generic parameters it accepts.

The scope of the proof is therefore the whole safe interface. A constructor might establish an invariant that a later mutator accidentally breaks. A destructor might assume the invariant after another method has partially changed it. A derived implementation might expose copying or comparison behavior that does not fit the ownership model. Reviewing isolated unsafe blocks misses these interactions.

Start by writing the representation invariant in ordinary language. State which allocation is owned, which elements are initialized, which pointer is authoritative, what counts as an active borrow, and when destruction is permitted. Then classify every method as establishing, preserving, temporarily changing, or consuming that invariant. A temporary change is acceptable only when no safe observation can exploit an invalid state and every failure path remains sound.

Unsafe is not an effect that contaminates all callers. The purpose of the abstraction is precisely to contain the proof so callers can use a safe contract. An unsafe public method is appropriate when a precondition genuinely cannot be checked or represented internally and must remain the caller's responsibility.

Begin with the safe implementation

Before introducing raw access, implement the desired behavior with existing safe operations if possible. This supplies a behavioral reference and may already produce optimal machine code. Unsafe code is easier to justify when it removes a concrete limitation rather than expressing a vague expectation that pointer arithmetic must be faster.

For obtaining two distinct mutable elements, safe slice splitting is usually sufficient. An unsafe implementation can nevertheless illustrate the proof boundary clearly: bounds and distinctness checks occur in safe code, and the raw operation relies only on those facts and the input slice contract.

Listing 31.1 Rust example

```
fn pair_mut<T>(
    values: &mut [T],
    left: usize,
    right: usize,
) -> Option<(&mut T, &mut T)> {
    if left == right || left >= values.len() || right >= values.len() {
```

Listing 31.1 continued

```
        return None;
    }
    let pointer = values.as_mut_ptr();
    // SAFETY: each index selects an initialized in bounds element.
    // Distinct indices denote disjoint logical elements. Both borrows
    // are tied to the exclusive input slice and no other access is used.
    unsafe {
        Some((&mut *pointer.add(left), &mut *pointer.add(right)))
    }
}

fn main() {
    let mut values = [10, 20, 30];
    let (left, right) = pair_mut(&mut values, 2, 0).unwrap();
    *left += 1;
    *right += 2;
    assert_eq!(values, [12, 20, 31]);
    assert!(pair_mut(&mut values, 1, 1).is_none());
    assert!(pair_mut(&mut values, 0, 3).is_none());
}
```

The safe wrapper cannot rely on a `debug_assert` for either condition. Release builds must enforce every fact required for soundness. A method may use unchecked internals after an unconditional check, or expose an unsafe unchecked variant whose documented preconditions assign those checks to callers.

Zero sized elements deserve explicit consideration. Their addresses may be equal even when their logical indices differ. The proof should be about disjoint elements under the slice model, not a simplistic pointer inequality test. This is one reason using a standard safe splitting operation can reduce proof complexity.

A concrete owner has a small proof surface

Consider a minimal heap allocated `u64` owner. A production program should normally use `Box<u64>`. Reimplementing this tiny case is useful because it isolates allocation ownership without generic destructors, zero sized values, or unsized metadata.

The invariant is that the stored pointer owns exactly one allocation created with the global allocator for `Layout::new::<u64>()`, the allocation contains one initialized `u64`, and the pointer is not duplicated into another owner.

Borrowed access is mediated by the owner's shared or exclusive borrow. Destruction releases the allocation exactly once.

Listing 31.2 Rust example

```
use std::alloc::{alloc, dealloc, handle_alloc_error, Layout};
use std::mem::ManuallyDrop;
use std::ops::{Deref, DerefMut};
use std::ptr::NonNull;

struct OwnedWord {
    pointer: NonNull<u64>,
}

impl OwnedWord {
    fn new(value: u64) -> Self {
        let layout = Layout::new::<u64>();
        // SAFETY: u64 has a nonzero valid allocation layout.
        let raw = unsafe { alloc(layout) };
        let pointer = match NonNull::new(raw.cast::<u64>()) {
            Some(pointer) => pointer,
            None => handle_alloc_error(layout),
        };
        // SAFETY: allocation has sufficient size and alignment and
        // is uniquely owned. write initializes its sole value.
        unsafe { pointer.as_ptr().write(value) };
        Self { pointer }
    }

    fn into_inner(self) -> u64 {
        let owner = ManuallyDrop::new(self);
        // SAFETY: the invariant supplies one initialized value.
        let value = unsafe { owner.pointer.as_ptr().read() };
        // SAFETY: this consumes the sole allocation ownership using
        // the same allocator and layout that created the block.
        unsafe {
            dealloc(owner.pointer.as_ptr().cast(), Layout::new::<u64>());
        }
        value
    }
}

impl Deref for OwnedWord {
    type Target = u64;

    fn deref(&self) -> &u64 {
        // SAFETY: the invariant supplies an initialized live value,
        // and shared owner access prevents conflicting mutation.
        unsafe { self.pointer.as_ref() }
    }
}
```

Listing 31.2 continued

```
impl DerefMut for OwnedWord {
    fn deref_mut(&mut self) -> &mut u64 {
        // SAFETY: exclusive owner access supplies unique access to
        // its sole initialized value for this borrow.
        unsafe { self.pointer.as_mut() }
    }
}

impl Drop for OwnedWord {
    fn drop(&mut self) {
        // SAFETY: this owner still owns the allocation. u64 needs
        // no destructor, and deallocation uses the original layout.
        unsafe {
            dealloc(self.pointer.as_ptr().cast(), Layout::new::<u64>());
        }
    }
}

fn main() {
    let mut value = OwnedWord::new(40);
    *value += 2;
    assert_eq!(*value, 42);
    assert_eq!(value.into_inner(), 42);
}
```

`ManuallyDrop` prevents ordinary owner destruction after `into_inner` manually deallocates the block. Suppression is established before that deallocation. If the owner were allowed to drop normally afterward, it would deallocate the same block twice. The pointer field is private, and the type does not implement `Clone` or `Copy`, preventing safe clients from creating another owning wrapper around it.

This example intentionally omits raw pointer export and adoption. Each additional operation would need a precise ownership transfer contract. `into_raw` would surrender automatic cleanup; `from_raw` would normally be unsafe because the caller must prove unique ownership, allocator compatibility, layout, and initialization. Accepting a pointer from any allocator would not be justified merely because it points to a `u64`.

Genericizing multiplies obligations

Replacing `u64` with `T` is not a mechanical edit. A generic type may be zero sized, require a destructor, carry borrowed references, have strict validity requirements, or be unavailable across threads. An owner must run the inner destructor before deallocation, including when that destructor panics. It must preserve variance and drop checking relationships even if the runtime representation contains only a raw pointer.

A raw owning pointer type may need a `PhantomData<T>` relationship. That marker is only part of the proof; it does not allocate storage or run destruction. A generic owner also needs to handle allocation layout correctly and avoid issuing forbidden zero sized allocation requests to APIs that require a nonzero size.

Specialization can therefore be a useful design decision. A component owning one fixed foreign resource can expose a small sound wrapper without becoming a universal container. Generality should be introduced when clients need it and when the expanded proof is understood. Reusing `Box`, `Vec`, or a mature arena can preserve broad correctness while leaving custom code focused on the domain protocol.

The same caution applies to deriving traits. Deriving `Clone` on a raw owner would duplicate the pointer instead of duplicating the allocation. A manually written clone could allocate and copy appropriately, but its panic and failure behavior still needs analysis. A derived `Debug` that only prints a pointer may be harmless yet unhelpful; a custom implementation that dereferences storage depends on the representation invariant remaining established.

Thread safety is a separate proof

Raw pointer fields can prevent automatic `Send` or `Sync` implementations. Adding an unsafe implementation asserts that the type's ownership or shared access contract is valid across threads. The proof must cover the inner value and every method, not merely the reference count or allocation mechanism.

For the concrete `OwnedWord`, moving the sole owner between threads can be justified because no aliasing owner remains and `u64` is transferable. Shared

references can support reads across threads because mutation requires an exclusive owner borrow and `u64` is safe for shared access. If such implementations are added, each deserves a documented argument. They should not be copied to a wrapper containing thread affine handles or unsynchronized interior mutable state.

For a generic owner, bounds such as `T: Send` and `T: Sync` often participate in those arguments, but the wrapper's own behavior can impose additional requirements. A destructor may need to run on a specific thread. A foreign library may prohibit sharing handles even when the numeric representation looks harmless. Thread safety follows the resource contract, not the apparent simplicity of its fields.

Unsafe functions still need explicit operation boundaries

An unsafe function assigns preconditions to its caller. Its body must still make local unsafe operations explicit under a disciplined Rust 2024 style. This separates the caller obligation from the implementation step relying on it and makes audits easier to follow.

Listing 31.3 Rust example

```
#![deny(unsafe_op_in_unsafe_fn)]

/// Reads one initialized u32 through a valid shared pointer.
///
/// # Safety
/// pointer must be aligned, live, initialized, and readable for this
/// operation without conflicting access or a data race.
unsafe fn read_word(pointer: *const u32) -> u32 {
    // SAFETY: supplied by this function's caller contract.
    unsafe { pointer.read() }
}

fn main() {
    let word = 123_u32;
    // SAFETY: a live shared reference supplies all read requirements.
    assert_eq!(unsafe { read_word(&word) }, 123);
}
```

An unsafe block should be small enough that the safety argument identifies the actual operation. A comment attached to a hundred line block often fails to explain which facts hold before and after each pointer change. Conversely, mechanically repeating the word valid next to every

operation adds little. State the particular allocation, bounds, initialization, and alias facts supplied at that point.

Safety comments are also maintenance interfaces. If a future edit changes a length update or introduces a callback, the nearby argument should make the invalidated assumption visible. A proof that exists only in the original author's memory is difficult to preserve through routine refactoring.

Safe callers may violate every nonbinding expectation

If a safe method requires an index to be in bounds for memory safety, it must check that condition or obtain it through a safe type whose invariant guarantees it. Documentation saying callers should provide a valid index does not make a safe unchecked dereference sound. Use an unsafe method when the caller genuinely bears that precondition, or return an error on invalid input.

The same reasoning applies to order requirements. A safe caller can call `finish` twice if the type still makes the method available, drop an iterator early, forget a guard, supply a closure that panics, and implement a trait in any way permitted by its safe contract. If an unsafe implementation assumes more, the trait may need to be unsafe with explicit obligations, or the implementation needs runtime validation.

An unsafe trait should be rare and purposeful. It expresses that implementing the trait incorrectly can let otherwise safe users trigger undefined behavior. It is not a marker for an implementation that happens to contain unsafe code. Many traits can remain safe if consumers validate the relevant properties rather than trusting implementors for memory safety.

Testing challenges the invariant

Useful tests include empty inputs, maximum boundaries, equal indices, early drops, repeated consumption attempts expressed through compile failures, and element types whose destructors record behavior or panic. Generic structures should include zero sized types and types containing borrowed data. Property tests can compare safe observable behavior with a straightforward model.

Miri can detect many invalid accesses, initialization errors, and aliasing violations along executed paths. Sanitizers can expose other failures in supported builds. Neither replaces the interface proof. A thousand passing inputs do not establish that no safe client can violate a hidden precondition.

When a tool reports a failure, do not merely adjust the operation until the test stops failing. Identify which invariant was false and whether other methods rely on it. A correct repair often changes a constructor, state transition, or public contract rather than one pointer expression.

Audit the smallest complete boundary

A practical review proceeds from safe entry points to unsafe operations. For each operation, find the facts it needs and trace them backward to checks or established invariants. Then follow state changes forward through success, error, panic, and destruction. Finally challenge the interface with a safe caller that ignores every advisory expectation.

The result should be a concise explanation of why the abstraction is sound, not merely a list of unsafe lines. Keeping the boundary small makes that explanation shorter and more durable. The best unsafe abstraction lets the rest of the program forget raw pointers because all relevant obligations have been captured in one coherent ownership and access policy.

Training Encounters

1. Why is a safe method containing an unchecked index access unsound if it only uses `debug_assert` to validate the index?

2. List the new obligations introduced by changing `OwnedWord` into an owner of arbitrary `T`. Explain why adding a phantom field alone is insufficient.

3. A container trusts a safe trait implementation to report the exact number of elements that an iterator will yield, and uses that count for unchecked writes. What must change if a lying implementation can overflow allocated storage?

Training Debriefs

1. Debug assertions can disappear in release builds, while safe callers remain allowed to provide invalid indices. A memory safety precondition needs an unconditional check, a trusted safe type relationship, or an explicitly unsafe caller contract. Testing debug mode does not justify release behavior.

2. Handle zero sized allocation policy, inner destruction and panic paths, borrowed lifetimes, variance, automatic thread safety, layout, and ownership markers. A phantom field records static relationships but does not perform cleanup, validate pointers, or enforce allocation behavior.

3. Either validate or dynamically limit writes independently of the reported count, or use a genuinely unsafe trait with precise obligations and controlled implementations. A safe trait's advisory behavior cannot be trusted as a memory safety proof when arbitrary safe implementors can violate it.

Further Reading

• The Rust Reference on unsafety describes unsafe functions, blocks, and traits.

• The Rust standard library allocation module documents allocation contracts used by custom owners.

• The Rustonomicon develops the obligations of safe abstractions implemented with unsafe code.

The Friendly Corruptor

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design AsciiLabel with private owned bytes, a checked constructor, an as_str accessor, and replacement of a byte at a requested index. The proposed implementation checks ASCII only during construction and uses from_utf8_unchecked in as_str. Its replacement method accepts any u8. Exhibit the safe caller sequence that invalidates the conversion's contract. Repair the interface while retaining useful mutation, and define what happens on invalid index or invalid replacement. Add an append operation and review it under the same invariant. Finally compare the repaired unsafe accessor against a completely safe implementation. The boss is defeated only when the explanation covers every safe constructor and mutator, rather than one successful test of the accessor.

Encounter Constraints

- Fields remain private and safe callers need no safety preconditions.
- Errors must leave the previous valid label usable.
- Do not treat documentation of a safe method as permission to cause undefined behavior.

Hint one / Hint two / Reveal debrief

Side Quest

Relax the wrapper from ASCII to general UTF-8. Replace a byte-oriented mutation API with operations on valid character boundaries, and decide which operations require reallocation. Explain why a valid u8 replacement at one position is no longer a sufficient local check. Design an edit operation that either produces one valid complete string or leaves the previous string unchanged, without weakening error behavior.

Capability unlocked Invariant Auditor

Choose Your Exit

Descend into unfinished storage / Level 32
Track validity when complete values do not exist yet.

Open the validation vault / Level 36
Choose tests and dynamic tools that attack the abstraction contract.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 32

The Scattered Assembly

Ferrum's archive reconstructs records from fragments delivered in an unpredictable order. The old builder counts how many fragments arrived and assumes that many leading slots are initialized. A late fragment can occupy slot nine while slot zero is still empty. This room replaces a convenient prefix assumption with a real state machine. The challenge is to know which values exist, who owns them, and what cleanup remains legal when construction stops.

Mission Build an out-of-order fixed-size assembly representation with explicit initialization and destruction accounting.

Suggested loadout Level 2, Level 6, Level 30, Level 31. These are guidance, not locked gates.

Mastery Targets

• Track initialization per slot instead of treating a count as a contiguous prefix.

• Define duplicate arrival, failure cleanup, and transfer of a complete result.

• Demonstrate that no empty slot is read as T and no owned value is dropped twice.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 32

Initialization as a State Machine

Storage can exist before a value exists

Low level code often needs to allocate space before it can construct every value that will occupy it. A foreign function may fill an output record. A collection may reserve capacity and initialize elements gradually. A fixed array constructor may fail after producing only a prefix. These situations require representing storage without pretending that a complete `T` already exists.

`MaybeUninit<T>` provides such storage. A `MaybeUninit<T>` can be valid even when its bytes do not represent a valid initialized `T`. That permission belongs to the wrapper type, not to `T`. Converting to `T`, obtaining a reference to `T`, or running `T`'s destructor requires proof that the appropriate value has actually been established.

An uninitialized integer is not an integer with a random but stable numeric value. Reading uninitialized storage as an ordinary initialized integer violates the required contract. Similarly, a null reference is not an empty reference waiting to be repaired. Types can have validity requirements that apply before any explicit dereference.

The state machine has at least two states: storage without a `T`, and storage containing one valid `T`. More complex structures track which fields or elements have reached the second state. Correct code updates that state alongside every write, read that transfers ownership, replacement, and destruction.

Write establishes a value without destroying an old one

`MaybeUninit::write` initializes the slot and returns a mutable reference to the newly written value. It does not drop any previous inner value. This makes it suitable for initially empty storage, but writing twice without consuming or dropping the first value can leak resources.

Listing 32.1 Rust example

```
use std::mem::MaybeUninit;

fn main() {
    let mut storage = MaybeUninit::<String>::uninit();
    storage.write(String::from("prepared"));
    // SAFETY: write established a valid String and nothing has moved
    // or dropped that value since initialization.
    let value = unsafe { storage.assume_init() };
    assert_eq!(value, "prepared");
}
```

`assume_init` consumes the wrapper and returns its initialized value. The unsafe operation asserts a fact the compiler does not track for this wrapper. It does not perform initialization, inspect the bytes for validity, or convert an arbitrary representation into a valid string.

Dropping the wrapper does not drop the inner `T`. That behavior is essential when storage is uninitialized, and it means implementations must track cleanup explicitly after initialization. If the wrapper in the example simply went out of scope after the write, the string allocation would be leaked. The wrapper's validity makes that memory safe, but it does not make it desirable.

The same distinction applies to `assume_init_read`, which reads an initialized value without consuming the wrapper. The caller must account for ownership afterward. Calling it twice on the same owning string would duplicate ownership unless the state machine prevents the second use. `assume_init_drop` destroys an established value but does not automatically record that the slot has become uninitialized.

Zeroed storage is not universal initialization

Filling bytes with zero is appropriate only when zero bytes form a valid value of the target type and satisfy any additional abstraction invariants. Integers and some simple records can have valid all zero representations. References, nonzero integers, and many enums cannot. Even a representation that satisfies a narrow language validity condition may fail a type's deeper safe method invariants.

Therefore `MaybeUninit::zeroed` does not make `assume_init` universally safe. It provides storage with zeroed contents under its contract. The implementation still needs a type specific proof before interpreting it as `T`. Padding may also have behavior that prevents treating returned object representations as a stable all zero byte array.

For ordinary construction, prefer real constructors and safe array helpers. `Default` is a semantic operation that produces a valid default value; it is not a claim that a type's representation is all zero. A default string is constructed through its type's valid initialization behavior, not through a general memset convention.

When a foreign API specifies zero initialized configuration structures, verify the exact foreign layout and validity rules at that boundary. Use an explicit boundary type rather than assuming an arbitrary Rust struct is suitable. A boolean, enum, pointer, and reserved byte field can each have different requirements.

A prefix length is an initialization invariant

A common representation stores capacity for `N` values and tracks how many leading elements are initialized. The invariant is that indices below `len` contain valid owned `T` values, while indices from `len` onward are not treated as initialized. The length is therefore part of the memory safety proof, not merely a convenient count.

Listing 32.2 Rust example

```
use std::mem::{ManuallyDrop, MaybeUninit};

struct Prefix<T, const N: usize> {
    storage: [MaybeUninit<T>; N],
    len: usize,
}

impl<T, const N: usize> Prefix<T, N> {
    fn new() -> Self {
        Self {
            storage: [const { MaybeUninit::uninit() }; N],
            len: 0,
        }
    }

    fn push(&mut self, value: T) -> Result<(), T> {
        if self.len == N {
            return Err(value);
        }
        self.storage[self.len].write(value);
        self.len += 1;
        Ok(())
    }

    fn pop(&mut self) -> Option<T> {
        if self.len == 0 {
            return None;
        }
        self.len -= 1;
        // SAFETY: the previous initialized prefix included this
        // slot. Decrementing len removes its old drop obligation.
        Some(unsafe { self.storage[self.len].assume_init_read() })
    }

    fn as_slice(&self) -> &[T] {
        // SAFETY: exactly this prefix contains initialized T values.
        // Array storage is aligned and contiguous, and the returned
        // shared borrow is tied to this owner.
        unsafe {
```

Listing 32.2 continued

```
            std::slice::from_raw_parts(
                self.storage.as_ptr().cast::<T>(),
                self.len,
            )
        }
    }

    fn into_array(self) -> Result<[T; N], Self> {
        if self.len != N {
            return Err(self);
        }
        let owner = ManuallyDrop::new(self);
        // SAFETY: every element is initialized, array layouts match,
        // and suppressing owner Drop transfers each value exactly once.
        let array = unsafe {
            owner.storage.as_ptr().cast::<[T; N]>().read()
        };
        Ok(array)
    }
}

impl<T, const N: usize> Drop for Prefix<T, N> {
    fn drop(&mut self) {
        while let Some(value) = self.pop() {
            drop(value);
        }
    }
}

fn main() {
    let mut prefix = Prefix::<String, 2>::new();
    assert!(prefix.push(String::from("left")).is_ok());
    assert_eq!(prefix.as_slice(), &[String::from("left")]);
    assert!(prefix.push(String::from("right")).is_ok());
    assert!(prefix.push(String::from("overflow")).is_err());
    let values = match prefix.into_array() {
        Ok(values) => values,
        Err(_) => panic!("expected a complete array"),
    };
    assert_eq!(values, [String::from("left"), String::from("right")]);
}
```

The write occurs before incrementing `len`. If the counter advanced first and initialization could fail, cleanup might treat an unwritten element as a `T`. The current `write` itself does not call user code, but keeping the order aligned with the invariant makes future changes easier to review.

`pop` decrements the count before transferring ownership out. Once the value is returned, the old storage must no longer be dropped as initialized. Updating the count first ensures the container's cleanup no longer includes that slot. The old bytes may remain physically present, but bytes remaining in memory do not imply an active owned value remains there.

`into_array` requires a complete prefix. It suppresses the original owner's destructor before reading the complete array, transferring each destruction responsibility to the returned array. A partial prefix is returned to the caller as the error value, preserving ownership instead of silently discarding the partially built collection.

This teaching container has a deliberate limitation: if an element destructor panics during its `Drop` loop, the remaining elements can leak. The state update still prevents double destruction, so the representation remains memory safe. A production container may add a cleanup guard that continues dropping remaining elements during unwinding, while considering the consequences of a second panic. Matching a standard collection's full panic behavior requires additional work.

Partial construction needs partial cleanup

The prefix representation supports fallible construction naturally. If an element builder returns an error or panics, the initialized prefix owns the values already produced. Its destructor drops those values on ordinary unwinding. No completed `[T; N]` is claimed until all elements exist.

A field based struct can use a similar state machine, but the state is a set of initialized fields rather than one prefix count. If the first field owns a string and constructing the second field fails, the first must be dropped or intentionally leaked. A guard with explicit state can provide cleanup. An ordinary temporary struct or tuple of already constructed values is often simpler because safe ownership handles partial evaluation automatically.

This is an important design filter. If the only reason for manual field initialization is aesthetic control over placement, the unsafe complexity may not be justified. If constructing a very large object directly in allocated storage avoids an unacceptable temporary or satisfies a foreign interface, the additional state machine can be worthwhile. Verify the generated behavior and actual requirement before taking on the proof.

Do not create `&mut T` to uninitialized storage and call ordinary methods hoping they will establish its fields. The reference asserts that a valid `T` already exists. Initialization should operate through `MaybeUninit`, raw field addresses, or a safe constructor that owns the required state transition.

Out parameters must specify success precisely

A foreign function may receive an output pointer and return a status code. The Rust wrapper must know exactly which statuses guarantee full initialization, whether partial initialization requires cleanup, and whether the function writes within the supplied storage. These facts cannot be inferred from a conventional zero return code without an interface contract.

The following local simulation makes the boundary explicit. It is an ordinary Rust function with an unsafe output contract, rather than a claim about any particular foreign library.

Listing 32.3 Rust example

```
use std::mem::MaybeUninit;

#[repr(C)]
#[derive(Debug, PartialEq)]
struct Statistics {
    accepted: u64,
    rejected: u64,
}

/// # Safety
/// output must be aligned and writable for one Statistics value.
/// On true, it is fully initialized. On false, it is untouched.
unsafe fn fill_statistics(output: *mut Statistics, available: bool) -> bool {
    if !available {
        return false;
    }
    // SAFETY: the caller supplies writable storage, and write does
    // not read or drop its previous uninitialized contents.
    unsafe {
        output.write(Statistics { accepted: 12, rejected: 3 });
    }
    true
}

fn statistics(available: bool) -> Option<Statistics> {
    let mut output = MaybeUninit::<Statistics>::uninit();
    // SAFETY: output supplies aligned writable storage of the exact
```

Listing 32.3 continued

```
    // required type; no initialized value is being overwritten.
    let initialized = unsafe {
        fill_statistics(output.as_mut_ptr(), available)
    };
    if initialized {
        // SAFETY: true is documented to establish every field.
        Some(unsafe { output.assume_init() })
    } else {
        None
    }
}

fn main() {
    assert_eq!(statistics(true), Some(Statistics {
        accepted: 12, rejected: 3,
    }));
    assert_eq!(statistics(false), None);
}
```

A real wrapper should also respect the foreign ABI and resource ownership rules. If the output contains pointers, successful initialization may still require validating lengths or adopting resource handles through appropriate constructors. A fully written record is not necessarily semantically valid input for every safe Rust type.

If failure leaves partially initialized owned fields, the status protocol must identify enough state to release them correctly. A foreign cleanup function may be required. Assuming the output is untouched on failure when the library documents partial results can leak resources or cause invalid destruction. Keep that contract next to the wrapper rather than in a distant comment.

Initialization tracking should survive future edits

The dangerous bugs often arise when a previously infallible step gains user code. A counter update once adjacent to a raw write may become separated by logging, allocation, conversion, or a callback. A panic between them can expose a state the original proof never considered.

Keep initialization state updates close to the operations they describe, and explain why their order is correct. Prefer transitions that leave a conservative state on interruption: undercounting an initialized element can leak, while overcounting an uninitialized element can cause undefined

behavior during a read or drop. Both deserve repair, but they differ in severity and in the guards needed to recover.

Testing with destructor counters can reveal leaks and double destruction. A test type can record a unique identifier when dropped, and a fallible builder can fail after each possible prefix length. Include zero capacity, full capacity, failed completion, early pop, and a value type that is not `Copy`. Miri can challenge invalid reference formation and use of uninitialized storage along those tested paths.

`MaybeUninit` is most effective when it is hidden behind a small safe state machine. Callers should manipulate meaningful operations such as push, pop, finish, and abort. They should not need to remember whether one particular slot was written before another method was called. That knowledge belongs in the owner, where it can be checked once and preserved by construction.

Training Encounters

1. Why must a prefix container increment its initialized count after writing a new element, and decrement it before returning a popped owner?

2. A programmer calls `assume_init_mut` on an uninitialized object to invoke a method that fills every field. Explain why the plan is invalid before the method body begins.

3. A foreign function returns failure after allocating a string in one output field but before filling a second field. What information must its contract provide for a correct Rust wrapper?

Training Debriefs

1. The count authorizes reads and destruction. Advancing it too early can expose uninitialized storage as a value. Removing a value's count before ownership transfer prevents later cleanup from dropping the old slot again. The physical bytes need not be erased; the logical ownership state must change.

2. Forming the mutable reference already asserts a valid initialized object with appropriate type invariants. A later method cannot retroactively justify that assertion. Write through uninitialized storage APIs or raw field

pointers until the complete valid value exists, then form ordinary references.

3. The wrapper must know which fields are initialized on each failure path, who owns any allocated resources, and which cleanup operation can release partial results. A status code alone is insufficient unless it unambiguously specifies that state. Calling `assume_init` on the complete output would be invalid when a field remains uninitialized.

Further Reading

• The Rust standard library MaybeUninit documentation specifies initialization, reads, writes, and destruction obligations.

• The Rust standard library ManuallyDrop documentation explains suppression of automatic destruction.

• The Rust Reference on destructors provides the cleanup model that manual initialization must preserve.

The Ninth Fragment

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a generic assembler for N slots. insert(index, value) may receive slots in any order; choose and document whether duplicate arrivals replace the old value or return the new value as an error. finish consumes the assembler and succeeds only when every slot is initialized. A failed finish should preserve enough ownership for retry or clean disposal according to your stated API. First implement a safe model using optional elements. Then outline a MaybeUninit representation and its initialization map. Include an early-return construction error and a destructor that panics in one stored element. Explain what memory-safety guarantee remains, even if completing cleanup of every other element cannot be promised by your teaching implementation.

Encounter Constraints

• Use stable Rust mechanisms rather than assuming generic const-expression support.

• Do not create a reference to an incomplete T.

• Keep logical initialization state consistent before transferring or destroying an owner.

Hint one / Hint two / Reveal debrief

Side Quest

Add replacement as a transactional operation: a user callback computes a new element from the old one and may return an error or panic. Choose whether the old value must remain available during computation. Explain which signatures can support failure preservation without cloning T, and which require a weaker contract or a separately prepared replacement. Compare the safe model with your initialization bookkeeping.

Capability unlocked Initialization Ledger

Choose Your Exit

Cross the foreign boundary / Level 33
Foreign out-parameters can expose similarly partial construction states.

Test the hostile sequence / Level 36
Generate out-of-order arrivals and verify destruction accounting against a model.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 33

The Two Allocators

A foreign maintenance tool connects to Ferrum's Rust archive. Its first integration returned a pointer, length, and capacity, then invited callers to free the pointer however they preferred. Different allocators and ambiguous ownership turned a convenient bridge into a trap. The tool needs a small explicit protocol instead: borrowed input, owned output, and one matching destruction path. Your task is to make the crossing understandable from both sides of the boundary.

Mission Design a narrow C-compatible ownership protocol for Rust-created byte results and borrowed foreign input.

Suggested loadout Level 6, Level 29, Level 30, Level 31. These are guidance, not locked gates.

Mastery Targets

- Specify creation, read-only access, and matching release of one opaque result handle.
- Keep Rust containers and references out of the public C ABI.
- Document foreign-pointer preconditions, error ownership, and panic containment at each export.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 33

Foreign Interfaces as Ownership Protocols

A Calling Convention Does Not Define a Safe API

An FFI declaration connects two systems that may disagree about representation, ownership, failure, and concurrency. extern C selects a calling convention appropriate to the target. It does not establish how long a pointer remains valid, which allocator owns it, whether callbacks can arrive later, or whether a foreign function mutates memory behind an apparently shared reference.

The productive way to design an FFI boundary is as a protocol. Every pointer has an owner and an access duration. Every allocation has a matching destruction operation. Every callback registration has a lifetime and an unregistration rule. Every status code has a postcondition. Write these rules before converting raw pointers into Rust references, because reference creation asserts stronger facts than a foreign pointer alone provides.

There are two different layers of unsafety. Declaring a foreign signature asserts that its ABI and types match the foreign implementation. Calling it may require additional preconditions about arguments and global state. A safe wrapper must discharge those preconditions for every safe caller, not merely for the examples the author expects.

Edition 2024 makes some of these assertions more visible through unsafe extern blocks and unsafe attributes such as no_mangle. The syntax records responsibility; it does not validate a header or detect an allocator mismatch. The review still needs both sides of the boundary.

Use Explicit Representations for Shared Data

Rust's ordinary struct layout is not a foreign ABI contract. repr(C) requests C-compatible field layout rules for the target, but it does not make every field type suitable for C. String, Vec, references with hidden validity assumptions, and trait objects are not automatically portable C interface types.

A common borrowed representation is a pointer and a length. The pair still needs a validity contract. In particular, a zero-length foreign buffer may use a null pointer even though Rust slice constructors require a nonnull aligned pointer, including for empty slices. A wrapper can handle the empty case before constructing a slice.

Listing 33.1 Rust example

```
#[repr(C)]
pub struct ByteView {
    pub data: *const u8,
    pub len: usize,
}

/// # Safety
/// For nonzero len, data must identify len initialized readable bytes
```

Listing 33.1 continued

```
/// in one live allocation, with total size at most isize::MAX.
/// The bytes must not be mutated for the duration of this call.
pub unsafe extern "C" fn sum_bytes(view: ByteView) -> u64 {
    if view.len == 0 {
        return 0;
    }
    // SAFETY: The caller supplies the documented readable allocation.
    let bytes = unsafe {
        std::slice::from_raw_parts(view.data, view.len)
    };
    bytes.iter().fold(0_u64, |sum, byte| {
        sum.wrapping_add(u64::from(*byte))
    })
}
```

The function returns a wrapping sum deliberately, avoiding an arithmetic panic for very large inputs. It borrows the bytes only during the call and returns no reference. It does not validate an arbitrary nonnull pointer. A pointer can be nonnull and still be dangling, too short, or concurrently modified. Those facts cannot generally be recovered by inspecting the address.

The length uses usize because the interface is intended to pair with the target C size type in a correctly matched declaration. A serialized file format or cross-process wire format should use explicitly specified widths and byte order instead. ABI compatibility on one target is not a portable serialization design.

For tagged data, accept an integer tag and validate it rather than accepting an arbitrary foreign integer directly as a Rust enum. An invalid Rust enum discriminant can violate value validity before a match has a chance to reject it. repr(u8) controls representation; it does not make every u8 bit pattern a valid enum value.

Opaque Handles Keep Representation on One Side

An opaque pointer lets C retain ownership of a Rust-created object without understanding its layout. The interface must provide operations and a matching destructor. The following complete Rust API uses a deliberately prefixed symbol family. Its C header would declare Counter as an incomplete struct.

Listing 33.2 Rust example

```
pub struct Counter {
    value: u64,
}

#[unsafe(no_mangle)]
pub extern "C" fn rbook_counter_new() -> *mut Counter {
    Box::into_raw(Box::new(Counter { value: 0 }))
}

/// # Safety
/// If either argument is null, no memory is accessed. Otherwise
/// counter must be a live, exclusive handle from rbook_counter_new
/// and output must be aligned, disjoint u64 storage for exclusive writes.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rbook_counter_add(
    counter: *mut Counter,
    delta: u64,
    output: *mut u64,
) -> i32 {
    if counter.is_null() || output.is_null() {
        return 1;
    }
    // SAFETY: The caller guarantees a live exclusively accessible handle.
    let counter = unsafe { &mut *counter };
    let Some(next) = counter.value.checked_add(delta) else {
        return 2;
    };
    counter.value = next;
    // SAFETY: output is valid, aligned, writable, and disjoint.
    unsafe { output.write(next) };
    0
}

/// # Safety
/// A nonnull pointer must come from rbook_counter_new and must not have
/// been destroyed. No call or reference may still use the allocation.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rbook_counter_free(counter: *mut Counter) {
    if !counter.is_null() {
        // SAFETY: This call receives the unique original allocation.
        drop(unsafe { Box::from_raw(counter) });
    }
}
```

The null checks support a defined error for missing arguments. They do not make the add function safe for arbitrary pointers, so the Rust function remains unsafe. The output pointer must be disjoint from the Counter allocation because creating &mut Counter establishes exclusive access to

the entire value during its use. A C caller cannot bypass that contract by pointing output at an internal field.

Status 2 leaves the counter and output unchanged because checked arithmetic happens before mutation. Status 0 updates both. The destructor accepts null as a no-op, but a stale pointer is still invalid. Double destruction is not made safe by checking nonnull because freeing an allocation does not automatically clear every foreign copy of its address.

The no_mangle attribute exposes global symbol names. The prefix reduces collision risk but does not prove uniqueness across an arbitrary linked process. A real library needs a deliberate naming and versioning strategy. The unsafe attribute is a reminder that symbol identity is part of correctness, not merely linker decoration.

Allocation Must Return to the Correct Owner

A pointer does not identify which allocator can free it. Memory allocated by Rust must be returned through the matching Rust ownership protocol unless an explicit compatible allocation contract says otherwise. Memory allocated by a foreign library must be destroyed with that library's corresponding operation. Even if two allocators happen to use the same system heap today, relying on that coincidence makes the interface fragile.

CString offers a useful example. as_ptr borrows a nul-terminated buffer owned by the CString. into_raw transfers ownership to a raw pointer. from_raw takes that ownership back under specific requirements. It is not a generic constructor for any C string pointer.

Listing 33.3 Rust example

```
use std::ffi::{CStr, CString};

fn main() {
    let owned = CString::new("worker-ready").unwrap();
    let borrowed = owned.as_ptr();
    // SAFETY: owned remains alive and the bytes remain unchanged.
    let view = unsafe { CStr::from_ptr(borrowed) };
    assert_eq!(view.to_bytes(), b"worker-ready");

    let transferred = owned.into_raw();
    // A foreign owner may retain transferred under an agreed protocol.
    // SAFETY: This is the exact pointer from into_raw, returned once,
    // with its terminating nul and string length preserved.
```

Listing 33.3 continued

```
    let recovered = unsafe { CString::from_raw(transferred) };
    assert_eq!(recovered.to_str().unwrap(), "worker-ready");
}
```

The borrowed CStr view ends before ownership is transferred in this example. Foreign code must not use the borrowed pointer after the recovered CString is dropped. If ownership is transferred, the foreign side must eventually call a designated Rust release function or otherwise return the exact pointer according to the contract.

Interior nul bytes are rejected by CString::new because C string length is determined by the first nul. CStr::to_str separately checks UTF-8. Nul termination and text encoding are different invariants. A foreign API using a platform-specific encoding may require conversion rather than treating every C string as UTF-8.

For raw byte buffers, include the information required for correct destruction in an opaque owner or a carefully specified descriptor. A Vec's pointer alone is insufficient to reconstruct it; length and capacity must match the original allocation contract. An opaque buffer handle is often safer than asking C callers to preserve a Rust vector's exact capacity metadata.

A Callback Recreates Code Plus Environment

C callback APIs often receive a function pointer and a void context pointer. Together they represent a closure's code and captured state. A trampoline casts the context back to its original type and invokes the callback. Its safety depends on lifetime, type, exclusive access, threading, and panic behavior.

The following complete example simulates a synchronous foreign driver in Rust. It demonstrates a safe wrapper around a callback that may panic, without permitting unwinding through the C ABI callback frame.

Listing 33.4 Rust example

```
use std::any::Any;
use std::ffi::c_void;
use std::panic::{self, AssertUnwindSafe};

struct Context<F> {
```

Listing 33.4 continued

```
    callback: F,
    panic: Option<Box<dyn Any + Send>>,
}

/// # Safety
/// The context must be a live Context<F> on visit's thread.
/// Each call must have exclusive access through its return.
unsafe extern "C" fn trampoline<F: FnMut(i32)>(
    context: *mut c_void,
    value: i32,
) {
    // SAFETY: The caller guarantees validity and exclusive access.
    let context = unsafe { &mut *context.cast::<Context<F>>() };
    if context.panic.is_some() {
        return;
    }
    let result = panic::catch_unwind(AssertUnwindSafe(|| {
        (context.callback)(value);
    }));
    if let Err(payload) = result {
        context.panic = Some(payload);
    }
}

fn visit<F: FnMut(i32)>(values: &[i32], callback: F) {
    let mut context = Context { callback, panic: None };
    let raw = (&mut context as *mut Context<F>).cast::<c_void>();
    for &value in values {
        // SAFETY: Same thread, serial calls, and the exact context type.
        unsafe { trampoline::<F>(raw, value) };
    }
    if let Some(payload) = context.panic {
        panic::resume_unwind(payload);
    }
}

fn main() {
    let mut total = 0;
    visit(&[2, 3, 5], |value| total += value);
    assert_eq!(total, 10);
}
```

The driver must call on the thread running visit, with no overlap or reentry, and retain no context pointer after visit returns. Each call gets exclusive access to the live Context<F> with the exact type. A driver can wait for a callback on another thread and still be synchronous. That design needs F: Send and a synchronized transfer and return of exclusive access.

catch_unwind records an unwinding panic and suppresses subsequent callback bodies. After the driver returns through the C boundary, visit resumes unwinding in ordinary Rust. AssertUnwindSafe is justified here by the narrow policy: after a callback panic, the wrapper does not treat the callback's state as successfully usable and resumes the panic rather than continuing the workflow. It does not magically repair captured application state.

Aborting panics are not caught. A real foreign API may need a status return that requests early termination rather than continuing no-op callbacks. It may also have cleanup requirements before the wrapper can resume unwinding. The bridge must match those semantics, not merely reuse this trampoline mechanically.

Asynchronous Registration Requires a Stronger Lifetime Protocol

If foreign code stores the context pointer, a stack-local context is insufficient. The owner must keep it alive until registration ends and every in-flight callback has finished. Unregistering future callbacks is not necessarily the same as waiting for current callbacks. Read the foreign library's exact guarantee.

A robust registration guard often owns a heap-allocated context and a foreign registration token. Drop or an explicit close operation unregisters the callback and synchronizes with in-flight calls before destroying the context. If unregistration can fail or block, an explicit close method may be necessary to expose that behavior. A destructor cannot conveniently report an error to its caller.

Persistent registration also needs a contract that remains sound if safe code forgets the guard. A destructor alone cannot protect borrowed captures: mem::forget can suppress unregistration, and leaking the boxed callback does not keep its borrowed referents alive. Persistent registrations normally own their callback state and require F: 'static, together with whatever Send, Sync, and invocation restrictions the foreign threading model requires. Here static constrains captured borrows; it does not require the callback allocation to live forever. A genuinely scoped registration may accept shorter borrows only if the scope itself guarantees

teardown and completion before returning, even when an inner registration handle is forgotten. Do not make memory safety depend on a safe caller eventually running Drop.

Threaded callbacks also change access requirements. A raw context pointer does not make FnMut safe to invoke concurrently. Use synchronization, require a thread-safe callback contract, or forward events into a Rust-owned channel. Reentrancy on one thread can violate exclusive access just as concurrency can. A foreign callback that recursively triggers another callback needs an explicit model.

If the foreign library cannot guarantee that callbacks stop before destruction, a safe wrapper may need shared ownership with a separate shutdown protocol, or it may be impossible to provide the desired safe interface without leaking the context. A small leak can preserve memory safety in a deliberately limited situation, but it is not a substitute for understanding the protocol.

Unwinding Must Match the ABI Contract

An ordinary C ABI boundary is not a general exception transport. A Rust panic that attempts to escape a nonunwinding ABI causes an abort under the documented behavior. A foreign exception entering Rust through a boundary that does not permit it can cause undefined behavior. C-unwind exists for interfaces intentionally designed to permit unwinding, but both sides and all intervening frames must support the chosen protocol.

Rust 1.99 stabilized definitions of C-variadic functions on supported targets. Each `VaList::next_arg` requires an available argument of compatible promoted type and a value the read type can represent. Variadics need a calling protocol. `catch_unwind` does not provide portable foreign exception recovery.

Panic containment is also not error rollback. If an exported operation mutates a handle and then catches a panic, it must decide whether the handle remains usable. Returning an ordinary failure code while leaving a broken invariant can turn later calls into confusing failures. Prepare changes before committing, or invalidate and destroy the affected object when recovery cannot be guaranteed.

Version the Protocol Rather Than the Compiler's Internals

A C-compatible interface should describe its own stable types and functions. Do not expose Rust's trait object representation, default enum layout, or native Rust ABI as a plugin standard. Recompiling both sides with the same toolchain may make an internal interface workable, but that is a deployment constraint, not a general ABI guarantee.

For extensible descriptors, include an explicit version or structure-size field with documented negotiation rules. Define whether unknown fields are ignored, whether the caller zero-initializes reserved fields, and who owns any pointed-to data. Fixed-width integers help wire protocols, while target ABI types may fit in-process interfaces. Keep those purposes separate.

Test the actual boundary with the foreign compiler and target. Rust-only tests cannot validate a hand-written C header's calling convention, packing, or integer types. Exercise defined failure cases, callback shutdown races, and encoding edges. For the raw destructor above, test null and exactly-once release. A second free violates its safety contract.

Training Encounters and Debriefs

Exercise 1. A C API returns a nonnull pointer and a length. Is that enough to construct a safe Rust slice?

Solution. No. The allocation must remain live, cover the full range within one allocation, contain initialized values, satisfy alignment and size limits, and have no conflicting mutation during the borrow. Null checking establishes only one small part of the contract. Handle zero length explicitly if the foreign convention permits null empty buffers.

Exercise 2. A registration guard unregisters a callback and immediately frees its context. What additional guarantee is needed?

Solution. Unregistration must also ensure that no callback is currently using the context or can begin afterward. Preventing future scheduling alone is insufficient if an in-flight callback still holds the pointer. The guard needs synchronization with completion before destruction.

Exercise 3. Why should a foreign caller not use free on a pointer returned by CString::into_raw?

Solution. The allocation belongs to Rust's CString ownership protocol. It must be returned through the matching Rust destruction path, with the exact pointer and required length properties preserved. Allocator compatibility cannot be assumed from the pointer's appearance or from one platform's current implementation.

Further Reading

• Rustonomicon FFI discusses ownership, callbacks, representation, and unwinding at foreign boundaries.

• Standard library CString specifies borrowed pointers and raw ownership transfer.

• Rust Reference on external blocks defines foreign declarations and ABI strings.

• Rust Edition Guide on unsafe attributes explains the explicit responsibility associated with exported symbols.

• Standard library mem::forget explains why unsafe abstractions must remain sound when safe callers suppress destruction.

The Borrowed Bridge Toll

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Sketch a C-facing API that accepts borrowed input bytes, transforms them into an owned Rust result, and returns an opaque handle. Separate the operation that exposes a temporary data pointer and length from the operation that releases the result. Specify exactly how long the exposed bytes stay valid, whether the result may mutate, and what a null input means for zero length. A failed creation must not leave an ambiguous partially owned output. Include the signatures and ownership table a C caller needs, plus a safe Rust owner that uses the matching release operation when wrapping the foreign interface in the reverse direction. Do not claim that null checking can validate arbitrary pointer liveness.

Encounter Constraints

• The allocating side supplies the matching destruction function.

- Foreign callers must not copy an owning handle and release it twice.
- No unwinding may cross an ABI boundary that does not permit it.

Hint one / Hint two / Reveal debrief

Side Quest

Add a completion callback that foreign code can invoke after registration returns. Specify which threads may call it, whether calls can overlap, which state must be owned and static, and what unregister guarantees about in-flight calls. A forgotten guard can leak the context without extending borrowed captures. A scoped alternative needs teardown guaranteed by the scope boundary.

Capability unlocked Boundary Keeper

Choose Your Exit

Forge a public library / Level 41
Turn an ownership protocol into a versioned compatibility promise.

Choose the archive representation / Level 34
Evaluate storage and retention behind the opaque boundary.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 34

The Archive That Remembers Too Much

The Endgame routes pass through an archive that answers three questions: find one station, list stations in name order, and show recent events. Engineers keep adding maps and cached copies whenever a query slows down. Ferrum now stores more indexing data than original records. This chamber asks you to choose a representation from actual operations, account for retained ownership, and decide where an index earns its memory cost.

Mission Choose a coherent collection layout for mixed queries and explain the cost of maintaining each retained index.

Suggested loadout Level 8, Level 14, Level 29. These are guidance, not locked gates.

Mastery Targets

• Define canonical record ownership and make secondary indexes refer to stable logical identities.

• Explain update, deletion, sorted traversal, and event-retention behavior.

• Propose measurements that compare complete workloads rather than lookup complexity alone.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 34

Collections Allocation and Data Representation

Choose a Representation for the Whole Workload

Collection choice is an ownership and access-pattern decision before it is a complexity-table decision. A Vec stores a contiguous sequence and moves elements when it reallocates. A HashMap organizes keyed lookup without promising iteration order. A BTreeMap maintains sorted keys and supports ordered ranges. A VecDeque represents efficient operations at both ends while its logical sequence may wrap across two contiguous regions.

These structures differ in allocation behavior, locality, stable-address properties, destruction cost, and the kinds of borrowing their APIs permit. A linked structure with attractive insertion complexity may lose to a vector because traversal follows scattered allocations. A hash table with expected constant-time lookup may be inferior for a tiny fixed set where hashing costs more than a short scan. A sorted vector can provide compact lookup for data built once and read many times.

Begin with the workload's phases. How is data constructed? How often does it change? Are reads sequential or random? Is ordering observable? How long are references retained? Is peak memory or tail latency more important than average throughput? A representation suitable for one phase may be worth converting before the next.

Avoid optimizing a single method in isolation. Reducing lookup time by keeping a second index increases update work and creates a consistency invariant between structures. Sharing strings through Arc reduces repeated content allocation but adds reference counting and indirection. Every representation buys some operations by charging others.

Length Capacity and Ownership Are Different Facts

A Vec owns initialized elements up to len and allocated storage up to capacity, subject to special handling for zero-sized types. Spare capacity is not a collection of initialized T values. Growing len requires establishing initialization; shrinking len without dropping removed elements can leak resources. Safe methods maintain these distinctions automatically.

Reallocation can move the element buffer. A pointer or reference into that buffer cannot remain usable across an operation that may reallocate unless an abstraction provides the necessary guarantee. Rust's borrowing rules reject many such patterns with ordinary references. Raw pointers do not gain stability merely because the programmer bypasses those checks.

Listing 34.1 Rust example

```
fn main() {
    let mut values = Vec::with_capacity(8);
    values.extend([10, 20, 30]);
    let capacity = values.capacity();
    values.clear();
    assert!(values.is_empty());
    assert_eq!(values.capacity(), capacity);

    values.extend([7, 8]);
    assert_eq!(values, [7, 8]);
}
```

clear drops the elements while retaining the allocation for reuse. That is valuable in a repeated batch loop. It can be undesirable if one unusually large batch causes a long-lived worker to retain excessive memory. A deliberate retention policy might shrink after a large outlier or replace the buffer when idle. Shrinking on every batch can instead create allocation churn.

with_capacity and reserve request enough storage for a lower bound; they do not promise an exact growth factor. reserve takes an additional element count relative to the current length, not a desired total capacity. reserve_exact avoids deliberate speculative over-allocation by the collection, but the allocator may still provide more space. Do not build correctness on exact capacity values unless a specific API guarantees them.

Fallible reservation can report capacity overflow or allocation failure where supported, but it does not turn every later operation into a

completely fallible allocation protocol. Formatting, cloning, and nested element allocations may still allocate independently. If an application needs a hard memory bound, model all owned allocations rather than only the outer vector.

Stable Handles Are Different From Stable Addresses

A vector index remains a number when the vector reallocates, so an index-based handle can survive buffer movement. It does not automatically survive removal, reordering, or reuse. swap_remove changes which element occupies an index. A generational arena adds identity information so a reused slot does not silently validate an old handle.

For nonzero-sized T, Box<T> inside a vector gives each T its own allocation. Moving the box handle leaves that allocation in place. This can justify the allocation and indirection when pointees need stable addresses. An index already survives growth without boxes.

A Box alone permits moving its value out. Pin adds Level 24's movement and destruction obligations. Rust 1.99 provides `Box::into_non_null` and `Vec::into_parts` for ownership handoffs, with unsafe reconstruction APIs. Use them for allocations you plan to reclaim. Reserve `Box::leak` for leaking.

If readers need snapshots while writers continue modifying data, a shared immutable allocation can simplify lifetime management. Copy-on-write, reference-counted snapshots, or versioned structures trade update cost for stable read views. A borrowed view into a mutating vector cannot provide the same independence without an ownership change.

Strings Are Byte Storage With a UTF Eight Invariant

`String` owns UTF-8 bytes. Its length and capacity count bytes. `str` is the unsized UTF-8 string type, and `&str` is a borrowed view of it. A `char` is a Unicode scalar value. A grapheme cluster can contain several scalar values. Choose the level that matches the operation.

Listing 34.2 Rust example

```
fn main() {
    let text = String::from("aé");
    assert_eq!(text.len(), 3);
    assert_eq!(text.chars().count(), 2);
    assert_eq!(text.get(0..1), Some("a"));
    assert_eq!(text.get(1..3), Some("é"));
    assert_eq!(text.get(0..2), None);
}
```

The final range ends inside a multibyte encoding, so get rejects it. Direct indexing with that invalid range would panic. Byte offsets are useful in parsers and diagnostics, but slicing must respect UTF-8 boundaries. If the protocol is byte-oriented, parse &[u8] first and validate text where the protocol requires it.

Repeatedly finding the nth char by scanning from the beginning can create quadratic work. A single chars traversal, stored byte offsets, or a domain-specific index may be more appropriate. For user-visible cursor movement and truncation, scalar-value boundaries may still split a grapheme cluster. The standard string types provide valid UTF-8, not a full text-layout model.

String::from_utf8 can validate an owned byte vector without necessarily copying its contents. A lossy conversion changes invalid bytes into replacement text and may allocate depending on the input. That is a semantic choice, not just a convenient way to avoid an error. In identifiers, protocol signatures, or exact round trips, replacement can destroy information the application needs.

Borrowed Lookup Can Avoid Temporary Key Allocation

HashMap<String, V> can look up an entry with &str because the borrowed key representation has compatible equality and hashing behavior. This lets an API accept a borrowed query without allocating a String solely for the lookup.

Listing 34.3 Rust example

```
use std::collections::HashMap;

fn add_count(counts: &mut HashMap<String, u64>, name: &str) {
    if let Some(count) = counts.get_mut(name) {
        *count += 1;
```

Listing 34.3 continued

```
    } else {
        counts.insert(name.to_owned(), 1);
    }
}

fn main() {
    let mut counts = HashMap::new();
    add_count(&mut counts, "compile");
    add_count(&mut counts, "compile");
    assert_eq!(counts.get("compile"), Some(&2));
}
```

This implementation allocates a key only when inserting a new name. A straightforward entry(name.to_owned()) implementation is more concise and performs one entry search, but it constructs an owned key even when the entry already exists. The tradeoff depends on repetition rate, key length, hashing cost, and the map API available in the supported compiler range.

The example's count addition can overflow in an extremely long-running process. A production counter should choose checked, saturating, wrapping, or wider arithmetic according to its semantics. Collection ergonomics do not remove numeric policy decisions. If overflow is a recoverable condition, report it before changing the count.

Borrow-based lookup requires compatible Hash and Eq behavior between the stored key and query. A custom case-insensitive key wrapper cannot simply borrow an ordinary str if its equality and hashing semantics differ. The Borrow relationship has a semantic contract. Use an appropriate query representation or normalize keys consistently.

Hashing Is an Algorithm and a Trust Decision

HashMap's expected lookup behavior depends on hashing and collision patterns. The standard default hasher is selected with resistance to collision attacks in mind, but its precise algorithm is not a stable contract. Replacing it with a faster deterministic hasher can be appropriate for trusted numeric keys and inappropriate for adversarial text input.

Do not benchmark only randomly distributed integers and assume the result applies to user-controlled strings. Include realistic key lengths,

repeated prefixes, misses, load factors, and insertion patterns. Hashing a long string is linear in its byte length even when table access is expected constant time with respect to entry count.

HashMap iteration order is unspecified. If output order affects a file, signature, cache key, or test snapshot, sort explicitly or use an ordered representation. A deterministic custom hasher does not create a documented stable iteration order across implementation changes. Representation details can still change.

Capacity also affects traversal in the current standard implementation: iteration visits table storage rather than only occupied entries, so a sparse map can cost more to scan than len suggests. Treat that as an implementation-sensitive performance fact and verify it against the supported standard library. The broader design lesson is stable: excess retained capacity can affect both memory use and access cost.

A Complete Interning Representation

Interning replaces repeated string contents with compact symbols. It is useful when many records repeat a smaller vocabulary and equality or indexing is frequent. The interner must own the canonical strings and maintain a relationship between text lookup and symbol lookup.

Listing 34.4 Rust example

```
use std::collections::HashMap;
use std::sync::Arc;

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
struct Symbol(usize);

struct Interner {
    by_text: HashMap<Arc<str>, Symbol>,
    texts: Vec<Arc<str>>,
}

impl Interner {
    fn new() -> Self {
        Self { by_text: HashMap::new(), texts: Vec::new() }
    }

    fn intern(&mut self, text: &str) -> Symbol {
        if let Some(&symbol) = self.by_text.get(text) {
            return symbol;
```

Listing 34.4 continued

```
        }
        let symbol = Symbol(self.texts.len());
        let owned: Arc<str> = Arc::from(text);
        self.texts.push(Arc::clone(&owned));
        self.by_text.insert(owned, symbol);
        symbol
    }

    fn resolve(&self, symbol: Symbol) -> Option<&str> {
        self.texts.get(symbol.0).map(AsRef::as_ref)
    }

    fn shared_text(&self, symbol: Symbol) -> Option<Arc<str>> {
        self.texts.get(symbol.0).cloned()
    }
}

fn main() {
    let mut interner = Interner::new();
    let first = interner.intern("parser");
    let second = interner.intern("parser");
    let third = interner.intern("scheduler");
    assert_eq!(first, second);
    assert_ne!(first, third);
    assert_eq!(interner.resolve(first), Some("parser"));
    let retained = interner.shared_text(first).unwrap();
    drop(interner);
    assert_eq!(&*retained, "parser");
}
```

Each distinct string has one shared content allocation referenced by both indices. Cloning Arc increments a reference count rather than copying the text. The vector maps a dense symbol to text. The hash map finds an existing symbol from a borrowed str. The strings remain immutable, preserving hash-key consistency.

The design retains every interned string until the interner is dropped, apart from external Arc owners that can retain selected strings longer. That is appropriate for a bounded compiler vocabulary and potentially dangerous for an unbounded service receiving arbitrary new names. An interner is a retention policy. A cache with eviction needs a different identity and ownership model.

Symbols are local to one interner, but the type does not encode which one. Passing a Symbol from another interner may resolve to unrelated text at

the same index. Returning Option prevents out-of-bounds access but cannot detect that semantic mix-up. A production design can attach an owner identity, use branded lifetimes, or keep symbols within a component boundary that controls their origin.

The two indices also create an update invariant. Every stored text should have a matching map entry and every map symbol should index the corresponding text. This example relies on ordinary allocation behavior and a fixed standard hasher; a more general fallible or user-hasher version should analyze failure between updates. A compact representation is useful only while its indices remain consistent.

Data Layout Can Matter More Than Container Choice

An array of structs stores each record's fields together. A structure of arrays stores each field in a separate sequence. If an algorithm repeatedly reads only positions from a large record containing strings and metadata, separate position storage can reduce memory traffic. If it frequently processes complete records, an array of structs may offer better locality and simpler ownership.

Neither layout is universally faster. A structure of arrays introduces a shared-length invariant and makes insertion or removal update several vectors. It can improve vectorization and cache use for numeric kernels while complicating general application code. Measure the actual access pattern and include update cost, not just one read loop.

Large enum variants can similarly dominate memory footprint. Boxing a rare large variant makes common values smaller but adds an allocation for that variant. Small-string optimization and inline small-vector crates make related tradeoffs, but their exact layouts and thresholds are library-specific. Adopt them when measured allocation patterns justify the extra representation complexity.

Reference counting is another layout choice. Arc<T> shares one allocation across owners and uses atomic reference counts. Rc<T> is thread-confined and avoids that atomic sharing protocol. Neither prevents mutation races by itself; mutation needs an appropriate access strategy. Replacing copies with shared ownership can reduce memory while increasing contention or extending retention.

Destruction Is Part of the Workload

Dropping a collection drops its elements. For a large graph of owned values, shutdown or eviction can become a latency event. Recursive ownership can also produce deep destructor call chains. A throughput benchmark that excludes destruction may hide a significant production cost.

Buffer reuse can defer allocation and destruction, but it can also retain memory and sensitive contents longer than expected. Clearing a vector changes logical length and drops elements; it does not promise to overwrite every byte of spare storage. If erasure is a requirement, use a mechanism designed to provide that guarantee rather than relying on ordinary clear or drop behavior.

When a service periodically replaces a large snapshot, consider which thread performs the final drop. The last Arc owner determines where destruction happens. A seemingly cheap reference-count decrement can become expensive when it releases the entire object graph. Ownership placement therefore affects tail latency even when steady-state reads are efficient.

Training Encounters and Debriefs

Exercise 1. A Vec has enough capacity for one more element. Can a stored raw pointer into it be used after any arbitrary mutation?

Solution. No. Avoiding reallocation addresses only one hazard. Removal, reordering, element replacement, aliasing, and destruction can invalidate the pointer's intended use. A raw pointer requires a complete validity and access argument, not just a capacity check. Prefer a handle abstraction when identity is the actual requirement.

Exercise 2. Why might interning make an unbounded server use more memory over time even though repeated strings become cheaper?

Solution. The interner retains every distinct string. Repetition savings do not bound the number of unique inputs. Add a bounded vocabulary, eviction-aware design, per-request lifetime, or another retention policy. Compact symbols are not a memory limit.

Exercise 3. A benchmark replaces HashMap with BTreeMap and becomes slower for lookup but faster overall. Is that contradictory?

Solution. No. The complete workload may need sorted output or range scans, avoiding a separate sorting pass and temporary storage. Cache behavior, key costs, and update patterns can also differ. Evaluate total work and required semantics rather than one operation's expected complexity.

Further Reading

- Standard library collections compares collection capabilities and complexity.
- Standard library Vec documents ownership, capacity, allocation, and pointer guarantees.
- Standard library HashMap documents lookup, ordering, hashing, and implementation-sensitive traversal costs.
- Standard library String specifies UTF-8 storage and byte-based length and capacity.

The Three Index Hydra

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design an in-memory station archive supporting lookup by numeric ID, alphabetical listing, renaming, deletion, and a bounded history of recent events. Choose a canonical record owner and any secondary indexes. Renaming must not leave the old alphabetical entry visible, and deletion must not make an identifier silently refer to a different station. Decide whether alphabetical output is maintained continuously or sorted on demand. Explain how updates preserve consistency if preparation fails, and describe what happens to externally retained snapshots or shared payloads after deletion. Your deliverable is a representation sketch, operation walkthroughs, and a benchmark plan with realistic query proportions. No collection wins merely because one operation has attractive asymptotic complexity.

Encounter Constraints

- State whether IDs can be reused and how stale identity is detected.

• Bound event history in a way that accounts for payload bytes.

• Do not use interior mutation to change an ordering key while it remains indexed.

Hint one / Hint two / Reveal debrief

Side Quest

Add an export snapshot that readers can retain while stations change. Compare cloning owned records, sharing immutable payloads, and preserving an entire historical index. Measure peak live memory when one slow reader holds a snapshot across many updates. Explain how the representation that minimizes immediate copying can nevertheless retain the most memory in that workload. Include the lifetime of the historical index.

Capability unlocked Representation Strategist

Choose Your Exit

Enter the measurement arena / Level 35
Test whether your representation improves the complete workload.

Explore immutable worlds / Level 38
Give historical reads an explicit snapshot and validation contract.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 35

The Benchmark Crown

Ferrum offers a crown to whoever makes its message router fastest. One contestant replaces dynamic handlers with a large generic dispatch tree and reports a dramatic win. Another measures the full service and sees longer startup, larger binaries, and uneven latency. Neither result is automatically wrong. The arena rewards a reproducible experiment that explains which cost moved, who pays it, and which workload the conclusion actually describes.

Mission Design and report a fair performance comparison across runtime cost, compilation, code size, and workload behavior.

Suggested loadout Level 10, Level 11, Level 34. These are guidance, not locked gates.

Mastery Targets

- Keep input generation and observable output policies equivalent across compared designs.
- Measure a representative router workload and distinguish it from a dispatch microbenchmark.
- Report uncertainty and tradeoffs without inventing a universal winner.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 35

Performance Is a Measured Property of a Design

Rust gives a programmer unusually direct control over ownership, allocation, representation, and dispatch. That control can remove unnecessary work, but it does not make every Rust program fast or every low level implementation better. Performance belongs to a workload running in an environment. A claim such as faster needs an operation, an input distribution, a build configuration, and a measurement boundary before it becomes useful engineering information.

The strongest optimizations often change a design constraint. Borrowing a view can remove a copy when the consumer does not need ownership. Batching can reduce synchronization when immediate delivery is unnecessary. Immutable snapshots can shorten read lock duration when readers accept versioned state. These improvements arise from understanding what the application requires. Rewriting an iterator as a loop is much less likely to matter if both compile to similar machine code and allocation dominates the request.

This level develops a measurement process that connects observed cost to ownership decisions. It treats benchmarks as experiments with failure modes, examines allocation and layout through concrete examples, and explains when compiler configuration or dispatch choices deserve attention. No timing result is asserted for the examples: their purpose is to make hypotheses testable on the reader's actual target.

Define the Question Before the Timer

A throughput benchmark asks how much work completes per unit time under a specified load. A latency benchmark asks how long individual operations take, often with attention to the tail of the distribution. A memory benchmark may care about peak retained bytes rather than total allocations. An optimization can improve one metric while worsening another. Batching often raises throughput and increases waiting time; caching can reduce CPU work while retaining more memory.

Choose a boundary that matches the decision. If considering a parser implementation, isolate parsing enough to understand its cost, then validate the effect in an end to end path. If considering a service admission policy, a tiny parser microbenchmark cannot reveal queueing or overload behavior. Measure both the component and the product behavior when the connection between them is uncertain.

Input distribution matters as much as size. A branch predictor can perform well on uniform data and poorly on a mixed production distribution. A hash table with short repeated keys behaves differently from one with long unique keys. A parser receiving mostly valid records may take a different path from one defending against malformed input.

Use representative distributions and include adversarial boundaries that the service must handle acceptably.

State what remains outside the experiment. A warm in memory benchmark excludes cold filesystem behavior. A single threaded loop excludes contention. A steady state run excludes startup and code loading. These limitations are not flaws if they match the question, but they must not disappear when reporting the result. A narrow experiment supports a narrow conclusion.

Build Configuration Changes the Program

Debug builds prioritize a different development experience from optimized builds. Bounds check elimination, inlining, constant propagation, and other transformations can change the relative costs of two implementations. Benchmark the configuration that will actually ship, including relevant target settings and feature flags. Running an unoptimized binary and extrapolating its ratios to a release build is unreliable.

Cargo profiles expose settings such as optimization level, link time optimization, code generation units, debug information, and panic behavior. Changing several at once makes it difficult to explain an observed difference. Keep a baseline, change one hypothesis at a time, and record the toolchain and target. A configuration that improves one binary can worsen build time, code size, or another workload.

Useful inspection commands include the following shell commands, shown as a workflow rather than an executable Rust listing.

Listing 35.1 Sketch

```
cargo build --release
cargo test --release
cargo rustc --release -- --emit=asm
```

Select the intended package and target explicitly in a workspace. Emitted assembly can explain whether a proposed abstraction difference survives optimization, but reading assembly is not a substitute for workload measurement. A shorter instruction sequence can still be slower because of dependencies, cache behavior, or branch prediction. Inspection is evidence for a causal explanation, not a standalone score.

Target specific CPU features can improve one deployment and reduce portability. Do not benchmark with a local machine specific target setting and silently ship the same binary to machines that lack the required features. Performance configuration belongs in the deployment contract just as ABI and feature requirements do.

A Small Comparative Harness

Timing one invocation is usually too noisy for a tiny operation. Repeating a workload amortizes timer overhead, but repetition can also create unrealistically warm caches or allow the compiler to simplify known input. `black_box` is a best effort tool for discouraging certain optimizations; it is not a semantic barrier on which correctness or security may rely.

The following complete harness compares equivalent iterator and loop implementations. It checks semantic agreement before timing, alternates measurement order, and reports medians across several samples.

Listing 35.2 Rust example

```
use std::hint::black_box;
use std::time::{Duration, Instant};

fn iterator_sum(values: &[u64]) -> u64 {
    values.iter().copied()
        .filter(|value| value & 1 == 0)
        .fold(0, u64::wrapping_add)
}

fn loop_sum(values: &[u64]) -> u64 {
    let mut sum = 0_u64;
    for &value in values {
        if value & 1 == 0 {
            sum = sum.wrapping_add(value);
        }
    }
    sum
}

fn measure(operation: fn(&[u64]) -> u64, input: &[u64]) -> Duration {
    let start = Instant::now();
    let mut result = 0_u64;
    for _ in 0..64 {
        result ^= operation(black_box(input));
        black_box(result);
    }
    start.elapsed()
```

Listing 35.2 continued

```
}

fn median(mut samples: Vec<Duration>) -> Duration {
    samples.sort();
    samples[samples.len() / 2]
}

fn main() {
    let input: Vec<u64> = (0..2_000).collect();
    assert_eq!(iterator_sum(&input), loop_sum(&input));
    for _ in 0..4 {
        black_box(iterator_sum(black_box(&input)));
        black_box(loop_sum(black_box(&input)));
    }
    let mut iterator = Vec::new();
    let mut explicit = Vec::new();
    for round in 0..11 {
        if round & 1 == 0 {
            iterator.push(measure(iterator_sum, &input));
            explicit.push(measure(loop_sum, &input));
        } else {
            explicit.push(measure(loop_sum, &input));
            iterator.push(measure(iterator_sum, &input));
        }
    }
    println!("iterator median {:?}", median(iterator));
    println!("loop median {:?}", median(explicit));
}
```

This harness is intentionally modest. It does not calculate confidence intervals, control CPU frequency, isolate competing processes, or model multiple input distributions. Repeating it can still reveal whether a claimed large difference exists at all. For close results, use a more rigorous benchmarking setup and inspect variation rather than treating the smaller printed number as a durable winner.

Alternating order reduces one simple bias from warming or environmental drift. It does not eliminate all bias. The function pointer measurement boundary can also affect inlining relative to a real caller. If the production call is statically dispatched and inlined, a benchmark through a function pointer may test a different optimization context. The experiment must eventually reproduce the relevant call shape.

Allocation Is Both Work and Lifetime

An allocation has an immediate cost, but retained ownership can matter more than allocation frequency. A small view stored inside an `Arc` backed object may keep a large buffer alive. Reusing a buffer can reduce allocator traffic while preserving a large peak capacity indefinitely. Counting allocations without measuring retained bytes can therefore reward a design that increases steady state memory.

Trace ownership through the hot path. Does each layer clone a `String` because it needs ownership, or because the API requires it unnecessarily? Can a parser return borrowed fields while the input remains valid? Would an owned compact result let the input buffer be released earlier? Borrowing avoids copies but can extend the owner's lifetime; ownership can cost a copy while reducing long term retention. The right choice follows the lifecycle of the data.

Serialization often exposes a simple reuse opportunity. The following library function writes records into a caller owned string, retaining its capacity for later calls.

Listing 35.3 Rust example

```
use std::fmt::Write;

struct Record<'a> {
    id: u64,
    name: &'a str,
}

fn render(records: &[Record<'_>], output: &mut String) {
    output.clear();
    for record in records {
        writeln!(output, "{}\t{}", record.id, record.name)
            .expect("writing to String cannot fail");
    }
}

fn main() {
    let records = [
        Record { id: 1, name: "alpha" },
        Record { id: 2, name: "beta" },
    ];
    let mut output = String::new();
    render(&records, &mut output);
    assert_eq!(output, "1\talpha\n2\tbeta\n");
```

Listing 35.3 continued

```
    let capacity = output.capacity();
    render(&records, &mut output);
    assert_eq!(output.capacity(), capacity);
}
```

The API communicates reuse directly through `&mut String`. It also makes the caller responsible for deciding when retained capacity should be reduced or the buffer dropped. If one exceptional response grows the buffer to hundreds of megabytes, reusing it forever may be undesirable. A size based retention policy can preserve ordinary reuse while releasing exceptional capacity.

Preallocation should follow a credible estimate. Reserving a maximum possible size for every small request can waste memory and increase pressure on caches or the allocator. Repeated growth may be acceptable if inputs are small or unpredictable. Measure the distribution and total memory budget rather than assuming that one reserve call is always an improvement.

Data Layout Determines What Traversal Touches

Cache behavior depends on which bytes a loop needs and how those bytes are arranged. An array of large records can force a scan over widely separated hot fields even when the loop ignores most payload data. Separating frequently scanned metadata from cold payloads can improve locality. The transformation changes representation and may complicate updates, so it should serve a measured access pattern.

The following library code shows two representations of the same conceptual records.

Listing 35.4 Rust example

```
struct Row {
    score: u64,
    payload: [u8; 128],
}

struct Columns {
    scores: Vec<u64>,
    payloads: Vec<[u8; 128]>,
}
```

Listing 35.4 continued

```
fn sum_rows(rows: &[Row]) -> u64 {
    rows.iter().fold(0, |sum, row| sum.wrapping_add(row.score))
}

fn sum_columns(columns: &Columns) -> u64 {
    columns.scores.iter().copied().fold(0, u64::wrapping_add)
}

fn split_rows(rows: Vec<Row>) -> Columns {
    let mut scores = Vec::with_capacity(rows.len());
    let mut payloads = Vec::with_capacity(rows.len());
    for row in rows {
        scores.push(row.score);
        payloads.push(row.payload);
    }
    Columns { scores, payloads }
}
```

The column representation makes score scans contiguous, but now every insertion and removal must preserve equal lengths and matching indices. An owning type should enforce that invariant rather than exposing both vectors for arbitrary mutation. If most operations need the score and payload together, the original row representation may be better. Layout optimization is a choice about access patterns, not a universal preference for one arrangement.

False sharing is another layout effect. Independent counters updated by different threads can contend when they occupy one cache line. Padding or sharding may help, but the relevant cache line behavior is target dependent and padding increases memory use. More importantly, sharing one global counter may be unnecessary if the application can combine per worker counts periodically. Removing an unnecessary synchronization requirement is often more effective than tuning its placement.

Dispatch Trades Runtime Work for Code Shape

Static dispatch can let the compiler specialize and inline operations for concrete types. It can also create many versions of a generic function, increasing code size and build work. Dynamic dispatch introduces an indirect call and may limit inlining, but it can reduce duplication and place a stable boundary around heterogeneous behavior. The effect depends on the operation's size and the surrounding optimization opportunities.

This complete library example isolates the two call shapes without claiming one will dominate every workload.

Listing 35.5 Rust example

```
trait Transform {
    fn apply(&self, value: u64) -> u64;
}

struct Multiply(u64);

impl Transform for Multiply {
    fn apply(&self, value: u64) -> u64 {
        value.wrapping_mul(self.0)
    }
}

fn sum_static<T: Transform>(transform: &T, values: &[u64]) -> u64 {
    values.iter().fold(0_u64, |sum, &value| {
        sum.wrapping_add(transform.apply(value))
    })
}

fn sum_dynamic(transform: &dyn Transform, values: &[u64]) -> u64 {
    values.iter().fold(0_u64, |sum, &value| {
        sum.wrapping_add(transform.apply(value))
    })
}
```

The transform is tiny, so call overhead and optimization visibility may be material. If `apply` instead performs a network operation, the indirect call is unlikely to dominate end to end latency. If the compiler can see the concrete dynamic value at the call site, it may optimize more than the source spelling suggests. Inspect the actual build rather than assuming that `dyn` guarantees a particular instruction sequence.

Avoid forcing inlining everywhere. An inline annotation can influence optimization, but excessive duplication can hurt instruction cache behavior and binary size. Large generic stacks can produce similar effects. A useful boundary is often to keep the hot inner loop concrete while erasing implementation types at a coarser service boundary. That is an architectural hypothesis to test, not a style rule.

Profiles Should Explain the Bottleneck

A CPU profile attributes sampled execution to code paths, but a slow service can spend most of its time waiting rather than consuming CPU. Wall clock tracing, lock contention measurements, queue depth, and dependency latency may be needed to explain the full path. A flat CPU profile can make a small local function look important simply because all the expensive waiting is absent from the sample.

Separate self time from inclusive time. A wrapper can appear expensive because it calls the actual bottleneck. Likewise, allocation cost may be spread across many call sites while one ownership choice causes all of them. Use the profile to form a causal hypothesis: repeated cloning of a request buffer, a contended lock around a tiny critical section, or a scan over cold fields. Then change the design and predict which measurements should move.

The maximum possible benefit is constrained by the fraction of total cost affected. If parsing accounts for one tenth of request time, making parsing twice as fast improves total time by much less than twofold. This simple accounting prevents spending days on a visually interesting hot loop while a serialized dependency dominates the product metric. Recompute the bottleneck after a substantial change because the limiting component may shift.

Correctness Must Survive the Optimization

A benchmark only compares implementations usefully when they implement the required semantics. A handwritten parser that skips validation may be faster because it accepts a different language. A cache that omits invalidation may be faster because it returns stale data outside the contract. A lock free structure with an invalid reclamation scheme may appear fast until undefined behavior changes the experiment entirely.

Preserve an executable reference model where feasible. Compare optimized output against a simple implementation across representative and boundary inputs. Keep overflow, Unicode, error handling, and cancellation behavior in the contract. If the optimization intentionally relaxes a guarantee, publish that as a product or API decision and measure the new behavior honestly.

Unsafe code requires a soundness argument independent of its timing result. Passing a benchmark or a test suite does not establish pointer validity or aliasing correctness. Prefer a safe representation change when it captures most of the benefit. The best performance work often removes work or improves locality while keeping the compiler's strongest checks available.

Training Encounters

1. A microbenchmark shows a parser is twice as fast, but request latency barely changes. Give three explanations that should be investigated before concluding the benchmark is useless.

2. Reusing one response buffer removes most allocations but increases steady state memory after a large response. Explain the ownership cause and propose a retention policy.

3. A generic implementation beats a dynamic one in a function pointer benchmark. Explain why the call shape may invalidate the intended comparison and describe a better experiment.

Training Debriefs

1. Parsing may be a small fraction of total latency, the benchmark input distribution may differ from production, or the service may be limited by queueing or dependencies. The microbenchmark can still establish a local improvement. Measure the parser's contribution in the real path and verify that the optimized version is actually used under the same build configuration.

2. `clear` removes contents but retains capacity, so one exceptional allocation remains owned by the reusable buffer. Drop or shrink buffers above a measured retention threshold, use size classes, or return oversized buffers to a pool with its own byte budget. The policy should balance future reuse against memory pressure rather than minimize allocation count alone.

3. A function pointer can prevent the inlining and specialization that a concrete production caller allows, or affect the two implementations differently. Benchmark the real static call shape and the real dynamic

boundary, inspect optimized output, and include end to end measurements. Report code size and build cost if those are part of the design tradeoff.

Further Reading

- Cargo profiles documents optimization and code generation configuration.
- black_box specifies its best effort optimization barrier role.
- Rust compiler command options describes emitted artifacts for inspection.

The Tournament of Unequal Rules

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Compare a router using erased handlers with a statically specialized alternative across a small and a large set of handler types. Define identical work, input distributions, error behavior, and output checks. Plan one experiment that isolates dispatch overhead and another that measures the full route, including realistic batching and payload processing. Record build configuration, binary size, compile cost, allocation behavior, and latency or throughput as appropriate. A proposed benchmark constructs input inside one timed loop, reuses it outside the other, and never consumes the computed result. Repair that protocol before interpreting any numbers. Victory is a defensible experiment and explanation; no particular timing improvement is required.

Encounter Constraints

- Do not assign arbitrary speedup numbers to unrun experiments.
- Avoid changing required semantics merely to simplify one contestant.
- Treat profiling evidence separately from a claim of causation.

Hint one / Hint two / Reveal debrief

Side Quest

Give the router a memory ceiling and a tail-latency objective. A batching optimization improves throughput but retains more payloads and delays small requests. Design a load experiment that reveals the tradeoff near saturation. Include admission rate, queue size, completed throughput, and latency distribution, so an overloaded system cannot appear fast merely by failing or dropping most work. Report rejected work alongside successful completions.

Capability unlocked Measurement Skeptic

Choose Your Exit

Challenge the correctness oracle / Level 36
Ensure the faster implementation still satisfies its behavioral contract.

Return to the dispatch forge / Level 10
Revisit where type erasure belongs after identifying the real cost.

Mark your quest log / Return to the map

ZONE 4 / THE UNSAFE DEPTHS / LEVEL 36

The Fault Vault

The final vault of the Unsafe Depths contains a pipeline that survives ordinary traffic and loses work during a rare failure. Its tests mostly repeat successful jobs and sleep until shutdown seems finished. Ferrum needs evidence that can challenge the contract deliberately. This level asks you to invent hostile but controlled histories, preserve reproducible traces, and distinguish a useful test result from a proof the test cannot supply.

Mission Build a validation strategy around explicit acceptance, completion, and failure-accounting invariants.

Suggested loadout Level 8, Level 20, Level 21, Level 31. These are guidance, not locked gates.

Mastery Targets

- Define an independent state model for job ownership and accepted-work accounting.
- Control at least one failure interleaving without relying on sleeps.
- Explain which properties need compile checks, runtime tests, dynamic tools, or protocol reasoning.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 36

Validation Should Attack the Contract

Rust's compiler establishes important properties before execution, but it does not establish that a parser accepts the intended language, a retry is idempotent, a shutdown completes, or an unsafe abstraction is sound. Validation works best when each technique attacks a specific remaining uncertainty. A unit test checks a concrete observation. A property test explores a relationship over many inputs. A model checks a simplified state machine. An interpreter or sanitizer detects certain invalid executions. None of these should be asked to prove more than its method can support.

Begin by writing the contract in observable terms. A bounded queue never contains more than its capacity, preserves accepted item order, and returns rejected items unchanged. A transaction either commits its intended state or leaves the previous state available. A parser either returns a valid representation or a defined error without losing bytes outside its consumption contract. Such statements generate useful tests because they describe behavior independent of the implementation's private steps.

Tests that merely repeat the algorithm can share its mistakes. If production computes an offset with a complicated expression and the test computes the expected value with the same expression, agreement provides little evidence. Prefer a simpler reference model, independently constructed examples, or algebraic relationships that constrain the result from another direction. Validation should challenge the implementation's assumptions rather than narrate its source code back to itself.

Choose Tests From Invariants and Boundaries

Boundary values are where representation choices become observable. Empty collections, zero capacity, maximum lengths, integer overflow, missing delimiters, Unicode boundaries, and truncated frames often exercise paths absent from ordinary examples. Derive these cases from the type and protocol. A queue with capacity zero is not simply a smaller ordinary queue if the implementation performs modulo by capacity before checking fullness.

Also test transitions, not only states. A queue may work when first filled but fail after its indices wrap several times. A service may reject correctly after shutdown but race while closing admission. A transaction may restore state on one error path while leaking a reservation on another. The sequence that reaches a state can matter as much as the state's current field values.

Classify failures so that tests assert the intended recovery behavior. Invalid input, resource exhaustion, dependency uncertainty, and internal invariant failure can require different responses. A test should distinguish rejection before admission, failure of accepted work, and an unknown

external outcome. That distinction informs which operations a caller can safely retry.

Keep tests deterministic when the property does not require environmental variation. Determinism makes a failure reproducible and reduces the chance that a flaky test is ignored. Randomized exploration is valuable, but retain the seed or minimized input that produced a failure. A discovered bug should become a stable regression case after the broader search has explained it.

Compare a Ring Buffer With a Simpler Model

A reference model can be slower and more obvious than the optimized structure. Its purpose is to describe behavior through a different representation. The following complete program implements a small safe ring buffer and compares operation sequences against `VecDeque`. It exercises empty, full, wraparound, and rejected insertion behavior at several capacities.

Listing 36.1 Rust example

```
use std::collections::VecDeque;

struct Ring<T, const N: usize> {
    slots: [Option<T>; N],
    head: usize,
    len: usize,
}

impl<T, const N: usize> Ring<T, N> {
    fn new() -> Self {
        Self {
            slots: std::array::from_fn(|_| None),
            head: 0,
            len: 0,
        }
    }

    fn push(&mut self, value: T) -> Result<(), T> {
        if self.len == N {
            return Err(value);
        }
        let tail = (self.head + self.len) % N;
        self.slots[tail] = Some(value);
        self.len += 1;
        Ok(())
```

Listing 36.1 continued

```
    }

    fn pop(&mut self) -> Option<T> {
        if self.len == 0 {
            return None;
        }
        let value = self.slots[self.head].take();
        self.head = (self.head + 1) % N;
        self.len -= 1;
        value
    }
}

fn exercise<const N: usize>() {
    let mut ring = Ring::<u32, N>::new();
    let mut model = VecDeque::new();
    let mut state = 0x1234_5678_u64;
    for _ in 0..5_000 {
        state = state.wrapping_mul(6_364_136_223_846_793_005)
            .wrapping_add(1);
        let value = (state >> 32) as u32;
        if value & 1 == 0 {
            let actual = ring.push(value);
            let expected = if model.len() == N {
                Err(value)
            } else {
                model.push_back(value);
                Ok(())
            };
            assert_eq!(actual, expected);
        } else {
            assert_eq!(ring.pop(), model.pop_front());
        }
        assert_eq!(ring.len, model.len());
    }
    while let Some(expected) = model.pop_front() {
        assert_eq!(ring.pop(), Some(expected));
    }
    assert_eq!(ring.pop(), None);
}

fn main() {
    exercise::<0>();
    exercise::<1>();
    exercise::<2>();
    exercise::<7>();
}
```

The zero capacity case is safe because `push` returns before modulo and `pop` returns before advancing the head. The model checks ownership of rejected values as well as queue order. The deterministic sequence generator provides varied histories while remaining reproducible. It is not a substitute for exhaustive exploration, and a different generator may discover different patterns.

This test still has limits. It uses `u32`, so it does not check destruction of nontrivial elements. It does not explore every capacity or every operation sequence. It also inspects the private length field rather than a public method because the example keeps the contract small. A production test suite should add drop counting, explicit boundary histories, and public API tests where compatibility matters.

The ring buffer is implemented safely, so many memory errors are excluded by construction. If a later optimization replaces `Option<T>` with `MaybeUninit<T>`, the same behavioral model remains useful, but new obligations appear: exactly the initialized elements must be read and dropped, panic paths must preserve accounting, and zero capacity must remain valid. Reusing the model preserves semantic evidence while adding tools targeted at the unsafe representation.

Property Testing Needs Meaningful Properties

Useful properties often relate several operations. Encoding then decoding should reproduce a value within the format's domain. Sorting should preserve the multiset and produce an ordered result. Splitting and recombining a valid byte stream at arbitrary boundaries should not change the parsed messages. A queue's outputs should equal its accepted inputs in the promised order. These properties constrain behavior more strongly than checking that a function does not panic.

Round trip tests can still hide paired bugs. An encoder and decoder that share the same wrong field order may agree with each other while violating an external format. Combine round trips with independently specified fixtures or differential checks against an authoritative implementation. For a public protocol, bytes on the wire are part of the contract, not merely an implementation detail between two functions in one crate.

Generate valid and invalid inputs deliberately. Uniform arbitrary bytes can spend most of its time failing at the first header check, leaving deep parser states unexplored. A structured generator can reach valid nested records and then mutate one length or delimiter. Conversely, generating only valid values misses rejection and resource limit behavior. The generator's distribution is part of the test's coverage strategy.

Shrinking or minimizing a failing input makes the underlying invariant easier to see. Preserve enough context to reproduce stateful failures: the initial state, operation sequence, scheduling decisions where relevant, and toolchain configuration. A five operation queue history is more actionable than an unexplained failure after a million random steps. The minimized case should become a named regression test describing the violated behavior.

Failure Injection Tests the Unhappy Transition

Ordinary success tests rarely visit the exact point at which an invariant is temporarily fragile. Inject failures between stages to verify that ownership based cleanup restores the promised state. Use an explicit error injection interface in tests or a small dependency that can return controlled failures. Avoid production switches that accidentally expose test behavior to users unless the system intentionally supports fault injection.

This complete example uses a transaction guard to restore a vector's original length unless committed. It tests failures after different numbers of appended values.

Listing 36.2 Rust example

```
struct AppendTransaction<'a> {
    target: &'a mut Vec<u32>,
    original_len: usize,
    committed: bool,
}

impl<'a> AppendTransaction<'a> {
    fn new(target: &'a mut Vec<u32>) -> Self {
        let original_len = target.len();
        Self { target, original_len, committed: false }
    }

    fn commit(mut self) {
        self.committed = true;
```

Listing 36.2 continued

```
    }
}

impl Drop for AppendTransaction<'_> {
    fn drop(&mut self) {
        if !self.committed {
            self.target.truncate(self.original_len);
        }
    }
}

fn append(
    target: &mut Vec<u32>,
    values: &[u32],
    fail_after: Option<usize>,
) -> Result<(), &'static str> {
    let transaction = AppendTransaction::new(target);
    for (index, &value) in values.iter().enumerate() {
        if fail_after == Some(index) {
            return Err("injected failure");
        }
        transaction.target.push(value);
    }
    transaction.commit();
    Ok(())
}

fn main() {
    for stage in 0..3 {
        let mut target = vec![9];
        assert!(append(&mut target, &[1, 2, 3], Some(stage)).is_err());
        assert_eq!(target, vec![9]);
    }
    let mut target = vec![9];
    append(&mut target, &[1, 2, 3], None).unwrap();
    assert_eq!(target, vec![9, 1, 2, 3]);
}
```

The rollback policy is valid because the transaction only appends. If it overwrote existing elements, truncation would not restore their old values. A test that merely checks length would then miss damaged contents. This illustrates why the invariant must precede the test: cleanup must restore the actual promised state, not one convenient metric.

Panic injection adds another dimension when unwinding is part of the supported configuration. A guard can restore local state during unwinding, but process abort does not run ordinary unwind cleanup. Foreign

exceptions and ABI boundaries add their own constraints. Test the failure behavior that the application actually promises, and avoid presenting unwind tests as evidence for an aborting deployment's cleanup behavior.

Compile Time Rejections Are API Tests

Some of Rust's best API guarantees are that invalid programs cannot be written through the safe interface. A typestate transition, borrow relationship, or trait bound can be tested with an example that is expected not to compile. Rustdoc supports compile failure examples, and dedicated test harnesses can check diagnostics more precisely when needed.

The following complete example should fail because a read only view does not expose the mutation method.

Listing 36.3 Rejected example

```
struct ReadOnly;
struct Writable;

struct Store<State> {
    values: Vec<u32>,
    state: std::marker::PhantomData<State>,
}

impl Store<Writable> {
    fn append(&mut self, value: u32) {
        self.values.push(value);
    }
}

fn main() {
    let mut store = Store::<ReadOnly> {
        values: vec![],
        state: std::marker::PhantomData,
    };
    store.append(3);
}
```

A compile failure test can pass for the wrong reason. A missing import or syntax error may reject the program before it reaches the intended bound. Keep such examples complete and minimal, and inspect the diagnostic during development. If a precise diagnostic is part of the assertion, account for compiler version changes so that harmless wording updates do not overwhelm meaningful failures.

Also test positive neighboring cases. A type system restriction that rejects every program is not a useful API. Show that writable stores can append, borrowed views remain usable for their valid interval, and sendable service operations satisfy the intended scheduler bound. Positive and negative examples together describe the boundary that the abstraction is meant to enforce.

Fuzzing Explores Inputs Beyond Examples

Coverage guided fuzzing uses execution feedback to search for inputs that reach new behavior. It is particularly useful for parsers, decoders, and stateful operation sequences. A fuzz target should be small enough to execute many times, deterministic enough to reproduce failures, and strict about resource limits so that arbitrary lengths cannot consume unbounded memory or time.

Do more than call the parser and ignore the result. On success, validate invariants of the returned representation. If the format supports reencoding, compare a canonicalized result. For incremental parsing, feed the same bytes under several chunk boundaries and compare outcomes. For a data structure, decode bytes into operation sequences and compare against a model. These targets turn silent logical corruption into observable failures.

Keep malformed input handling separate from infrastructure failure. An allocation limit exceeded by a declared length should produce the format's defined error rather than crashing the harness. A timeout caused by intentionally huge work may indicate an algorithmic denial of service risk or simply a missing test bound; investigate the contract before discarding it. Fuzzing's value includes resource behavior, not only memory faults.

Every discovered failure needs triage. Determine whether it is a harness bug, an unsupported input assumption, a logic error, or an unsafe execution. Minimize the input and retain it in a regression corpus. Fixing one crash without adding its invariant to the validation strategy leaves the same category of defect ready to reappear elsewhere.

Miri Detects Executed Undefined Behavior

Miri interprets Rust programs and can detect many classes of undefined behavior on the executions it explores, including invalid memory access and certain aliasing violations. It is especially valuable for unsafe abstractions whose ordinary tests may appear to work on real hardware. A pointer bug can remain invisible when an allocator happens to retain old bytes; an interpreter can identify the invalid operation instead of waiting for a visible crash.

Miri is not a proof of soundness. It observes tested executions under its supported model and capabilities. Some platform interactions are unavailable or require substitutes, and Rust's full unsafe semantics continue to have areas under development. Passing Miri does not mean every possible safe client of an abstraction has been considered. The written safety argument remains necessary.

Use small focused tests that exercise initialization, destruction, reborrowing, zero sized types, empty collections, and unusual operation histories. A generic container tested only with integers may miss double drops or invalid reads of uninitialized elements. Types with drop counters and types with stricter alignment can expose different obligations. Run the tool on the tests that stress the representation rather than assuming one happy path is representative.

The typical command shape is shown below. The required toolchain and component availability should be checked against the project's environment.

Listing 36.4 Sketch

```
rustup component add miri --toolchain nightly
cargo +nightly miri test
```

Do not automatically rewrite correct code merely to silence an unexplained interpreter report. Understand whether the report identifies a real violation, an unsupported operation, or a model limitation. Conversely, do not dismiss an inconvenient report because optimized native execution looks fine. Native success is weak evidence when the alleged defect concerns undefined behavior.

Concurrency Tests Need Schedule Reasoning

Sleeping for a few milliseconds and hoping another thread reached a state is a fragile test protocol. Use barriers, channels, or controlled hooks to establish the intended interleaving. A test for closing admission should deliberately pause a request around its registration boundary and then run shutdown, rather than rely on a race occurring by chance.

Repeated stress tests can discover real bugs but rarely establish their absence. Schedule exploration tools can systematically vary supported synchronization interleavings within a bounded model. Such tools require using their modeled primitives and have limits on the behaviors explored. The model must include the synchronization edges on which the algorithm's proof depends; replacing only one atomic in a larger protocol is not enough.

Memory ordering bugs deserve particular caution. A test on one architecture can pass because that machine's common behavior is stronger than the algorithm's required portable guarantee. Increasing stress iterations does not repair a missing happens before relationship. Combine a written ordering argument with appropriate model exploration and tests for the state machine's visible behavior.

Progress properties also need tests and analysis distinct from memory safety. A queue can return correct values in all completed operations while one participant starves indefinitely. Use bounded scenarios to detect obvious deadlocks and retain timeouts as test escape hatches, but do not confuse the timeout threshold with a proof of fairness. State the scheduling assumptions under which progress is expected.

A Validation Plan Should Remain Economical

Not every change needs every tool. A private wording change does not need fuzzing, and a small safe refactor may need only existing behavior tests and a targeted new case. An unsafe storage rewrite, parser format change, or admission protocol change creates different uncertainties and deserves stronger evidence. Match effort to the contract that could be broken.

Organize evidence so that failures are interpretable. Fast deterministic tests should cover ordinary invariants and regressions. Slower randomized or interpreter runs can target representation and state space exploration. Integration tests should verify boundaries with real dependencies where their behavior matters. Avoid several suites that all assert the same implementation detail while leaving failure and cancellation paths unexamined.

When a bug is found, ask which assumption escaped validation. A missing boundary case suggests a generator or invariant gap. A flaky timing test suggests an uncontrolled interleaving. A Miri finding suggests a safety argument that omitted a client behavior or lifetime transition. Improving the validation model is more durable than adding one opaque test with the original incident's incidental constants.

Training Encounters

1. A ring buffer passes thousands of integer operation sequences. It is then rewritten with uninitialized storage. Identify three new classes of evidence needed beyond rerunning the same sequences.

2. A parser's encoder and decoder pass round trip tests but disagree with another implementation. Explain how paired bugs can survive and propose complementary validation.

3. A concurrency test uses a sleep to make shutdown race with admission and fails once a month. Redesign the test and explain what uncertainty remains after it becomes deterministic.

Training Debriefs

1. Test destruction accounting with nontrivial elements, initialization boundaries under errors or panics, and validity under a tool such as Miri. Include zero sized and aligned element types where relevant. Maintain a written proof of which slots are initialized and who drops each element. Behavioral agreement with a model remains valuable but does not by itself prove the unsafe representation sound.

2. The two functions can share the same wrong field order, length rule, or normalization assumption. Add independent byte fixtures from the

specification, differential tests against an authoritative implementation, and properties that inspect the returned representation. Round trips constrain internal agreement; interoperability requires agreement with the external contract.

3. Add a controlled hook or barrier around the admission registration point, pause the request, run shutdown to the relevant phase, and release the request. Assert the defined acceptance outcome and final active count. This tests one deliberate interleaving reproducibly. Other schedules and portable memory ordering still need reasoning or broader model exploration.

Further Reading

- Rustdoc documentation tests describes executable and compile failure examples.
- Miri documents supported checks, execution limits, and usage.
- Rust Fuzz Book explains fuzz target construction and corpus workflows.
- Loom describes modeled concurrency exploration and its assumptions.

The Missing Job

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

A bounded service accepts jobs, transfers them to workers, and collects results. Introduce a controlled fault after one job is accepted but before its result reaches the collector. Count admission rejections separately. Classify accepted jobs as completed, failed, abandoned, or uncertain if external effects are possible. Build a small reference model and generate operation sequences that compare implementation accounting against it. Add synchronization hooks so a test can pause a worker at the precise fault boundary and trigger shutdown. Record the sequence and seed needed to reproduce a failure. Your evidence plan must also include a compile-time API property and identify why passing runtime tests does not prove an unsafe representation or a distributed delivery guarantee.

Encounter Constraints

- Acceptance must have one explicitly named point.

• The test controls ordering through synchronization rather than timing guesses.

• Do not count uncertain external effects as confirmed successful work.

Hint one / Hint two / Reveal debrief

Side Quest

Create three deliberate mutants: omit the final sender drop, release a byte-budget token too early, and reuse a deleted generation. Name the specific test that should catch each mutant. If a mutant survives, decide whether your observable contract excludes the behavior or whether your evidence is missing a relevant property. Do not treat mutation scores as a substitute for the contract itself.

Capability unlocked Failure Cartographer

Choose Your Exit

Enter the endgame / Level 37
Use adversarial reasoning while building a complete borrowing parser.

Audit one invariant chain / Level 42
Combine tool evidence with ownership, state, and boundary arguments.

Mark your quest log / Return to the map

ZONE 5 / THE ENDGAME / LEVEL 37

The Living Document

Ferrum's operators keep old search results open while editing the source archive. The original parser returns borrowed fields from a buffer that the editor wants to replace. One quick fix copies every field; another stores references inside the object that owns the editable string. Your endgame begins by deciding what an old result means after an edit. Storage, identity, and the version of the text must tell the same story.

Mission Design parsed document snapshots whose records remain meaningful while a newer document version is edited.

Suggested loadout Level 4, Level 8, Level 13, Level 14, Level 34. These are guidance, not locked gates.

Mastery Targets

- Keep each retained record tied to the exact source version from which its ranges were produced.
- Preserve UTF-8 range validity without storing movable internal self references.
- Explain the memory cost of retaining a tiny result from a large old document.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 37

Designing a Borrowing Parser

A parser is an unusually good test of whether an ownership design matches the problem. The input already contains most of the information that the output needs. Copying every field into a new allocation may be unnecessary, but returning references introduces a relationship between the input buffer and the parsed result. Streaming introduces another relationship: the buffer may need to change while callers still want to retain records. Neither relationship is a syntax problem. Each is an architectural choice about which component controls the storage.

This level starts with one configuration line at a time. Split a document into lines before calling the parser. Each line contains an ASCII identifier, an equals sign, and a UTF-8 value. Identifiers use letters, digits, and underscores and begin with a letter or underscore. Both fields must be nonempty after trimming Unicode whitespace.

The objective is to preserve a clear chain of responsibility. The caller owns the input. A successful parse borrows slices from that input. An error owns its diagnostic position and classification, so reporting the error does not require retaining an invalid parse. A later conversion can deliberately allocate when the record must outlive the input.

Start With the Output Contract

The lifetime on a parsed record describes where its text comes from. It does not make parsing faster by itself. It prevents the record from being used after the backing input becomes unavailable. If the caller passes a slice into a large file buffer, a tiny record can therefore keep that entire buffer logically borrowed. The allocation may be large even though the record has only two references.

An owned diagnostic is useful because failure often crosses more boundaries than success. An application may collect errors from many files, close the files, and render a report later. Offsets and an enum can survive that process. A diagnostic containing a borrowed substring would force the reporting layer to retain the original text or convert the diagnostic first.

The following complete program implements this contract. Offsets are byte positions. The trimming calculation works because the returned slices remain boundaries in the original UTF-8 string. Identifier validation uses bytes because the grammar defines identifiers in ASCII. Value text remains Unicode.

Listing 37.1 Rust example

```
#[derive(Debug, PartialEq, Eq)]
struct Record<'a> {
    name: &'a str,
    value: &'a str,
}
```

Listing 37.1 continued

```
#[derive(Debug, PartialEq, Eq)]
enum Kind {
    MissingEquals,
    EmptyName,
    InvalidName,
    EmptyValue,
}

#[derive(Debug, PartialEq, Eq)]
struct ParseError {
    offset: usize,
    kind: Kind,
}

fn parse_line(input: &str) -> Result<Record<'_>, ParseError> {
    let equals = input.find('=').ok_or(ParseError {
        offset: input.len(),
        kind: Kind::MissingEquals,
    })?;
    let left = &input[..equals];
    let right = &input[equals + 1..];
    let name = left.trim();
    let name_start = left.len() - left.trim_start().len();
    if name.is_empty() {
        return Err(ParseError {
            offset: name_start,
            kind: Kind::EmptyName,
        });
    }
    for (index, byte) in name.bytes().enumerate() {
        let valid = byte == b'_'
            || byte.is_ascii_alphabetic()
            || (index > 0 && byte.is_ascii_digit());
        if !valid {
            return Err(ParseError {
                offset: name_start + index,
                kind: Kind::InvalidName,
            });
        }
    }
    let value = right.trim();
    if value.is_empty() {
        return Err(ParseError {
            offset: equals + 1,
            kind: Kind::EmptyValue,
        });
    }
    Ok(Record { name, value })
}
```

Listing 37.1 continued

```
fn main() {
    let input = String::from("  retry_limit = 12  ");
    let record = parse_line(&input).unwrap();
    assert_eq!(record.name, "retry_limit");
    assert_eq!(record.value, "12");
    let bad = parse_line("9retries=12").unwrap_err();
    assert_eq!(bad.offset, 0);
    assert_eq!(bad.kind, Kind::InvalidName);
}
```

There is no allocation on the success path. That statement is a property of this implementation, not a universal property of parsers returning references. A parser can return borrowed slices while allocating temporary stacks, error contexts, or indexes. When performance matters, count those allocations separately from the output representation.

Notice the validation order. A missing equals sign is detected before identifier validation. The same malformed line could violate several rules, so the parser defines a precedence between errors. This precedence becomes observable behavior if callers test exact error kinds or highlight exact positions. Decide whether it is part of the public contract before downstream tools depend on it.

Explain the Borrow Rather Than Fighting It

Suppose a caller wants to append more data to the input after parsing. If the record will be used afterward, the append conflicts with its borrow. The important fact is that the append may relocate the string allocation. Even an append that happens to fit the current capacity is not an appropriate public explanation for allowing the operation: the API grants mutation of the whole string, and the type system does not encode a capacity-dependent exception for these slices.

The following complete example is intentionally rejected. Its failure is useful. The API has detected an incompatible storage policy before it becomes an intermittent invalid-reference bug.

Listing 37.2 Rejected example

```
fn main() {
    let mut input = String::from("mode=fast");
    let value = input.split_once('=').unwrap().1;
    input.push_str("\nnext=slow");
    println!("{value}");
}
```

There are several legitimate repairs, and they express different requirements. Finish using the record before mutating the buffer when processing is strictly immediate. Copy selected fields when a small amount of text must survive. Transfer the whole buffer into an owner when many fields will survive together. Store byte ranges instead of references when one object must own the buffer and describe regions inside it. These choices should be made from retention and mutation requirements, not from a wish to silence one diagnostic.

Borrowed output is especially effective in batch processing. Read an immutable document, parse many records, perform analysis, then release all of them together. Streaming processing is different. A decoder usually wants to reuse a mutable buffer indefinitely. Retaining borrowed records across the next read prevents that reuse. If a service needs arbitrary downstream retention, some component must own the retained bytes independently of the reusable input buffer.

Own the Buffer and Store Ranges

A structure containing both a string and references into that string is difficult to construct and maintain safely. Moving the structure is only one concern. Mutating or replacing the string can invalidate the references even if the outer structure never moves. Pinning alone does not prohibit every operation on an inner string. An ordinary owned buffer with validated ranges avoids this entire family of internal reference relationships.

The next example stores byte ranges. A record view is constructed only when the caller asks for one. Its lifetime is then a conventional borrow of the owning structure. The structure can be moved freely because the stored integers do not point into the old location of anything.

Listing 37.3 Rust example

```
use std::ops::Range;

struct StoredRecord {
    source: String,
    name: Range<usize>,
    value: Range<usize>,
}

impl StoredRecord {
    fn parse(source: String) -> Result<Self, String> {
        let Some(eq) = source.find('=') else {
            return Err(source);
        };
        let left = &source[..eq];
        let right = &source[eq + 1..];
        let name_start = left.len() - left.trim_start().len();
        let name_end = left.trim_end().len();
        let value_start = eq + 1
            + right.len() - right.trim_start().len();
        let value_end = eq + 1 + right.trim_end().len();
        if name_start >= name_end || value_start >= value_end {
            return Err(source);
        }
        Ok(Self {
            source,
            name: name_start..name_end,
            value: value_start..value_end,
        })
    }

    fn fields(&self) -> (&str, &str) {
        (&self.source[self.name.clone()],
         &self.source[self.value.clone()])
    }
}

fn main() {
    let record = StoredRecord::parse("  theme = dark ".into())
        .ok().unwrap();
    let moved = Box::new(record);
    assert_eq!(moved.fields(), ("theme", "dark"));
}
```

This example validates only the structural split and nonempty fields; it deliberately leaves identifier validation to an additional grammar step. That distinction matters when combining examples. A range representation does not automatically inherit every validation rule from an

earlier parser. A production constructor should run all required checks before making the value available.

The fields must remain private. A public method that mutates `source` without updating the ranges could make indexing panic or return the wrong region. Safe Rust would still prevent memory unsafety, but the abstraction would stop preserving its semantic promise. Encapsulation is therefore valuable even when no unsafe code exists.

Returning the original `String` on failure is also a policy. It lets the caller recover ownership without a second allocation. A richer API could return a pair containing the original source and a structured error. If the source is large and error reporting is small, the application may instead want to discard the source immediately. Error representation is part of the ownership architecture.

Streaming Requires a Progress Protocol

A streaming parser must distinguish malformed input from incomplete input. Receiving the bytes `name=` does not establish that the value is empty unless the enclosing protocol says the line has ended. Treating every temporary end of a network buffer as end of input produces errors that depend on packet boundaries.

Model the outcome as complete, incomplete, or invalid. An incomplete result may carry the minimum additional information needed, but it should not promise an exact number of bytes unless the format makes that knowable. For a length-prefixed frame the next required length may be exact; for a delimited UTF-8 line it may not be.

The interface below is an architectural sketch rather than an implementation. It returns a consumed count only on completion. That count gives the buffer owner a clear point at which it can remove or reuse bytes after the returned view is no longer needed.

Listing 37.4 Sketch

```
enum Decode<T> {
    Complete { value: T, consumed: usize },
    Incomplete,
    Invalid { offset: usize, reason: ErrorKind },
```

Listing 37.4 continued

```
}

fn decode(input: &[u8]) -> Decode<Frame<'_>>
```

Do not discard partial input merely because a future waiting for more bytes is cancelled. Buffer ownership and parse progress must live somewhere that survives the cancellation boundary. One approach places the buffer and cursor in the connection object while each read future borrows that object. Another transfers an owned incomplete frame through the state machine. The correct choice depends on whether cancellation means retry, connection closure, or abandoning the message.

UTF-8 decoding adds another boundary. A network chunk may end in the middle of a multibyte code point. If the protocol is textual, validate complete regions and preserve any incomplete suffix. If framing is byte based, finding the complete frame before UTF-8 validation often produces a cleaner separation. Byte offsets remain useful internally; converting them into line and column positions can happen when displaying diagnostics.

Escapes Change the Allocation Story

Consider extending values to support escape sequences. The input text `hello\nworld` does not contain the newline that the logical value should contain. A borrowed slice cannot represent the decoded result directly. You can always allocate, or use `Cow<'a, str>` so unescaped values borrow while escaped values own their decoded form.

That optimization adds a second mode to the output. Callers should work through `&str` where possible and request ownership only at a real retention boundary. Calling `into_owned` immediately after every parse defeats the allocation saving while retaining the API complexity. Conversely, if almost every field needs decoding and long-term retention, an owned representation can be simpler and just as appropriate.

Avoid pretending that `Cow` makes ownership disappear. It relocates the choice into a value. Its lifetime still matters when it contains the borrowed variant. Storing it in a long-lived application model may require conversion even if only one particular record borrowed. This is a useful

application of the lifetime reasoning developed earlier in the book: the type describes all values permitted by the interface, not only the branch observed in one test.

Verification Follows the Representation

For the borrowing parser, verify that accepted names satisfy the grammar and that the returned text equals the intended regions. For the range representation, verify that every range is ordered, inside the source, and on UTF-8 boundaries. Test whitespace-only sides, multiple equals signs, non-ASCII names, multibyte values, and no delimiter. Define whether the first or last equals sign separates the fields; this level uses the first.

A useful property test compares two implementations with different representations: a simple allocating reference parser and the borrowing implementation. Agreement across generated inputs is evidence against a shared boundary mistake, although it does not prove the grammar is correct. A second property can reconstruct an accepted canonical line from the parsed fields and parse it again. That tests normalization behavior, not original-byte preservation, so name the property accordingly.

Performance measurements should include retention. Parsing one million lines without allocation may look excellent until a single retained field forces a gigabyte input buffer to remain alive. Compare peak live memory under the real retention pattern. A selective copy can reduce total memory even though it increases allocation count. The ownership model determines which measurements matter.

Training Encounters

1. Extend `parse_line` with a comment rule in which a line whose first nonspace character is `#` produces no record. Design the return type and explain why a valid comment should not be a parse error.

2. Add quoted values with escapes. Identify exactly when the result must allocate and describe how an API using `Cow` changes the lifetime of a record retained beyond the input.

3. Design a decoder that receives a four-byte big-endian length followed by UTF-8 payload bytes. Specify incomplete, invalid, and complete outcomes,

including a maximum accepted length and checked arithmetic for total frame size.

Training Debriefs

1. `Result<Option<Record<'a>>, ParseError>` distinguishes accepted absence from malformed input. Check the comment prefix before looking for an equals sign. Keep blank-line policy explicit as well. Callers can filter `None` without treating routine document structure as a failure.

2. Borrow an unquoted or quoted region when its logical value is exactly an input slice. Allocate when decoding changes the bytes. A record containing `Cow<'a, str>` still cannot generally outlive `'a`; convert all retained fields to owned strings, or construct a separate owned record at the storage boundary. Escapes must be validated before accepting the record.

3. Before four bytes arrive, return incomplete. Decode the length without assuming host endianness. Reject values above the protocol limit before reserving memory. Use checked addition for header plus payload and return incomplete until that many bytes exist. Validate only the complete payload as UTF-8. Return the total consumed length while leaving any following frame untouched.

Further Reading

• The str type documents UTF-8 boundaries and string slicing operations used by the examples.

• Range describes the half-open range representation used for stored fields.

• Cow documents the borrowed and owned representation used when decoding can change text.

The Stale Span Doppelganger

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Extend the borrowing-parser design into an editor model with immutable source snapshots. A reader can retain a record from version A while the editor produces version B. Design owners and byte ranges so the old record still resolves to A and cannot silently index B. An API accepting a free-standing span must reject use with an unrelated snapshot or make that mismatch impossible through ownership. Define how invalid text produces owned diagnostics without exposing an invalid record. Show the lifetime and retention consequences of returning a temporary borrowed view versus an independently owned record handle. Your deliverable should include a complete ownership diagram and tests for edits that change byte lengths before a retained field.

Encounter Constraints

- Do not mutate a source allocation while retained views rely on its old contents.
- Avoid a structure storing ordinary references into its own movable owned String.
- Do not confuse Unicode character positions with byte offsets.

Hint one / Hint two / Reveal debrief

Side Quest

Add normalized search keys while preserving exact original spelling for display. For ASCII-only case normalization, decide whether keys borrow or own bytes and where they are cached. Then discuss what changes when normalization can change byte length. Keep normalized offsets separate from source spans, and measure the cost of caching keys for records that are never searched. Keep display text independent from lookup policy.

Capability unlocked Snapshot Pathfinder

Choose Your Exit

Make history transactional / Level 38
Add an explicit validation rule when snapshots become the basis of writes.

Revisit lending views / Level 13
Compare reusable producer storage with independently retained document records.

Mark your quest log / Return to the map

ZONE 5 / THE ENDGAME / LEVEL 38

The Transaction Within

The restored archive lets an operator preview a repair, try a risky adjustment, and discard only that adjustment. Ferrum calls these inner experiments savepoints. The existing store knows how to validate one transaction against its snapshot, but the operator now needs nested local work that does not leak into the published map. Your challenge is to distinguish committing a child proposal from publishing a new global version.

Mission Add nested local savepoints while preserving the outer transaction's snapshot and global commit validation.

Suggested loadout Level 14, Level 20, Level 34, Level 37. These are guidance, not locked gates.

Mastery Targets

- Child rollback leaves parent staged reads and writes unchanged.
- Child commit merges only into the parent proposal and never independently publishes store state.
- A conflicting outer commit rejects the complete nested proposal without partially publishing it.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 38

Building a Versioned In Memory Store

A versioned store connects ownership, shared data, errors, and concurrency in one small design. Readers want stable views. Writers want to make changes without exposing half a transaction. The program must decide when a snapshot becomes stale, what a conflict means, and whether retrying an operation is safe. Rust can help enforce exclusive access to a commit operation, but it cannot choose the consistency model for us.

We will build a process-local key-value store with immutable snapshots and optimistic transactions. The implementation is intentionally conservative. Every committed write advances one global version. A transaction can commit only when that version still matches the version from which it began. This rejects some nonconflicting writes, but makes the validation rule easy to state and inspect. It is a useful first design because its correctness argument fits in a few explicit steps.

The store is not durable. It has no write-ahead log, crash recovery, replication, or remote transaction protocol. A successful commit means that this in-memory object accepted a new state. Calling that operation durable would be an application bug even if every Rust reference were valid.

State the Consistency Contract First

A snapshot contains an immutable map and the version associated with that map. The store owns one `Arc` to the current map. Readers clone that `Arc`, so an old map remains available while readers use it. Writers create a replacement map and publish it only after validation. Existing snapshots continue to observe the old map.

The map stores owned keys and values. This avoids borrowing from request buffers or temporary transactions. Values could instead be `Arc<[u8]>` to make map copies cheaper for large payloads. That change would alter copying cost, but it would not alter the conflict rule. Separating these questions is valuable: consistency concerns which versions may commit; representation concerns how much work those versions require.

A transaction owns its staged changes and a snapshot of its starting state. Reads first consult staged changes and then the snapshot. A staged deletion must be distinguishable from no staged change, so the staging map stores `Option<String>` values. `Some(value)` means replacement and `None` means deletion. The outer map lookup tells us whether the transaction has an entry at all.

A Complete Conservative Implementation

The following program has no external dependencies. `commit` requires `&mut self`, so safe callers cannot publish two commits through the same store simultaneously. Wrapping the store in a mutex would extend that serialization to multiple threads. The global version check remains necessary because transactions can be created, held independently, and submitted later.

Listing 38.1 Rust example

```
use std::collections::BTreeMap;
use std::sync::Arc;

type Map = BTreeMap<String, String>;

#[derive(Clone)]
struct Snapshot {
    version: u64,
    data: Arc<Map>,
}

struct Store {
    version: u64,
    data: Arc<Map>,
}

struct Transaction {
    base: Snapshot,
    changes: BTreeMap<String, Option<String>>,
}

#[derive(Debug, PartialEq, Eq)]
enum CommitError {
    Conflict,
    VersionExhausted,
}

impl Store {
    fn new() -> Self {
        Self { version: 0, data: Arc::new(Map::new()) }
    }

    fn snapshot(&self) -> Snapshot {
        Snapshot {
            version: self.version,
            data: Arc::clone(&self.data),
        }
    }
```

Listing 38.1 continued

```
    fn begin(&self) -> Transaction {
        Transaction {
            base: self.snapshot(),
            changes: BTreeMap::new(),
        }
    }

    fn commit(&mut self, tx: Transaction)
        -> Result<u64, CommitError>
    {
        if tx.base.version != self.version
            || !Arc::ptr_eq(&tx.base.data, &self.data)
        {
            return Err(CommitError::Conflict);
        }
        if tx.changes.is_empty() {
            return Ok(self.version);
        }
        let next = self.version.checked_add(1)
            .ok_or(CommitError::VersionExhausted)?;
        let mut replacement = (*self.data).clone();
        for (key, value) in tx.changes {
            match value {
                Some(value) => { replacement.insert(key, value); }
                None => { replacement.remove(&key); }
            }
        }
        self.data = Arc::new(replacement);
        self.version = next;
        Ok(next)
    }
}

impl Transaction {
    fn get(&self, key: &str) -> Option<&str> {
        match self.changes.get(key) {
            Some(value) => value.as_deref(),
            None => self.base.data.get(key).map(String::as_str),
        }
    }

    fn put(&mut self, key: String, value: String) {
        self.changes.insert(key, Some(value));
    }

    fn delete(&mut self, key: String) {
        self.changes.insert(key, None);
    }
```

Listing 38.1 continued

```
}

fn main() {
    let mut store = Store::new();
    let other_store = Store::new();
    let foreign = other_store.begin();
    assert_eq!(store.commit(foreign), Err(CommitError::Conflict));
    let before = store.snapshot();
    let mut first = store.begin();
    let mut stale = store.begin();
    first.put("mode".into(), "fast".into());
    assert_eq!(first.get("mode"), Some("fast"));
    assert_eq!(store.commit(first), Ok(1));
    stale.delete("mode".into());
    assert_eq!(store.commit(stale), Err(CommitError::Conflict));
    assert!(before.data.is_empty());
    assert_eq!(store.snapshot().data["mode"], "fast");
}
```

Validation checks both the global version and the identity of the snapshot's map. The pointer-identity check matters because two different stores can have the same numeric version. Without it, a transaction could read one store and commit to another. A durable or distributed implementation would normally use an explicit store identity and a version protocol appropriate to its storage model.

There are four important transitions. Before validation, the transaction is only a proposal. After validation, the new map is prepared privately. Publishing replaces the store's map pointer. Updating the version completes the new state before the mutable borrow is released. No user callback executes between pointer replacement and version update, and those assignments themselves do not allocate. A reader cannot interleave through a safe shared reference while this mutable borrow is active.

Building a replacement first gives a strong structural benefit: ordinary panics during preparation do not partially update the currently published map. The example uses strings and their ordinary ordering. A more generic key type can execute user-defined comparison code; that is another reason to keep preparation separate from publication. Allocation failure may abort the process under the allocator's policy, so an argument about unwind safety does not establish recovery from every failure mode.

Understand What the Global Version Proves

If a transaction commits, no earlier writer has changed the store since the transaction's snapshot was taken. Therefore all of its snapshot reads correspond to the state immediately preceding its commit. Its staged changes can be applied as one serialized operation in commit order. This is a conservative serializable execution model for these transactions, provided all relevant reads and writes occur through the store.

The final condition is easy to overlook. Suppose the transaction reads a balance from this store but obtains an exchange rate from a mutable external cache. The store's version says nothing about the exchange rate. A proof is only as broad as the state it actually validates. Application invariants that depend on time, files, remote services, or another database require an additional protocol.

The conservative model also rejects harmless overlaps. Two transactions that write different keys still conflict when the first commits. This can be acceptable for a small configuration service with infrequent writes. It can be expensive for a high-volume independent-counter workload. Correctness comes before granularity: establish a model whose invariants are clear, then measure whether its conflicts justify a more complicated validator.

Read-only transactions currently validate their version too. A read-only transaction submitted after a writer receives a conflict even though it has no effects. One alternative is to return its snapshot version without validation. That is appropriate when a historical consistent read is sufficient. It is inappropriate if the caller interprets a successful validation as evidence that a decision still reflects the current state. Method names should make that distinction visible.

Finer Validation Introduces New State

Per-key versions can reduce conflicts, but tracking only written keys is insufficient when decisions depend on reads. Imagine two operators who may be on call, with an invariant that at least one remains. Two transactions each observe both operators, then each removes a different operator. Their write sets are disjoint. Validating only write-write conflicts allows both to commit and breaks the invariant.

A finer optimistic design records the versions of all keys read and validates them at commit. Missing keys need a representation too. If a transaction reads that a key is absent, another transaction inserts it, and the first relies on the absence, the validator must detect that change. Deletion and reinsertion can create another trap when a generation number is reused. Use monotonically advancing versions or another identity mechanism that does not silently equate different histories.

Range queries are harder still. Recording the versions of returned keys does not detect a new matching key that was absent during the original query. That is a phantom. A range-level generation, predicate lock, index validation, or another explicitly designed mechanism is needed if range-dependent decisions require serializable behavior. A `BTreeMap` gives ordered traversal; it does not automatically give a database isolation level.

These issues illustrate the boundary between Rust correctness and algorithm correctness. The compiler can establish that a map reference is not used after invalidation. It cannot infer that a transaction forgot to validate the absence of an operator. Tests should describe the application invariant directly and arrange schedules that attempt to violate it.

Shared Ownership Has Retention Costs

An `Arc` snapshot is cheap to create but can be expensive to retain. The current map may be replaced many times while an old snapshot keeps one historical map alive. The example clones all key and value strings for every write transaction, so peak memory includes every still-retained version plus the replacement under construction. This is predictable but can be substantial.

Structural sharing can reduce copying. A persistent tree copies only changed paths while sharing unchanged nodes. Its ownership graph is more complex, but `Arc` can represent shared immutable nodes. Another design stores large values behind `Arc` while cloning the top-level map. That preserves a simple validator and reduces payload copying without implementing an entirely new collection.

Reference counting itself is not free. Cloning `Arc` updates a shared count, and dropping the final reference runs destruction. If the last reader releases a huge historical map on a latency-sensitive thread, that thread

may pay the destruction cost. Some systems deliberately defer reclamation to another thread. Such a change needs a bounded retirement policy; otherwise delayed reclamation merely becomes another unbounded queue.

The complete example below shows the essential snapshot behavior with copy-on-write. It is a separate experiment, not an optimization already present in the store implementation.

Listing 38.2 Rust example

```
use std::collections::BTreeMap;
use std::sync::Arc;

fn main() {
    let mut current = Arc::new(BTreeMap::from([
        (String::from("limit"), 10_u32),
    ]));
    let old = Arc::clone(&current);
    Arc::make_mut(&mut current).insert("limit".into(), 20);
    assert_eq!(old["limit"], 10);
    assert_eq!(current["limit"], 20);
    assert!(!Arc::ptr_eq(&old, &current));
}
```

When another strong owner exists, copy-on-write must preserve that owner's view. Do not assume `Arc::make_mut` avoids cloning under a snapshot-heavy workload. Measure the actual number of live readers at write time. A representation optimization can be ineffective precisely when the versioning feature is most heavily used.

Retry the Decision Carefully

After a conflict, a caller may begin another transaction and repeat its computation. Repeating pure calculations is usually straightforward. Repeating external effects is not. If the transaction sent an email before commit, retrying can send it twice even though only one state change is accepted. If it charged a payment before commit, the consequences are larger.

One useful pattern computes a proposal and an effect description without executing the effect. Commit both the state change and a durable or otherwise appropriate outbox record in the same consistency domain, then process the outbox with deduplication. Our in-memory store cannot make

that outbox survive process failure, so it is only an illustration of where the protocol belongs. Another approach uses an external service's idempotency key and records its result under a carefully specified recovery protocol.

A retry policy also needs a limit. Immediate retries under contention can create repeated work and poor tail latency. Backoff, admission control, or a serialized command processor may be more effective than increasingly fine validation. Optimistic concurrency is attractive when conflicts are rare and transactions are cheap. It becomes less attractive when every failed transaction performs expensive work that must be discarded.

Returning the consumed transaction inside `CommitError` can help callers inspect or reuse a proposal. It should not imply that the old reads are still valid. A safer retry API accepts a closure that reruns the whole read-compute-write step against a fresh snapshot, while clearly restricting external side effects. Even that closure-based API cannot prove semantic idempotence through an ordinary trait bound.

Training Encounters

1. Change the commit error to return the original transaction on conflict. Explain which parts can be reused without recomputation and which cannot safely be assumed current.

2. Design per-key version validation for point reads, including absent keys and deletion. Give a history that breaks a validator that checks only the write set.

3. Add a snapshot age metric. Specify whether age means elapsed time, number of commits behind, or both, and describe how you would detect retained historical memory without making every snapshot mutable.

Training Debriefs

1. Use an error variant carrying the transaction. Owned input values or an expensive parsed request can be reused. A proposed numeric result derived from old reads cannot simply be applied to the new state. Recompute the decision or establish a domain-specific merge rule. The transaction's snapshot remains a valid historical view even after rejection.

2. Associate each logical key with a generation that survives deletion, or validate against a separate key-version table. Record the observed generation for every point read, including absence. Validate read generations before applying writes. The two-operator example demonstrates why disjoint writes can still violate a shared predicate.

3. A snapshot can carry its creation `Instant` and version as immutable metadata. The store can expose the current version so monitoring computes version lag. Retained-memory estimates require tracking live generations, weak references to historical maps, or explicit snapshot registration. Avoid inferring byte size directly from strong reference counts because one map may have many readers while another has few but much larger data.

Further Reading

• BTreeMap documents ordered key operations; the transaction and validation rules in this level are separate design choices.

• Arc documents shared ownership, weak references, and copy-on-write operations.

• Mutex describes one way to serialize access when adapting the store to multiple threads.

The False Inner Commit

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design savepoints for the versioned store. A parent transaction stages a value, a child reads it and changes two keys, and the child can either merge or roll back. Reads should consult the nearest staged change before older layers and the base snapshot. A child deletion must hide an older value rather than look like no staged entry. Only the outer transaction may call the store's publication operation. Provide histories covering child rollback, child merge followed by parent rollback, and a background writer invalidating the outer base before publication. Explain what remains a valid historical snapshot after conflict. You may use an owned transaction stack or a scoped borrowing API, but identify who owns every staged value at each transition.

Encounter Constraints

• Do not advertise independent database isolation for these local savepoints.

• Preserve store-origin validation as well as version validation.

• No external side effects occur merely because an inner savepoint merges.

Hint one / Hint two / Reveal debrief

Side Quest

Allow a long-running preview to report which keys it changed without claiming those values are still current. Design a conflict report that separates reusable owned input from calculations derived from stale reads. Then compare retrying the whole decision with merging a domain-specific commutative operation. State the conditions that make the latter legitimate rather than treating every conflict as mergeable. Test a conflicting concurrent writer as well.

Capability unlocked Savepoint Architect

Choose Your Exit

Move proposals through workers / Level 39
Preserve ownership and accounting while work crosses bounded queues.

Review the consistency claim / Level 42
Check precisely which state the version rule validates.

Mark your quest log / Return to the map

ZONE 5 / THE ENDGAME / LEVEL 39

The Collector Goes Dark

Ferrum's workers are fast enough to overwhelm their collector. Bounded channels keep queues small until the collector fails, after which the coordinator waits for workers and the workers wait for result capacity. Every allocation still has a valid owner, yet the city stops. This boss tests the difference between safe memory and a live shutdown protocol. The escape route follows who can close each queue and who observes that closure.

Mission Make collector failure terminate a bounded finite pipeline without detached workers or a cyclic wait.

Suggested loadout Level 19, Level 20, Level 21, Level 26, Level 28. These are guidance, not locked gates.

Mastery Targets

- Trace the wait graph when the result receiver disappears under backpressure.
- Join every worker and classify undelivered versus previously accepted jobs honestly.
- Reproduce collector failure with deterministic synchronization at small queue capacities.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 39

A Bounded Worker Pipeline

Concurrency becomes easier to understand when every queue and owner has a name. A worker pipeline is a useful case study because its apparent simplicity hides several independent contracts. Who owns work while it is queued? What happens when a worker fails? Which component drains results while the producer is blocked? Does shutdown discard work or finish it? Is the memory bound expressed in messages or in bytes?

We will build a finite pipeline using standard-library scoped threads and bounded channels. A coordinator distributes numeric jobs among workers. Each worker computes one result and sends it to a collector. The collector reduces results into a summary instead of retaining every result. This makes the buffering behavior visible and avoids requiring a runtime or an external channel implementation.

The pipeline's domain is intentionally pure arithmetic. That lets us reason about ownership transfer and lifecycle without confusing successful computation with durable side effects. Later sections explain what changes when a job sends a network request or modifies storage.

Draw the Wait Relationships

The pipeline has one bounded input channel per worker and one shared bounded output channel. Each worker owns its receiver. The coordinator owns the input senders. The collector owns the output receiver. Workers own clones of the output sender. The coordinator drops the original output sender once worker creation is complete.

The extra collector thread is essential. If the coordinator filled every input queue before starting to drain a bounded output queue, workers could block while sending results, stop receiving jobs, and leave the coordinator blocked while sending jobs. Every component would be waiting for another component to free capacity. Bounded queues expose this wait cycle; they do not solve it by themselves.

Listing 39.1 Sketch

```
coordinator -> worker input queues -> workers -> result queue -> collector
      |                                                          |
      +---------------- final joins ----------------------------+
```

In this design the collector runs while dispatch is active, so output pressure can be relieved independently. Worker input queues eventually have space if each computation finishes and the collector continues consuming. That is a progress argument under stated assumptions. It is not a claim that the operating system guarantees fair scheduling or that an arbitrary user callback cannot block forever.

A Complete Pipeline

This example returns a summary or an error for invalid worker counts, dispatch failure, or a joined thread panic. If the OS cannot create a thread, `Scope::spawn` panics. Use `Builder::spawn_scoped` for a returned startup error and close the channels for any workers already started. The summary counts failed squares separately from overflow in the aggregate sum.

Listing 39.2 Rust example

```
use std::sync::mpsc::sync_channel;
use std::thread;

#[derive(Debug, Default)]
struct Summary {
    completed: usize,
    job_failures: usize,
    sum: u64,
    sum_overflowed: bool,
}

fn run(jobs: u64, worker_count: usize)
    -> Result<Summary, &'static str>
{
    if worker_count == 0 {
        return Err("at least one worker is required");
    }
    thread::scope(|scope| {
        let (result_tx, result_rx) = sync_channel::<Option<u64>>(8);
        let collector = scope.spawn(move || {
            let mut summary = Summary::default();
            for result in result_rx {
                match result {
                    Some(value) => {
                        summary.completed += 1;
                        match summary.sum.checked_add(value) {
                            Some(sum) => summary.sum = sum,
                            None => summary.sum_overflowed = true,
                        }
                    }
                    None => summary.job_failures += 1,
                }
            }
            summary
        });

        let mut senders = Vec::new();
        let mut workers = Vec::new();
        for _ in 0..worker_count {
```

Listing 39.2 continued

```
            let (job_tx, job_rx) = sync_channel::<u64>(2);
            senders.push(job_tx);
            let output = result_tx.clone();
            workers.push(scope.spawn(move || {
                for value in job_rx {
                    let result = value.checked_mul(value);
                    if output.send(result).is_err() {
                        break;
                    }
                }
            }));
        }
        drop(result_tx);

        let mut dispatch_failed = false;
        let mut next = 0;
        for value in 0..jobs {
            if senders[next].send(value).is_err() {
                dispatch_failed = true;
                break;
            }
            next = (next + 1) % worker_count;
        }
        drop(senders);

        let worker_panics: Vec<_> = workers.into_iter()
            .filter_map(|worker| worker.join().err())
            .collect();
        let summary = collector.join()
            .map_err(|_| "collector panicked")?;
        if dispatch_failed || !worker_panics.is_empty() {
            Err("worker failed before pipeline completion")
        } else {
            Ok(summary)
        }
    })
}

fn main() {
    let summary = run(10, 3).unwrap();
    assert_eq!(summary.completed, 10);
    assert_eq!(summary.job_failures, 0);
    assert_eq!(summary.sum, 285);
    assert!(!summary.sum_overflowed);
}
```

The example is finite and assumes the counts fit in `usize`, which is practical for the demonstration but still a representation assumption. A production

long-running collector should use checked or saturating counters according to its metric policy. The numeric range of a job is independent of the numeric range of a metric counting jobs.

Round-robin routing is simple and deterministic, but it can leave the coordinator waiting on a slow worker while another worker is idle. That is head-of-line blocking in the dispatcher. A shared work queue or work-stealing scheduler can improve utilization for uneven jobs. Changing the scheduling policy should preserve the ownership and shutdown contracts established here.

Follow Ownership Through Shutdown

Normal shutdown begins when dispatch finishes and the coordinator drops all job senders. A worker continues receiving buffered jobs, then sees the channel disconnected once its queue is empty. It exits and drops its result sender. After every worker has exited, no result sender remains, so the collector drains the last buffered results and ends its loop. The coordinator joins workers and then the collector.

The order in which values disappear is the shutdown protocol. There is no special stop message in this version. That is appropriate because the desired policy is to finish all work already accepted by the input channels. A stop message could accidentally sit behind ordinary work, and multiple worker queues would require separate stop messages. Channel closure already expresses that no additional work will arrive.

Dropping the original result sender is easy to miss. If the coordinator retained that sender while joining the collector, the collector could wait forever for another result even after every worker finished. The channel remains connected because an unused sender is still an owner. This is a lifecycle bug that the type system permits: the ownership is valid, but the program's waiting policy is wrong.

The join pass retains worker panic payloads until the collector has joined. This arithmetic example can then drop ordinary panic payloads. If you adapt the pipeline to run arbitrary callbacks, choose a disposal policy as in Level 28. Implicit scope joins enforce lifetimes and can panic when a child has failed.

If a worker panics, its receiver is dropped. Sending to that worker then fails and returns ownership of the undelivered job inside the send error. This example discards that error payload and returns an infrastructure failure after cleanup. A more capable implementation could recover that specific job and route it elsewhere: the failed send did not deliver it to the worker. Previously accepted or active jobs are different. A worker may have performed an external effect for one of those before panicking, so recovering the undelivered message does not establish the retry status of earlier work.

State the Actual Memory Bound

Each input channel stores at most two queued `u64` jobs. Each worker may hold one active job and one pending result. The result queue stores at most eight results. The coordinator may hold one job while waiting to send. The collector holds a small summary. Thread stacks and channel implementation overhead also consume memory, and creating an arbitrary number of workers would make that overhead unbounded with respect to the job count.

This is a bound on live queue entries, not a universal bound on bytes for arbitrary message types. Replacing `u64` with `String` permits one message to own a large allocation. Replacing it with `Arc<Vec<u8>>` can keep a shared large allocation alive even when only one small handle is queued. A byte budget may need a semaphore or explicit accounting carried by each message.

There is another subtle source of unboundedness: the producer. If the caller constructs a vector containing all jobs before calling the pipeline, bounded internal channels do not bound that vector. This implementation generates values incrementally from a range. Real systems should inspect the entire path from input acquisition through result retention, not only the most visible queue.

The collector's reduction is also deliberate. If it instead stored every output in a vector, output memory would grow with the workload even though every channel stayed bounded. That can be correct for a finite batch with an explicit output budget. It should not be described as a constant-memory streaming pipeline.

Ordering and Throughput Pull in Different Directions

Results arrive in channel delivery order. A worker can finish first and pause before sending. When `sum_overflowed` is set, the partial sum is incomplete, and skipping overflowing additions can make it depend on delivery order. To restore input order, give each job a sequence number and let the collector buffer out-of-order results. A slow early job can force the collector to retain many later results.

Bounded reorder storage creates an additional progress question. If the collector stops draining the shared output queue because its reorder buffer is full, the slow job may eventually try to send into that same full queue. A naive policy can deadlock even though all computations finish. Possible repairs include a bounded window of dispatched sequence numbers, separate control paths, or a scheduling rule that ensures the missing earliest result can always be delivered.

Do not add sequence numbers and assume ordering is solved. Decide the maximum distance between the oldest incomplete job and the newest admitted job. That distance controls both reorder memory and the amount of speculative work. Some applications can process results in delivery order and avoid reorder storage. When input order is part of the user-visible contract, give it an explicit budget.

Throughput measurements should separate computation from coordination. Tiny jobs can spend most of their time transferring through channels. Batching several items per message may help, but increases the work retained by one blocked sender and the delay before a partially filled batch is processed. Batch size is another latency-memory-throughput tradeoff, not an unconditional optimization.

Cancellation Changes Acceptance Semantics

A graceful drain means finish accepted work. Immediate cancellation means stop as soon as possible and account for abandoned work. These are different protocols. An atomic cancellation flag can tell a worker to stop before the next job, but it does not wake a worker blocked indefinitely in `recv`, and it does not interrupt arbitrary blocking I/O already in progress.

Closing the input channels wakes receivers when their queues are drained or disconnected, but it does not necessarily discard queued jobs. A worker can check a cancellation flag after receiving and choose to abandon the job. The component holding that job must then record whether it was cancelled, retried, or intentionally lost. Merely returning from the thread hides that distinction.

Timeouts create uncertainty around external effects. A worker may time out while a remote server continues processing a request. Retrying that job may duplicate the effect. Useful job envelopes often contain a stable operation identifier, an attempt number, and a domain-specific idempotency key. The pipeline can transport those values, but the remote protocol must actually honor them.

For a service, acceptance should have a precise point. Is a job accepted when received from the socket, when inserted into an internal queue, or when durably recorded? A bounded channel can acknowledge only process-local transfer. If the process crashes afterward, the message may be lost. Rust's ownership transfer is exact about memory responsibility and silent about durable delivery unless the application adds that mechanism.

Test the Lifecycle Rather Than the Scheduler

Tests should assert observable contracts under deliberately constrained conditions. Use one worker and capacity zero to expose rendezvous behavior. Pause one worker after computing to control delivery order. Inject a job error without panicking and verify later work completes. Inject a panic in a controlled test and verify the coordinator returns. Use test timeouts to catch lifecycle regressions that could otherwise wait indefinitely.

Avoid tests that require a particular interleaving merely because it happened once. A sleep can encourage an interleaving but is a weak synchronization primitive. Barriers, additional test channels, or instrumented hooks can establish the exact state before triggering a failure. For example, wait until a worker has received a job, then close the result receiver and verify the worker handles a failed send.

Track queued jobs, active jobs, completed jobs, job failures, abandoned jobs, and wait times. Count admission rejections separately. After shutdown, every accepted job needs one disposition. In this process-local model, accepted equals completed plus failed plus abandoned. If an external effect is possible, add an uncertain category until the outcome can be resolved.

Training Encounters

1. Add a job identifier and preserve input order. Specify a bounded admission window and explain why the collector can always receive the earliest missing result.

2. Change graceful shutdown into immediate cancellation. Describe what happens to queued, active, and completed-but-undelivered jobs. Identify a blocked operation that a flag alone cannot interrupt.

3. Replace numeric jobs with buffers of variable size. Design a byte-budget token that travels with each job and returns capacity when the final responsible owner drops it.

Training Debriefs

1. Admit at most `w` unretired sequence numbers and ensure the collector continues draining results for that window. Retain at most `w` results while waiting for the earliest. Advance the admission window only as ordered results retire. Sending new work must not prevent the coordinator or another component from draining the result channel.

2. Stop admitting new jobs, signal cancellation, and close senders. Workers check cancellation before beginning received jobs and report abandoned identifiers where accounting requires it. Active blocking I/O needs its own cancellation or timeout mechanism. Completed results need an explicit delivery or discard policy, because cancellation of a consumer does not erase completed work.

3. Acquire capacity before allocating or accepting a buffer whenever possible. Put the acquired budget in an owned guard inside the job envelope. Transfer the guard with the bytes and release it after processing or final discard. Shared payloads require accounting for the underlying

allocation rather than charging or releasing the full size independently for every `Arc` clone.

Further Reading

- sync channel documents bounded buffering and rendezvous channels.
- Scope::spawn documents scoped spawning and OS startup failure.
- ScopedJoinHandle::join documents scoped joins and panic reporting.

The Silent Result Chamber

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Adapt the worker pipeline so its collector can return an ordinary error after receiving a chosen result. The result queue and input queues are bounded. Arrange a test where dispatch is still active at that moment. Specify how closing the result receiver reaches workers, how workers stop accepting further work, and how a coordinator blocked in send eventually observes disconnection. Join every thread even after the first failure. Distinguish the message returned by a failed send from jobs that a worker previously accepted or started. For pure finite computations, define which abandoned jobs may be retried; for external effects, allow an uncertain classification. Your solution must not rely on a cancellation flag waking an unrelated blocking operation.

Encounter Constraints

- Assume each active computation eventually returns; do not promise interruption of arbitrary blocking I/O.
- Every remaining sender and receiver owner has a documented shutdown role.
- Error reporting must not introduce a second bounded queue that nobody drains.

Hint one / Hint two / Reveal debrief

Side Quest

Replace pure arithmetic with a simulated remote operation that can complete before its acknowledgment is lost. Record job IDs and attempts in the simulator. Show why replaying a previously accepted job may duplicate an effect even though replaying the payload of a failed input send does not. Add a domain-specific idempotency policy and test its required peer behavior separately. Keep the admission boundary explicit throughout.

Capability unlocked Shutdown Warden

Choose Your Exit

Negotiate uncertain outcomes / Level 40
Represent what the peer may have done when local confirmation disappears.

Return to service supervision / Level 28
Generalize finite pipeline shutdown into a service lifecycle.

Mark your quest log / Return to the map

ZONE 5 / THE ENDGAME / LEVEL 40

The Lost Acknowledgment

Ferrum reopens a communication route, sends a repair command, and loses the connection before an acknowledgment arrives. One operator calls the command failed and retries with a new identifier. Another calls it successful because the bytes were written. Both know less than they claim. Your protocol map needs a place for uncertainty, and the permission to retry must depend on an explicit peer contract rather than optimism.

Mission Represent reconnect and retry decisions without confusing local transport status with confirmed remote effects.

Suggested loadout Level 15, Level 23, Level 26, Level 27, Level 39. These are guidance, not locked gates.

Mastery Targets

- Model pending, confirmed, and unknown outcomes distinctly.
- Preserve operation identity across a retry when the chosen protocol requires deduplication.
- Test a peer that executes a command but drops its acknowledgment.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 40

Protocol State Machines and Resource Ownership

Protocol code often begins with an enum named `State` and ends with checks scattered across dozens of methods. A method verifies that negotiation has finished. Another verifies that shutdown has not begun. A third assumes a request identifier has already been assigned. As the protocol grows, the same conditions become duplicated and occasionally disagree.

Rust offers several ways to represent these conditions. An enum can represent a runtime state and force exhaustive handling of its variants. A

generic state parameter can make different operations available on different types. Ownership can make a transition consume the previous state, preventing later use of the old capability. These techniques are most useful when they expose the actual protocol, including failure and retry behavior.

This level develops a local model of a framed connection. It is not a networking implementation. Its transport records frames in memory so that the ownership and state transitions are visible without an I/O runtime. We will then examine what changes when a transition sends bytes to another process, where local type changes alone cannot establish remote agreement.

Separate State From Evidence

A connection is initially negotiating. Successful negotiation permits data frames. Closing consumes that permission. The important question is what evidence justifies each transition. A local caller passing a supported version is evidence only in this model. A real connection would need to validate a received peer message, perhaps after several reads and writes.

The state type should therefore describe a fact the implementation has established, rather than a fact the caller merely wishes were true. If a public constructor can create `Connection<Established>` without negotiation, the type no longer proves negotiation happened. Private fields and controlled constructors are part of the design.

State also includes runtime data. An established connection needs a next sequence number. Using a marker type alone would hide that relationship in an unrelated optional field. Instead, the established state owns the counter it needs. The negotiating state owns its offer count. The closed state owns neither because no further protocol operation is permitted.

A Complete Local Model

The model below keeps unsupported negotiation recoverable by returning the original connection. Data send validates its arguments before changing the sequence number. Closing consumes the established connection and returns a closed value whose transcript can be inspected.

Listing 40.1 Rust example

```
#[derive(Debug, PartialEq, Eq)]
enum Frame {
    Offer(u16),
    Data { sequence: u32, payload: Vec<u8> },
    Close,
}

#[derive(Debug)]
struct Negotiating {
    offers: usize,
}

#[derive(Debug)]
struct Established {
    next_sequence: u32,
}

#[derive(Debug)]
struct Closed;

#[derive(Debug)]
struct Connection<S> {
    transcript: Vec<Frame>,
    state: S,
}

#[derive(Debug, PartialEq, Eq)]
enum SendError {
    PayloadTooLarge,
    SequenceExhausted,
}

impl Connection<Negotiating> {
    fn new() -> Self {
        Self {
            transcript: Vec::new(),
            state: Negotiating { offers: 0 },
        }
    }

    fn negotiate(mut self, version: u16)
        -> Result<Connection<Established>, Self>
    {
        self.state.offers += 1;
        self.transcript.push(Frame::Offer(version));
        if version != 1 {
            return Err(self);
        }
        Ok(Connection {
```

Listing 40.1 continued

```
            transcript: self.transcript,
            state: Established { next_sequence: 0 },
        })
    }
}

impl Connection<Established> {
    fn send(&mut self, payload: Vec<u8>)
        -> Result<u32, (SendError, Vec<u8>)>
    {
        if payload.len() > 1024 {
            return Err((SendError::PayloadTooLarge, payload));
        }
        let sequence = self.state.next_sequence;
        let Some(next) = sequence.checked_add(1) else {
            return Err((SendError::SequenceExhausted, payload));
        };
        self.transcript.push(Frame::Data { sequence, payload });
        self.state.next_sequence = next;
        Ok(sequence)
    }

    fn close(mut self) -> Connection<Closed> {
        self.transcript.push(Frame::Close);
        Connection { transcript: self.transcript, state: Closed }
    }
}

impl Connection<Closed> {
    fn transcript(&self) -> &[Frame] {
        &self.transcript
    }
}

fn main() {
    let negotiating = Connection::new();
    let negotiating = negotiating.negotiate(2).unwrap_err();
    let mut ready = negotiating.negotiate(1).unwrap();
    assert_eq!(ready.send(vec![10, 20]), Ok(0));
    let closed = ready.close();
    assert_eq!(closed.transcript().len(), 4);
    assert_eq!(closed.transcript().last(), Some(&Frame::Close));
}
```

The method sets differ between states. There is no `send` method on a negotiating or closed connection. There is no successful way to keep using the original established connection after `close`, because closing takes

ownership. An alias to the same resource would undermine that exclusivity, so the connection itself is not cloneable.

The error returns the unsent payload. This is often preferable to discarding a caller-owned buffer on a validation failure. It also makes a useful distinction: the error means the local model did not enqueue a data frame. A real transport error might occur after some bytes were written and could not make the same promise without tracking write progress.

The model rejects a send when incrementing the sequence would overflow. Consequently the maximum integer value is never used as a frame sequence. That is an explicit exhaustion policy. An alternative could use the maximum once and transition to an exhausted state. Wrapping silently would reuse identifiers and could confuse duplicate detection, so it should not be introduced casually as an arithmetic convenience.

A Transition Consumes a Capability

An established connection is a capability to emit data under the modeled protocol. Consuming it during close revokes that capability locally. This is more precise than a Boolean checked on every send, but it also changes ergonomics. Callers must handle a different return type, and a container cannot store all states under one concrete generic instantiation.

That is a useful trade when application control flow already has clear phases. A setup function can return an established resource. A worker can own it for the duration of a session. Shutdown can consume it and return a report. The compiler then helps prevent phase confusion across those boundaries.

Typestate becomes less convenient when many independently scheduled events can change the state at runtime. A connection manager may store thousands of connections in different phases. Representing the manager as a generic type for every possible combination would be impractical. An enum containing state-specific structures is usually a better representation at that boundary.

The two approaches can cooperate. A dynamic manager matches on a runtime enum and invokes phase-specific methods on the contained typed resource. That preserves a finite runtime representation while keeping

each state's operations locally precise. The objective is to make illegal transitions hard to express in the part of the program that performs them, not to maximize the number of generic parameters.

Runtime State Must Be Exhaustive

The next complete example uses an owned enum to represent a request that can be started or completed. Failed transitions return the original request unchanged. This makes the rejected event recoverable and keeps ownership explicit.

Listing 40.2 Rust example

```
#[derive(Debug, PartialEq, Eq)]
enum Request {
    Queued { id: u64, body: Vec<u8> },
    Running { id: u64, body: Vec<u8> },
    Done { id: u64, status: u16 },
}

impl Request {
    fn start(self) -> Result<Self, Self> {
        match self {
            Self::Queued { id, body } => {
                Ok(Self::Running { id, body })
            }
            other => Err(other),
        }
    }

    fn finish(self, status: u16) -> Result<Self, Self> {
        match self {
            Self::Running { id, .. } => {
                Ok(Self::Done { id, status })
            }
            other => Err(other),
        }
    }
}

fn main() {
    let request = Request::Queued { id: 7, body: vec![1, 2] };
    let running = request.start().unwrap();
    let done = running.finish(200).unwrap();
    assert_eq!(done, Request::Done { id: 7, status: 200 });
    assert!(done.start().is_err());
}
```

Ownership makes it easy to move payloads between variants without cloning. A transition taking `&mut self` would need a temporary replacement, an `Option`, or a more specialized representation to move out the payload. Such designs can be valid, but the temporary state then becomes part of the panic-safety argument. Taking `self` is often the simplest expression when the caller can accept a returned state.

Returning `Err(original)` is not always the right policy. A malformed network event may require closing the connection rather than preserving the old state. A timeout may move a request into an indeterminate state because the peer could have accepted it. The return type should express those possibilities directly. Using the same generic error for every failed transition can erase the most important information.

Local State and Remote State Can Diverge

Consider sending a close frame over a socket. The local method may serialize the frame, write some bytes, and then encounter an error. Did the peer receive the close? It may have received all of it even if the local operation failed to observe completion. Dropping the local connection cannot answer that question. A `Closed` marker proves only the condition that the implementation actually establishes.

Useful protocols often distinguish closing, locally closed, peer acknowledged, and fully finished. Whether all these states need separate Rust types depends on which distinctions affect allowed operations. If application code may retry a close but may not send more data, a closing state with a retry method can be appropriate. If the resource will simply be discarded, a detailed public state machine may add complexity without changing useful behavior.

The same issue appears with requests. A cancelled future may have written a request completely before being dropped. Retrying with a new identifier can execute the request twice. Retrying with the same identifier only helps if the peer stores and recognizes that identifier under the required failure conditions. Types can carry the identifier through transitions, but they cannot create a deduplication guarantee on the remote side.

This is why protocol review should include a table of observable events and uncertain outcomes. For each write, timeout, disconnect, and cancellation,

ask what the local endpoint knows and what the peer may have done. An unknown outcome is a legitimate state. Pretending it is a simple failure often causes incorrect automatic retries.

Cancellation Must Leave a Representable State

An async transition may suspend after modifying local state or after sending part of a frame. If the future owns the connection and is dropped, dropping the connection may be an acceptable cancellation policy. If the future merely borrows the connection, cancellation must leave the borrowed object in a state that later methods understand.

One robust approach stores write progress in the connection itself. The future advances an offset into an owned frame buffer. If cancelled, the offset and buffer remain, and a later operation can resume or explicitly abort the connection. Another approach makes the resource unusable for further protocol traffic after any interrupted write. That is simpler but should be visible in the API's failure behavior.

Avoid placing indispensable progress only in a temporary future while leaving the connection marked established. Dropping that future could lose the information needed to continue the byte stream correctly. This is a semantic invariant, even if the underlying memory remains safe. The cancellation analysis from the async levels applies directly to state-machine representation.

Dropping a connection also cannot perform arbitrary asynchronous cleanup. A destructor can release local resources, but it cannot wait through ordinary async suspension points. If graceful protocol completion matters, provide an explicit async close operation and define what ordinary drop does when close is skipped. Blocking indefinitely inside drop would move the problem into unrelated scopes and can deadlock an executor thread.

Keep State Spaces Manageable

It is tempting to encode every independent property as a generic parameter: authenticated, compressed, encrypted, writable, registered, draining. Six Boolean properties already describe sixty-four combinations before runtime counters enter the picture. Many combinations may be impossible, but the public API can still become difficult to name and understand.

Group properties that transition together into one semantic state. Keep independent runtime policy in ordinary fields when it does not affect method availability. Use newtypes for important validated values such as a nonempty stream identifier or checked frame length. A small number of meaningful capabilities usually communicates more than a large product of marker parameters.

Error design can simplify the state space as well. If any transport failure permanently invalidates a session, consuming the session and returning an error is honest and easy to reason about. If recovery is required, return the exact recoverable resource. Do not force a complex recovery type into an API whose callers always discard it, and do not hide a recoverable buffer when callers genuinely need to retry.

Testing should follow transitions and invariants. Generate sequences of events and assert that accepted transitions preserve identifier uniqueness, frame ordering, and resource accounting. Compile-fail tests can check that a closed typed connection has no send operation. Runtime tests must still examine malformed peer messages and interrupted I/O because those enter through values rather than through the method set.

Training Encounters

1. Modify the local connection model so the maximum `u32` sequence number may be used exactly once. Decide whether exhaustion is a field inside the established state or a separate state type.

2. Design an async write operation that may be cancelled after a partial write. Specify which object owns the frame and offset, and which subsequent operations remain legal.

3. A manager must store both negotiating and established connections. Design an enum boundary that permits polling either state while exposing data send only after the established variant has been matched.

Training Debriefs

1. Store `Option<u32>` as the next sequence, taking `Some(max)` for the last send and replacing it with `None`. Future sends return exhaustion without emitting a frame. A separate exhausted type gives a stronger method distinction but complicates a method that normally borrows `&mut self`; it is most useful if callers benefit from owning the exhausted resource explicitly.

2. Put the frame buffer and current offset in the connection's pending-write state. A future borrows and advances that state. After cancellation, allow only resuming the pending write or aborting the connection until framing is restored. Do not allow a new data frame to interleave with the unfinished bytes.

3. Use an enum with variants containing `Connection<Negotiating>` and `Connection<Established>`, plus any closing states required by the protocol. A match gives access to the state-specific value. Transition code consumes one variant and returns another; rejected negotiation either restores the negotiating resource or produces a terminal error according to policy.

Further Reading

- Enums documents the representation of runtime alternatives.
- The Future trait defines the polling interface at which async transitions can suspend.
- Drop documents synchronous destruction and its role in resource cleanup.

The Echo That May Already Exist

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Design a client-side command state machine for a fake peer. The peer may execute a command and then lose its reply. The client supports reconnect, querying status when the peer protocol permits it, and an explicit retry policy. Choose which transitions should consume typed capabilities and which belong in a runtime enum. A disconnect after sending must not automatically become a confirmed failure. Define the stable operation ID, the peer's deduplication or status-retention contract, and what happens when that retention expires. Build a deterministic simulation of lost acknowledgment, duplicate delivery, and confirmed rejection before execution. Explain what your Rust types guarantee locally and what remains dependent on the peer's behavior.

Encounter Constraints

• Do not claim exactly-once execution from ownership or a successful local write.

• An unknown outcome cannot silently convert into safe retry authorization.

• Scope the simulator explicitly; it is not a deployed distributed transaction system.

Hint one / Hint two / Reveal debrief

Side Quest

Add a process restart between command execution and acknowledgment handling. Decide where the client stores operation identity and recovery state, and what the peer must retain. A process-local HashMap is not durable history. Draw the crash windows around each persistence step and describe which outcomes remain uncertain when either side loses its remembered state. Treat forgotten recovery records as uncertainty, too.

Capability unlocked Uncertainty Navigator

Choose Your Exit

Ship the protocol as an API / Level 41
Expose uncertainty and recovery without making accidental compatibility promises.

Return to cancellation terrain / Level 26
Connect dropped local computations to effects that may continue elsewhere.

Mark your quest log / Return to the map

ZONE 5 / THE ENDGAME / LEVEL 41

The Downstream Gauntlet

The city's repaired components are ready to become a library used outside Ferrum. The release candidate passes its own tests, but an integration crate no longer spawns one returned future, a minimal build imports an unavailable module, and the package omits a schema file. This gate tests whether a release is the artifact downstream users actually compile. Public promises include configurations and properties that never appear in the happy-path demonstration.

Mission Validate a release through representative downstream callers and the packaged artifact rather than only the development checkout.

Suggested loadout Level 9, Level 10, Level 14, Level 16, Level 27, Level 33. These are guidance, not locked gates.

Mastery Targets

• Create downstream compile fixtures for the public ownership, error, and future bounds you intend to support.

• Exercise minimal and relevant feature configurations instead of assuming all-features covers them.

• Build the packaged artifact with its declared inputs and compiler support policy.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 41

Shipping a Coherent Rust Crate

A crate is more than the code that compiles in the author's preferred configuration. It is a set of public promises: which types downstream code may name, which compiler versions are supported, which feature combinations work, which platforms are tested, and what happens when a dependency changes. These promises are part of the architecture because downstream programs compile against them.

This level follows a fictional protocol library named `framecore`. Its central algorithm validates and describes frames. Some users run it in a server with the standard library. Others want the same frame logic in a constrained environment without operating-system services. The goal is to share a coherent semantic core while keeping transport integration and deployment policy outside that core.

The interesting questions are not how to run `cargo new`. They are where a boundary belongs, how a feature changes obligations, and what evidence supports a compatibility claim. Build configuration is executable architecture: a field in a manifest can enable code paths that the ordinary development build never checks.

Choose Boundaries From Responsibilities

The frame model should own validation rules, lengths, identifiers, and structured protocol errors. A socket adapter should own reads, writes, buffering, and cancellation behavior. A service should own admission control, retries, logging policy, and shutdown. These layers may live in separate crates when independent dependency or platform requirements justify it. They need not become separate crates merely to imitate a diagram.

One advantage of a small protocol core is that it can be tested without a runtime or network. Another is that its public API need not expose runtime-specific error types or buffer implementations. If an adapter returns a framework's concrete buffer in every public method, that framework becomes part of the library's compatibility surface. Replacing it later can affect downstream code even when the protocol semantics stay identical.

Avoid making the lowest layer artificially generic over every imaginable transport. A byte-slice decoder and an owned frame encoder may be sufficient. Adapters can combine them with their own I/O abstractions. A generic transport trait belongs in the core only when the core genuinely needs to drive transport operations and when its error, lifetime, and async contracts are clear.

There is also a cost to splitting crates. Each crate has a manifest, version, feature graph, documentation surface, and release relationship. A private

module can provide encapsulation without introducing those responsibilities. Start from dependencies and independent consumers, then decide whether separate compilation and distribution boundaries are useful.

Features Add Capabilities

Cargo features are combined across a dependency graph. A library should generally design them as additive capabilities so two downstream users can enable different features without making the shared dependency unusable. A feature that silently changes the meaning of an existing operation is harder to compose than one that exposes an additional adapter.

For `framecore`, `std` can enable operating-system adapters. Error types can implement `core::error::Error` in either configuration. The `trace` feature depends on `std` for diagnostics. Frame validation follows the same protocol in every build.

This illustrative manifest is a project configuration excerpt. Its version and compiler baseline are choices for this example, not a claim about the newest available toolchain.

Listing 41.1 Sketch

```
[package]
name = "framecore"
version = "0.1.0"
edition = "2024"
rust-version = "1.85"

[features]
default = ["std"]
std = []
trace = ["std"]
```

The baseline here would need to be tested before publication. Merely writing `rust-version` does not prove the source and every dependency actually build on that compiler. Our book examples are checked on the locally available compiler identified in the reading guide; that is separate from the hypothetical crate's support policy.

Mutually exclusive features deserve particular caution. Suppose one feature selects backend A and another selects backend B by defining the

same public implementation twice. A downstream graph can enable both even if each direct user enables only one. Prefer exposing both adapters under distinct names or choosing a backend through an explicit type or constructor. If exclusivity is unavoidable, document the composition limit and detect it clearly, but recognize that it restricts downstream use.

Defaults are conveniences, not a reliable global off switch. One dependency can request no default features while another enables them. Review the resolved graph when behavior surprises you. The relevant unit is the dependency instance and Cargo's resolver rules, not just the line in one manifest.

Keep the Semantic Core Small

The following complete library example demonstrates a core that needs neither allocation nor operating-system services. It accepts only lengths that can be represented in a four-byte length field and computes total encoded size with checked arithmetic. Optional tracing does not change the result.

Listing 41.2 Rust example

```
#![cfg_attr(not(feature = "std"), no_std)]

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum LengthError {
    PayloadTooLarge,
    AddressSpaceOverflow,
}

pub fn encoded_length(payload_len: usize)
    -> Result<usize, LengthError>
{
    if u32::try_from(payload_len).is_err() {
        return Err(LengthError::PayloadTooLarge);
    }
    let length = payload_len.checked_add(4)
        .ok_or(LengthError::AddressSpaceOverflow)?;
    #[cfg(feature = "trace")]
    {
        use std::io::Write;
        let _ = writeln!(std::io::stderr(), "encoded length {length}");
    }
    Ok(length)
}
```

The two errors are distinct because a payload length may fit the protocol field while header plus payload exceeds the platform's addressable `usize` range. On common wide platforms the second branch is unreachable after the first check, but the source expresses the platform-independent contract. Tests on a wide host alone do not exercise every narrow-target boundary.

`no_std` does not mean no heap, no runtime support of any kind, or suitability for every embedded target. It changes the automatically linked standard-library environment. A library may separately use `alloc` when an allocator is available. Target support also depends on atomic availability, panic strategy, linker setup, and any foreign dependencies. Document the actual requirements rather than using `no_std` as a broad performance or deployment claim.

The trace is best effort. A failed write leaves the length result intact. Writing to standard error can still block. A reusable library can let the application choose the diagnostic sink and timing policy. Test the relevant feature combinations when tracing affects timing or synchronization.

Public Types Carry Dependency Commitments

If a public function returns a type from another crate, downstream users may need that same crate version to name, construct, or match the result. Two incompatible versions of a dependency define distinct Rust types even when their displayed names look identical. This becomes especially visible in plugin systems, serialization adapters, and public error enums containing foreign error types.

A wrapper can reduce exposure, but it must preserve the operations callers need. Wrapping every useful type behind an opaque handle may force users into awkward conversions or eliminate zero-copy access. Review the boundary from both sides: which dependency details are incidental, and which are intentionally part of the API?

Auto traits also participate in the promise. A future returned by a function may be expected to be `Send`. A public type may currently be `Sync` because of its fields. Adding a non-thread-safe field later can break downstream uses even if no method signature changes. The API levels explain these

implications at the type level; release engineering must actually check them across intended changes.

Error evolution needs the same care. An exhaustive public enum makes adding a variant a source-compatibility concern. A non-exhaustive enum leaves room for additional variants but requires downstream wildcard handling. Hiding all structure in a string is easy to evolve but prevents reliable programmatic recovery. Choose based on expected caller decisions, not only on implementation convenience.

Build Scripts Cross Another Execution Boundary

A build script runs on the build host and may generate source, probe libraries, or invoke native compilers. The final library may target a different platform. Confusing host and target assumptions can produce generated constants or linker flags that work locally and fail in cross-compilation.

Keep generated output in the designated output directory and make input dependencies explicit. A generator that reads an undeclared schema or environment variable can leave stale output after a change. A generator that embeds the current time can make equivalent builds differ even when source inputs are identical. Reproducibility requires deciding which inputs are legitimate and how they are recorded.

Build dependencies are executable dependencies. A procedural macro and a build script can run code during compilation, before the resulting application starts. Reviewing only runtime dependencies therefore misses part of the supply chain. A smaller dependency set can reduce review and build cost, but rewriting mature functionality locally can introduce its own correctness burden. Evaluate the actual code and maintenance requirements.

Native integration adds another version axis: headers, libraries, target ABI, and compiler flags must agree. A successful Rust type check cannot establish that a linked C library uses the expected structure layout. The FFI level's ownership and ABI contracts continue to apply during packaging and deployment.

Define a Support Matrix With Evidence

A useful continuous integration matrix follows supported claims. If the crate supports the minimum compiler, test it. If it supports no default features, compile and test that configuration. If adapters are optional, exercise each supported adapter and meaningful combinations. If the crate promises multiple targets, include target-specific compilation and runtime tests where execution is available.

Testing only `--all-features` is insufficient. Code excluded by the maximal configuration can contain broken imports or missing bounds. Testing only the default configuration misses optional integrations. At the same time, testing every mathematical subset becomes impractical as features grow. Reduce the configuration space by designing additive features and identify representative combinations from real dependency interactions.

The following command sequence is a starting point for this fictional crate, not a universal complete matrix. A project with foreign libraries, runtime integrations, or platform-specific behavior needs additional checks tied to those promises.

Listing 41.3 Sketch

```
cargo test --locked
cargo test --locked --no-default-features
cargo test --locked --all-features
cargo check --locked --all-targets
cargo doc --locked --no-deps
cargo package --locked
```

`--locked` asks Cargo to use the recorded dependency resolution rather than updating it as part of the command. This improves repeatability for that resolution, but it does not by itself make the entire build hermetic. Compiler versions, target toolchains, native libraries, environment inputs, and build scripts can still matter.

Package validation catches another class of problems: files present in a development checkout may be absent from the distributed package. Examples may reference a workspace-only path. Generated code may depend on an untracked file. Reviewing and building the actual packaged artifact tests the boundary that users receive.

Documentation Is Executable API Design

Examples should demonstrate the intended ownership path. If every example clones a buffer before calling the API, investigate whether the API forces unnecessary ownership or whether the example hides an important borrow. If every error is unwrapped, readers cannot see which failures are expected and which are invariant violations.

Document the dimensions that types do not express: cancellation behavior, allocation behavior where promised, ordering, blocking, fairness, panic conditions, and external side effects. Avoid promising exact implementation details unless callers need them. A documented guarantee can become a compatibility obligation even when no compiler checks it.

Rustdoc examples can compile as tests, which helps prevent drift. Compile-fail examples can show forbidden usage, but a test that merely fails for any reason may pass after an unrelated syntax error. Keep rejected examples small and inspect the intended diagnostic category during maintenance. The book's example checks follow this distinction between accepted programs, rejected programs, and nonstandalone sketches.

Finally, release notes should explain behavior changes from the caller's perspective. A new internal data structure matters when it changes latency, allocation, ordering, or requirements. A compiler baseline change matters even if the runtime behavior stays identical. A clear compatibility story is a technical feature because it lets downstream developers make reliable integration decisions.

Training Encounters

1. Your library's default feature enables a network adapter. Two downstream dependencies disagree about whether defaults should be enabled. Explain the likely composition and redesign the feature layout if the core must remain usable without networking.

2. A public future becomes non-`Send` after a refactor introduces `Rc` across an await point. Describe a regression check that captures the intended API property without depending on the future's concrete anonymous type.

3. A build script generates protocol constants from a schema file and an environment variable. Specify the declared inputs, generated output location, and cross-compilation considerations.

Training Debriefs

1. One user's request can enable the default feature for the shared dependency, so another user's disabled defaults do not necessarily remove it. Keep the core independent, make networking an explicit additive adapter feature, and avoid changing core validation semantics when networking is enabled. Inspect the resolved feature graph when diagnosing integration behavior.

2. Write a compile-time helper accepting `T: Send` and pass the returned future into it using representative borrowed inputs. The check tests the property callers need while leaving the concrete future opaque. Add separate checks for any intended lifetime constraints; `Send` alone does not imply `'static`.

3. Declare schema and environment changes as rerun triggers, write generated files under `OUT_DIR`, and distinguish host execution from target properties. Use target configuration supplied by the build environment rather than assuming the host's sizes or ABI describe the target. Keep nondeterministic data out of generated protocol constants.

Further Reading

- Cargo features documents feature composition and optional dependencies.
- Cargo dependency resolution explains resolution and version identity concerns.
- Rust version documents declaring a compiler support baseline.
- Build scripts documents host-side generation and change tracking.
- Cargo package describes validating the distributable package.

The Release Candidate Hydra

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Prepare a small library release with a protocol core and an optional adapter. Create two consumer fixtures: one uses the minimal configuration and one requires a returned future to be Send while borrowing suitable input. Now review a proposed change that adds Rc across a suspension point, narrows a generic bound, adds a variant to an exhaustive error enum, and references a workspace-only schema in a build script. Decide which changes violate your stated promises and which require an explicit compatibility decision. Validate the actual packaged source, not just local files. Your deliverable includes the support matrix, meaningful compile checks, the package contents you inspected, and release notes written around observable caller consequences.

Encounter Constraints

• Do not claim minimum compiler support without checking the declared baseline.

• Keep optional features composable according to Cargo's resolved feature behavior.

• A local successful build is not evidence that omitted package inputs are available to users.

Hint one / Hint two / Reveal debrief

Side Quest

Design an upgrade guide for a deliberate breaking change to error recovery and compiler baseline. Include one short before-and-after caller example and a migration check. Then identify one implementation change that should remain invisible to consumers. Explain how documentation can accidentally turn an incidental layout, allocation count, or cancellation detail into a future compatibility commitment. Make each promised migration step verifiable.

Capability unlocked Compatibility Sentinel

Choose Your Exit

Face the final audit / Level 42
Combine published promises with the invariants and evidence behind them.

Reforge the API boundary / Level 16
Reconsider which extensibility and compatibility choices belong in public types.

Mark your quest log / Return to the map

ZONE 5 / THE ENDGAME / LEVEL 42

Restore Ferrum

Ferrum's map is connected again, but restoring each district separately did not prove the whole city coherent. A borrowed parser can retain old storage. A bounded worker queue can hide an unbounded producer. A valid local state machine can still retry an uncertain remote effect. The final encounter brings these boundaries together. Your artifact is a small working simulation and an explanation a skeptical developer can inspect, challenge, and extend.

Mission Build and defend one coherent in-memory system whose ownership, state, progress, and verification claims agree.

Suggested loadout Level 6, Level 14, Level 20, Level 26, Level 31, Level 36, Level 37, Level 38, Level 39, Level 40, Level 41. These are guidance, not locked gates.

Mastery Targets

• Trace one request from source ownership through validation, work admission, state change, and reported outcome.

• Demonstrate a deterministic failure history and preserve the documented accounting or uncertainty state.

• Separate type-enforced guarantees, runtime invariants, and assumptions about another component.

Enter the manual / Attempt the boss / World map

FIELD MANUAL 42

Reviewing Rust Through Invariants

Understanding advanced Rust means being able to explain why a design works when the compiler has already accepted it. Type checking removes many invalid programs, but accepted programs can still deadlock, violate domain rules, allocate without bound, duplicate effects, or expose an unsound abstraction. A useful review method follows invariants across ownership boundaries and asks what evidence supports each promise.

This final level revisits three small designs. Each is ordinary safe Rust. None needs an exotic language feature. The point is to connect the book's concepts into a practical way of reading code: identify the capability a function receives, the state it changes, the failures it can encounter, and the observations available to callers. When unsafe code appears, the same method continues with additional memory obligations.

Begin With the Claim a Caller Relies On

A function signature is a partial contract. `&mut Store` establishes exclusive access through that borrow. It does not establish that a transaction preserves a balance invariant. `Result<T, E>` exposes failure, but it does not say whether an error leaves the input unchanged. `Arc<T>` shares ownership, but it does not establish a bounded retention policy or make arbitrary interior state thread-safe.

Write the semantic claim in a sentence that can be contradicted by an example. For a transfer function: if it returns an error, balances are unchanged; if it succeeds, the requested amount moved between two distinct valid accounts without arithmetic overflow. For a cache: returned values remain valid independently of later map mutation, and loading does not hold the cache lock. These claims are more useful than saying the code is clean or idiomatic.

Next identify all operations that can interrupt progress: ordinary errors, panics, cancellation points, callbacks, blocking I/O, lock acquisition, and foreign calls. A claim about failure preservation must survive the relevant interruptions. It need not survive conditions outside the stated model, such as process termination, unless durability is part of the contract.

Review One Mutation at a Time

Consider transferring an amount between balances. A tempting implementation subtracts from the source and then checks whether the destination can accept the addition. If the second check fails, the source is already changed. Rust prevents many aliasing mistakes here, but it permits this domain error.

The complete implementation below computes every fallible arithmetic result before committing either mutation. Indices are validated before

indexing. Distinctness is explicit. Assignments of the resulting `u64` values do not call user code, allocate, or perform arithmetic that could fail.

Listing 42.1 Rust example

```
#[derive(Debug, PartialEq, Eq)]
enum TransferError {
    InvalidAccount,
    SameAccount,
    InsufficientFunds,
    DestinationOverflow,
}

fn transfer(balances: &mut [u64], from: usize,
            to: usize, amount: u64)
    -> Result<(), TransferError>
{
    if from == to {
        return Err(TransferError::SameAccount);
    }
    let source = *balances.get(from)
        .ok_or(TransferError::InvalidAccount)?;
    let destination = *balances.get(to)
        .ok_or(TransferError::InvalidAccount)?;
    let new_source = source.checked_sub(amount)
        .ok_or(TransferError::InsufficientFunds)?;
    let new_destination = destination.checked_add(amount)
        .ok_or(TransferError::DestinationOverflow)?;
    balances[from] = new_source;
    balances[to] = new_destination;
    Ok(())
}

fn main() {
    let mut balances = [50, 10];
    transfer(&mut balances, 0, 1, 20).unwrap();
    assert_eq!(balances, [30, 30]);
    let before = balances;
    assert_eq!(transfer(&mut balances, 0, 1, 100),
        Err(TransferError::InsufficientFunds));
    assert_eq!(balances, before);
    let mut overflow = [1, u64::MAX];
    assert_eq!(transfer(&mut overflow, 0, 1, 1),
        Err(TransferError::DestinationOverflow));
    assert_eq!(overflow, [1, u64::MAX]);
}
```

The mutation appears to occur in two assignments, but no safe observer can read the slice through another live conflicting reference during the exclusive borrow. There is also no callback between the assignments. For

this local in-memory contract, the prepared values are committed without an observable intermediate state through the function's safe interface.

That conclusion would need revision if balances were atomic values observed concurrently, if each assignment wrote a remote database row, or if setting a balance invoked user-defined code. The proof depends on the representation and operations, not merely on the visual proximity of two lines.

The function rejects same-account transfers even for zero amount. That is a policy, and the order of checks makes it take precedence over invalid-index reporting when equal invalid indices are supplied. A reviewer should decide whether that behavior matches the API rather than silently assuming another precedence. Exact error behavior often becomes part of downstream tests.

Use Types to Expose Resource Relationships

Now review a cache. Returning `&String` from behind a mutex is generally incompatible with releasing the guard: later mutation could remove or replace the entry while the reference remains in use. Returning a guard would preserve access but hold the lock for the caller's entire use. Returning an owned `Arc<String>` gives the caller an independent ownership share while allowing the map lock to be released.

The next complete program uses this approach. It deliberately permits duplicate loading under contention. That policy avoids holding the lock while executing a potentially slow or reentrant loader. The first installed value wins, and every successful caller receives the stored value.

Listing 42.2 Rust example

```
use std::collections::BTreeMap;
use std::sync::{Arc, Mutex};

#[derive(Default)]
struct Cache {
    values: Mutex<BTreeMap<String, Arc<String>>>,
}

impl Cache {
    fn get_or_load<F>(&self, key: &str, load: F) -> Arc<String>
    where
        F: FnOnce() -> String,
```

Listing 42.2 continued

```
    {
        let existing = {
            let guard = self.values.lock().unwrap();
            guard.get(key).cloned()
        };
        if let Some(value) = existing {
            return value;
        }
        let candidate = Arc::new(load());
        let mut guard = self.values.lock().unwrap();
        Arc::clone(guard.entry(key.to_owned()).or_insert(candidate))
    }
}

fn main() {
    let cache = Cache::default();
    let first = cache.get_or_load("mode", || "fast".into());
    let second = cache.get_or_load("mode", || panic!("unused"));
    assert!(Arc::ptr_eq(&first, &second));
    assert_eq!(first.as_str(), "fast");
}
```

Reviewing this implementation requires separating three properties. The returned value remains alive because the caller owns an `Arc`. The map is protected because access occurs while holding the mutex. The loader may execute more than once for one key because it runs outside the lock. None of those properties implies the others.

If duplicate loads have unwanted effects, coordinate one active loader and its waiters per key. Publish failure on panic or abandonment. Releasing the map lock still permits A to wait for B while B waits for A. Give keys a fixed total order and allow each loader to request only higher-ranked keys. Reject other requests before starting or waiting on a load.

The cache is also unbounded. A correct ownership story does not establish an eviction policy. Even with eviction, callers can retain values after the map removes them, so resident-map size and total live payload memory are different metrics. If the application needs a hard memory budget, shared ownership and retained external handles must participate in that budget.

Mutex poisoning is handled here by panicking through `unwrap`. That is an explicit simplification of the example's recovery contract. A reusable cache should decide whether poisoned state can be inspected and repaired,

discarded, or treated as fatal. Blindly replacing every `unwrap` with recovery can be as incorrect as blindly adding them, because recovery requires understanding the protected invariant.

Treat Unsafe Boundaries as Proof Boundaries

When unsafe code appears, ask why safe operations cannot express the required implementation and which extra claim the caller or abstraction makes. A raw pointer does not record allocation lifetime, initialized extent, or permitted aliasing. Those facts must come from somewhere: a constructor, a borrow, a foreign contract, or an invariant maintained by earlier operations.

The review should follow pointer creation, transformation, use, and destruction. An index bounds check proves only part of a dereference obligation. The pointer must also have appropriate provenance, alignment, initialization, validity for the accessed type, and compatible aliasing. Creating a reference may impose stronger requirements than merely manipulating a raw pointer. A safety comment should connect the operation to these concrete facts.

Do not assume a private unsafe block is safe because all public methods are safe. A safe abstraction must preserve its invariants for every safe caller, including callers that provide unusual trait implementations, panic in callbacks, forget guards, or pass zero-sized types. If correctness requires a trait implementation to uphold memory safety, an ordinary safe trait bound may be insufficient; the contract needs an appropriate unsafe boundary or additional defensive checks.

Conversely, do not add unsafe code merely to make the borrow checker accept an implementation that can be redesigned safely. A helper using `split_at_mut`, a separate owner and view, a stable handle, or a staged mutation can often make the proof visible in the types. The best reason to use unsafe is a requirement that has been identified and justified, such as an implementation primitive or measured performance need that safe alternatives cannot meet acceptably.

Match Verification to the Claim

Compilation checks type relationships, trait obligations, and many ownership constraints. Unit tests check selected outcomes. Property tests explore invariants across generated inputs. Concurrency model checking explores schedules under a modeled set of primitives. Miri can detect certain undefined behaviors in executed paths. Profiling and benchmarks measure particular workloads. Each tool contributes evidence for a different claim.

For the transfer function, useful tests include insufficient funds, destination overflow, invalid indices, same-account policy, and preservation on error. A property can compare the total before and after a successful transfer using a wider integer type. For the cache, use synchronization to arrange two concurrent misses and test the documented winner policy. Counting only successful lookups would miss duplicate loader execution.

For unsafe code, passing tests is not a replacement for the invariant argument. Tests rarely cover every aliasing history, layout, panic point, or target. A successful Miri run means the executed paths passed that tool's checks under its current model; it is not a language-level proof of soundness. The unsettled details of Rust's memory model make precise source references and conservative implementation choices especially important.

Performance claims require their own evidence. A generic iterator chain may optimize well, but that does not establish the application's latency distribution. An allocation-free parser may retain a huge source buffer. A lock-free structure may spend significant time on retries and reclamation. Review the actual workload and the cost that matters to the caller.

Read Compiler Diagnostics as Missing Relationships

An ownership error often says that the program has not expressed a relationship clearly enough. A method borrowing all of `self` may hide field disjointness. A returned reference may depend on a temporary guard. A spawned task may retain a borrow of a stack frame that can end first. A generic trait object may lack a lifetime or auto-trait bound needed by its consumer.

Before adding a clone, mutex, box, or `'static` bound, explain what relationship is missing. Sometimes cloning is exactly right because the consumer needs independent ownership. Sometimes a mutex is right because shared mutation is the requirement. Sometimes a narrower borrow or a handle-based representation is more accurate. The error message is a starting point for that analysis, not a request for one universal repair.

The same discipline applies when code compiles unexpectedly. Nonlexical lifetimes may end a borrow before the end of a block. Reborrowing may temporarily restrict an existing mutable reference without consuming its underlying capability forever. A bound may be satisfied through a blanket implementation. Explain the accepted program in terms of these relationships rather than assuming the compiler overlooked a rule.

A Practical Review Sequence

First identify the owned resources and the exact points where responsibility transfers. Then identify borrowed views and which mutation or destruction would invalidate them. Next examine state transitions and failures, including cancellation and callbacks. After that inspect concurrency progress and memory bounds. Finally examine public compatibility promises and the evidence supplied by tests and measurements.

This sequence is useful because later questions depend on earlier ones. You cannot assess cancellation correctly without knowing which object owns partial progress. You cannot bound memory without knowing which shared handles retain allocations. You cannot audit a raw pointer without knowing which allocation and lifetime it comes from. Technical vocabulary helps only when it connects these facts.

Rust's concepts become most powerful when used together. Ownership defines responsibility. Borrowing grants temporary access. Traits express obligations and relationships. Enums represent alternatives. Async exposes suspended computations. Pinning constrains movement for address-sensitive values. Unsafe code permits operations whose proof lies outside ordinary checks. A strong design uses each mechanism to express one part of a coherent argument.

Training Encounters

1. Generalize `transfer` to a balance type whose arithmetic can return errors and whose assignment may run a destructor. Identify which parts of the original failure argument no longer follow automatically.

2. Change the cache to return errors and allow one active loader per key. Model loading, ready, and failed. Explain panic cleanup and what happens when two active loaders request each other's keys.

3. Choose a function from your own codebase and write three claims: one enforced by types, one requiring runtime validation, and one requiring a protocol with another component. Name a verification method for each.

Training Debriefs

1. Fallible arithmetic can still be staged, but replacing old values may invoke destructors that panic or affect external state. A generic rollback promise needs stronger assumptions or a representation that commits by swapping a prepared aggregate with carefully defined destruction behavior. The simple `u64` proof cannot be transferred merely by replacing the type parameter.

2. Give each load a shared entry and publish ready or failed once, including when a panic guard cleans up. Wake waiters. Apply the same strict key order to starting nested loads and waiting for existing ones. Every dependency increases in rank, so a cycle would have to increase and return to its starting rank. Test A and B loading together and reject the backwards request before it can block.

3. An example type claim is that a returned view cannot outlive its source, checked through compilation. A runtime claim is that an index is within the initialized prefix, checked through validation and boundary tests. A protocol claim is that retries do not duplicate a remote effect, checked through operation identifiers, peer behavior, and fault-injection tests. No single tool establishes all three.

Further Reading

• The Rust Reference is the primary language reference for checking the exact rule relevant to a review.

• The Rustonomicon discusses the obligations involved in unsafe abstractions.

• Miri documents an interpreter-based tool and its supported checks and limitations.

• Rust API Guidelines collects public API design considerations maintained in the Rust project ecosystem.

The Last Unstated Assumption

BOSS ENCOUNTER / FORGE 40 / EXPLAIN 40

Build a finite repair simulator using the book's ideas: immutable parsed configuration, a versioned local store, bounded work admission, and a fake remote peer that can lose replies. Choose a modest scenario such as updating station settings. Implement enough of the path to run a successful request, a stale transaction, and a lost acknowledgment. Supply an ownership diagram, a wait graph, and three precise invariants. For each invariant, name the implementation mechanism and evidence that supports it. Introduce one controlled failure after acceptance and show the reported outcome without silently losing the request or claiming the remote effect never occurred. Unsafe code is optional; if used, include its proof obligations and explain why the safe representation was insufficient.

Encounter Constraints

• Keep the scope finite and runnable without a real network or deployed database.

• Do not claim durability, exactly-once delivery, or proof of soundness from the simulator.

• Any memory bound must name queues, payload retention, snapshots, and admitted work.

Hint one / Hint two / Reveal debrief

Side Quest

Invite another developer to choose one hostile change: a retained old snapshot, a forgotten guard, a collector failure, a malformed frame, or a reused operation ID. Predict which invariant it challenges before running anything. Repair the design or narrow the promise, then update the evidence. Your final achievement is the ability to explain the repair, not merely to recover a passing test.

Capability unlocked Keeper of Explicit Invariants

Choose Your Exit

New Game Plus as an Architect / Level 9
Rebuild one subsystem around a different trait or erasure boundary and compare its obligations.

New Game Plus as a Delver / Level 29
Revisit representation choices with the complete system's ownership and measurement evidence.

Mark your quest log / Return to the map

The Hint Vault

Choose how much help to reveal. Nudges, tactics, and debriefs occupy separate sections so the answer is not immediately below the first hint. Every entry links back to its boss encounter.

01	Nudge	Tactic	Debrief	22	Nudge	Tactic	Debrief
02	Nudge	Tactic	Debrief	23	Nudge	Tactic	Debrief
03	Nudge	Tactic	Debrief	24	Nudge	Tactic	Debrief
04	Nudge	Tactic	Debrief	25	Nudge	Tactic	Debrief
05	Nudge	Tactic	Debrief	26	Nudge	Tactic	Debrief
06	Nudge	Tactic	Debrief	27	Nudge	Tactic	Debrief
07	Nudge	Tactic	Debrief	28	Nudge	Tactic	Debrief
08	Nudge	Tactic	Debrief	29	Nudge	Tactic	Debrief
09	Nudge	Tactic	Debrief	30	Nudge	Tactic	Debrief
10	Nudge	Tactic	Debrief	31	Nudge	Tactic	Debrief
11	Nudge	Tactic	Debrief	32	Nudge	Tactic	Debrief
12	Nudge	Tactic	Debrief	33	Nudge	Tactic	Debrief
13	Nudge	Tactic	Debrief	34	Nudge	Tactic	Debrief
14	Nudge	Tactic	Debrief	35	Nudge	Tactic	Debrief
15	Nudge	Tactic	Debrief	36	Nudge	Tactic	Debrief
16	Nudge	Tactic	Debrief	37	Nudge	Tactic	Debrief
17	Nudge	Tactic	Debrief	38	Nudge	Tactic	Debrief
18	Nudge	Tactic	Debrief	39	Nudge	Tactic	Debrief
19	Nudge	Tactic	Debrief	40	Nudge	Tactic	Debrief
20	Nudge	Tactic	Debrief	41	Nudge	Tactic	Debrief
21	Nudge	Tactic	Debrief	42	Nudge	Tactic	Debrief

The Nudge Gallery

A change of attention before a solution.

Level 01 The Double Delivery

Draw a box around the bytes and put exactly one owner's name above it at every step. A receipt should not accidentally become another editable owner of the same capsule.

Return to the encounter / Need a tactic

Level 02 The Partial Inventory

Do not try to repair one partly emptied struct for all branches. Start by naming the different complete results each branch should produce, then move fields directly into those results.

Return to the encounter / Need a tactic

Level 03 The Two Crews

The useful boundary is between the earlier region's end and the later region's start. Sort requests for the implementation, then undo that ordering when presenting arguments to the caller. Preserve that distinction.

Return to the encounter / Need a tactic

Level 04 The Smuggled Page

Ask who chooses the lifetime of the temporary slice. The caller should choose the result type, while the inspection function chooses a short borrow lifetime that fits its local storage.

Return to the encounter / Need a tactic

Level 05 The Portal That Both Takes and Gives

Imagine the caller writes the most dangerous legal input after your proposed substitution. Then imagine the caller retains the most demanding legal output. A portal must satisfy both sides of its contract.

Return to the encounter / Need a tactic

Level 06 The Interrupted Replacement

List every operation that can execute user code before you decide where the commit point belongs. Preparation and destruction can both be user-code boundaries in a generic abstraction. Trace those boundaries.

Return to the encounter / Need a tactic

Level 07 The Reentrant Revision

There are two proofs here. One says no conflicting dynamic borrow is active during the callback. The other says the document has not changed since the proposal's input snapshot was taken.

Return to the encounter / Need a tactic

Level 08 The Compaction Mimic

A public identifier and a vector index do not have to be the same number. Add one lookup step so compaction can update a private location without changing the public name.

Return to the encounter / Need a tactic

Level 09 The Charter of Excessive Demands

Write down who selects each type. If the planner implementation selects one result, make that relationship explicit. If one policy supports several independent inputs, let the input help select its implementation.

Return to the encounter / Need a tactic

Level 10 The Shapeshifter Registry

Describe what the desk can do without knowing the tool's type. Construction and arbitrary generic configuration may belong outside that smaller contract, even if concrete users still need them. Choose deliberately.

Return to the encounter / Need a tactic

Level 11 The Commutative Gate

Walk the implementation's types in the rule's relevant order rather than counting how many local names appear. Mathematical operand order and legal implementation ownership are separate questions. Respect that boundary.

Return to the encounter / Need a tactic

Level 12 The Retrying Oracle

Ask what happens to each captured value during one call, not merely how it entered the closure. Ownership of settings and consumption of settings are different operations. Track both explicitly.

Return to the encounter / Need a tactic

Level 13 The Vanishing Collection

Give the temporary page view a name whose lifetime changes on each call. Then ask what the reader is allowed to mutate while that view is still being used. Follow that dependency.

Return to the encounter / Need a tactic

Level 14 The Ambiguous Import

Write a table in your notebook with phase, state after failure, recoverable value, and permitted next action. If one error string cannot answer those questions reliably, give the distinctions structure.

Return to the encounter / Need a tactic

Level 15 The Receipt That Never Happened

Draw the arrows first. Put an owned value on each arrow and write the exact fact that permits the transition. The arrow for an absent acknowledgement should not end at the same state as a local refusal to begin.

Return to the encounter / Need a tactic

Level 16 The Semver Mimic

Imagine the proposed crate as a replacement linked into each client. What operation used to be expressible that no longer is? Write that operation as a tiny function rather than judging the change by the privacy of its new fields.

Return to the encounter / Need a tactic

Level 17 The Mirror Expansion

Write the nonmacro version first. Mark acquisition, the guard's last possible use, each early exit, and the final result expression. Your macro should compress that program without quietly choosing a different control-flow boundary for the user.

Return to the encounter / Need a tactic

Level 18 The Capacity Doppelganger

Write two different statements: how much storage this type contains, and how many bytes this value currently uses. Only the first needs to distinguish types. Then list the checks that must succeed before the old payload may be overwritten.

Return to the encounter / Need a tactic

Level 19 The Unreturning Survey Team

Trace two owners separately: the owner of each string and the owner of a vector containing references to those strings. The worker may own the second while borrowing from the first. Joining can move that vector without moving the catalog's text.

Return to the encounter / Need a tactic

Level 20 The Bell Without a Forecast

Write the sentence a sleeping reader needs to become true. A bell is only one possible reason to reevaluate that sentence. Include a second exit condition for a target that shutdown has made unreachable, and decide what data must be read together.

Return to the encounter / Need a tactic

Level 21 The Half Delivered Decree

Replace the Boolean result with a record containing one outcome for each destination. Write down what send returning Ok actually proves. Then place a crash between the two sends and ask which facts the caller can recover without consulting another authority.

Return to the encounter / Need a tactic

Level 22 The Counter Hydra

Place every operation on one ordered timeline. Draw a reader's first and second generation loads as brackets. Can any writer payload store fit between the brackets without one of the writer's generation markers becoming visible to that acceptance test?

Return to the encounter / Need a tactic

Level 23 The Endless First Poll

Erase async syntax and list the state a paused loop needs: input owner, next index, accumulated value, and completion status. Then ask who will request another poll if the operation is waiting only for its own remaining CPU work.

Return to the encounter / Need a tactic

Level 24 The Twin Room Collapse

Draw a box around each child allocation and a different box around the parent struct. Move the parent box on paper. Which addresses change? Then identify the operation that would remove the actual child value rather than merely transfer its owner.

Return to the encounter / Need a tactic

Level 25 The Unrung Completion

Enumerate states before writing poll. There must be a persistent difference between nobody has sent yet and nobody can ever send. Now place the empty check and waker installation on a timeline with a producer attempting to complete between them.

Return to the encounter / Need a tactic

Level 26 The Vanishing Cursor

Ask what survives when the caller drops only the temporary operation. Any byte buffer or cursor needed by the next attempt must belong to something that survives that drop. A connection marked simply ready is missing evidence about its unfinished wire state.

Return to the encounter / Need a tactic

Level 27 The Contract Hydra

Write three different questions: can the scanner move between threads, can this operation move between threads, and can this operation outlive the current stack frame? Find a distinct trait or lifetime relationship for each rather than expecting one bound to answer all three.

Return to the encounter / Need a tactic

Level 28 The Supervisor That Declared Victory

Draw one row for every accepted identifier. Walk the failure script and refuse to erase a row merely because a worker or reply channel disappears. For each final label, write the evidence that lets the supervisor use that label instead of unknown.

Return to the encounter / Need a tactic

Level 29 The Mimic Packet

Write down the first seven byte positions before writing a struct. Ask which positions exist when the buffer ends after the sequence number, and which error can already be known from the first byte.

Return to the encounter / Need a tactic

Level 30 The Overlapping Observatory

Treat each window as a capability to access one region. Draw the regions before and after splitting. Which value should disappear when two child capabilities are created, and which owner must remain unavailable?

Return to the encounter / Need a tactic

Level 31 The Friendly Corruptor

Begin from the assumptions of the unchecked conversion. Work backward to the constructor, then forward through every method that changes the bytes. Find a replacement byte that is not valid as a standalone text character.

Return to the encounter / Need a tactic

Level 32 The Ninth Fragment

Write the state of four slots after arrivals 3, 1, and 3. A count of three tells you almost nothing about which values exist. Decide the duplicate policy before designing cleanup.

Return to the encounter / Need a tactic

Level 33 The Borrowed Bridge Toll

Make a table with one row per exported operation: input ownership, output ownership, and validity interval. If the foreign caller cannot tell who frees a value from that table, the interface is incomplete.

Return to the encounter / Need a tactic

Level 34 The Three Index Hydra

Name the authoritative copy first. Then imagine renaming a station halfway through an update: which indexes disagree, and can a caller observe them before the update finishes? Add a byte budget to the event log.

Return to the encounter / Need a tactic

Level 35 The Tournament of Unequal Rules

Before reaching for a timer, write what must be equal between the contenders. Then identify every cost currently included for only one side. Ask what observable result prevents the measured computation from disappearing.

Return to the encounter / Need a tactic

Level 36 The Missing Job

Write the accounting equation before generating random operations. Then identify the exact transition where a job becomes accepted. If the test cannot tell whether that transition happened, its shutdown assertion is probably ambiguous.

Return to the encounter / Need a tactic

Level 37 The Stale Span Doppelganger

Give the old text and the new text different names. Then follow one retained field after an insertion at the beginning. Which owner or identity prevents its old byte range from resolving against the new bytes?

Return to the encounter / Need a tactic

Level 38 The False Inner Commit

Rename child commit to merge while designing it. That word makes the missing distinction visible: one action changes a local proposal, while another publishes a new store state for all readers.

Return to the encounter / Need a tactic

Level 39 The Silent Result Chamber

Draw arrows from each blocked operation to the component that can release it. Then remove the result receiver. Which workers wake immediately, which must first finish computation, and which owner keeps the coordinator's send connected?

Return to the encounter / Need a tactic

Level 40 The Echo That May Already Exist

List what the client observed and what the peer could have done. If two remote histories produce the same local observation, the client's next state must accommodate both unless another protocol event distinguishes them.

Return to the encounter / Need a tactic

Level 41 The Release Candidate Hydra

Pretend the release archive is all the next developer receives. Which tests depend on files, features, or compiler properties that exist only in your working checkout? Write a consumer that states one promised bound directly.

Return to the encounter / Need a tactic

Level 42 The Last Unstated Assumption

Choose one request and draw every owner it passes through. At each boundary ask what can fail, who still owns partial progress, and which observation authorizes the next transition. One unexplained arrow is a useful place to attack.

Return to the encounter / Need a tactic

The Tactics Workshop

A concrete reasoning step for a blocked attempt.

Level 01 The Double Delivery

Write the success and failure types before the method body. If failure must permit resubmission, the error needs to own the rejected capsule. A reference or formatted error string cannot return the ownership you consumed. Keep the accepted storage inside the dock, and let the consuming call enforce the local handoff.

Return to the encounter / Reveal debrief

Level 02 The Partial Inventory

A match on the inspection mode can occur before any field leaves the shipment. Inside one arm, destructure the shipment and construct that arm's result. For the Drop variation, ask whether cleanup truly belongs to the aggregate or whether independently owned fields can perform it without needing a complete Shipment.

Return to the encounter / Reveal debrief

Level 03 The Two Crews

After bounds validation, suppose A lies before B. Split the whole slice at B.start. Select A from the left half and B from the right half using an offset relative to that half. Neither selected region needs a borrow through the original whole slice after the split. Keep both regions inside the callback's temporary access.

Return to the encounter / Reveal debrief

Level 04 The Smuggled Page

Look for the shape F: for<'scope> FnOnce(&'scope [Record]) -> R. The important feature is that R does not mention 'scope. If Record owns its text, borrowing that text through the temporary slice cannot fit into an escaping result. A captured configuration reference has a different owner and a different relationship.

Return to the encounter / Reveal debrief

Level 05 The Portal That Both Takes and Gives

For an input-only function, broader accepted inputs are useful to a restrictive caller. For an output-only view, a longer-valid reference can satisfy a shorter use. Combining those directions through one type parameter removes either unilateral substitution. A short reference to the wrapper changes the outer access duration, not the wrapper's internal type argument.

Return to the encounter / Reveal debrief

Level 06 The Interrupted Replacement

A candidate value can exist beside the published value until validation succeeds. The guard's cleanup may restore availability, but forgetting it must leave a state that remains safe even if permanently inconvenient. When evaluating a strong failure claim, inspect replacement's old-value destruction as well as the callback that builds the candidate.

Return to the encounter / Reveal debrief

Level 07 The Reentrant Revision

Give snapshot acquisition, callback execution, and publication visibly separate scopes. Store an ordinary generation value beside the copied text. At publication, compare that value and prepare its checked successor before assigning either field. A failed freshness check should return the candidate without pretending the callback never ran. Check every exit.

Return to the encounter / Reveal debrief

Level 08 The Compaction Mimic

When the last dense element moves into a hole, you need to know that element's slot. Store or recover that reverse relationship before removal. The generation belongs to the logical slot history, not the record's current dense position. Validate presence and generation before producing any borrowed view. Preserve that ordering.

Return to the encounter / Reveal debrief

Level 09 The Charter of Excessive Demands

The approve function's key obligation is shaped like Policy<P::Plan>. Only the reporting function needs P::Plan: Display. Demonstrate the distinction with a plan containing a private opaque field that deliberately has no textual representation; ordinary approval should still be available. This becomes a test of interface scope, not mere formatting syntax.

Return to the encounter / Reveal debrief

Level 10 The Shapeshifter Registry

Give the registry an explicit lifetime for borrowed implementations. Make the erased operation return a common result rather than Self. Compare a local &mut dyn Tool with a Box<dyn Tool + 'host>: the dispatch boundary can be identical while storage ownership and allocation differ. Do not accidentally change the required borrow duration.

Return to the encounter / Reveal debrief

Level 11 The Commutative Gate

Compare impl<T> Mul<T> for Vector<T> with impl<T> Mul<Vector<T>> for T. The second begins with an uncovered generic self type. A local Scalar<T> self wrapper changes that relationship; a concrete primitive self type removes that particular uncovered parameter. Both alternatives still need ordinary overlap analysis. That second check is an independent obligation.

Return to the encounter / Reveal debrief

Level 12 The Retrying Oracle

A mutable counter requires mutable access to the environment, which points to FnMut. A captured configuration read through shared access can remain owned across every call. If the body transfers a unique resource outward, that resource is absent on the next attempt; no stronger trait bound can recreate it. That absence matters.

Return to the encounter / Reveal debrief

Level 13 The Vanishing Collection

The common consumer should turn a page into an owned count, checksum, or other summary before proceeding. A stable-source reader may separately expose a longer source relationship, but requiring that relationship from scratch storage would contradict its reuse policy. Make retention an explicit API choice rather than a hidden assumption inside collect.

Return to the encounter / Reveal debrief

Level 14 The Ambiguous Import

An I/O cause can remain a source beneath a domain-level import category. A conflict should identify stale decision context rather than pretend the candidate is malformed. Returning the candidate preserves ownership, but applying it requires a fresh validation protocol. Stage all fallible preparation before publishing the accepted aggregate. Test failed publication.

Return to the encounter / Reveal debrief

Level 15 The Receipt That Never Happened

Separate validation failure, admission failure, and uncertain delivery into different result shapes. Have the first two return the relevant owned input when reuse is legitimate. Have the uncertain result carry a stable operation identifier instead. Check the public constructors and accessors: a caller who can replace a validated field can manufacture evidence the transition methods were supposed to protect.

Return to the encounter / Reveal debrief

Level 16 The Semver Mimic

Use helpers that require Send, functions that hold one borrowed entry while requesting another, an external trait implementation, and an erased plugin value. These checks target properties without naming hidden iterator types. Keep new metadata behind an evolution strategy appropriate to the existing public enum or struct. A private representation gives freedom only within the public capabilities already promised.

Return to the encounter / Reveal debrief

Level 17 The Mirror Expansion

A local binding gives the acquisition expression one evaluation and gives the guard a scope. A nested closure may make error typing convenient, but it intercepts return and propagation. Decide that behavior openly. For cross-crate helpers, inspect both path qualification and visibility: hygiene cannot grant access to a private function. Include an intentionally rejected escaping-reference example alongside successful cases.

Return to the encounter / Reveal debrief

Level 18 The Capacity Doppelganger

A fixed initialized array and a used counter avoid partial-initialization machinery. Return a slice ending at the validated counter. Wire length is a protocol value, not automatically a usize. Keep conversion to u32 and checked total-size arithmetic separate. If an array needs room for both header and payload, consider a different output representation instead of making stable const arithmetic do more than it supports.

Return to the encounter / Reveal debrief

Level 19 The Unreturning Survey Team

Use contiguous partitions so ordered assembly is simple. Keep the caller's catalog alive outside thread::scope, and create worker closures that capture references into it. Return each worker's matches through its scoped join handle, then consume the handles in partition order. Do not collect references into a temporary cloned catalog. Arrange a test interleaving with synchronization rather than relying only on sleep timing.

Return to the encounter / Reveal debrief

Level 20 The Bell Without a Forecast

The predicate can be current_revision at least target, or phase stopped. Check the ready condition first if your policy permits returning the final snapshot after shutdown. Otherwise distinguish stopped before satisfying the target. Keep the revision and snapshot update within one critical section. Returning an Arc clone lets the reader release the guard before doing expensive work, while the snapshot remains alive independently.

Return to the encounter / Reveal debrief

Level 21 The Half Delivered Decree

Keep the same message identifier in both recipient envelopes. Record each queue-acceptance result as it happens, and retain returned payload ownership on failed sends. Distinguish that local report from a recipient acknowledgement of completed processing. A caller who never receives the report cannot infer which transfer happened. If retries must be safe across that uncertainty, the recipients or a stronger coordinator must recognize repeated operation identifiers.

Return to the encounter / Reveal debrief

Level 22 The Counter Hydra

An even generation identifies a completed publication only while that identifier is not reused. The writer must reserve the next odd and even values before beginning. Readers reject odd first observations and unequal second observations. Keep the payload fields atomic even on discarded attempts, because detecting interference afterward does not retroactively legalize a data race. A second writer would invalidate the simple protocol unless separately serialized.

Return to the encounter / Reveal debrief

Level 23 The Endless First Poll

Compute an end index from the cursor and a bounded remaining length, avoiding overflow-prone unchecked cursor arithmetic. Update the accumulator and cursor before returning. When the end has not been reached, call wake_by_ref on the current context's waker and return Pending. A later poll can receive a different waker. Do not store references into your own Vec just to remember where the next byte lives.

Return to the encounter / Reveal debrief

Level 24 The Twin Room Collapse

Store optional pinned box handles, plus a terminal flag or enum. Parent mutability can access those handles because moving a handle does not move its pinned pointee. Poll with each handle's pinned mutable view. On completion, take and drop the owned handles; do not extract F from its box. Check terminal state before polling any child again, and keep the output outside stored parent state unless its own bounds are accounted for.

Return to the encounter / Reveal debrief

Level 25 The Unrung Completion

One mutex can make checking and registering one coherent operation. The sender takes the registered waker while publishing readiness, releases the guard, then wakes. If send happens first, the receiver sees the stored value; if registration happens first, send obtains the waker. Endpoint destruction must publish its own state transition too. Move payloads and obsolete notification handles out before expensive or reentrant destruction when that behavior matters.

Return to the encounter / Reveal debrief

Level 26 The Vanishing Cursor

Represent idle and pending-frame states in the exporter. The future borrows the pending state and advances an offset into its owned buffer. After Ready(n), validate n against the remaining slice and record progress before any further return. After Pending, preserve all state. A resume method can borrow it again; a new-frame method must reject or queue according to an explicit policy rather than overwrite the pending bytes.

Return to the encounter / Reveal debrief

Level 27 The Contract Hydra

Use an explicit returned-future Send bound at the generic boundary. At the registry boundary, box and pin the future while retaining a named lifetime for borrowed receiver and input. Test that value with a Send-only helper. A static helper should reject short-lived borrowed backing data. To cross that later boundary, move an owner of the bytes into the outer future instead of pretending move turns a reference into owned storage.

Return to the encounter / Reveal debrief

Level 28 The Supervisor That Declared Victory

Separate admission state, job state, and worker state. Closing admission must coordinate with registration. A job can stop locally while its external outcome remains uncertain; a worker can stop without reporting a normal result. Join or otherwise observe each owned worker, retain enough active-job identity to classify interrupted work, and ensure the supervisor does not block worker completion by abandoning a bounded result channel it still expects workers to use.

Return to the encounter / Reveal debrief

Level 29 The Mimic Packet

The decoder needs three outcomes: incomplete, invalid, or complete with a consumed length. A complete header does not imply a complete packet. Use the decoded length only after converting it safely, and leave trailing bytes untouched. Compare your output against a hand-calculated sequence whose integer bytes differ, rather than testing only zero and one.

Return to the encounter / Reveal debrief

Level 30 The Overlapping Observatory

Start with a struct containing an exclusive slice and a consuming split method. You can check the offset before calling the standard splitting operation. If you later replace the field with a pointer and marker, list every fact that the original slice supplied automatically. Your unsafe implementation must preserve those facts for every safe method, including empty cases.

Return to the encounter / Reveal debrief

Level 31 The Friendly Corruptor

Separate validation from commitment. Check the index and permitted byte class before changing storage. For a bulk operation, validate the complete proposed input first or stage a replacement. Your argument must include error paths, because returning an error after partially writing invalid bytes still breaks the next safe call to as_str.

Return to the encounter / Reveal debrief

Level 32 The Ninth Fragment

Use the safe optional-slot model as your behavioral oracle. For each operation, identify the ownership transition and update its initialization flag at the correct point. In finish, checking the number of received messages is insufficient: duplicate arrivals may inflate that number. Distinguish the number of initialized distinct slots from total arrival count.

Return to the encounter / Reveal debrief

Level 33 The Borrowed Bridge Toll

Separate representation from ownership. repr(C) can help describe a simple pointer-and-length view, but it does not make the pointed-to bytes live. Keep the owning result opaque, expose immutable borrowed bytes, and route destruction through the allocating implementation. For zero-length input, branch before constructing a slice if your foreign convention allows null.

Return to the encounter / Reveal debrief

Level 34 The Three Index Hydra

Compare two complete designs: one hash-based canonical store with sorting at listing time, and one with a maintained alphabetical index. Include the work needed to keep the second index correct. Store logical IDs in indexes rather than raw pointers into a growable vector, and define stale-ID behavior independently from memory safety.

Return to the encounter / Reveal debrief

Level 35 The Tournament of Unequal Rules

Use the same prepared datasets and verify equivalent outputs outside the timed region. Add distinct small-work and realistic-work cases so dispatch cost is not mistaken for end-to-end impact. Record compiler and optimization settings and compare code size as the number of concrete handler types grows. Keep claims tied to the tested workload and machine.

Return to the encounter / Reveal debrief

Level 36 The Missing Job

Represent each job by an ID and an independent model state. Pause execution immediately after admission is recorded, fail the collector or initiate shutdown, and then release the worker. Assert the documented terminal classification. Keep the failure trace small enough to replay deterministically, and use shrinking or manual reduction to isolate the essential history.

Return to the encounter / Reveal debrief

Level 37 The Stale Span Doppelganger

Separate immutable source snapshots from the editor's current-snapshot handle. Store validated ranges, not internal references, and construct borrowed views on demand. A retained record can own a shared snapshot handle and an index. Check every boundary before slicing, and make source identity part of span resolution when the span does not already own its source.

Return to the encounter / Reveal debrief

Level 38 The False Inner Commit

Use a stack of maps whose values distinguish replacement from deletion. A lookup checks the newest map first and stops at the first staged entry, including a tombstone. Merging moves entries into the immediately older map. The base snapshot and origin identity remain attached to the outer transaction until its final validation.

Return to the encounter / Reveal debrief

Level 39 The Silent Result Chamber

Treat channel disconnection as part of the protocol. Result-send failure should lead to dropping the relevant job receiver, rather than waiting for more jobs forever. Join every handle explicitly after admission closes. Add a test hook that confirms dispatch is active before the collector fails, and use capacities zero or one to expose hidden waiting cycles.

Return to the encounter / Reveal debrief

Level 40 The Echo That May Already Exist

Give the command a stable logical ID separate from its connection and attempt number. Simulate execution followed by a dropped reply. Reconnection alone supplies no missing acknowledgment. An explicit unknown state forces callers to choose status lookup, deduplicated retry under a real contract, or manual recovery instead of treating every error as safely repeatable.

Return to the encounter / Reveal debrief

Level 41 The Release Candidate Hydra

Use small compile fixtures rather than copying production implementations into tests. Require Send on the produced future through a generic helper, then separately exercise supported borrowed inputs. Test the minimal feature set and meaningful combinations, inspect packaged files, and build those files under the declared support conditions. Configuration coverage should follow concrete promises.

Return to the encounter / Reveal debrief

Level 42 The Last Unstated Assumption

Keep three columns in your review: enforced by types, checked at runtime, and assumed from a protocol. Put each invariant in the correct column, then name its test or argument. A bounded queue belongs only to part of a memory bound; a local Result belongs only to part of an outcome protocol. Use the fake peer's independent effect log to expose that distinction.

Return to the encounter / Reveal debrief

The Debrief Archive

Compare arguments after producing your own evidence.

Level 01 The Double Delivery

Give the capsule an owned payload and keep its fields private. Inspection borrows it; editing requires exclusive access; submission takes it by value. The dock can return Result<Receipt, (Capsule, RejectReason)>, so both outcomes assign responsibility explicitly. On success, the dock stores or consumes the payload and the caller keeps only a receipt. On rejection, the error owns the capsule and can return it to the correction path. The compiler prevents reuse of a moved capsule, but it does not know whether an external destination received anything. Keep that distinction in the receipt's contract. A second constructed capsule with the same identifier is a separate replay problem, so private construction or external deduplication may be needed when the model expands.

Return to the encounter / Record your evidence

Level 02 The Partial Inventory

Choose the branch before destructuring the owned shipment. Each branch can then move exactly the fields it needs into a variant whose fields describe the surviving inventory. The old shipment is partially moved only inside a branch where no later operation requires the whole value. Cleanup remains attached to each initialized value's actual owner; a move does not create a second destructor obligation. A custom Drop implementation changes the argument because its destructor receives the complete aggregate and may inspect every field. A cleaner alternative is to place cleanup responsibilities in the individual owned field types, remove unnecessary aggregate Drop, or expose an explicit consuming transition that preserves a valid representation. Compiler-generated cleanup is conditional on initialization state, not on an invented placeholder value.

Return to the encounter / Record your evidence

Level 03 The Two Crews

Validate each range's ordering and bounds before creating selected mutable slices. Choose a simple empty-range policy, such as rejecting empty requests, to keep the first implementation's overlap rule unambiguous. Sort the two validated ranges by start position while remembering the caller's order. Require the earlier range's end to be at most the later range's start. Split at the later start; the earlier selected range belongs to the left slice, and the later selected range begins in the right slice. These independently borrowed slices carry structural nonoverlap into the callback. Reverse the callback arguments when the original requests were reversed. Runtime validation establishes the geometry; split_at_mut turns that established geometry into the safe access shape. A later version may support empty ranges with an equally explicit policy.

Return to the encounter / Record your evidence

Level 04 The Smuggled Page

The inspection function owns the decoded vector. A higher-ranked bound can require the callback to accept a record slice for the borrow lifetime chosen inside the function, while the result type R remains independent of that quantified lifetime. Owned counts, copied identifiers, or other owned summaries can leave normally. A reference into the local vector cannot become R because that would require the result type to depend on a lifetime that exists only inside the callback invocation. This does not prohibit all borrowed results: a callback may return a reference captured from a longer-lived caller owner when R already contains that valid relationship. Quantification protects the temporary dependency, not every conceivable reference. A direct function returning a view into its own destroyed vector would still lack a surviving owner.

Return to the encounter / Record your evidence

Level 05 The Portal That Both Takes and Gives

A shared reference preserves substitution direction, so a read-only view can generally shorten an embedded usable lifetime when the surrounding relationship permits it. Function arguments reverse that direction: an operation able to accept every sufficiently short input can serve a more restrictive caller, while an operation requiring a static input cannot serve arbitrary temporary inputs. In a function pointer using the same lifetime parameter in both input and output, the parameter receives opposing variance requirements and becomes invariant. That does not prevent borrowing the wrapper itself for a short period; it prevents substituting a different embedded relationship. A for<'a> function pointer states a separate universal capability across calls and must be analyzed as quantification rather than silently treated as the same type. Concrete rejected witnesses expose which promise a substitution would break.

Return to the encounter / Record your evidence

Level 06 The Interrupted Replacement

Keep the currently published configuration owned by the service while the callback builds a separate candidate. An ordinary preparation error drops only temporary state and returns before publication, so the old configuration remains available. A reservation guard can release a logical resource during normal drop or unwinding, but safe forgetting may leak that resource; represent the leak as reduced availability rather than invalid access. Publication should be a small step without user callbacks. For ordinary owned strings or another constrained representation, this makes the error argument clear. A fully generic replacement also destroys the old value, and an arbitrary destructor can panic or perform side effects. Therefore do not promise universal rollback merely because preparation was staged. Distinguish the exact Result error contract, unwind behavior, and conditions outside the process-local model.

Return to the encounter / Record your evidence

Level 07 The Reentrant Revision

Copy the current text and generation under a short shared borrow, then release that guard before calling user code. The callback works from the owned snapshot, so reentrant access does not conflict with a still-active document borrow. Reacquire exclusive access only for validation and publication. Compare the generation before replacing text, and reject stale candidates while preserving the newer document. Compute the next generation with checked arithmetic before mutation so exhaustion cannot create an unversioned change. RefCell enforces temporary access compatibility; it does not track whether a proposal was derived from current state. The generation is that separate semantic protocol. Returning an owned rejected candidate supports inspection, but a retry must reconsider the new document and any external effects already performed by the callback.

Return to the encounter / Record your evidence

Level 08 The Compaction Mimic

Separate the handle namespace from the dense position. The slot table validates the handle generation and yields the current dense index; a reverse mapping stored with each dense record identifies which slot to update when that record moves. During swap removal, update the moved survivor's slot before making the operation available to later lookups. Mark the deleted slot absent and advance its generation before permitting reuse. If the generation is exhausted, leave that slot retired rather than wrapping into an old identity. Cross-atlas safety requires an owner identity or a scoped brand if it is part of the public guarantee. None of this keeps a record alive after a lookup borrow ends; handles defer access checks, while a returned reference resumes Rust's ordinary borrowing constraints.

Return to the encounter / Record your evidence

Level 09 The Charter of Excessive Demands

A planner with one output relationship can expose associated Plan and Error types. A policy parameterized by its input can implement Policy<PlanA> and Policy<PlanB> independently, because the input choice participates in selecting the implementation. The pipeline requires the policy to accept the particular associated plan produced by its planner. Its reporting variant adds Display for that associated plan; the underlying planner trait need not exclude nonprintable plans. If one planner type truly participates in several independent input relationships, a trait parameter may identify those relationships while associated types describe each relationship's results. Decide this from the intended implementation space. Trait signatures are evidence available to generic code, and a useful implementation body cannot supply an obligation omitted from its visible bounds.

Return to the encounter / Record your evidence

Level 10 The Shapeshifter Registry

Expose a command method whose parameter and result can be understood without the concrete implementor. A shared outcome enum makes the dynamic result contract explicit. Store Box<dyn Tool + 'host> when owned heterogeneous storage is useful, allowing a tool's captured references to remain tied to host configuration. A temporary &mut dyn Tool can instead point to a concrete local tool and requires no box allocation. Generic configuration and concrete Self construction do not have to be dynamically callable: move them to constructors or extension APIs, or exclude appropriate methods with Self: Sized. Erasure preserves the stated lifetime and auto-trait bounds; it does not create missing ones. Rust trait-object dispatch is an in-program abstraction and does not establish a stable ABI between independently versioned foreign modules.

Return to the encounter / Record your evidence

Level 11 The Commutative Gate

With a local Vector<T> self type, your crate has the local implementation anchor required for the intended foreign trait relationship, subject to the remaining rules. In the reversed generic form, an uncovered T appears as Self before the local Vector<T> is encountered, so the mere presence of that local operand does not authorize the implementation. Making Scalar<T> a local self type restores a clear local anchor and gives callers an explicit adapter. Implementing particular primitive scalar cases can also avoid the uncovered generic parameter issue, at a cost in supported cases and maintenance. Neither technique exempts the crate from overlap checks. Review blanket implementations as occupied design space, and keep an ergonomic free function available when operator notation would force an unnecessarily restrictive public commitment.

Return to the encounter / Record your evidence

Level 12 The Retrying Oracle

Repeated invocation with a mutable attempt counter needs FnMut. The closure may still own its configuration because reading an owned capture does not consume it during each call. A closure that moves a one-shot resource out when called generally offers only FnOnce for that behavior, so retries require a new resource factory, a different ownership protocol, or abandoning automatic repetition. Fn is not a purity marker: interior mutability and external operations can create effects behind shared access. Returning an owned proposal plus an effect description makes speculative work easier to repeat without immediately performing those effects. Execute the description only after the local acceptance point, while separately handling interruption and duplicate-delivery concerns if execution reaches another component. The closure bound describes callable access to the environment, not the whole retry proof.

Return to the encounter / Record your evidence

Level 13 The Vanishing Collection

The shared interface describes a view family parameterized by each receiver borrow, with the necessary Self: 'a relationship for that family. A generic consumer finishes inspecting one view and converts the needed information into an owned summary before requesting another. This preserves the scratch reader's right to reuse its buffer. Collecting all those borrowed views would retain overlapping access across later mutation and cannot be promised by the common abstraction. An owned collection copies or transfers retained bytes; a specialized stable-source API can instead return views tied to an external immutable owner whose lifetime is independent of a short cursor borrow. An ordinary associated type can already vary by implementation. A GAT lets each implementation choose a view type that also varies with the lifetime of each call.

Return to the encounter / Record your evidence

Level 14 The Ambiguous Import

Use domain variants or another structured classification to identify the phase and disposition while retaining source errors when cause traversal is useful. Keep parsing, policy checks, and proposal construction separate from publication so an ordinary preparation error leaves the published in-memory state unchanged. Return owned rejected inputs when callers can correct them; a prepared proposal rejected for conflict may remain inspectable but must not be applied blindly to new state. Its earlier reads and policy decision can be obsolete. Render the final human message at the reporting boundary while preserving classification beneath it. Error is an outcome category, not an automatic rollback or retry guarantee. Tests should compare state before and after each failure and verify that recovery values contain the resources the contract promises.

Return to the encounter / Record your evidence

Level 15 The Receipt That Never Happened

Validation belongs at the constructor that first establishes the facts. An approved ticket can own a checked destination and payload, with shared inspection and consuming transition methods. A pre-handoff error may carry that same approved value because no delivery attempt was accepted. After handoff, return a different outcome containing an operation identifier and whatever status evidence exists. That value must not silently convert back into permission to issue a fresh operation. A receipt requires explicit confirmation under the chosen transport contract. Typestate is useful for the local phase changes; an enum is often simpler for branching outcomes and heterogeneous storage. Private fields make those types credible evidence instead of decorative names.

Return to the encounter / Record your evidence

Level 16 The Semver Mimic

Start from the client evidence rather than from the proposed implementation. A non-Send scratch field can remove a previously available transfer capability even behind private fields. Iterator changes can restrict how long a view is usable or which other borrows may coexist. A plugin trait affects implementors as well as callers, and erased use depends on dyn compatibility. Preserve those properties with an internal scratch strategy, a precise borrowing surface, and an extension mechanism that does not silently add impossible obligations. Where preservation is impractical, record the deliberate break and release it accordingly. Client compilation tests catch important regressions, while documentation and behavioral tests still carry promises such as ordering, allocation, and failure effects.

Return to the encounter / Record your evidence

Level 17 The Mirror Expansion

Bind acquisition once in a scope whose lifetime is visible in the expanded code. Match its error path explicitly or use propagation according to the chosen control-flow contract. An introduced closure changes what return and question mark target, so that is API behavior rather than harmless implementation detail. The guard should be dropped on the exits supported by the ownership model; attempting to return a reference to a guard that dies with the expansion must be rejected. Qualify defining-crate helpers appropriately and ensure an external expansion can legally access them. Test the actual downstream context instead of relying on a unit test that inherits local imports. A readable expansion is part of the earned artifact.

Return to the encounter / Record your evidence

Level 18 The Capacity Doppelganger

Keep capacity in the array type and actual content length in a checked field. Validate the incoming slice length before copying, then update the used length after the copy; ordinary byte copying introduces no user callback that could invalidate the staged argument. Expose only the used prefix. For encoding, convert the runtime length to the protocol's fixed-width field and use checked addition for the header size in usize. A const function is eligible for compile-time evaluation when called in a const context, while runtime inputs still require runtime computation. Zero capacity naturally admits the empty payload. Separate header and payload or write into caller-provided output rather than requiring unsupported generic array-length arithmetic.

Return to the encounter / Record your evidence

Level 19 The Unreturning Survey Team

Each worker borrows a contiguous region of the caller-owned catalog and returns references into that region. Its local vector owns only the collection of references, not the referenced strings, so moving or consuming that vector does not invalidate the names. Join handles in partition order and append their results in that same order. The scope establishes that workers finish before the function returns, while the returned references remain tied to the input lifetime. Clamp or reject worker count according to an explicit policy, avoid empty partitions when unnecessary, and propagate or represent panic deliberately. Scoped lifetime validity and Send obligations still matter independently; neither justifies claiming fairness or equal work across partitions.

Return to the encounter / Record your evidence

Level 20 The Bell Without a Forecast

Store revision, snapshot, and phase together because they define one observable state. A waiter holds the mutex while checking whether its target is satisfied or shutdown makes it impossible; otherwise Condvar::wait atomically releases the guard and waits, then the loop checks again. Clone the snapshot handle while protected and process it after releasing the guard. Publication changes the predicate before notification, so an early notification is harmless: a later reader sees the ready state. Stopping changes the phase under that same lock and wakes all blocked readers. Checked revision advancement avoids accidental history reuse. Poison recovery needs an invariant-based decision; mechanically taking the inner value is not evidence that interrupted state is trustworthy.

Return to the encounter / Record your evidence

Level 21 The Half Delivered Decree

Represent the fanout as separate ownership transfers with a shared operation identity. A successful send establishes that one queue accepted its message; a failed send returns the undelivered copy and says nothing that undoes a prior success elsewhere. A per-recipient report preserves that distinction. If the relay disappears before reporting, the caller has uncertainty and needs recipient deduplication, status lookup, or an authoritative log to retry safely. A blocking policy can couple throughput to the slower tower; a nonblocking policy needs explicit retry or rejection behavior. Shared immutable payloads avoid duplicate byte copies but retain their allocation until the final relevant owner releases it. Atomic broadcast would require additional coordination and failure assumptions beyond these channels.

Return to the encounter / Record your evidence

Level 22 The Counter Hydra

In the SeqCst total order, an accepted reader has matching even generation observations around its payload loads. A writer's odd marker precedes its payload stores and its final even marker follows them. Without generation reuse, an intervening publication would change a generation observation or expose odd state, so acceptance excludes an overlapping write across the observed interval. This proof depends on every relevant operation participating in the stated ordering and on the single-writer discipline. Precheck that both required generation steps fit before changing any state. A finite retry budget gives a useful Busy result under contention; it does not guarantee that a later attempt succeeds. The protocol orders observations but does not retain an independently allocated pointee.

Return to the encounter / Record your evidence

Level 23 The Endless First Poll

The owned buffer, cursor, accumulator, and terminal policy are the suspended computation. Since the representation contains ordinary movable fields and no address-sensitive self references, safe mutable access through the applicable Unpin relationship is sufficient here. Process one bounded slice, update state, and either finish or wake the current task before returning Pending when more CPU work is already available. A Pending without a future notification can strand the computation. Destruction releases the stored Vec whether the body was polled zero times or many times; cleanup cannot rely on reaching the successful final branch. This exercise makes cooperation explicit while leaving scheduling fairness to the executor and its own work-budget policy.

Return to the encounter / Record your evidence

Level 24 The Twin Room Collapse

Box::pin establishes independent stable storage for each child. Pin<Box<F>> is a movable owning handle even when F needs pinning, so the parent can move handles without moving the child values out of their allocations. An Option lets completion take and drop those handles safely. Poll children through pinned mutable projections obtained from the handles, not unrestricted references to their pointees. Store the parent's terminal status before a later poll could touch a completed child. A left-first polling policy is biased and should be documented or deliberately alternated. Dropping the loser performs its local destruction policy, while cancellation semantics for its remote work remain a separate contract. The extra allocations purchase a smaller and auditable proof boundary.

Return to the encounter / Record your evidence

Level 25 The Unrung Completion

Check readiness and register interest under the same mutex. Poll takes a ready value, reports closure, or installs the current waker. The sender stores readiness or closure under the lock and takes the registered waker to invoke after unlocking. Receiver drop marks abandonment and takes both the retained payload and the registered waker under the lock. Drop them after unlocking because either destructor can run arbitrary code. Clone the incoming waker outside the lock and drop any displaced waker after unlocking. Keep readiness or closure in state even with no waker registered so a later poll can observe it.

Return to the encounter / Record your evidence

Level 26 The Vanishing Cursor

The exporter owns both the bytes and progress, so destroying one borrowed attempt releases access without losing the resumable record. After the sink returns an accepted count, update the owner's offset immediately in the same poll execution; cancellation occurs when the future is dropped between polling calls, not at an arbitrary source instruction inside a running poll. Pending leaves the acknowledged offset unchanged under the chosen sink contract. Define zero accepted bytes with nonempty input as a no-progress error rather than spinning. An unfinished state permits resume or explicit abort and prevents another frame from interleaving. Completing local transmission can clear the pending frame, but a lost response still leaves application-level outcome uncertain. That uncertainty requires a peer protocol or stable operation identifier beyond local cancellation safety.

Return to the encounter / Record your evidence

Level 27 The Contract Hydra

A static method can return impl Future with an explicit Send bound, making the operation promise available to generic callers. The erased adapter can return Pin<Box<dyn Future<Output = Report> + Send + 'a>>, tying the future to the receiver and input borrows that it actually retains. The scanner object's bounds describe transfer or shared access to the scanner, while the future's bounds describe suspended operation state. Neither implies static lifetime. For an unscoped handoff, ensure the retained scanner and request contain no shorter borrowed state, then move those owners into an outer async block. That wrapper can borrow its owned data internally while its complete future owns the backing storage. An owned handle can still contain nonstatic borrows, so establish the required static bounds explicitly. Erasure and ownership handoff remain separate decisions.

Return to the encounter / Record your evidence

Level 28 The Supervisor That Declared Victory

Use one authoritative admission transition to reject new work or register its identifier before it can become active. A work envelope owns its capacity through the stage the bound is meant to cover. The supervisor retains completion handles and gathers their results, including panic or explicit failure, while continuing to drain any channels needed for worker progress. Terminal accounting can partition accepted identifiers into completed, ordinary failed, locally abandoned before effects, and externally uncertain, with more categories when useful. A worker failure is not automatically proof that its active operation had no effect. Deadline escalation changes the shutdown result rather than retroactively turning it into a clean drain. Report local termination separately from remote certainty and persistent recovery obligations.

Return to the encounter / Record your evidence

Level 29 The Mimic Packet

The format is a sequence of specified bytes, so parse fields by explicit ranges and integer conversion functions. Read the fixed header only when it is available, decode the length, calculate total size with checked arithmetic, and wait for the entire payload. Validate the kind before returning the message. Encoding performs the inverse after checking representable length. Independent fixtures catch paired encoder-decoder mistakes. repr(C) describes certain target layout relationships; it does not remove padding, choose protocol byte order, or make arbitrary bytes a valid Rust value. Keep an ordinary message model separate from the wire format and document whether unknown kinds are errors or future extension points.

Return to the encounter / Record your evidence

Level 30 The Overlapping Observatory

A wrapper around an exclusive slice can delegate checked splitting to split_at_mut and preserve the borrow directly. Consuming the parent prevents an independent parent capability from surviving alongside its children. Return the parent on a failed consuming split, or choose a clearly documented alternative. A pointer implementation adds obligations without changing the desired contract: each nonempty region must stay within the original live allocation, while an empty view still needs a nonnull aligned pointer but need not have backing storage, and mutable regions must remain disjoint. A lifetime parameter records the borrow relationship but cannot create it from a naked address. Prefer the safe representation unless a concrete requirement justifies the additional implementation proof.

Return to the encounter / Record your evidence

Level 31 The Friendly Corruptor

ASCII bytes are valid UTF-8, but that implication helps only while all stored bytes remain ASCII. The original replacement method can store a non-ASCII continuation byte and make the unchecked conversion violate its contract. Validate both index and new byte before mutation, and apply the same policy to append and bulk replacement. A private field narrows access but does not repair an unsafe public mutation method. A safe conversion or a String-based representation may remove the unsafe step entirely. If retaining it, explain the invariant beside the type and conversion, and review future methods against the same statement. Runtime tests support this reasoning but do not establish soundness for arbitrary future call sequences.

Return to the encounter / Record your evidence

Level 32 The Ninth Fragment

The safe model makes empty and full slots explicit. A MaybeUninit version needs the same logical information in a separate map; total count alone cannot authorize reading a particular index. On insertion, write the new value before marking an empty slot initialized. On transfer or removal, revoke the old slot's ownership before normal cleanup can revisit it. finish checks every slot, then transfers values without leaving a second destruction obligation behind. Error handling must retain or clean precisely the initialized slots. If an element destructor panics, a simple loop may leak later elements; that limitation must not permit reading uninitialized storage or dropping an already transferred owner. Optimize only after the safe model and invariant are clear.

Return to the encounter / Record your evidence

Level 33 The Borrowed Bridge Toll

An opaque handle keeps Vec layout and allocator details private. A successful creation transfers one ownership obligation, while an accessor temporarily borrows the result without transferring its allocation. The bytes remain valid only while the owner lives and the documented mutation restrictions hold. The matching release path consumes the obligation exactly once; foreign misuse remains an explicit caller contract, not something a pointer check can generally detect. Validate lengths and status paths before forming Rust slices, with documented provenance and lifetime requirements for foreign input. Define panic handling for exports and preserve outputs on failure. A Rust RAII wrapper can encode the matching release, but its unsafe constructor must receive a genuinely valid owned handle.

Return to the encounter / Record your evidence

Level 34 The Three Index Hydra

One owner should define the authoritative station data. Secondary indexes store IDs or handles and must be updated under the same logical mutation protocol. A rename can prepare new owned data before committing coordinated changes; generic panic and allocation assumptions still need a stated policy. Sorting on demand can avoid a permanently maintained tree when listings are rare, while a maintained ordered index can reduce repeated sorting when they dominate. Deletion revokes lookup identity, but externally retained Arc payloads may stay alive. Recent-event capacity measured only in entry count does not bound bytes for arbitrary payloads. Benchmark update cost, listing cost, retained memory, and the actual mixed workload before choosing the extra indexes.

Return to the encounter / Record your evidence

Level 35 The Tournament of Unequal Rules

Begin by making the two implementations compute the same observable result on the same inputs. Separate setup from measurement consistently and prevent the intended work from becoming irrelevant to the optimized program. The isolated dispatch test answers a narrow question; a full router test reveals interactions with payload work, instruction-cache pressure, allocation, and batching. Static specialization can increase code size and build effort while improving some runtime paths, so record those dimensions rather than hiding them in one score. Repeat measurements under a documented configuration and investigate variance. A profile identifies where time is observed, while changing one suspected cause and measuring again supplies stronger evidence for that explanation.

Return to the encounter / Record your evidence

Level 36 The Missing Job

Start with a model whose state changes are simpler than the implementation: each job identity has a known owner or terminal classification. Compare those observable transitions rather than copying the production queue code into the test. A barrier or test channel can establish that acceptance occurred before the injected fault; another releases the blocked participant. This makes one intended history reproducible, not every possible schedule. Compile checks can establish a bound such as Send for a returned future, while runtime histories test accounting and shutdown. Miri or another dynamic tool may find executed memory violations, but neither a clean run nor model agreement proves all unsafe behavior sound or an external operation exactly once.

Return to the encounter / Record your evidence

Level 37 The Stale Span Doppelganger

An immutable snapshot can own one source string and its validated ranges. A temporary view borrows that snapshot; an independently retained record can carry an ownership share of the exact snapshot plus a record index. Moving handles leaves ranges meaningful because resolution uses their associated owner. If spans travel independently, attach and validate a snapshot identity rather than assuming equal numeric offsets imply equal text. Editing constructs a new snapshot instead of mutating A beneath its readers. Owned diagnostic positions survive rejection without granting a valid record. Sharing avoids copying every field but retains the complete old source, so selective copying may still be the best policy for a few long-lived search results.

Return to the encounter / Record your evidence

Level 38 The False Inner Commit

A savepoint layer is a proposal overlay, not a new published store. Reads search from the newest layer outward, distinguishing a tombstone from absence in that layer. Rollback discards the child's owned changes. Merge moves those changes into its parent while leaving the same base snapshot and outer validation requirements intact. Only a successful outer publication advances the store version and replaces visible state. If a background writer invalidates the base, rejecting the outer transaction discards the whole combined proposal; its historical snapshot remains valid for reading. Retain the identity check that prevents a transaction from another store passing on an equal numeric version. Nested structure does not make external effects reversible.

Return to the encounter / Record your evidence

Level 39 The Silent Result Chamber

Dropping the collector's receiver causes workers' pending or later result sends to fail. Workers then exit their input loops and drop receivers, allowing a coordinator blocked on an affected input send to observe disconnection. This progress argument depends on active computations eventually reaching that send or another shutdown check. Drop coordinator input senders and retain panic payloads until all joins finish. A send error returns the specific undelivered message. Track earlier accepted jobs by ID and classify their outcomes separately. An atomic cancellation flag alone does not wake a blocked send or recv. Keep any final reporting path bounded by a justified amount or independently drainable.

Return to the encounter / Record your evidence

Level 40 The Echo That May Already Exist

A local state machine can prevent phase confusion and preserve the operation ID, but it cannot infer an unseen remote outcome. After an ambiguous disconnect, move to an unknown state. Querying status or retrying with the same ID is useful only under a peer contract that retains and recognizes the necessary history. Expiration may restore uncertainty rather than prove nonexecution. Confirmed rejection before execution is a different outcome and can permit a new attempt under the stated policy. Use typed capabilities when a transition genuinely removes an operation, and runtime alternatives when a manager must hold mixed states. The fake peer should independently record effects so tests can detect duplicate execution behind lost replies.

Return to the encounter / Record your evidence

Level 41 The Release Candidate Hydra

Downstream fixtures test the interface properties callers rely on without naming anonymous future types. A helper requiring Send can detect the Rc regression; a borrowed-input fixture separately tests lifetime flexibility. Narrower bounds and exhaustive error changes need compatibility review rather than being dismissed as internal edits. Minimal configurations expose imports hidden by default or maximal builds. Package inspection and building from the package detect workspace-only assumptions. Declare generated inputs and distinguish host execution from target properties in build scripts. The compiler baseline is a separate tested dimension. Release notes should explain changed caller obligations and behavior, while avoiding claims of universal compatibility based only on one preferred configuration.

Return to the encounter / Record your evidence

Level 42 The Last Unstated Assumption

A strong solution makes each boundary explicit rather than maximizing features. Parsed views resolve against their owning snapshot. Admission transfers a job identity and the resources needed to account for it. Store publication validates the right origin and version before committing prepared state. Worker and collector shutdown release the channel owners needed for progress. The fake peer records effects independently, so a lost reply produces an unknown outcome until the chosen protocol resolves it. Tests attack these claims with deliberate histories. Types prevent particular invalid accesses and transitions; runtime checks enforce lengths, versions, and budgets; peer assumptions govern remote semantics. The final argument should state those limits directly and connect every claimed guarantee to actual evidence.

Return to the encounter / Record your evidence

Consequences in the Branching Rooms

The Vanishing Archive / Choice 1

The borrowing relationship prevents reuse while the slices remain usable. That compiler error identifies the incompatible storage policy. Extending annotations does not make the buffer independent. This choice works if consumers finish before reuse or if you change the owner and stop reusing this allocation; it does not meet the requirements as stated.

Return to the room / Explore Level 4

The Vanishing Archive / Choice 2

Shared ownership keeps the original bytes alive for views or validated ranges. It also prevents overwriting those bytes while readers still need them, and one tiny identifier retains the entire 256 MiB. This design fits a different contract that allows a separate receive buffer and makes dense retention worth its memory cost.

Return to the room / Explore Level 8

The Vanishing Archive / Choice 3

The queue owns compact strings independent of the batch. This deliberately spends allocations and byte copies to release the much larger source and permit reuse. Measure selected-byte volume and allocator costs, but compare them against peak retained memory rather than declaring zero-copy the winner by definition.

Return to the room / Explore Level 37

The Ghost Payment / Choice 1

A new identifier can make the retry look like a second purchase. Dropping the future did not reverse the remote effect. This action is suitable only if duplicate execution is acceptable or another layer prevents it; the stated purchase invariant does not permit that assumption.

Return to the room / Explore Level 26

The Ghost Payment / Choice 2

A stable key is useful only under the peer's documented scope, retention period, payload matching, and failure behavior. If the record expires before retry or different payloads share a key, your local type cannot repair the protocol. First establish the remote contract rather than treating the presence of a key field as a guarantee.

Return to the room / Explore Level 40

The Ghost Payment / Choice 3

Keep the original operation identity and the exact request meaning. Consult the peer's status and idempotency behavior, then retry or reconcile according to that contract. An inconclusive lookup may leave the outcome uncertain. This is an explicit protocol state, not a reason to manufacture a success or failure locally.

Return to the room / Explore Level 28

The Hollow Lock / Choice 1

The recursive call can attempt to acquire the mutex already held by the loader. A global lock also serializes unrelated keys. This does not satisfy the stated reentrancy and contention requirements. A non-reentrant contract could be a different design, but it must be explicit and still needs an argument for lock duration.

Return to the room / Explore Level 20

The Hollow Lock / Choice 2

This removes the global lock from user code and can be a sound, useful cache policy. It also allows duplicate loads for the same key, so it fails the one-active-loader requirement as stated. If duplicate pure work is acceptable, this simpler policy may be the right tradeoff; external effects need additional care.

Return to the room / Explore Level 42

The Hollow Lock / Choice 3

Publish ready or failed and wake waiters even if a load panics or is abandoned. Run loaders outside the global lock and impose a fixed key order: a loader may request only higher-ranked keys. Reject other nested requests before starting or waiting on a load. This prevents self-waits and cross-key cycles such as A waiting for B while B waits for A.

Return to the room / Explore Level 25

Appendix A A Diagnostic Reasoning Guide

Compiler diagnostics are most useful when translated into a missing relationship. The entries below describe recurring advanced situations. They are not a replacement for the exact diagnostic or the smallest reproducing example. Use them to decide which design question to answer before changing a type.

A Mutable Borrow Appears to Last Too Long

First identify the last use of the value derived from the borrow. A returned reference, iterator, guard, or closure can keep access active after the method call itself ends. If the derived value is needed later, ending the borrow earlier would invalidate that use. If it is not needed, a smaller scope or explicit drop can make the intended boundary clear. Do not assume lexical block size is always the borrow's actual duration; nonlexical lifetime analysis follows uses and constraints.

A second possibility is an overly broad method signature. A helper taking `&mut self` claims access to the whole object even if its implementation touches one field. Borrowing disjoint fields directly or passing a narrower field reference can expose the independence the caller needs. The helper's body is not normally substituted into every call site for borrow checking.

A Reference Cannot Be Returned From Behind a Guard

Ask who prevents mutation or destruction after the guard is released. Returning the guard preserves synchronization for the caller's use. Returning a copied value or an `Arc` transfers independent ownership. Running a callback while the guard is held offers temporary access but introduces lock-duration and reentrancy concerns. Returning a bare reference after releasing the only protection generally loses the relationship that made access valid.

The same pattern appears with `RefCell` guards, mapped file views, transaction snapshots, and temporary owners. Identify the owner or

capability that must travel with the view. If no practical return type can carry it, redesign the API around an owned result or scoped operation.

A Spawned Task Requires Static Data

An unscoped task can outlive the stack frame that creates it. A borrow of a local variable cannot support that possibility. Move owned data into the task, share ownership with an appropriate `Arc`, or use a scoped concurrency mechanism whose lifetime boundary is enforced. A `'static` bound on a type means it contains no borrowed data that expires sooner; it does not mean its value must remain alive forever.

Adding `move` changes how a closure or async block captures its environment. It does not turn a borrowed reference into owned backing storage. A moved `&str` can still point into a local string. Follow the capture one level deeper and name the actual owner of the bytes.

A Future Is Not Send

Inspect the state retained across suspension points. A value that is used and dropped before an await may not enter the suspended state; a value kept across the await does. Common causes include `Rc`, non-`Send` guards, trait objects without required auto-trait bounds, and borrowed data whose referent does not satisfy the necessary relationship.

Decide whether thread migration is actually required. A local executor may accept a non-`Send` future. If migration is required, shorten the lifetime of the offending value, change ownership or synchronization representation, or adjust the trait contract that produces the future. Do not replace every local type with a synchronized equivalent unless shared access is part of the requirement.

A Trait Cannot Be Used as a Dyn Type

Inspect which operations require information unavailable through an erased receiver. Generic methods, unconstrained uses of `Self`, and associated items can prevent the desired dynamic interface. Sometimes a method can be excluded from dynamic dispatch with an appropriate `Self: Sized` condition. Sometimes the public trait should be split into a small erased core and a generic extension.

Associated types may need explicit choices in the trait object. Erasure removes the concrete implementing type; it does not remove every relationship the interface needs. An adapter can package a generic implementation behind a simpler object-safe operation when dynamic composition is a real requirement.

An Implementation Conflicts With Another Implementation

Rust must select a coherent implementation under its overlap rules, including implementations that a blanket rule covers. Adding a more specific implementation does not generally specialize a stable blanket implementation. A newtype creates a distinct type on which a different implementation may be legal. A policy parameter or explicitly named adapter can make the choice visible rather than relying on overlapping inference.

For orphan-rule errors, identify which trait or type is local and how generic parameters appear in the implementation. The rules are more precise than saying that one mentioned type is local. When designing a library, remember that a broad blanket implementation can reserve implementation space that downstream users might otherwise need.

A Lifetime Annotation Does Not Fix a Returned Temporary

An annotation describes a relationship among valid references. It does not extend the storage duration of a local allocation or temporary owner. If a function creates a string and wants to return its contents, return the owned string, place the storage in a caller-provided owner, or use another representation whose owner survives.

When the return value genuinely borrows from an input, make that relationship explicit enough for the compiler and caller. Avoid assigning one lifetime to every parameter merely to make the signature shorter. Unrelated inputs can then become unnecessarily coupled, restricting calls that should be valid.

An Iterator Cannot Yield the Borrow You Expected

Determine whether each item borrows from the iterator's original backing data or from the current mutable borrow of the iterator itself. Ordinary `Iterator` has one associated `Item` type for the implementation. A lending interface can use a generic associated type tied to each call, which prevents retaining one item while requesting another when the buffer is reused.

Changing to owned items is sometimes the right API, especially when consumers need to retain them. Collecting borrowed items from a reused buffer is not fixed by annotation because it conflicts with the storage policy. Decide whether the producer or consumer should own the retained bytes.

Pin Does Not Permit a Field Operation

Ask whether the field is structurally pinned and whether exposing a pinned projection is part of the type's contract. A pinned outer value does not automatically authorize arbitrary projection, replacement, or movement of its fields. `Unpin` changes which restrictions matter, and a pointer wrapper's movability is different from moving its pointee.

Use a documented safe projection facility where appropriate. If implementing projection with unsafe code, explain why the projected value remains pinned until its required destruction and why no other safe operation can move it. Address stability is a whole-abstraction promise, not a one-line cast property.

A Const Expression Is Rejected

Separate the syntax of a const context from the operations permitted during compile-time evaluation. A function callable at runtime is not necessarily callable in const evaluation. A const generic parameter does not make every arithmetic expression over that parameter legal in every type position on stable Rust.

Consider whether the computation truly belongs in the type system. A constructor that validates a runtime value may offer a simpler contract. If compile-time evaluation is required, use supported const operations and

represent constraints within the stable feature set rather than silently requiring nightly behavior.

A Safe Wrapper Still Needs an Unsafe Trait

If an implementation may cause memory unsafety when a caller supplies a trait implementation that violates a semantic rule, ordinary safe implementors cannot be trusted to uphold that rule for safety. Either validate defensively, avoid depending on that semantic property for memory safety, or place the obligation behind an appropriately documented unsafe trait boundary.

An ordinary trait may still specify semantic laws such as ordering consistency, and violating them may produce logic errors. The critical distinction is whether violating the law can make safe caller code trigger undefined behavior. Review every place the implementation converts semantic expectations into unchecked memory operations.

A Program Compiles but Hangs During Shutdown

Inventory every owner that can keep a resource live. A forgotten channel sender prevents a receiver from observing disconnection. A guard held while joining a thread may prevent that thread from acquiring its lock. A result queue that is no longer drained can keep workers blocked while the coordinator waits for them to exit.

Draw the wait graph and the ownership graph separately. Memory safety can hold while the wait graph contains a cycle. Define which component closes admission, which drains accepted work, which drops the final sender, and which observes completion. Test shutdown as a protocol rather than as incidental cleanup.

A Clone Fixes Compilation but Memory Grows

Inspect what was cloned. Cloning a `String` duplicates bytes. Cloning an `Arc` duplicates a handle and retains the underlying allocation. Cloning a snapshot can preserve an entire historical structure. A cheap clone may therefore be expensive in lifetime and peak memory even when its immediate CPU cost is small.

Measure live retained data under the actual workload. A small selective copy can release a much larger shared source. A bounded queue containing shared handles can still retain large allocations. Decide whether independent ownership, shared retention, or a narrower borrowed scope matches the consumer's requirement.

Further Reading

- Rust compiler error index explains individual diagnostics and their examples.
- The Rust Reference defines the language rules behind type and lifetime relationships.
- Standard library documentation gives the exact contracts of guards, ownership wrappers, and synchronization operations.

Appendix B A Glossary of Relationships

These entries emphasize the relationship a term describes. The topic index points to chapters that develop the corresponding rules, examples, and limitations.

Address Sensitive State

A state whose correctness depends on a value remaining at a particular memory location. Self references and intrusive links are examples. Pinning can support an address-stability contract, but the implementation must still establish and preserve the relevant invariant.

Affine Ownership

A useful description of ownership that permits a value to be consumed at most once through a particular noncopyable binding while also allowing it to be discarded. Rust does not generally require every value to be used exactly once or every destructor to execute.

Aliasing

Multiple access paths that can reach overlapping storage. Whether particular accesses are permitted depends on reference and pointer rules, interior mutability, liveness, and the operations performed. The mere existence of two addresses is not a complete aliasing analysis.

Associated Type

A type selected by an implementation of a trait for a particular implementation relationship. It lets an interface describe connected types, such as an iterator and its item, without making every use choose an independent type parameter.

Auto Trait

A trait whose implementation is often derived structurally from a type's components, subject to the trait's rules and explicit implementations. `Send`, `Sync`, and `Unpin` are central examples. Auto-trait behavior can become part of a public type's compatibility surface.

Borrow

Temporary access to a value under constraints that preserve valid use. A borrow relates the access to an owner and to potentially conflicting operations. It does not transfer the owner's ordinary responsibility for the borrowed resource.

Capability

A value or access relationship that grants a particular operation. An exclusive reference grants access under exclusivity constraints; a typestate value may grant permission to send protocol data. The interpretation is useful only when construction and duplication preserve the intended authority.

Coherence

The set of rules that allows trait implementation selection to remain unambiguous across crates. Overlap checks and orphan rules shape which implementations may exist. A broad implementation can restrict future implementation choices even when its current body is small.

Contravariance

A relationship in which substituting a subtype reverses direction through a type constructor. Function argument positions provide an important example. Variance describes permitted substitutions; it does not mean the compiler can arbitrarily lengthen the lifetime of actual storage.

Covariance

A relationship in which subtype substitution is preserved through a type constructor. Shared references are covariant over relevant parameters. This supports shortening a usable lifetime when appropriate, while preserving the requirement that the underlying value remains valid.

Drop Check

Compiler analysis that considers whether borrowed data may still be accessed when an owning value's destructor runs. The fact that ordinary methods no longer use a reference does not always end the relationship relevant to destruction.

Drop Flag

A conceptual or implementation mechanism for tracking whether a place currently contains an initialized value requiring destruction. Conditional initialization and partial moves require cleanup behavior to depend on which values actually exist at the cleanup point.

Dyn Compatibility

The conditions under which a trait can support a trait-object interface. Erased dispatch must have enough information to invoke permitted methods without knowing the concrete implementing type. Some trait operations cannot be represented through that interface.

Erasure

Hiding a concrete type behind an interface that preserves selected operations and bounds. A trait object erases the implementor while retaining its trait contract, associated choices, lifetime constraints, and requested auto traits. Erasure is a design boundary rather than the absence of types.

Exclusive Access

Access whose validity depends on restricting conflicting access paths for the relevant duration. `&mut T` is the principal safe-language expression of this relationship. Exclusivity can matter even when an operation performs no write.

Future

A value representing a computation that can be polled toward completion. It may retain local state and borrows between polls. Creating a future does not generally execute its async body to completion or assign it an operating-system thread.

Generic Associated Type

An associated type that has its own generic parameters. A lending interface can use a lifetime parameter to connect each returned item to the particular borrow of the producer used by that call.

Happens Before

An ordering relationship used to reason about effects across threads, assembled from the relevant sequencing and synchronization rules. Atomicity alone does not establish every required edge. An ordering proof must identify the actual operations that publish and observe state.

Higher Ranked Bound

A bound quantified over a family of lifetimes, commonly written with `for<'a>`. It can require a callback or trait relationship to work for every lifetime selected under that quantification rather than one lifetime chosen by the caller.

Interior Mutability

Mutation made available through shared access under an abstraction's rules. `UnsafeCell` supports the core language exception, while types such as `Cell`, `RefCell`, and synchronization primitives impose different safe access protocols. Shared access does not thereby authorize arbitrary unsynchronized mutation.

Invariance

A relationship that does not permit subtype substitution through the relevant parameter. Mutable access and interior-mutability representations often require invariance over stored types so one lifetime or type choice cannot be replaced by an incompatible one.

Lending

An interface pattern in which an output borrows from the producer's current access, often preventing another operation while that output remains in use. It supports reusable internal storage whose contents cannot safely back several simultaneously retained items.

Lifetime Bound

A constraint on which borrowed data a type or reference may contain and how long it may be used. A lifetime annotation expresses a relationship; it does not allocate storage, extend a local variable's existence, or run a destructor.

Monomorphization

Generating specialized code for concrete generic instantiations. It can expose optimization opportunities and increase code size or compilation work. The resulting machine-code cost depends on optimization and usage rather than on generic syntax alone.

Nonlexical Lifetime

A borrow-analysis behavior in which relevant uses and control-flow constraints can determine the borrow's duration more precisely than the enclosing lexical block. It explains why an owner can sometimes be mutated after a reference's last use within the same block.

Pinning

A contract that constrains movement of a pointee once the relevant promise is established, including obligations around destruction. Moving a pinned pointer handle is different from moving its pointee. Projection determines which fields participate in structural pinning.

Place

An expression category describing a storage location, such as a local variable, field, or indexed element. Moves, borrows, assignments, and partial initialization operate on places. Distinguishing a place from a produced value clarifies many ownership rules.

Poisoning

A synchronization mechanism that records a possible invariant interruption after a panic while protected state is accessed. It is advisory evidence for a recovery decision. Clearing or ignoring poison does not itself repair the application's invariant.

Provenance

Information associated with pointers that concerns their origin and permitted relationship to allocations and accesses. An address alone is not a complete description of permission to dereference. Exact pointer operations must follow their documented contracts and Rust's evolving memory-model requirements.

Reborrow

A borrow created through an existing reference, often with a shorter usable duration. It can temporarily grant access while restricting use of the original reference. This allows mutable references to participate in repeated calls without being permanently consumed each time.

Representation

The layout and ABI-related choices for values, including size, alignment, field placement, and discriminants where specified. A representation attribute provides particular guarantees; it does not automatically validate bytes or make an arbitrary Rust type safe for every foreign interface.

Sound Abstraction

An abstraction whose safe use cannot cause undefined behavior when its unsafe implementation's obligations and documented unsafe caller contracts are respected. Soundness is stronger than passing tests and different from being free of ordinary logic errors.

Structural Pinning

A choice that a field participates in its enclosing type's pinning guarantee. Exposing a pinned projection commits the implementation to preserving that relationship across mutation, destruction, and `Unpin` behavior. It is not an automatic property of every field.

Subtyping

A relationship permitting one type to be used where another is required under language rules. Lifetime subtyping and variance are important in Rust. Subtyping should not be confused with inheritance of data fields or arbitrary implicit conversion.

Typestate

Representing a resource's protocol phase through types so different operations are available in different phases. Consuming transitions can prevent reuse of an old capability. Runtime state machines remain useful when states must be stored or selected dynamically.

Undefined Behavior

Behavior outside the program's permitted language and library contracts, invalidating ordinary reasoning about the resulting execution. Unsafe blocks do not make undefined behavior acceptable. Preventing it requires satisfying all relevant validity, access, lifetime, concurrency, and boundary obligations.

Unpin

An auto trait indicating that a type does not need the pinning restriction for soundness. It allows safe pinned access to recover ordinary mutable access in applicable APIs. It concerns the pointee's requirements rather than whether a pointer handle itself can move.

Validity

The requirements for bytes and metadata to constitute an allowed value of a type. A valid allocation and sufficient size do not imply a valid `bool`, reference, enum, or string. Creating a typed value may impose obligations before any later semantic use.

Waker

A handle through which a task can be scheduled for another poll after relevant progress becomes possible. A wake is a notification, not the result itself and not necessarily a one-to-one request for an immediate poll. Correct registration must avoid lost notifications.

Zero Sized Type

A type whose values require zero bytes of storage, though the type still has alignment and semantic behavior. Such values can participate in ownership, destruction, and generic algorithms. Zero size does not justify ignoring the contracts of references, pointers, or allocators.

Topic Index

Page numbers identify field manual openings. Each entry links to the relevant field manual.

ABI and foreign boundaries 330, 376

Allocation and retained memory 94, 389, 400, 427, 439, 450

Associated types and GATs 106, 152

Async interfaces 261, 305

Atomics and happens before 251

Backpressure and admission 242, 316, 450

Borrow splitting and reborrowing 39, 427

Cancellation and partial progress 294, 316, 450, 461

Cell RefCell and UnsafeCell 82

Closures and capture 141

Coherence and orphan rules 130, 186

Collections and hashing 389

Const evaluation and const generics 208

Destruction and panic safety 28, 71, 365

Dispatch and type erasure 118, 305

Errors and recovery 163, 439, 461

Executors and wakeups 261, 283

FFI ownership and callbacks 376

Generational handles and arenas 94

Initialization and validity 28, 341, 365

Iterators and lending 152

Layout alignment and representation 330

Lifetimes and higher ranked bounds 51, 60, 141, 152

Lock scope and deadlock 231, 316, 450, 484

Macros and hygiene 197

Memory ordering and reclamation 251, 341

Miri fuzzing and property tests 413, 484

Monomorphization and code size 118, 130, 400

Moves places and drop flags 18, 28

Pin Unpin and projection 273

Pointer provenance and aliasing 341, 353

Protocol state machines 174, 316, 461

Public APIs and compatibility 186, 473

RAII and resource ownership 18, 71, 294

Send Sync and scoped threads 221, 305

Snapshots and optimistic transactions 439

Soundness and unsafe review 341, 353, 365, 484

Trait obligations and bounds 106, 130

Typestate and newtypes 174, 461

Variance and PhantomData 60